Security Governance - Notes

IMPORTANT WARNING

- These notes have been made starting from my notes taken during lessons and the power points published by the professor, then rearranged through the help of Claude to produce a narrative flow.
- All the content belongs to the Professor of the course.
- While I try to ensure everything is inside these notes, please don't use them as a "source of truth", and study from official sources as well.

Chapter 1: Introduction

The Enterprise as a Complex System

Every organization exists to fulfil a **mission**: the fundamental business objective that justifies its existence and defines what it delivers. An energy provider produces and sells energy; a manufacturer produces and distributes goods; a bank safeguards and moves money. The mission is the anchor from which every other organizational choice descends. Once the objective is fixed, the enterprise must structure itself to reach that objective reliably and repeatably.

From the outside, an enterprise is a black box: it consumes inputs and produces goods or services for customers. Inside sits a structured machine built from three interlocking ingredients.

- **People** — the employees whose work moves the organization toward its objective.
- **Processes** — the structured sequences of activity through which work is carried out, formalized under Business Process Management (BPM), in which people and technology transform inputs into outputs.
- **Digital infrastructure** — the technological substrate of servers, networks, applications, and data on which people rely to perform their processes.

How these three are arranged depends heavily on scale. A small company tends toward a flat structure with few intermediaries; a large enterprise develops a deep hierarchy with many layers of coordination.

This composition is what makes the enterprise a **complex system**, and it is also the source of its first and most persistent vulnerability. Because the digital infrastructure is operated by human beings, the point of contact between people and technology becomes the weakest link. Managing the security of even a small company is never trivial, and the difficulty grows sharply with organizational size and complexity.

The Concept of Governance

Among the many processes an enterprise runs, this treatment concentrates on **governance**. In its broadest sense, governance is the totality of the processes of governing, whether undertaken by a government, a market, or a network, and whether they act over a family, a tribe, a formal or informal organization, a territory, or across territories. It operates through the laws, norms, power, and language of an organized society. In operational terms, governance is the set of processes of interaction and decision-making among the actors involved in a collective problem, and these processes lead to the creation, reinforcement, or reproduction of social norms and institutions.

Two words distill the essence of this definition: **processes** and **norms**. To govern is to look at the business objective and put in place everything necessary to reach it, which means identifying the required processes and then operating them so that the objective is achieved *in compliance with the surrounding environment.* Governance therefore carries a double burden. It is not enough to design processes from a purely functional point of view, asking only whether they work; the **compliance** dimension must be kept in view at the same time, ensuring the processes respect the applicable norms. Those norms may be external laws, internal company rules, contractual obligations, or regulatory prescriptions. Governance without compliance is incomplete governance.

Enterprise, Corporate, and Business Governance

The most general form of this discipline is **enterprise governance**: how to govern the company as a whole. Formally, it is the structure and the set of relationships that control, direct, or regulate the performance of an enterprise together with its projects, portfolios, infrastructure, and processes — the exercise of identifying the processes necessary to control the company itself. Enterprise governance splits into two complementary halves.

Corporate governance is the system of rules, practices, and processes by which a firm is directed and controlled. It is essentially concerned with **compliance processes**: a general, mission-independent approach that guarantees the firm is run in a controlled and accountable way, applicable to any company regardless of what it produces.

Business governance is the set of policies and business processes that determine the way the organization's business is actually run. It is concerned with **performance processes** and, unlike corporate governance, is highly specific to the mission. Where corporate governance asks whether the firm is directed and controlled properly in a generic sense, business governance asks whether the firm is performing well against its particular objective.

	Corporate governance	Business governance
Concern	Compliance processes	Performance processes
Scope	General, mission-independent	Specific to the mission
Orientation	Compliance-oriented	Performance-oriented

The Hierarchical Structure and the Direct/Control Loop

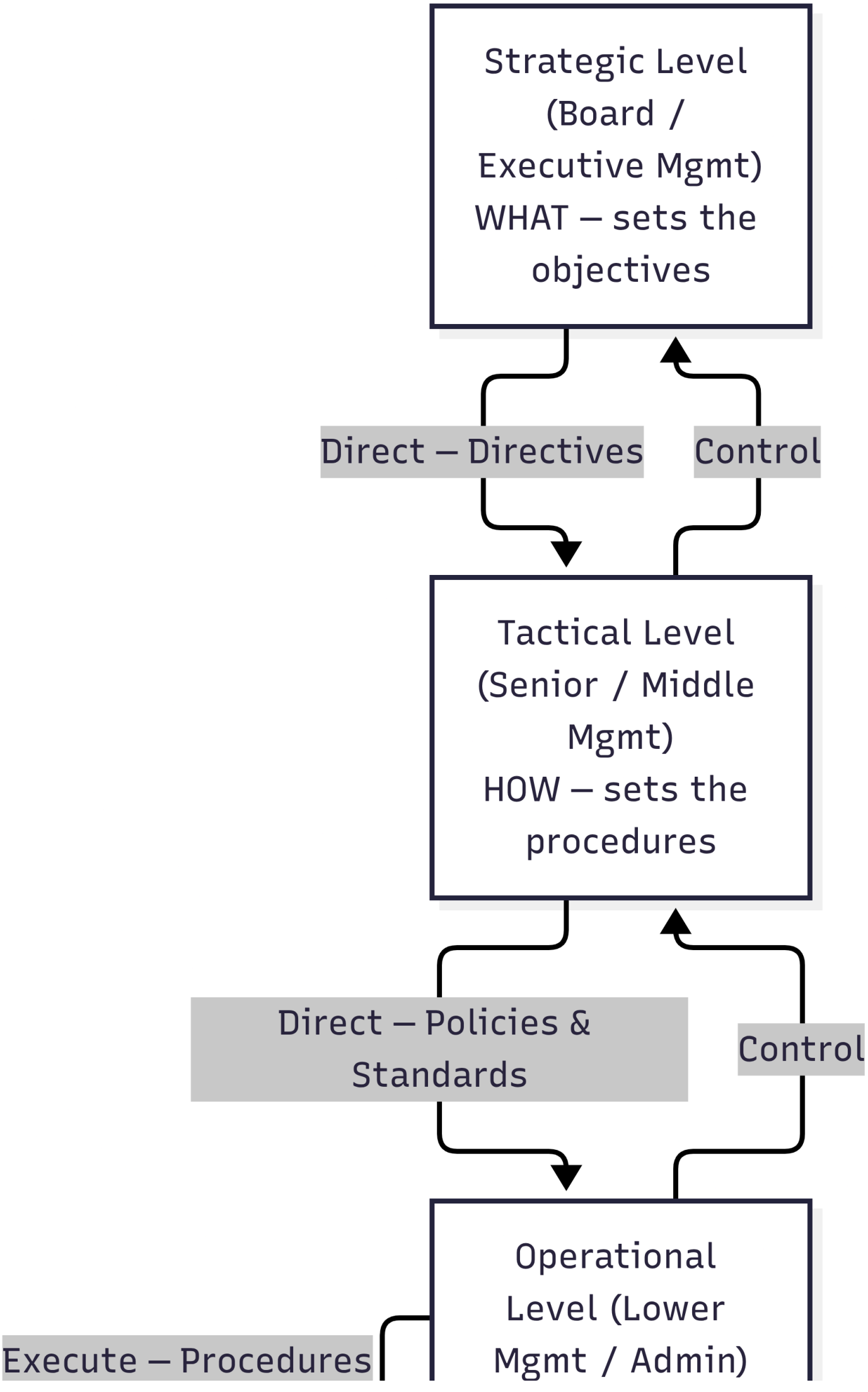
An enterprise is typically organized as a **pyramid**. At the apex sit a small number of people who make broad, high-consequence decisions; those decisions propagate downward, becoming progressively more concrete at each descent. This vertical arrangement gives rise to three management levels, and the mechanics of corporate governance can be read directly off them.

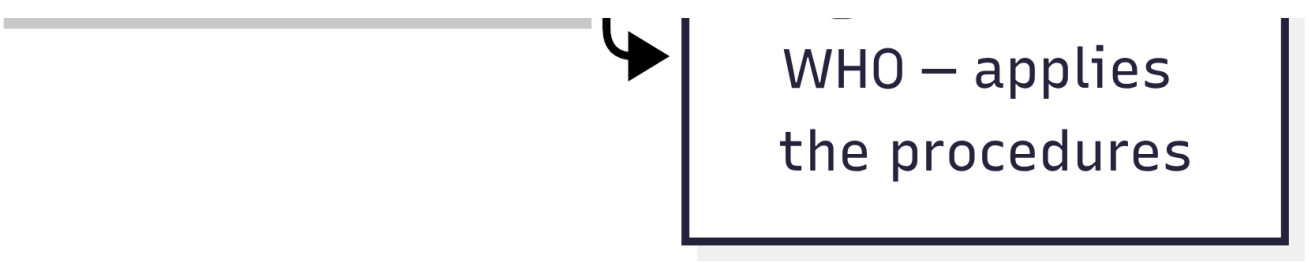
At the **strategic level** — the Board of Directors and executive management — decisions about cybersecurity are taken at their most abstract. Directors formalize them into **Directives**, documents that state the strategic objectives of the company. A directive at this altitude might declare that the infrastructure must be protected from external attackers, but it deliberately refrains from specifying *how*. The strategic level owns the **WHAT**: it sets the objectives.

At the **tactical level** — senior and middle management — the mandate received from above is translated into more concrete objectives. The discussion turns to how things should be done, and the abstract goal begins to acquire shape. Here one identifies the procedures needed to protect the assets, defining the **policies** and **company standards** that will govern behaviour. These standards are not necessarily imposed from outside by bodies such as ISO; they are frequently the internal standards the company chooses to adopt for itself. The tactical level is a *meta* layer: it defines the way things must be done without yet touching the machines. It owns the **HOW**: it sets the procedures.

At the **operational level** — lower management and administration — the middle-level objectives are finally converted into practical reality. The abstract instruction to deploy, say, an identity management system becomes a concrete decision about who does exactly what, on which machine, with which configuration. This is the layer where people put their hands on the systems. It owns the **WHO**: it applies the procedures, and it is where things are actually done.

These three levels are bound together by a continuous loop of three motions. **Direct** is the downward flow: it translates big, abstract objectives into concrete implementation procedures as they descend from strategic to tactical to operational. **Execute** is the practical act of doing the work at the operational level. **Control** is the upward flow: it observes whether what is being done actually works as expected and feeds the resulting information back up to management. When the feedback arrives at the top, management obtains a snapshot of the real situation — an understanding of how the company is genuinely behaving — and on that basis decides to hold course or to change direction, re-drafting the objectives as needed.


Strategic Level (Board / Executive Mgmt) WHAT – sets the objectives
Direct – Directives
Control
Tactical Level (Senior / Middle Mgmt) HOW – sets the procedures
Direct – Policies & Standards
Control
Operational Level (Lower Mgmt / Admin)
Execute – Procedures



Governance is therefore the implementation of a continuous loop. It is not a one-off act but a perpetual cycle of directing, executing, controlling, and re-directing.

Vulnerabilities as the Enemies of Governance

Once this loop is running inside a real company, it must contend with **vulnerabilities**, the natural enemies of the governance process. They arise for two broad reasons. The first is that the technical infrastructure itself changes over time and inevitably contains bugs, errors, and even backdoors. The second, and more insidious, is the interaction between the processes and the infrastructure, and above all the interaction between *people* and the infrastructure. The most dangerous moment in the whole governance enterprise is the moment at which people come into contact with the systems.

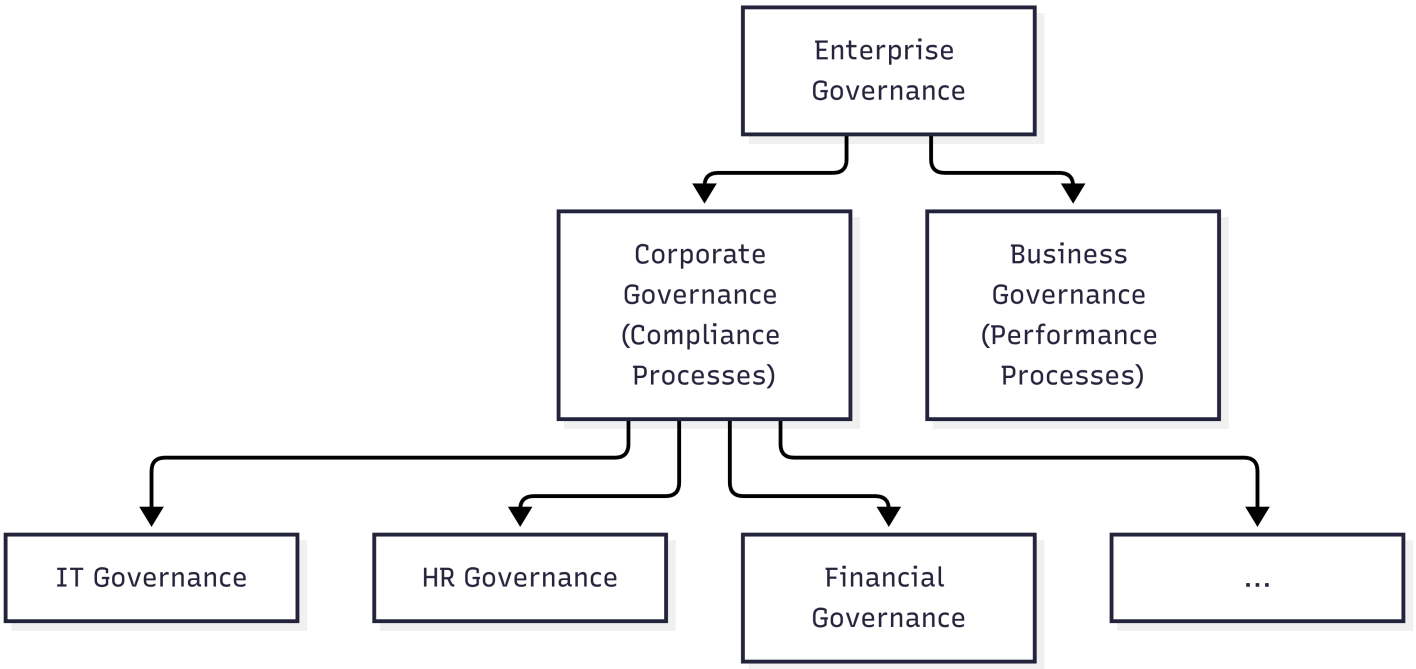
Risk as the Governing Metric

To steer the direct/control loop intelligently, it must be steered against a measurable quantity. That quantity is **risk**, which may take financial, production-related, cybersecurity, or other forms. The IT Governance Institute captured this centrality when it redefined enterprise governance as the set of responsibilities and practices exercised by the board and executive management with the aim of providing strategic direction, ensuring objectives are achieved, ascertaining that risks are managed appropriately, and verifying that the enterprise's resources are used responsibly.

Once risk is accepted as the governing metric, the governance problem collapses into a clean **optimization process**. The direct half of the loop is charged with a single objective: reduce risk, by putting in place actions that lower the risk level. The control half is charged with the complementary objective: measure the reduction, by taking measurements, recomputing the risk, and aggregating it into a coherent picture. Governance thus becomes the continual minimization of risk under observation.

Decomposing Governance: IT Governance and Assets

Supporting business and corporate governance in practice requires more specialized sub-governances, each dedicated to a particular domain. Enterprise governance decomposes into corporate and business governance at the top, and beneath these sit the domain-specific sub-systems: IT governance, HR governance, financial governance, and others.



IT governance is the set of processes that ensure the effective and efficient use of IT in enabling an organization to achieve its goals. It begins from the infrastructures and examines every kind of risk that arises from the digital environment, especially the risks that arise from the *offering of services*. IT-based systems create serious risks precisely because they manage and store the majority — if not the entirety — of a company's electronic assets: the data and information stored inside the organization, the data and information transmitted across its networks, and all the systems and applications required to store, transmit, and process that data. These electronic assets are exposed to a wide range of threats aimed at compromising their three cardinal properties: **Confidentiality, Integrity, and Availability**, abbreviated **CIA**.

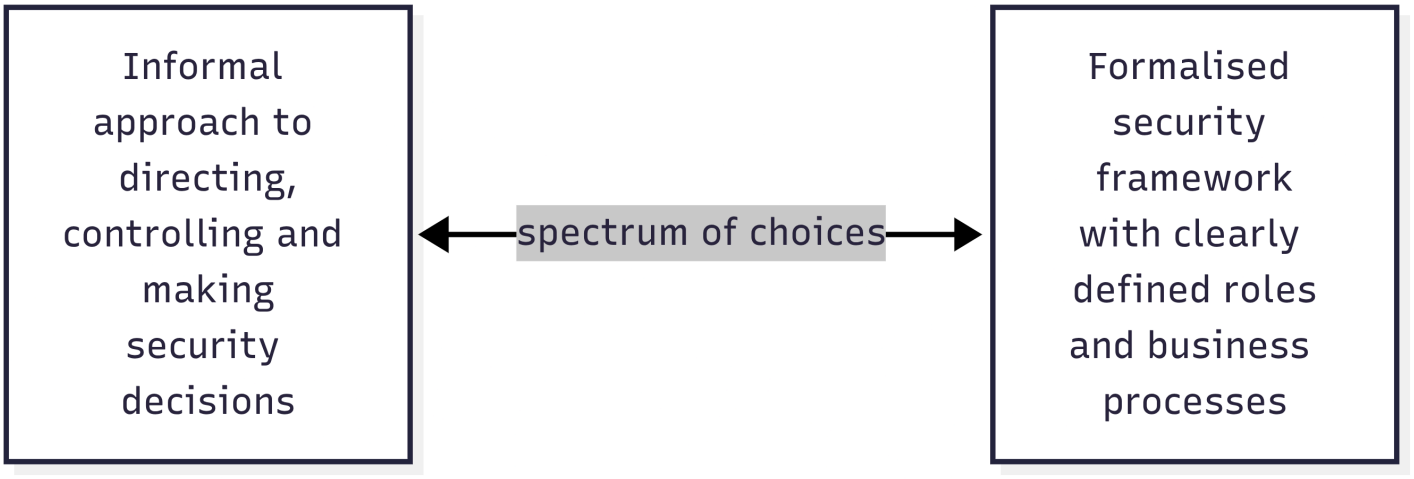
The decisive insight of IT governance is that the infrastructural view must be fused with the business view, because the two views assign radically different importance to physically identical machines. Consider a network of interconnected servers. On one server sits an archive of data; on another, identical machine sits the web server hosting the company's online shopping site. Through the infrastructure lens alone, the two servers look equivalent — both connected, both running, both nodes in the topology. The moment the business lens is applied, the equivalence collapses. If an attacker brings down the shopping site, the company can no longer serve its customers and immediately begins to lose money, which is a serious risk. If an attacker damages the archive, the data can very often be restored from other sources, which is a comparatively minor risk. The machines are the same; the risk they carry is not. This is why IT governance insists on considering all aspects of the digital world — not merely the machines but everything operating around them — and on first identifying which assets genuinely matter. Running a live service is an asset in the critical sense; the archive is not an asset of the same order.

The threat landscape is not confined to the outside. One must protect against external attackers arriving from the Internet, but one must equally guard against attackers already *inside* the organization. Employees who have become unhappy are a particularly grave danger: they know the assets intimately and understand how the infrastructure is managed, and this

superior knowledge makes them far more dangerous than an outsider who must first discover what an insider already knows.

Choosing an Approach: The Spectrum of Formalization

Governance is a risk-driven control loop operating over identified assets, so the practical question becomes how to implement that loop in a concrete company. There is no single universal answer and **no "one size fits all" approach**. Instead there is a spectrum. At one extreme lies a fully **formalized security framework**, with clearly defined roles and rigorously defined business processes. At the opposite extreme lies an **informal approach** to directing, controlling, and making security decisions. Every real company must find its proper place along this continuum.



Four questions guide a company toward the right level of formalization.

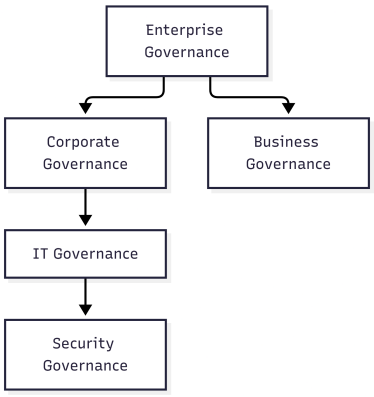
- **Size and complexity** — how large and how complex is the organization? Small companies must lean on flexibility because they typically lack the resources for heavy formalism, while large companies will most probably require a formalized framework simply to keep coordination tractable.
- **Resources** — what budget is actually available for security governance? A formalized approach is expensive, demanding certifications and sustained investment, so a company without money cannot realistically adopt it.
- **Mission and the importance of security to it** — what does the organization do, and how important is security to those aims? For a bank, security is a top priority and a matter of existential dependence; for a manufacturer, security still matters but the business does not hang on the same human-facing service dependencies.
- **External considerations** — are there contractual, legal, regulatory, or sector-specific requirements pushing the company toward a particular level of formalism? A small company might prefer an informal posture yet be obliged to obtain certification because it works for a public organization that demands it.

Answering these questions well requires analysing the **context**, and in practical terms the correct approach comes down to identifying three things: the **what** (the security decisions to be made), the **who** (the deciders who will make them), and the **data** (the information

required to make sensible, informed choices). Together, decisions, deciders, and the information feeding them constitute a **security framework**.

Positioning and Purpose of Security Governance

Security governance can now be located precisely within the larger structure. It is the means by which an organization controls and directs its approach to security, and when done well it coordinates all of the organization's security activities and enables the flow of security information and security decisions throughout the organization. Structurally, security governance is a **vertical** concern that cuts across the other governances: it sits within IT governance, which sits within corporate governance, which — alongside business governance — sits within enterprise governance.



The reasons to care about security governance follow directly from this position. It provides a structured approach to managing and mitigating cybersecurity risks within the organization. It helps protect sensitive data, maintain regulatory compliance, and strengthen the trust and confidence of stakeholders. By implementing effective security governance, an organization reduces both the likelihood and the impact of security incidents, thereby safeguarding its reputation and its business operations.

This defines the arc of the entire subject. The study of security governance proceeds through models for information security governance and the vertical nature of security; through cybersecurity best practices, guidelines, and frameworks and how to use them; through the assessment and management of risk, which entails understanding threats, vulnerabilities, and attacks, evaluating their impact, and finding proper countermeasures; through the modelling and understanding of threats; through the management of incidents; and finally through legislation, compliance, and insurance, closing on open research issues in security governance.

A Model for Information Security Governance

Recap: What Information Security Governance Is

Information Security Governance (ISG) is the process of establishing and maintaining a framework, together with the supporting management structure and processes, that provides assurance of several things at once:

- that information security strategies are aligned with and support the business objectives;
- that they are consistent with applicable laws and regulations through adherence to policies and internal controls;
- that responsibility is properly assigned throughout the organization.

All of this is undertaken in an effort to manage risk. Said more plainly, ISG is the global effort a company makes to ensure the well-being of its electronic resources.

The Principle of Cost-Effectiveness

The first foundational principle on which any ISG framework must be built is that it has to be **cost-effective**. Security is expensive, so there must be a deliberate balance between the cost of protecting electronic resources and the risk to which those resources are exposed. This balance has two failure modes, each equally to be avoided. **Overprotection** wastes money on defences whose cost exceeds the harm they prevent, generating unnecessary expense. **Underprotection** leaves genuine exposures unaddressed, allowing risks to materialize and strike the company. Good governance lives in the narrow band between these two errors.

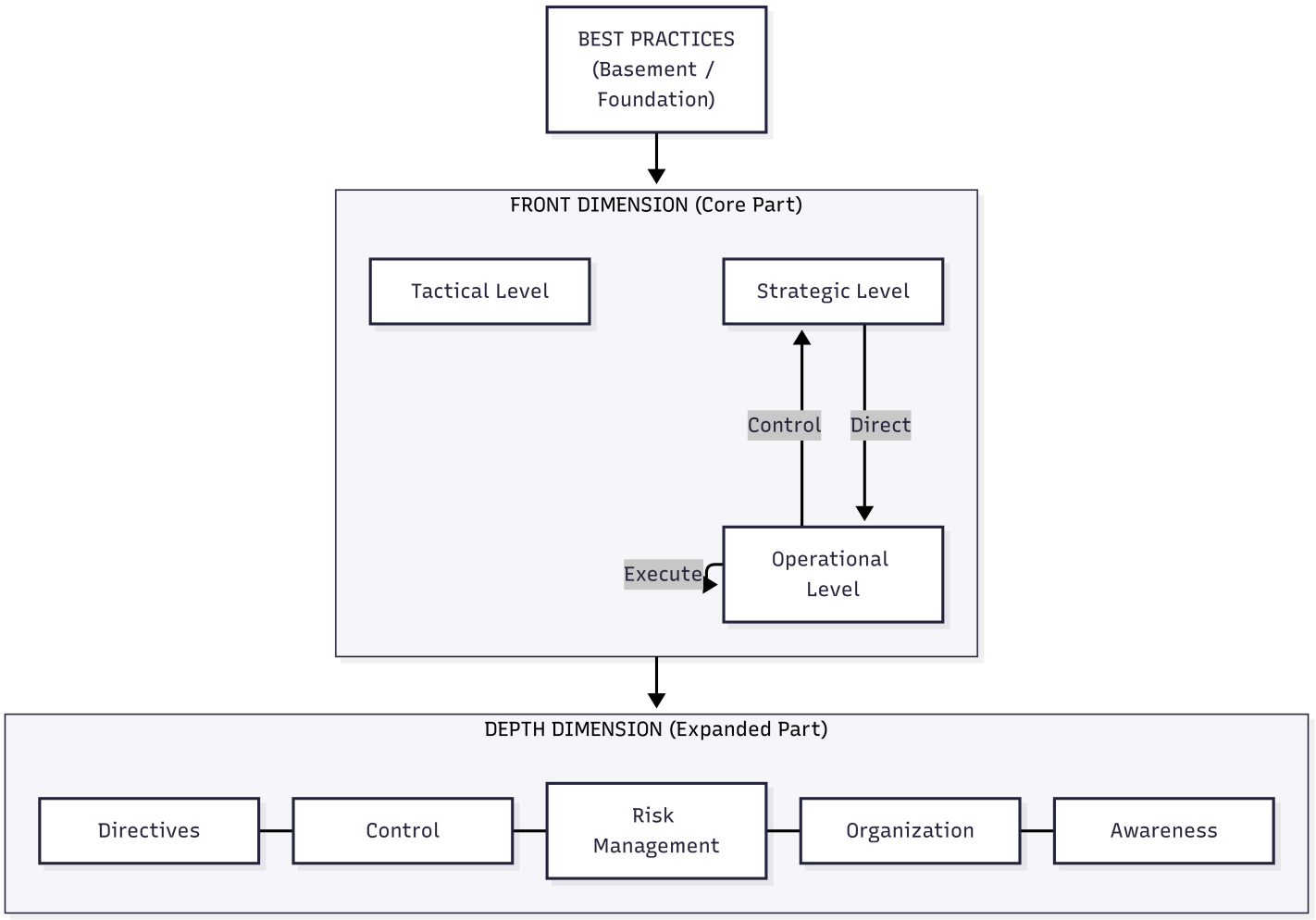
Consider how the balance is actually struck. Suppose a risk assessment has produced a list of risks, each tagged with an associated security level. In an idealized world with no budget limit, one could march down the list and remediate every risk until none remained. Real organizations operate under a **fixed budget** and must decide where that budget goes, starting from the most critical risks and working downward. A characteristic situation then arises in which most of the budget has been consumed fixing the critical problems and a residual amount remains. At this juncture the decision must be made carefully, weighing the cost of fixing a given remaining risk against the cost that would be incurred if that risk actually materialized, neither overspending on marginal exposures nor abandoning issues that quietly matter. Even the act of *identifying* a risk depends on acquiring and analysing data with the right instruments; if the proper instruments are not used, important points may be missed and the balance will be struck on incomplete information. For these reasons a good **IT risk management strategy** is mandatory as the precondition for a good ISG strategy, and good governance demands a strategy that weighs multiple aspects rather than fixating on the infrastructure alone.

The Von Solms Model: Overview

The model used to organize all of these elements was introduced by **R. Von Solms in 2006**. It can be pictured as a solid three-dimensional wedge that gathers every element one must keep in mind and arranges them in an interconnected structure. The wedge has three distinguishable parts.

- **Front Dimension (Core Part)** — the triangular face of the wedge; the implementation of the control loop, spanning the three management layers of the pyramid.
- **Depth Dimension (Expanded Part)** — five layers extending backward from the front face: Directives, Control, Risk Management, Organization, and Awareness.

- **Basement** — the **Best Practices** layer on which the whole wedge rests, the foundation from which everything else is constructed. Best practices are the documents that collect the common, accumulated knowledge of the field in cybersecurity and management.



The Front Dimension (Core Part)

The Front Dimension represents the execution of processes and actions and the influence of the Direct and Control loop upon those processes. It rests on two core principles.

Core Principle 1 states that the model covers the three management levels — strategic, tactical, and operational — and it answers the question of *why all three levels must be covered*: every level possesses its own distinct view of the company, and no single level can see the whole. At the top, a handful of people take general decisions and, when the direct process begins, they possess only ideas about what to do; the directive documents they write are therefore quite general. At the bottom, people are in direct contact with the system, laying their hands on the machines and producing procedures, and their scope is extremely detailed and correspondingly restricted. Because each person's view is limited, the intermediate tactical layer is indispensable: its function is to translate the general into the detailed on the way down and the detailed back into the general on the way up. This is why the processes must be able to flow *vertically* in both directions. On the control path, the operational level begins by collecting information of great precision — numbers of alerts, flows across different machines, and similar indicators — and this precise information is progressively correlated with the

procedures and abstracted as it climbs, so the upward movement is from a very detailed view toward an increasingly summarized one.

Core Principle 2 states that across these three levels there are three very distinct *actions*, and that control at the bottom is not the same activity as control at the top. The rationale is clear from the Board's perspective: the Board is not interested in learning that 321 virus attacks occurred last week, that 27 software patches were applied, and that 7 anti-virus updates took place, since this is far too much detail to plan with. What the Board wants to know is whether the company's access to the Internet causes any *unmitigated* risks. If there are none, the Board is content; if there are some, the Board wants to be told precisely which unmitigated risks these are. The action of control must therefore be qualitatively different at each altitude.

The Direct Part, Layer by Layer

The downward Direct process can be read as a matrix in which each management layer has a distinct meaning, a specific input, and a specific output, and in which the output of one layer becomes the input of the next.

Layer	To direct means	Input	Output
Strategic	Identify the assets, their relevance, and their required level of protection (the strategic level alone has the overview of the whole company)	**Context**: internal factors (strategic vision, role of IT, competitiveness, mission, constraints, production processes) and external factors (legal and regulatory prescriptions, other external risks)	A set of **Directives** indicating, at a high level, what the Board expects for the protection of information assets
Tactical	Expand the directives into relevant information security policies, company standards, and procedures; analyse the macro-objective against the situation and translate it into middle-level objectives	The set of directives produced above	A collection of **policies, procedures, and standards** the company must comply with (how digital resources are managed, how accounts are handled, and so on)

Layer	To direct means	Input	Output
Operational	Expand the inputs further into administrative guidelines and administrative procedures; physically implement and manage the technical measures	The policies, procedures, and standards above	A set of **operating procedures** specifying exactly how things must be done — the basis of execution at the lowest level

The Control Part, Layer by Layer

The upward Control process is governed by a single, decisive maxim: **you can only manage what you can measure**. To control one must measure, and to measure one must know in advance which information and data to collect. This *measurability* characteristic cannot be an afterthought; it must sit at the centre of all directives, policies, standards, and procedures produced during the Direct part, so that the objects being directed are constructed to be measurable in the first place. The Control matrix mirrors the Direct matrix but runs in the opposite direction, from operational up to strategic.

Layer	To control means	Input / Output
Operational	Extract measurement data from a wide range of entities, automatically or manually, by deploying monitors (**agents**) into the environment. Agents may live on the machines themselves or in the network, and their power depends on their observation range. A fundamental tension applies: **the need for monitoring must not compete with the needs of the business** — observing everything is expensive and degrades the system's speed, so monitoring must not smother the business it protects	Input: raw **measurements**. Output: a set of **specialized reports** built from the extracted operational data, passed upward
Tactical	Measurement and monitoring against the requirements of the relevant policies, procedures, and standards. The tactical layer receives all operational reports, aggregates them, and attempts to identify existing threats, evaluating whether the policies in force are sufficient	Output: a set of **tactical management reports** indicating the levels of compliance and conformance, passed to the strategic level

Layer	To control means	Input / Output
Strategic	Control culminates in **Situational Awareness**	Output: **reports reflecting compliance and conformance to the relevant directives, including risk considerations** — the snapshot the Board needs to hold course or re-direct

The Basement: Best Practices

Underneath both the Front and the Depth dimensions lies the layer of **best practices**, which sets the ground on which all operations are instantiated. Best practices are documents collecting the knowledge people have acquired over the years in security and management. The specific best practice one reaches for depends on the specific problem to be solved, and the required granularity depends on the management level being served.

The purpose of the whole ISG model, phrased memorably, is to help the organization in *'doing the right things right.'* This slogan resolves into two questions that every **Information Security Manager (ISM)** must be able to answer: *how do I know what the right things are?* and, supposing that is known, *how do I know I am doing them right?* The answer to both is to look to best practices. Formally, best practices — also called standards or guidelines — are a set of documents reporting the experiences and solutions arrived at by experts in information security management. They are the consensus of experts in the field and provide an internationally accepted framework usable as a building block for ISG. Many exist, each focused on related but different aspects, including the **ISO 27000 family**, **COBIT**, the **NIST SP 800-*** series, the **CSC** (Critical Security Controls), and the **NIST CSF** (Cybersecurity Framework), among others.

ISO 27002

Among these best practices, **ISO 27002** deserves close attention because companies use it to set up their infrastructures from the policy point of view. It is an International Standard designed to be used by organizations that intend to do any of three things: select controls within the process of implementing an Information Security Management System (ISMS) based on ISO/IEC 27001; implement commonly accepted information security controls; or develop their own information security management guidelines.

ISO 27002 is fundamentally a collection of **controls** — points that need to be verified. In its 2013 edition it contains **14 security control clauses**, which collectively hold a total of **35 main security categories** and **114 controls**. These controls are general security controls and are not themselves objects of certification; they function as instructions rather than as a pass/fail certificate. The structure is strictly hierarchical: each clause contains one or more main security categories, and each main category in turn contains a **control objective** stating

what needs to be achieved and one or more **controls** that can be applied to achieve that objective.

The description of each individual control is itself standardized into a fixed structure of three parts.

- **Control** — defines the specific control statement needed to satisfy the control objective.
- **Implementation Guidance** — a longer text providing more detailed information to support implementation of the control and the meeting of its objective, carrying an explicit caveat that it may not be entirely suitable or sufficient in every situation and may not fulfil an organization's specific control requirements.
- **Other Information** — further considerations such as legal factors and references to other standards, simply omitted when there is nothing further to say.

In this way a control functions as a security checklist item that matches the requirements for certification, accompanied by a guideline explaining one possible way to implement something compliant with the check.

ISO 27001 and the Distinction Between the Two Standards

ISO 27001 is the companion International Standard that specifies the requirements for establishing, implementing, maintaining, and continually improving an ISMS within the context of the organization. It also includes requirements for the assessment and treatment of information security risks tailored to the organization's needs, and it is deliberately generic, intended to be applicable to all organizations regardless of type, size, or nature. Crucially, ISO 27001 is the standard that grants **certification**: it certifies that a system is compliant with its requirements and operates as a checklist of security checks a company must satisfy to be formally certified.

The relationship between the two standards is a division of labour, and the difference between them is the difference between a requirement and a guideline. ISO 27001 is very specific and strict, spelling out in detail what a company must comply with and must have in place to be formally certified. ISO 27002, by contrast, is a *guideline* document that advises companies on what they should have in place for their information security management in order to follow best practice, guiding them to structure their management according to the accumulated experience of other companies. It is deliberately high-level and does not drill into very detailed specifics.

	ISO 27001	**ISO 27002**
Nature	Requirement	Guideline
Tone	Specific and strict	High-level and advisory
Role	Certifies	Guides

The Directive and the Information Security Policy Architecture (ISPA)

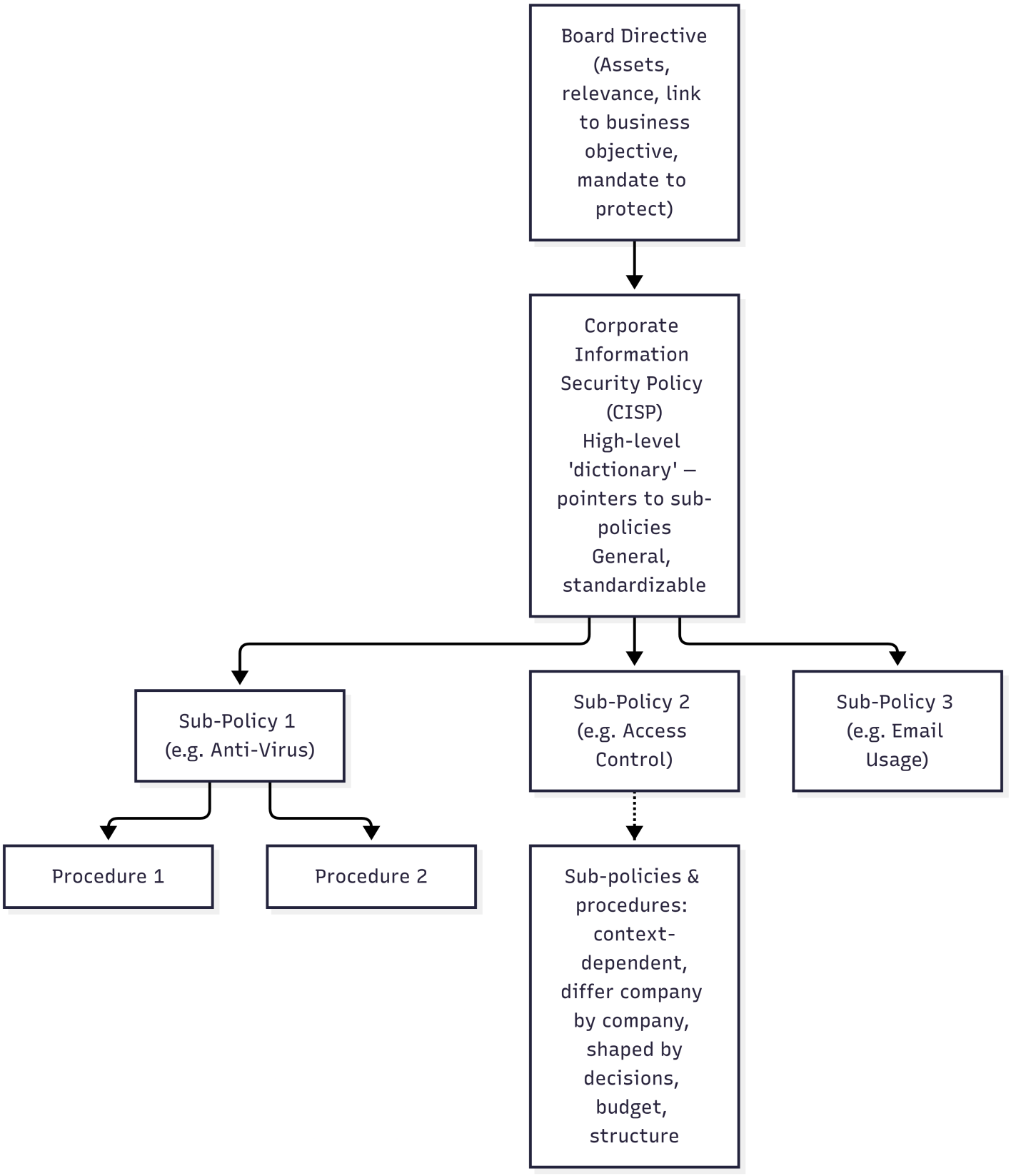
Why a Documented Direct Process Is Needed

Every best practice contains, in some form, the notion of a **Directive**, and the task it imposes is concrete: to set up the documents and policies that dictate how security is to be governed inside a company. Chapter 5 of ISO 27002, for instance, requires that decisions be taken and acted upon in a *structured* way, which in effect demands the design of a process that supports all three management layers. To meet that demand, an organization must develop a **framework** — a methodological instrument, a defined set of things that must be done, capable of supporting the Direct process. This framework is not a single document but an interconnected architecture of documents, each linked to the others, which together implement the guidelines of the downward direct flow.

Everything must be written down because documentation is what confers **trust and value**. By formalizing rules and policies on paper, the organization gives weight and accountability to what it does; an undocumented intention carries no assurance, whereas a documented one can be communicated, enforced, audited, and relied upon. Because the Direct process flows from top to bottom, and because each layer operates at a different degree of abstraction, the architecture must provide a set of documents calibrated to the right level of information at each layer. A general document is produced at the top and connected downward to progressively more specific documents produced in the layers beneath it.

The Document Tree

The architecture of the Direct process is best visualized as a **tree**. At the root sits the **Board Directive**. One level below sits the **Corporate Information Security Policy (CISP)**, which receives the directives from the board and transforms them into guidelines for the implementation of security controls. From the CISP a set of **sub-policies** branches out, one for each distinct domain of concern — access control, human resource management, device management, and so on. From those sub-policies are derived the **company standards**, which express the minimum level of specification that must be achieved, and finally, at the operational layer, the **procedures**, which are protocols: ordered lists of concrete actions.



A crucial distinction runs through this tree. The **Board Directive** and the **CISP** are relatively general and standardizable, because almost every company shares essentially the same fundamental need for protection, so the upper two documents can look broadly similar across organizations. The **sub-policies** and **procedures**, by contrast, are strictly context-dependent: they vary from company to company and are shaped by the particular decisions the organization wishes to enforce, by its structure, and by the budget it has available.

The Board Directive

At the top of the tree, the Board Directive carries the raw material of the entire Direct process. It must contain two things above all: the **set of IT assets identified by the Board**, together with their relevance and their connection to the business objective, and the **mandate from the Board to protect those assets**. The inclusion of relevance is essential, because protection must proceed *asset by asset* — not every asset carries the same importance, and the directive must make those differences explicit so that protection effort can be allocated accordingly.

The Corporate Information Security Policy (CISP)

One level lower sits the CISP, the **high-level document that provides the basis for all lower-level documents related to information security**. Its function is to take the objective expressed in the Board Directive, translate it into a middle-layer objective, and from that translation extend a set of pointers aimed at the specific sub-policy documents below. In this sense the CISP operates as a kind of **dictionary**: it lets the reader interpret and map the sub-policies against the original board directive, binding the detailed and the general together.

The methodology offers ten guidelines for writing a proper CISP, specifying both what it should contain and how it should be formalized.

1. It must indicate **board and executive management support and commitment** and make clear that the CISP flows from a higher-level directive; the link back to the Board Directive and the enforcement behind it must be visible.
2. It must be **accepted and signed by the CEO or an equivalent officer** — the signature of a board member is what gives the document its authority.
3. It must **not be a long document, nor written in technical form**: a maximum length of about four to five pages, containing high-level statements concerning information security rather than implementation detail.
4. It **should not change very often and must be stable** with respect to technical developments; as an intermediate working document bridging the direction and the operational levels, frequent revision would destabilize everything hanging beneath it.
5. As a direct consequence of stability, it **must not contain any references to specific technologies and must be technology-neutral**, since technology evolves far too frequently to be enshrined in a stable document.
6. It must indicate **who owns the policy** and what the responsibilities of other relevant people are; stating roles and responsibilities produces a map of who is supposed to do what and who is supposed to monitor it.
7. It must clearly indicate the **scope** of the policy — all the people who will be subject to it.
8. It must refer to possible **disciplinary actions for non-conformance**, both to the CISP itself and to its lower-level constituent policies; these disincentives are necessary because of the human factor, since people tend to look for an escape strategy when confronted with something difficult.

9. It must be **distributed as widely as possible** throughout the company and covered in all relevant awareness courses, so that employees understand both the policies and the reasons they exist.
10. It must contain a **Compliance Clause**, the element that ties the document directly into the Control process.

A representative CISP is organized into clearly named but deliberately general sections: the importance of information to the company, what information security is, why the company needs it, the information security commitment of the board, the scope, the information security statements that make up the policy, the responsibilities, and the compliance clause. The document is short and its section titles are generic. Where it becomes concrete is in the **scope**, which must define exactly who is affected: typically all of the company's IT assets stored, processed, and distributed via its information processing systems; any person granted authorization to access those resources, including permanent, temporary, third-party, and contractual employees and users; and all business partners and clients that access the company's IT assets in any way.

Sub-Policies

Descending one further level, the **sub-policies** take the mandate to protect the assets — already translated into macro-objectives by the CISP — and define specific policies for the specific aspects that must be considered. These documents are **company-dependent**, defined by each organization according to its own structure and circumstances. Certain categories, however, apply to essentially everybody and are commonly defined by all organizations: a **Malicious Software Control Policy** (an anti-virus policy), an **Acceptable Internet Usage Policy**, an **Acceptable Email Usage Policy**, a **Logical Access Control Policy**, a **Disaster Recovery (Backup) Policy**, a **Remote Access Control Policy**, and a **Third Party Access Control Policy**.

One requirement overrides all others at this level and is marked as critically important: the ISPA must **clearly indicate where each of these policies originates**. Every sub-policy must be *linked* or *traced back* to the CISP, so that the chain of authority from the board directive down to the individual policy is never broken. A well-formed sub-policy therefore carries an explicit **Reference** section pointing *up* to the statements in the CISP that give it origin and *down* to the lower-level procedure that supports it, along with a **Main Policy Statement** and its supporting sub-statements. For example, an Internet Acceptable Use sub-policy would state that use of the company's Internet service must be in support of official company work, that all user requests for connectivity must be approved and supported by the employee's line manager and made through a specified application procedure, and that it is the line manager's responsibility to ensure the user does not misuse the service.

Procedures

At the bottom of the tree lies the **operational level**, inhabited by the **procedures**, which are effectively **algorithmic steps**. Where a sub-policy declares what must be true, a procedure

specifies the exact ordered actions that make it true. In the running example, a user requesting Internet connectivity triggers an application procedure that states precisely what must be done: the applicant records identifying details, signs a declaration that they have read both the CISP and the relevant Internet Acceptable Use sub-policy and will act in line with them, and obtains the signatures of the line manager and the information security officer. A procedure of this kind must also clearly state **what it means to be compliant with it**, carrying its own compliance clause so that conformance can later be checked.

Once the full set of documents is in place — stating, from top to bottom, what the organization needs to do — the procedures are what get implemented in the **Execution** part of the model. Execution is simply the carrying-out of those procedures, and the role of the Control process that follows is to verify that everything is in fact working as the documents require.

The Control Part and Compliance

Compliance as the Scope of Control

Once the documents of the Direct process exist and the procedures have been executed, the model turns upward into the **Control** part, whose main scope is the **compliance check**. ISO 27002 makes this requirement explicit: after everything has been put in place, the organization must verify compliance at the various levels — compliance with external laws and contractual requirements, and compliance with the security policies and standards it has established internally. The standard devotes an entire clause to this, titled **Compliance**, with the objective of avoiding breaches of legal, statutory, regulatory, or contractual obligations related to information security and of any security requirements.

Two of its controls capture the essence of the task.

- **Identification of applicable legislation and contractual requirements** — all relevant legislative, statutory, regulatory, and contractual requirements, and the organization's approach to meeting them, must be explicitly identified, documented, and kept up to date for each information system and for the organization as a whole, with the specific controls and individual responsibilities also defined.
- **Compliance with security policies and standards** — managers must regularly review the compliance of information processing and procedures within their area of responsibility against the appropriate security policies, standards, and other security requirements.

ISO 27002 further requires management to initiate an **independent review**, carried out by individuals independent of the area under review — the internal audit function, an independent manager, or an external specialist — to ensure the continuing suitability, adequacy, and effectiveness of the organization's approach to managing information security. The results are recorded and reported back to the management that initiated the review, and if the approach is found inadequate, corrective actions must be considered.

The Measurability Problem

The central difficulty of the Control part is defining the **compliance criteria** — deciding how to verify that the organization is in fact behaving correctly. This difficulty flows directly from the nature of the documents themselves. Policies, directives, and standards are written in **natural language**, which is neither strict nor formalized, so it is genuinely hard to establish exactly what must be verified. The problem reduces to the governing maxim: **you can assess only what you can measure**. Things that cannot be measured cannot be controlled, so the organization must define measurements that make it possible to quantify how compliant it is. For this reason the compliance clauses, which encode these measurements, must provide a clear metric that can be evaluated and assessed.

Evaluating compliance is not equally hard at every level. At the **bottom** of the pyramid it is comparatively easy, because the operational layer deals in procedures and concrete data drawn directly from the system, which can be monitored. If an access control policy declares that only authorized users may access a certain type of data, compliance can be checked simply by inspecting the system's logs. As one climbs **upward**, the criteria become far more general, and the natural question arises of what metric could possibly apply at those altitudes. The answer is the concept of **risk**. At the higher levels, verifying compliance means assessing the risk that derives from non-compliance with the policies and directives. The task is to translate the technical data extracted from the environment into something that allows the risk level to be evaluated — to convert log entries and alert counts into a statement about exposure.

The Role of Time

In carrying out this exercise, **time plays a decisive and troublesome role**. Assessing risk is expensive in time for several compounding reasons. First, the necessary information is widely spread throughout the company and must be gathered from many places. Second, that data must then be analysed, which consumes time even when software is used to assist. Third, a conclusion must be derived from the analysis. Taken together, a full risk assessment can take **weeks or even months**. It cannot be performed daily; it is something that must be planned.

This temporal cost creates a structural tension with the domain itself. In cybersecurity, **weeks constitute a very large temporal frame** — the threat environment can change completely within such a window. Worse, the world is **dynamic**, so even if the organization chooses what seems like the right assessment interval, the context captured at the start of the assessment may no longer hold by the time it concludes. Traditionally, compliance monitoring has been performed through periodic **ICT audit sessions**, in which ICT risks are evaluated, an audit report is produced, and its results are compared against the compliance clauses. The abiding issue with this approach is precisely timing: the world is dynamic, and monitoring ought ideally to be done in near real time, which the periodic-audit model struggles to deliver.

Guidelines for Writing Compliance Clauses

Unlike the Direct process, which comes equipped with templates and suggestions for writing the CISP and sub-policies, the Control process offers **no standardized content** for compliance clauses; specifying them is not an exact science, and very few guidelines are available. Only a small number of principles can be stated. A compliance clause must be **clear and precise**, deliberately avoiding ambiguity; otherwise two people performing the same assessment against the same clause might reach different conclusions because the criteria were open to interpretation. A clause such as *'Employees must take Information Security seriously'* is a textbook failure, because the word *seriously* has no fixed meaning and cannot be assessed. Beyond clarity, a compliance clause should **express a way to measure the degree to which it is satisfied**, providing a usable metric rather than a vague aspiration.

Worked Example — A Clause for the Board Directive

Consider an extract from a Board Directive in which the board accepts the importance of the company's IT infrastructure to its strategic goals, recognizes the potential impact a compromise could have, and consequently demands a proper environment to protect the IT infrastructure against any disruptions or compromises, fully supporting a proper ISG environment. Here the **asset** is the IT infrastructure and the **protection need** is protection against compromises. Two candidate compliance clauses illustrate the difference between a weak and a strong formulation.

Option 1 states that the board requires proper feedback, every three months, of the level of compliance with the directive. Its merit is a clear **time window**. Its defect is that the phrase *level of compliance* is never defined anywhere, leaving it ambiguous exactly what is to be assessed. **Option 2** states that the board requires detailed feedback, every three months, of any serious risks that may compromise the protection of the company's ICT infrastructure. This option retains the same time window but is markedly better, because it also clarifies the **metric**: the thing to be reported is *risk*. Option 2 is therefore the superior clause.

Worked Example — Clauses for the CISP

The second example concerns the CISP, a document articulated as a collection of statements that act as pointers to sub-policies. Because of this structure, it is preferable to attach a compliance clause to **each statement** (or each category) rather than to the document as a whole. Take Statement 6.1, which declares that the company will have a proper information security organizational structure to manage information security, ensuring that security roles are assigned, that users are aware of the policy and of the disciplinary consequences of non-compliance, and that implementation is continuously coordinated and reviewed.

Option 1 states that the effectiveness of the company's information security organizational structure in managing and enforcing the requirements of the ISPA shall be evaluated every six months, with the results incorporated into higher-level reporting. **Option 2** states that the way the information security function is organized, and the reporting structures within it, shall be evaluated every twelve months to ensure the structure remains adequate to protect all the company's electronic assets, with the results forming part of the annual evaluation and

reporting of organizational structures. Both share the strength of a defined **time frame**. Option 1's weakness is that it does not specify *what* must be reported, so a change in the person writing the report would produce a different view each time. Option 2 improves on this because it refers to a document already expected to be produced annually, which will most probably be structured in a consistent way — yet it, too, leaves the **metric** unexpressed, failing to say explicitly what is to be measured. The honest conclusion is that no option here is perfect, but Option 2 is better than Option 1. The recurring lesson across both examples is that a good clause pins down both a time window and a measurable metric, and that most real clauses succeed on one dimension while remaining imperfect on the other.

Risk Management

Risk as the Element That Drives Control

From the control point of view, the single element that makes the control loop implementable is **risk**. Risk is perceived differently at every layer of the hierarchy, and to deal with it coherently the organization needs a dedicated process for its analysis: **Risk Management**. This process must be **vertical**, assessing risk at every level, and the closer it operates to the bottom of the pyramid, the more tightly it is bound to the technological layer.

Formally, risk management is the process of identifying and assessing all potential risks and introducing the controls that should mitigate those risks to acceptably low levels. It is one of the security processes required to identify the risks that may impact the well-being of an organization. Its defining spirit is to begin by looking for what might be dangerous, to establish to what extent it is dangerous, and then to seek possible solutions evaluated against a **cost–benefit principle** — the same cost-effectiveness discipline that governs the whole of ISG.

The Two Dimensions of Risk

Although there are several ways to perform the process, risk is generally understood through **two orthogonal dimensions**. The first is the **probability or frequency**: how likely it is that something bad will happen. The second is the **magnitude of gains or losses**, usually called the **impact**: how severe the consequence would be if it did happen. The aim of risk management is accordingly to determine what the impact will be if the risk materializes and how often the risk might materialize. These two dimensions combine into the canonical formula:

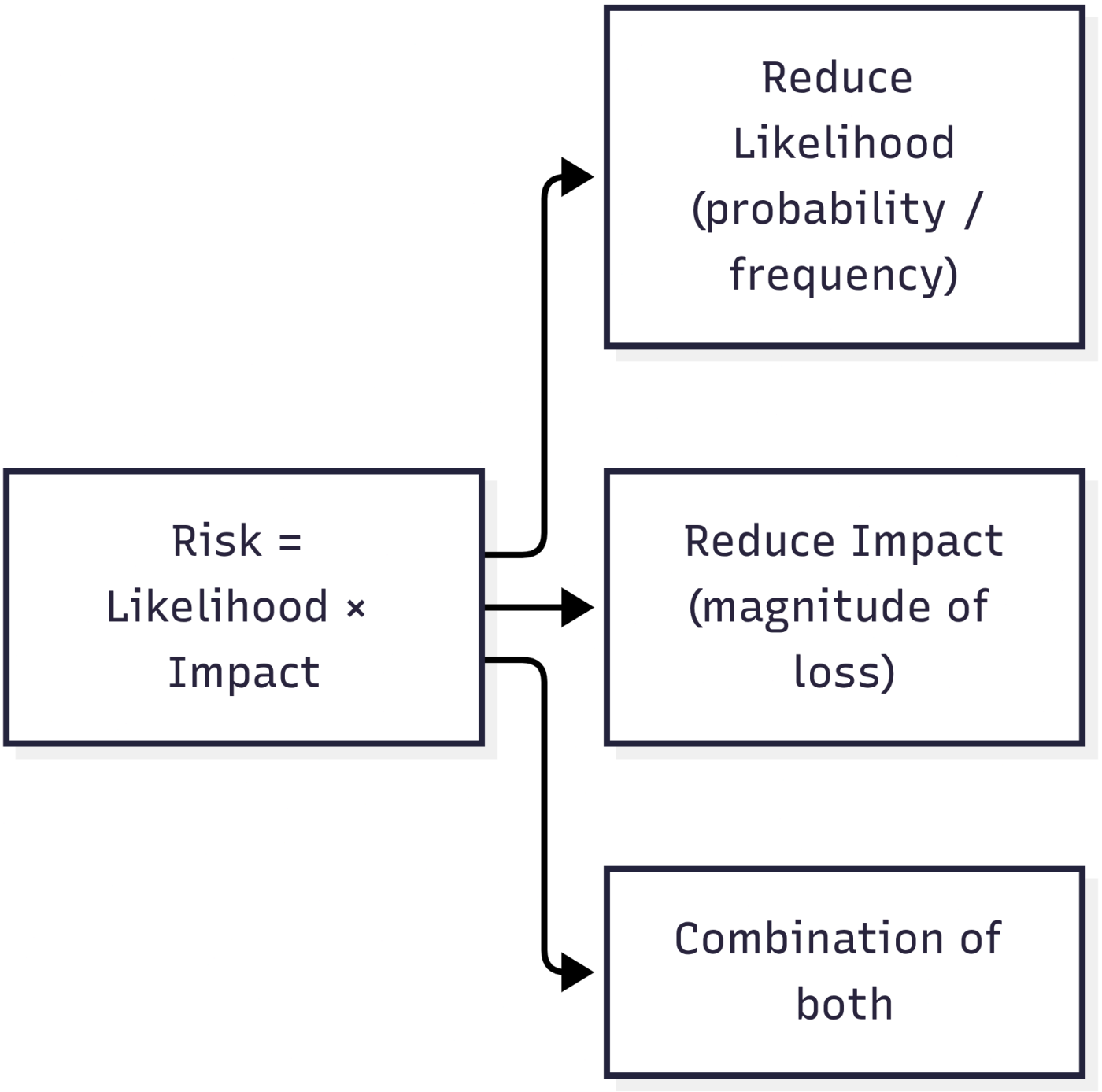
$$\text{Risk} = \text{Likelihood} \times \text{Impact} = (\text{Threat} \times \text{Probability}) \times \text{Impact}$$

The relationship is often displayed as a coloured matrix with likelihood on the vertical axis and impact on the horizontal axis, in which the combinations shade from green in the low–low corner, through amber in the middle band, to red where high likelihood meets high impact — a visual map that lets managers read off the severity of any risk from its coordinates.

Treating these dimensions as independent is best seen through examples. Suppose the incident of concern is **loss of data**. One cause might be a **fire** destroying the data centre: with proper security protections in place the probability is quite low, yet should it occur it could destroy the centre completely, giving a very high impact. The result is a critical risk even though the probability is low. The same loss of data might instead arise from **ransomware**: here the probability is higher than that of fire, while the impact depends on the type of data affected and may well be smaller. This gives a risk that is again medium-to-high but with a smaller impact. Because the two scenarios occupy different regions of the likelihood–impact space, the appropriate **countermeasures differ** — the organization may act one way against fire and an entirely different way against ransomware. Different models exist that decompose probability into more granular dominating factors, but in general the two governing quantities remain likelihood and impact.

How to Reduce Risk

The aim of the whole governance loop is to **reduce the level of risk**, and since risk is the product of likelihood and impact, there are exactly three levers available: reduce the **potential impact** of the risk, reduce the **probability or frequency** of the risk, or apply a **combination of both**. Which lever is pulled, and how hard, is itself a cost–benefit decision.



Risk Management, ISO 27002, and Governance

ISO 27002 treats IT risk analysis as an **essential part of best practice in IT**. It holds that an organization must identify its security requirements, and it names three main sources of such requirements.

- **Assessment of risks** to the organization, taking into account its overall business strategy and objectives; through a risk assessment, threats to assets are identified, vulnerability and likelihood of occurrence are evaluated, and potential impact is estimated.
- **Legal, statutory, regulatory, and contractual requirements** that the organization, its trading partners, contractors, and service providers must satisfy, together with their socio-cultural environment.
- The organization's own set of **principles, objectives, and business requirements** for handling, processing, storing, communicating, and archiving information in support of its

operations.

The standard stresses that the resources spent implementing controls must be balanced against the business harm that would likely result in their absence — once again the cost-effectiveness balance — and it points to **ISO/IEC 27005** for detailed guidance on risk assessment, treatment, acceptance, communication, monitoring, and review.

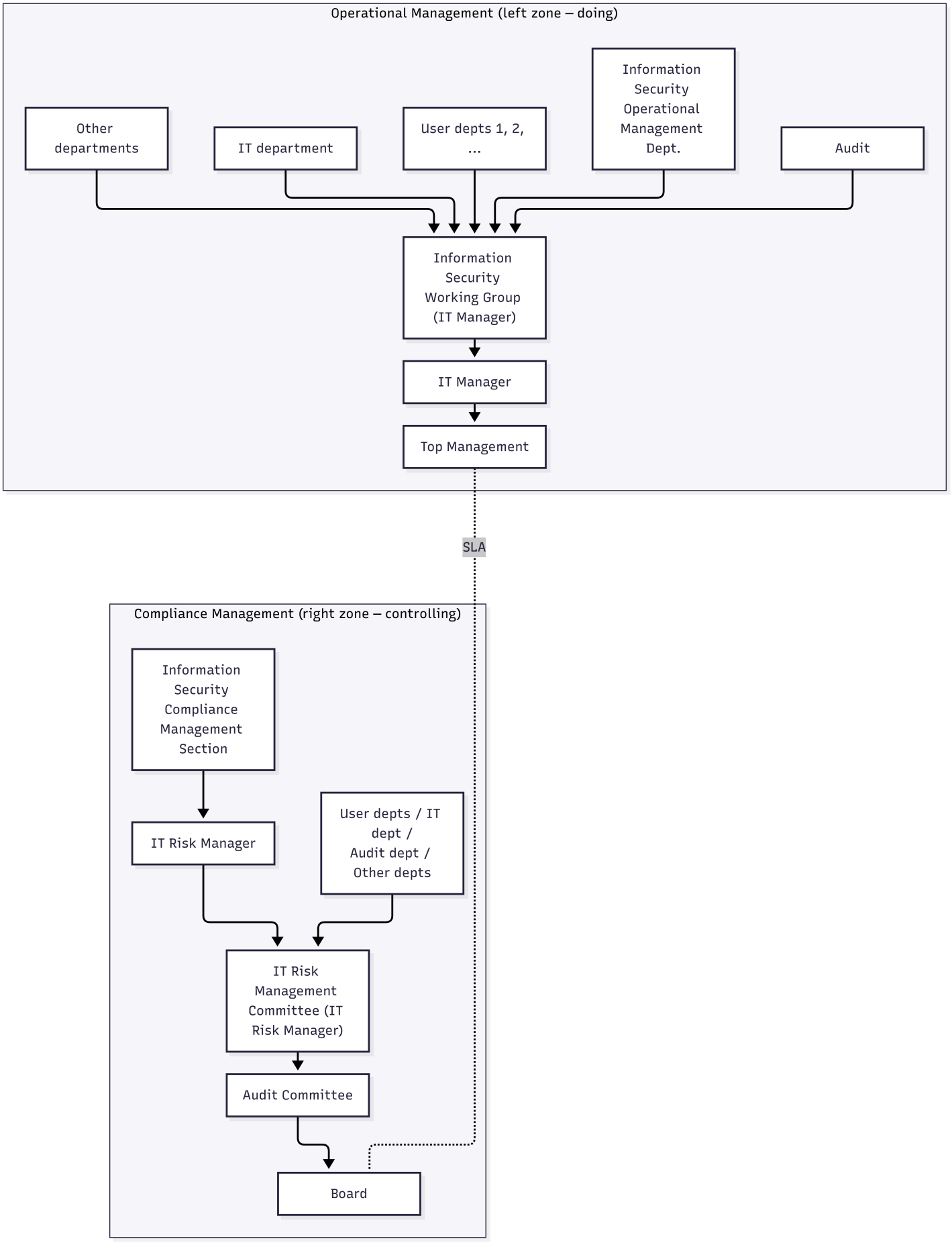
A structural principle closes the topic. All risks that could have a negative effect on the well-being of the organization, once they materialize, are definitively the **responsibility of management**. It follows that **all levels of organizational management** must be involved in the process of risk management, and that information and IT-related risks must be managed in an **integrated manner together with business risks** rather than in a technical silo. This is especially important in cybersecurity, because the people at the strategic level typically have no technical background; it therefore falls to the middle level to assess everything worth investigating, to highlight the critical issues, and to bring them to the attention of the managers who take decisions, since those managers must be made aware of all the potential bad situations that could occur.

Organization of Information Security

Two Orthogonal Security Functions

The way information security is organized inside a company is itself a matter of great importance, and every best practice document underlines it; ISO 27002, for instance, dedicates a clause titled *Organization of Information Security* to it. The guiding recommendation is that the organization should contain **at least two distinct components**, and that these two components are **orthogonal** to one another. The first looks after **day-to-day operational aspects** — operational security. The second is responsible for the **compliance monitoring function** — compliance management. The reason for the separation is structural: the people who *do* things must not be the same people who *check how things are done*, otherwise control would be grading its own work.

This produces a two-zone organizational chart. The **left zone**, Operational Management, is where the operational activities are actually carried out, implementing the bottom of the pyramid. The **right zone**, Compliance Management, is where the control process is implemented and feedback is provided back to the Direct process. The two zones meet at the top, at Top Management, and are connected through a **Service Level Agreement (SLA)** that governs the relationship between what is operated and what is audited.



In the operational chart, an **Information Security Working Group** led by the IT Manager aggregates input from the user departments, the dedicated Information Security Operational Management department, the IT department, and other departments, and reports upward through the IT Manager to Top Management. In the compliance chart, an **Information Security Compliance Management Section** feeds the **IT Risk Manager**, who sits on an **IT Risk Management Committee**; that committee collects input from all the departments,

assembles a consolidated view — the matrix of how things are done — and passes it to an **Audit Committee** that reports to the Board. In narrative terms, IT management on the operational side gathers information from the operational layer and composes it into a matrix view, which it hands to the person performing the audit; the audit side then collects input from across the departments and relays it to the risk management team, which in turn reports to top management.

Information Security Operational Management

The operational dimension is the set of people charged with ensuring that the system is functioning, organized into groups that monitor different departments, aggregate their information, and report to IT management. Its activities are the concrete, everyday actions of running security.

- **Logical access control management** — adding to, changing, and deleting user access rights from access control lists.
- **Identification and authentication management** — adding to, changing, and deleting entries in the user ID database and password files.
- **Firewall management** — configuring firewalls with the authorized access rights and connecting workstations to LANs and to the Internet.
- **Virus and malicious software management** — installing and updating antivirus software, together with the handling of antivirus and related security incidents.
- **Security configuration** — setting and updating the security settings and configurations of workstations and servers.
- **Availability** — ensuring availability through UPS systems.
- **Backups** — ensuring backups and the secure storage of those backups.

Information Security Compliance Management

The compliance dimension manages a different and orthogonal set of concerns, all directed at verifying that things are done in the right way and at assessing whether risk exists or persists. The activities managed here include:

- the level to which previously identified IT risks are managed and mitigated;
- the level of information security awareness of users;
- the availability, completeness, and comprehensiveness of the information security policies, procedures, and standards;
- the level of compliance to those policies, procedures, and standards;
- the impact on the company's IT risk level when policies are not complied with;
- the compliance with regulatory, legal, and statutory requirements;
- software licensing issues.

In practice this side also takes the compliance clauses defined in the documents and attempts to apply them, verifying not only that risks are controlled but that the whole documentary

edifice is current, complete, and actually followed.

Awareness and the SETA Programme

Why Awareness Matters at Every Level

A recurring principle of the whole model is that **all levels are involved in the ISG process**, and awareness is the mechanism by which that involvement reaches the human core of the company. Information security procedures, guidelines, and practices must be drafted and conveyed to **all users of information and IT** in the organization, which means every information worker must be made aware of, and trained in, the information security policy together with its associated procedures, guidelines, and practices. This is why the board directive and CISP documents must be distributed as widely as possible: awareness is the capability of the company to carry a genuine security attitude, to recognize security issues, and to know how to handle them, and it cannot exist unless the knowledge is spread.

The SETA Programme and Its Three Temporal Horizons

To build this capability the model recommends developing a **Security Education, Training and Awareness (SETA)** programme, an instrument operating at the human level of the company whose purpose is to inject the security factor into people. SETA should be an *extension* of the general knowledge employees already possess to do their jobs; its objective is to teach each of them to do that same job *securely*. The ambition is to move the awareness level from the basics to something that becomes automatic in the person's everyday life, creating a common understanding of both the issues and the solutions.

The programme has three stated objectives: to improve awareness of the importance and the need to protect organizational information resources; to acquire the necessary skills and know-how to do jobs more securely; and to create an understanding and insight into *why* it is important to protect organizational information assets. These objectives map onto three distinct activities, each operating over a different time horizon, captured by the differentiation between awareness, training, and education.

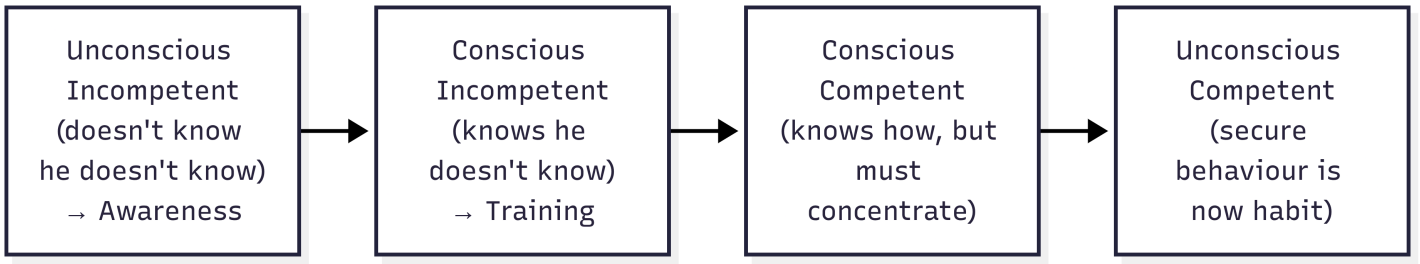
	Awareness	**Training**	**Education**
Attribute (question)	What	How	Why
Level	Information	Knowledge	Insight
Objective	Alert to	Skill	Understanding
Teaching method	Media (videos, newsletters, posters)	Practical instruction (lecture, workshop, hands-on)	Theoretical instruction (seminar, literature study)
Impact timeframe	Short-term	Medium-term	Long-term

Awareness is the first stage, developing the capability to understand what is dangerous. Training is the stage that explains how to behave. Education is the most challenging stage, making people understand why they should behave in a certain way, and it is effective only over the long run.

The Conscious Competence Learning Model

The journey an employee takes through a SETA programme is described by the **Conscious Competence Learning Model**, a four-stage progression from ignorance to instinct.

- **Unconscious Incompetence** — the person does not even realize that they do not know how to do a certain thing safely. The priority is to make the individual aware that they are incompetent for that specific task, the province of **awareness** exercises.
- **Conscious Incompetence** — the employee now knows that they lack the skill, and the priority becomes **training** them to do the job in a secure manner.
- **Conscious Competence** — the employee knows what to do and how to do it to keep the job secure, but still has to *concentrate* to perform the procedures correctly.
- **Unconscious Competence** — through enough practice and experience, all security-related practices and skills have become inherent to the person's normal daily behaviour. Security has become a **habit** requiring no conscious effort.



The Pillars of Training

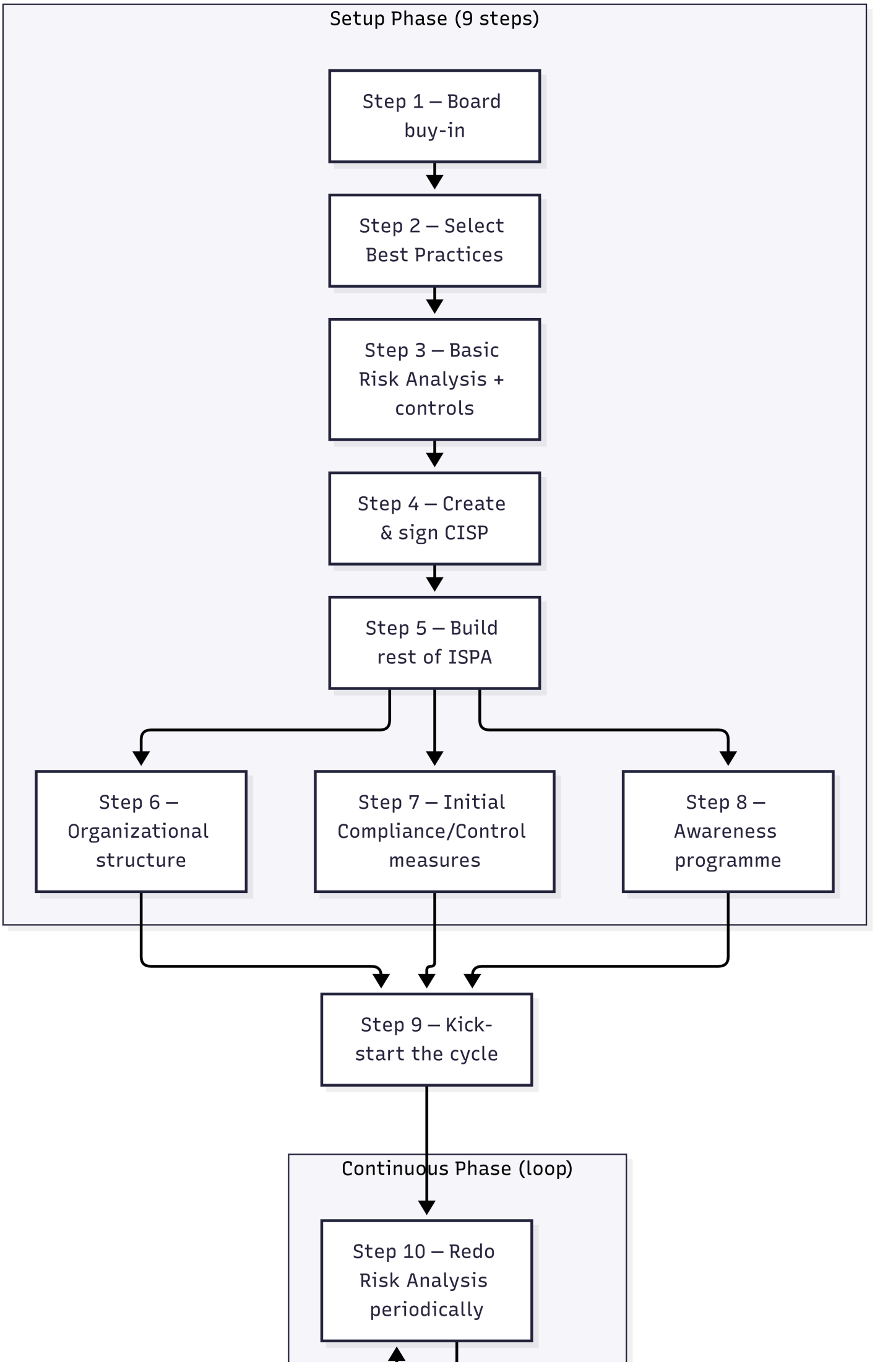
Underpinning the programme are the substantive topics that training must cover. At the level of **motivation**, employees must be taught why information is such an important asset, what the information security policy and procedures are, what the role and responsibility of every employee is, and what the consequences of not complying with the policy will be. At the level of **concrete practice**, the training should address user identification and authentication through passwords, how to choose a password, and the prohibition on sharing user IDs and passwords; virus control; backing up information; the sharing and storing of confidential information; secure email usage; secure Internet usage; social engineering; the handling of mobile devices such as notebooks and PDAs; the legal usage of software; office manners and discipline such as a clean-desk policy; and the required actions whenever a security incident or breach is suspected.

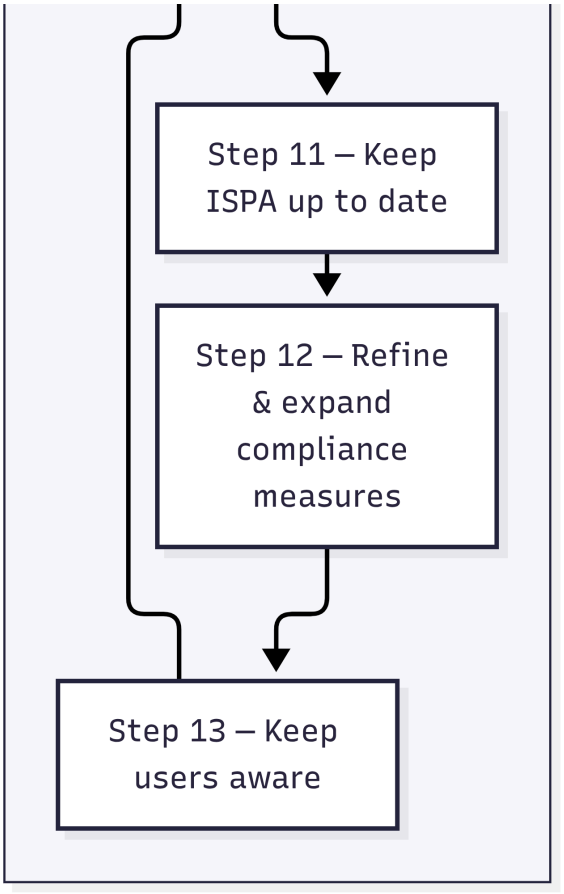
A Methodology for Establishing an ISG Environment

Putting the Whole Model Together

Having examined the Front Dimension, all five layers of the Depth Dimension, and the Best Practices basement, the final question is how to assemble the pieces into a working environment. The answer is a high-level **methodology of 14 steps**, deliberately written for the hardest case: a *complete* methodology that assumes no ISG environment exists at all and that everything must be started from scratch. In most real companies the situation is considerably better, so the methodology carries a standing instruction to **re-use as much as possible** to avoid wasting time and money — the actual situation must be compared against the steps and synchronized and integrated with them wherever possible.

The steps divide into two phases. The **Setup Phase** comprises the first nine steps (Steps 1 through 9), executed largely in sequence, although Steps 6, 7, and 8 branch out in parallel from Step 5. The **Continuous Phase** comprises the final five steps (Steps 9 through 13, with Step 9 acting as the bridge that kick-starts the loop), which then repeat perpetually, implementing the Direct–Control flow on an ongoing basis.


Setup Phase (9 steps)
Step 1 – Board buy-in
Step 2 – Select Best Practices
Step 3 – Basic Risk Analysis + controls
Step 4 – Create & sign CISP
Step 5 – Build rest of ISPA
Step 6 – Organizational structure
Step 7 – Initial Compliance/Control measures
Step 8 – Awareness programme
Step 9 – Kick-start the cycle
Continuous Phase (loop)
Step 10 – Redo Risk Analysis periodically



The Setup Phase (Steps 1–9)

1. **Get the Board's buy-in** about IT risk management and information protection. This is a critically important starting point, and without it the project will struggle to progress. It is acceptable that one cannot initially obtain a *clean* Board Directive; what is essential is a genuine commitment from the board to support the project, which is enough to begin. A basic, poor risk assessment may be performed first to identify the assets, under the approval of management, because the Direct process cannot start until it is known what must be protected. The formal Board Directive can come later, but it must be secured.
2. **Select some guiding Best Practices.** Choose an existing international best practice to serve as the foundation of the project and to provide motivation and direction. Many exist, and the essential point is simply to *have one.*
3. **Perform a basic Risk Analysis and determine all needed controls.** Some form of risk analysis is mandatory in order to identify the genuinely critical assets and to avoid wasting money and investment. It requires involving a wide spectrum of people — board members, line management, users, system owners — and must ensure that the relevant controls are installed and operational.
4. **Create a CISP and have it signed by the Chairman/CEO.** A CISP must be drafted, circulated among the stakeholders, and submitted to top management. A more formal Board Directive on information security is essential in order to write a consistent CISP. Once signed, the draft becomes official company policy and forms the basis and motivation for all future steps.
5. **Create the rest of the ISPA.** The documents composing the full architecture must now be defined, taking as input the risk analysis from Step 3 and the CISP from Step 4 and following the guidelines discussed earlier. Throughout, the **connections between the**

documents must be preserved, so that every sub-policy and procedure traces back up the tree.

6. **Create the organizational structure for ISG.** Specific attention must be paid to both the Operational Management and the Compliance Management sides, identifying the teams to be involved and the people responsible for them and assigning responsibilities. Because this step is closely related to Step 7, it is preferable to perform the two **in parallel**.
7. **Create an initial set of Compliance/Control measures** and begin using them to produce reports on all three management levels. It is important to obtain buy-in from the company's internal and external IT audit departments as well as the legal department; if these were already involved in Step 3, the process is much easier. Creating such measures is not straightforward, and their effectiveness and value will have to be refined over time — the point is to start with an initial set that forms the basis of the Control part of the model.
8. **Create an Awareness Programme**, including aspects such as information security job responsibilities. This step is core to the success of the whole effort and must be performed on a continuous basis, with all the documents of the ISPA forming part of the programme.
9. **Get the cycle going and kick-start the process.** The whole programme must now be initiated; if one already exists, the revised programme must be integrated with the existing one to make the new one operational. Step 9 is the hinge between the two phases.

The Continuous Phase (Steps 10–13)

10. **Redo the Risk Analysis from time to time** to identify changes in risks and controls. Risks are dynamic: old ones disappear as patches are installed, ports are closed, and access controls are deployed; new ones materialize as new software is installed or configuration parameters change; and existing ones shift in impact, since a single mitigation action may decrease the impact of several other threats at once. Periodic reassessment keeps the risk picture current and the controls operational.
11. **Keep the ISPA up to date** and aligned with the newly identified risks, ensuring that all changed risks are reflected by updating the content of the ISPA and its compliance clauses where necessary.
12. **Refine and expand the Compliance/Control measures** to cater for newly identified risks, enforce compliance, and keep reporting to top management. The choice and creation of compliance clauses and monitoring measures remain challenging, and the process never stops: experience is gathered continuously and used to refine existing measures and create new ones.
13. **Continue to make all users more information-security aware.** This process can never stop and must be enforced on a continuous basis, after which the loop returns to Step 10.

Chapter 2: Frameworks, Standards, Best Practices

1. The Problem of Governing Known Risk

In May 2017, hospitals across the United Kingdom found their computers displaying a single message: their files had been encrypted and would remain inaccessible unless a ransom was paid. The consequences were immediate and tangible — patients were redirected to other facilities, appointments were cancelled, and several medical services were disrupted. The malware responsible was **WannaCry**, a ransomware worm that propagated rapidly across unpatched Windows machines.

The most instructive feature of the incident is not the sophistication of the attack but its avoidability. The underlying Windows vulnerability had already been publicly disclosed, and Microsoft had released the corresponding security patch (MS17-010) *before* the attack took place. The vulnerability was **known**. This raises the central question that motivates the entire discipline of security governance: if the risk was known and a remedy existed, why was the organization not prepared to manage it?

The answer is rarely technical. Patching a vulnerability is a solved problem in engineering terms. The failure lies instead in the absence of a **structured, organization-wide process** for discovering assets, assessing their exposure, prioritizing remediation, and ensuring the remedy is actually applied across every machine. Managing known risk is a governance problem, and governance problems are addressed through frameworks, standards, and best practices. These three instruments form the conceptual backbone of this chapter.

2. Foundational Definitions

Three terms are frequently used interchangeably in casual speech but carry precise and distinct meanings in security governance.

A **cybersecurity framework** is a structured, risk-based approach that helps an organization organize, assess, and improve its cybersecurity activities and outcomes. Being *structured* matters because it provides a **common language**: when multiple organizations describe their security posture using the same vocabulary and categories, they can communicate, compare, and align with one another, which is indispensable for coordination and, ultimately, for certification. Being *risk-based* matters because it directs the organization toward doing the right things before damage occurs rather than reacting after the fact. A framework provides guidance — what to account for and how to think about security — without dictating the precise technical implementation.

A **cybersecurity standard** is a formal set of requirements or specifications that an organization can implement and audit against to demonstrate and certify conformity. The crucial distinction from a framework is formality and auditability. A standard contains specific requirements, and because those requirements are precise, an independent party can verify compliance and issue a certification. The canonical example is **ISO/IEC 27001**. Standards are standardized, so they supply a common language across organizations, and that shared language is what makes certification meaningful.

A **best practice** is a procedure or method proven by research and experience to produce effective or optimal results and suitable for widespread adoption. Best practices are the most concrete of the three: *actionable* and *less formal* than standards. They describe proven actions an organization can take to implement what a framework recommends, such as patch management, multi-factor authentication, vulnerability disclosure, access-control hygiene, and user awareness training. Over time, best practices that prove their value become embedded in standards and referenced by frameworks, so the three categories continuously feed into one another.

Instrument	Provides
Framework	Guidance for managing cybersecurity risk
Standard	Specific requirements that can be implemented and assessed
Best practice	Proven actions to implement

To illustrate the formality gradient with a mundane analogy, consider the 'policy' for how to contact a professor. A policy is a high-level set of rules defining what is allowed and expected: the first rule might define the acceptable channel (email), the second might require that the email be meaningful with a clear leading subject, and a third might require sufficient context. This structure — high-level rules refined into specific, verifiable expectations — mirrors exactly how governance instruments descend from abstract guidance to concrete, checkable requirements.

3. Why Frameworks and Standards Matter

Frameworks and standards together provide a **roadmap to assess, improve, and communicate an organization's security posture**; they are the foundations upon which security is governed. Three aspects explain their enduring value.

- **Standardized approaches** — they ensure that different teams, departments, and even different companies operate from a shared understanding.
- **Risk-based and adaptable** — effort is directed where exposure is greatest, and the same framework can serve organizations of radically different size and sector.
- **Continuous improvement** — they treat security not as a project with an endpoint but as a process that is perpetually refined.

The most widely recognized instruments of this kind include the NIST Cybersecurity Framework (CSF), ISO/IEC 27001 and 27002, the CIS Controls, and COBIT, among many others.

4. Historical Evolution: From Standards to Frameworks to Best Practices

The modern landscape did not emerge all at once; it is the product of roughly three decades of accumulation, in which each layer built upon the previous one.

The lineage begins in the United Kingdom with **BS 7799**. Its 1998 revision introduced the pivotal idea of a structured **Information Security Management System (ISMS)**: that security should be managed through a deliberate, documented, continuously governed system rather than a set of ad hoc technical measures. In parallel, best practices emerged as proven procedures that organizations could apply even before adopting a formal certification or framework. The authority that would come to formalize much of this thinking in the United States is the **National Institute of Standards and Technology (NIST)**, a federal agency founded in 1901 that develops scientific, technological, and measurement standards to promote innovation and industrial safety.

Year	Milestone
1990s	BS 7799 (UK) introduces one of the first structured approaches to information security management
1995	BS 7799-1 published as a code of practice — a catalogue of information security controls and guidance
1998	BS 7799-2 adds the management-system component, introducing the ISMS
2000	ISO/IEC 17799 adopts the British work at the international level
2005	ISO/IEC 27001 becomes the formal requirements standard for ISMSs; ISO/IEC 27002 evolves as companion control guidance
2013	Executive Order 13636 directs NIST to develop a *voluntary* framework for reducing cyber risk to critical infrastructure
2014	NIST issues Framework v1.0 (Improving Critical Infrastructure Cybersecurity); the Cybersecurity Enhancement Act reinforces NIST's role and explicitly connects standards, frameworks, and best practices
2018	NIST releases CSF v1.1, adding cyber supply-chain risk management, self-assessment guidance, and identity clarifications

The 2014 framework was designed explicitly as voluntary guidance built on existing standards, guidelines, and practices, a choice that made it flexible and broadly adoptable well beyond critical infrastructure. The NIST CSF is today recognized worldwide as a basis of security governance.

5. The Organizational Landscape and the Two Core Layers

The ecosystem of frameworks is vast. Beyond the general-purpose instruments, there exist sector-specific, national, and purpose-specific frameworks.

Domain	Frameworks
Finance	DORA, SWIFT Customer Security Programme
European Union	NIS2, EU Cybersecurity Act
U.S. federal & cloud	FISMA, FedRAMP
U.S. defense supply chain	CMMC
Healthcare	HIPAA Security Rule
Payment cards	PCI DSS
Industrial / operational technology	IEC 62443
Automotive	TISAX
Electricity	NERC CIP
National	Italian NCF, Germany's BSI IT-Grundschutz, France's ANSSI, UK's NCSC Cyber Assessment Framework, Australia's Essential Eight
Application security	OWASP SAMM, OWASP Top 10, BSIMM, NIST SSDF
Cloud security	CSA Cloud Controls Matrix, ISO/IEC 27017, FedRAMP
Zero Trust	NIST SP 800-207, CISA Zero Trust Maturity Model
Maturity models	C2M2, CMMC, COBIT
Threat modeling & intelligence	MITRE ATT&CK, Cyber Kill Chain, STRIDE, STIX/TAXII

The sheer breadth of this catalogue is itself a lesson: there is no single instrument, and competent governance means selecting and combining the right ones.

For structured study, the core instruments can be organized into two complementary layers that answer two different questions. The **framework layer** answers the strategic question — *what should I account for, and how should I organize security?* — and provides principles and practices for managing and improving security; its representatives are **ISO 27001, the NIST CSF, and the Italian Cybersecurity Framework**. The **implementation layer** answers the operational question — *how do I apply the strategy?* — and provides specific safeguards and measures to mitigate security risks; its representatives are **ISO 27002, NIST SP 800-53, and the CIS Controls**. Together, these two layers constitute security governance.

Layer	Main question	Instruments
Management system	How should information security be managed continuously?	ISO/IEC 27001

Layer	Main question	Instruments
Control guidance	Which information security controls can be implemented?	ISO/IEC 27002
Risk framework	How can cybersecurity risk be organized and communicated?	NIST CSF, Italian NCF
Detailed controls	Which safeguards can be selected, tailored, and assessed?	NIST SP 800-53
Practical safeguards	What should be prioritized and implemented first?	CIS Controls

The chapter follows these layers. First come the frameworks — the principles and standards giving the general idea of what to do within an organization. Then comes implementation — how to satisfy the security requirements the framework demands. Once implementation is understood, the governance pyramid can be reassembled in full.

6. ISO/IEC 27001

ISO/IEC 27001 is the first and best-known standard in the ISO 27000 family and the world's foremost standard for **Information Security Management Systems**. It is the most *international* of the governance instruments, recognized in over sixty countries, which is what gives it its value as a lingua franca of security certification.

6.1 The ISO 27000 Family

ISO 27001 does not stand alone; it sits at the head of a large family of related standards, each addressing a facet of information security management.

Standard	Scope
27000	Overview and vocabulary
27001	Requirements for an ISMS — defines *what* must be managed
27002	Code of practice for implementing the controls — explains *how*
27003	Implementation guidance
27004	Measurement
27005	Information security risk management
27006 / 27007	Audit and certification bodies
27011	Telecommunications
27017 / 27018	Cloud, and personally identifiable information in the cloud
27031	ICT business-continuity readiness
27032	Cybersecurity
27033 (series)	Network security

Standard	Scope
27035	Incident management
27037	Digital evidence
27799	Health information

For the study of governance, the two members that matter most are **27001** and **27002**.

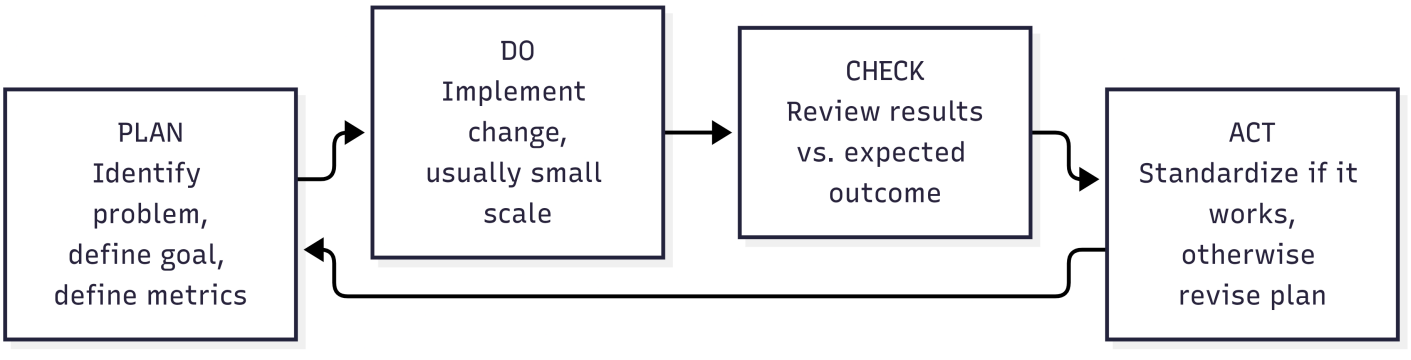
6.2 The Management-System Approach

The defining conceptual property of ISO 27001 is that it follows a **management-system approach**: security is treated as an **ongoing organizational process** rather than a one-time technical project. The standard specifies the requirements for establishing, implementing, maintaining, and *continually improving* an ISMS, so that information security is managed systematically through governance, risk assessment, controls, monitoring, and improvement. Concretely, an organization adopting ISO 27001 must define its scope, assign responsibilities, assess and treat its risks, measure its performance, and continually improve the system. The requirements it specifies are known as **security controls**, and organizations are expected to be compliant with them. These controls are deliberately not highly detailed, because they must cover a very wide range of topics: the scope and objectives of the organization, the human dimension, the assessment and management of risk, the measurement of performance, and the continuous improvement of the ISMS.

6.3 The Deming Cycle (PDCA)

The engine that drives the continual-improvement character of ISO 27001 is the **Deming Cycle**, also known as **PDCA**, a continuous-improvement method used to test changes in a structured way. Its four phases form a loop that never terminates.

- **Plan** — identify a problem, define a goal, and decide in advance how success will be measured.
- **Do** — implement the change, typically on a small scale to limit exposure.
- **Check** — review the results and compare them against the expected outcome, often on the basis of concrete measurements.
- **Act** — standardize the improvement if it worked, or revise the plan and repeat the cycle if it did not.

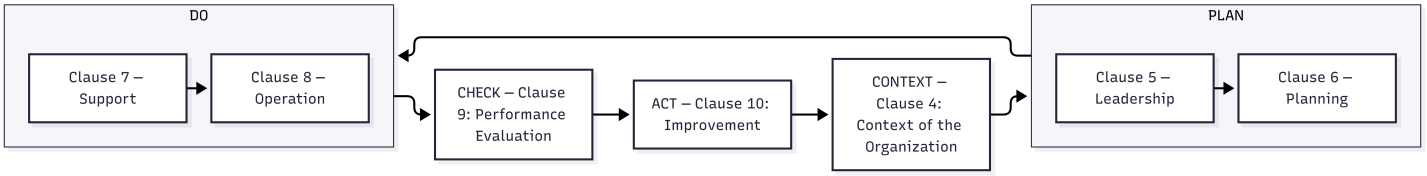


Security is never 'finished,' and each pass through the cycle refines the system a little further.

6.4 The Structure of ISO 27001: Clauses

ISO 27001 is organized into **clauses**, each a section of the document. Clauses 1 to 3 are introductory, providing context and definitions. The **mandatory requirements reside in Clauses 4 to 10**, and these seven clauses map directly onto the phases of the Deming Cycle. Clause 4, the context of the organization, underlies every phase of the cycle, while the remaining clauses occupy their positions around the loop.

Clause	Title	PDCA phase
4	Context of the Organization	Underlies all phases
5	Leadership	Plan
6	Planning	Plan
7	Support	Do
8	Operation	Do
9	Performance Evaluation	Check
10	Improvement	Act



Clause 4 — Context of the Organization. The organization must understand itself, the interested parties around it, and the scope of its ISMS. This clause establishes the boundaries within which everything else operates: what is being protected, who cares about it, and how far the management system reaches.

Clause 5 — Leadership. This clause makes information security a *leadership responsibility, not merely a technical activity.*

- *5.1 Leadership and commitment* — top management must actively support the ISMS, align it with the business strategy, provide the necessary resources, and ensure the system achieves its intended outcomes.
- *5.2 Policy* — the organization must define and communicate an information security policy that sets direction, supports objectives, and reflects a commitment to compliance and continual improvement.
- *5.3 Organizational roles, responsibilities, and authorities* — roles must be assigned clearly so that responsibilities are understood and reporting lines are effective. **Every person must be assigned a role and a responsibility, and this assignment must be formalized.**

Clause 6 — Planning. This clause turns context and leadership into a practical plan for managing risk and improving security.

- *6.1 Actions to address risks and opportunities* — identify the risks and opportunities related to the ISMS and plan actions to reduce threats, prevent problems, and improve effectiveness; this is where the security controls needed to assess and secure risks are brought to bear.
- *6.2 Information security objectives and planning to achieve them* — set **measurable** security objectives and define who will do what, by when, and with which resources.
- *6.3 Planning of changes* — changes to the ISMS are managed in a controlled way so that security is not weakened by poorly planned updates. Such change management may be performed **manually** or **automatically** through self-adaptive systems that detect and respond to vulnerabilities on their own.

Clause 7 — Support. This clause ensures the ISMS is supported by competent people, clear communication, and proper documentation.

- *7.1 Resources* — provide the people, tools, budget, and infrastructure needed to run the ISMS.
- *7.2 Competence* — ensure those performing security-related work have the necessary knowledge, training, and experience.
- *7.3 Awareness* — make sure staff understand the security policy, their responsibilities, and the consequences of non-compliance.
- *7.4 Communication* — define what must be communicated, to whom, when, and through which channels.
- *7.5 Documented information* — create, maintain, and control the documents and records needed to support and to *prove* that the ISMS works.

Clause 8 — Operation. This is the clause where the ISMS becomes operational — where risks are assessed, treatments are chosen, and controls are applied in practice.

- *8.1 Operational planning and control* — operational processes are planned and controlled so the ISMS is implemented as intended, with explicit reference to the implementation guidance of ISO 27002.
- *8.2 Information security risk assessment* — a consistent method for identifying and evaluating risks is defined.
- *8.3 Information security risk treatment* — controls or measures are selected and implemented to treat the risks deemed **unacceptable**.

Clause 9 — Performance Evaluation. This clause checks whether the ISMS is delivering results and whether leadership is using the evidence to steer improvements.

- *9.1 Monitoring, measurement, analysis, and evaluation* — track performance using indicators, metrics, and analysis.

- *9.2 Internal audit* — regularly check whether the ISMS conforms both to ISO 27001 and to the organization's own requirements, continuously searching for evidence of risk.
- *9.3 Management review* — top management reviews the ISMS to confirm it remains suitable, effective, and aligned with business needs.

Even here the standard remains non-prescriptive about exact technical steps: it tells the organization *that* it must monitor, audit, and review, without dictating the precise procedure.

Clause 10 — Improvement. This clause concerns conformity and the organization's capacity to improve continuously, including how it reacts after an incident.

- *10.1 Nonconformity and corrective action* — investigate problems, fix their root causes, and prevent recurrence.
- *10.2 Continual improvement* — keep strengthening the ISMS over time on the basis of audits, incidents, measurements, and changing risks.

6.5 Annex A and the Statement of Applicability

Beyond the ten clauses, ISO 27001 includes **Annex A**, a reference set of information security controls that functions as a **checklist**. In the 2022 revision, Annex A contains exactly **93 controls**. Organizations are not required to implement all of them indiscriminately; instead, they evaluate each control's applicability on the basis of their risk-treatment decisions. These decisions are documented in a central artefact called the **Statement of Applicability (SoA)**, which links risks, selected controls, and the justification for each inclusion or exclusion.

A well-constructed SoA records, for each control:

- the Annex A control identifier and name;
- whether the control is applicable;
- a justification for its inclusion or exclusion;
- the implementation status of the controls deemed applicable.

It normally also references the relevant policies, procedures, or evidence. The SoA is thus the document that connects abstract risk decisions to concrete controls and to the proof that those controls exist.

The relationship between Annex A and ISO 27002 is the natural bridge to the implementation layer: **ISO 27001 (through Annex A) defines *what* the 93 controls are, while ISO 27002 defines *how* each can be implemented.** ISO 27002 begins precisely from the controls catalogued in Annex A and expands each one into detailed guidance.

6.6 Could ISO 27001 Have Prevented WannaCry?

Returning to the motivating incident with the apparatus of ISO 27001 in hand reveals a critical lesson about the limits of certification. Imagine being the CISO of an organization with 2,000 Windows computers at the moment WannaCry strikes, having already implemented ISO

27001, and discovering that hundreds of machines remain unpatched despite Microsoft's prior release of MS17-010. How could this happen?

The decisive insight is that **possessing an ISO 27001 certificate does not prove that every computer was patched**. A certificate attests that a *management system* exists and functions, not that every technical control is perfectly executed at every instant on every asset. The standard, properly applied, nonetheless supplies the activities that should have prevented the situation:

- **asset inventory** — so that every Windows system is known;
- **risk assessment** — so that the exposure is recognized;
- **vulnerability management** — so that the available patch is identified and applied;
- **monitoring** — so that unpatched machines are flagged;
- **incident response** — so that an outbreak can be contained;
- **security awareness** — so that people understand their obligations.

The failure in a real organization is therefore not the absence of the framework but the gap between holding a certificate and genuinely operating every clause and control the framework demands — most of all the Clause 8 operational discipline that turns risk treatment into applied, verified safeguards.

7. The NIST Cybersecurity Framework (CSF) 2.0

If ISO 27001 is the most *international* governance instrument, the **NIST Cybersecurity Framework** is the most *widely used* security governance framework in the world. Where ISO 27001 is a certifiable standard built around a management system, the NIST CSF is a **voluntary, non-certifiable** framework built around the organization and communication of risk. The two are complementary rather than competing, and much of the CSF's power comes from its flexibility and universal applicability.

The framework focuses on using **business drivers** to guide cybersecurity activities and on treating cybersecurity risk as an integral part of the organization's broader risk-management processes. Although originally developed to improve cybersecurity risk management in critical infrastructures, it is deliberately constructed so that any organization, in any sector, can adopt it. It provides a common organizing structure that assembles standards, guidelines, and practices already shown to work in the field, and its applicability spans every kind of technology-dependent organization — information technology (IT), industrial control systems (ICS), cyber-physical systems (CPS), and the broader world of connected devices and the Internet of Things (IoT).

The framework consists of three parts, which answer three different governance needs: the **Core**, the **Implementation Tiers**, and the **Profiles**.

7.1 Features of the NIST CSF

The defining feature of the CSF is that it is **technology-neutral**. It remains effective regardless of the specific technologies in use and actively supports technical innovation, because it references a variety of existing standards, guidelines, and practices that themselves evolve alongside technology. Rather than prescribing particular products or configurations, it describes **outcomes**.

This neutrality lets the framework provide a common taxonomy and mechanism through which an organization can:

- describe its *current* cybersecurity posture;
- describe its *target* state for cybersecurity;
- identify and prioritize opportunities for improvement within a continuous and repeatable process;
- assess its progress toward the target state;
- **communicate** about cybersecurity risk among internal and external stakeholders.

Communication is not a peripheral benefit but one of the explicit purposes the framework is designed to serve.

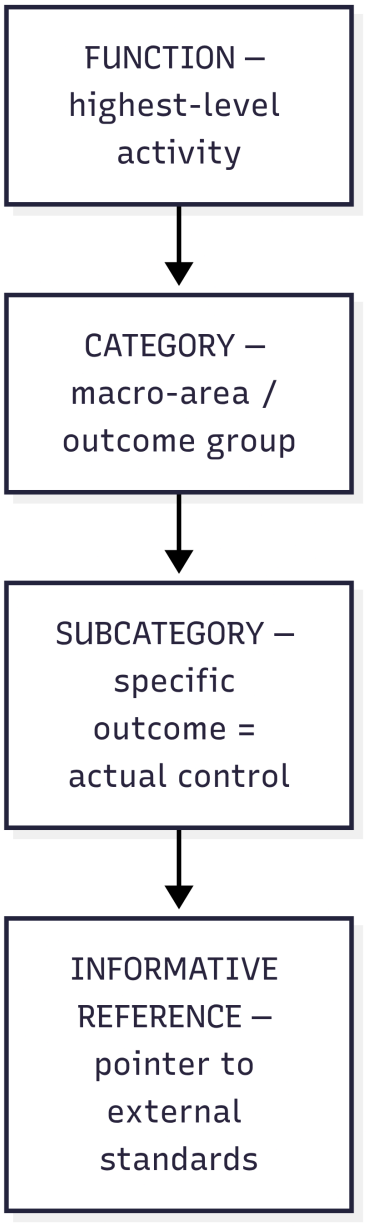
7.2 The Core and Its Four Elements

The **Core** is the heart of the framework. It provides a set of activities to achieve specific cybersecurity outcomes and references examples of guidance for achieving them. A crucial clarification distinguishes it from an ISO-style checklist: **the Core is not a checklist of actions to perform.** It presents key cybersecurity *outcomes*, identified by stakeholders as helpful in managing cybersecurity risk. The difference is more than semantic — a checklist enumerates tasks to tick off, whereas an outcome catalogue describes states to achieve, leaving the organization free to determine how best to reach them.

The Core is built from four nested elements, descending from the most general to the most specific.

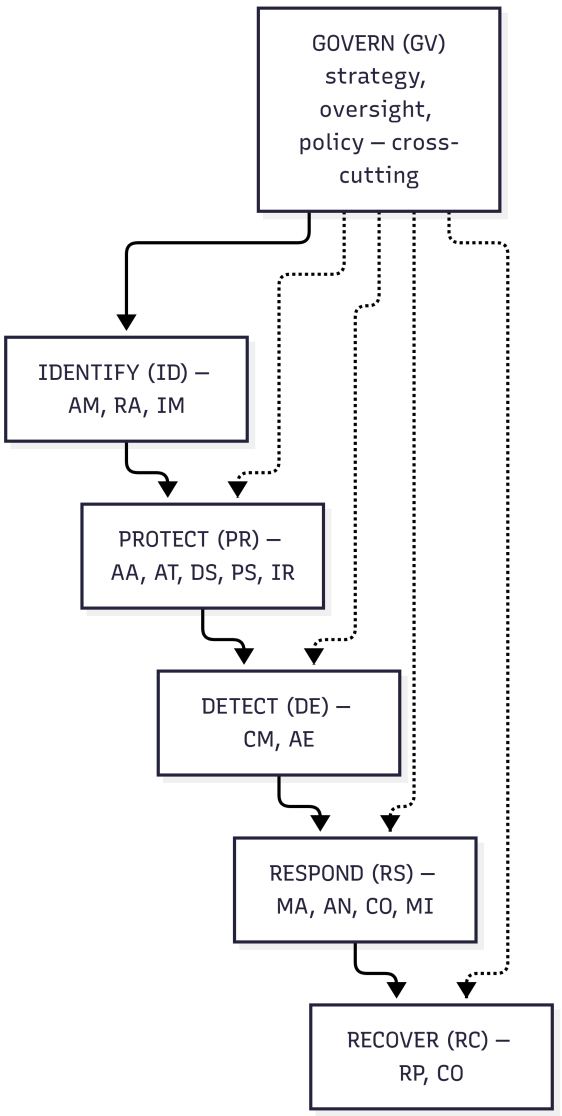
- **Functions** — organize basic cybersecurity activities at their highest level; the broad pillars of the framework.
- **Categories** — subdivisions of a Function into groups of outcomes tied to programmatic needs and particular activities; the macro-areas in which a Function operates. Examples: 'Asset Management,' 'Identity Management and Access Control,' 'Detection Processes.'
- **Subcategories** — specific outcomes of technical or management activities; the level at which the framework specifies the concrete actions — the actual security controls — that support each Category's outcomes. Examples: 'External information systems are catalogued,' 'Data-at-rest is protected,' 'Notifications from detection systems are investigated.'
- **Informative References** — specific sections of standards, guidelines, and practices that illustrate concrete methods for achieving the outcomes associated with each Subcategory;

in effect, pointers to other standards that operationalize the Categories and Subcategories.



7.3 The Six Functions

In its original form the CSF comprised five Functions. The 2.0 revision introduced a sixth, **Govern**, which occupies a special position: whereas the other five Functions are each specific to particular operational aspects of cybersecurity, Govern is **cross-cutting** and covers all the others. Governing security means possessing expertise across every function, with responsibility threading through the entire process — which is why a standardized, structured approach is indispensable.



Govern (GV) establishes and monitors the cybersecurity risk-management strategy, expectations, and policies, and sets the strategic conditions that make the other five Functions effective.

- *Organizational Context (GV.OC)* — understanding the mission, stakeholders, dependencies, and legal, regulatory, and contractual requirements.
- *Risk Management Strategy (GV.RM)* — establishing priorities, constraints, risk appetite, risk tolerance, and assumptions for cybersecurity decisions.
- *Roles, Responsibilities, and Authorities (GV.RR)* — defining accountability, decision-making authority, and responsibilities.
- *Policy (GV.PO)* — establishing, communicating, and enforcing cybersecurity policies.
- *Oversight (GV.OV)* — using risk-management results and performance information to review and adjust the strategy.
- *Cybersecurity Supply Chain Risk Management (GV.SC)* — identifying, assessing, managing, monitoring, and improving the risks arising from suppliers and third parties.

Identify (ID) develops the organizational understanding needed to manage risk to systems, people, assets, data, and capabilities; its maxim is that *you cannot protect what you have not identified.*

- *Asset Management (ID.AM)* — inventorying and managing data, devices, systems, and services.
- *Risk Assessment (ID.RA)* — identifying and evaluating cybersecurity risks.
- *Improvement (ID.IM)* — using lessons learned to improve risk management.

Protect (PR) develops and implements the safeguards that ensure delivery of critical services; its core principle is to *reduce the likelihood and impact of an attack.*

- *Identity Management, Authentication, and Access Control (PR.AA)* — managing identities, authentication, and permissions.
- *Awareness and Training (PR.AT)* — training users and partners to perform their cybersecurity duties.
- *Data Security (PR.DS)* — protecting the confidentiality, integrity, and availability of data.
- *Platform Security (PR.PS)* — hardening and securing platforms and devices.
- *Technology Infrastructure Resilience (PR.IR)* — ensuring the infrastructure can withstand and recover from disruption.

Detect (DE) develops and implements the activities that identify the occurrence of a cybersecurity event; its core principle is that *faster detection means faster response and less damage.*

- *Security Continuous Monitoring (DE.CM)* — continuously monitoring systems and networks.
- *Adverse Event Analysis (DE.AE)* — identifying and analyzing abnormal activity.

Respond (RS) develops and implements the activities taken on a detected incident; its governing idea is that *a good response reduces the operational and reputational impact of an incident.*

- *Incident Management (RS.MA)* — maintaining and executing incident-response procedures.
- *Incident Analysis (RS.AN)* — analyzing the incident to support response and recovery.
- *Incident Response Reporting and Communication (RS.CO)* — coordinating internal and external communications.
- *Incident Mitigation (RS.MI)* — containing, eradicating, and reducing the incident's impact.

Recover (RC) develops and implements the activities that maintain resilience plans and restore impaired capabilities or services; its essential insight is that *recovery is not only restoration but also improvement.*

- *Incident Recovery Plan Execution (RC.RP)* — executing recovery processes and restoring services.
- *Incident Recovery Communication (RC.CO)* — communicating recovery status and return-to-normal progress.

7.4 Implementation Tiers

The second part of the framework is the set of **Implementation Tiers**, which provide context on *how an organization views cybersecurity risk* and on the processes it has in place to manage that risk. They range from **Tier 1 (Partial)** through **Tier 2 (Risk Informed)** and **Tier 3 (Repeatable)** to **Tier 4 (Adaptive)**, describing an increasing degree of rigor and sophistication in risk-management practices.

Two caveats are essential to interpreting the Tiers correctly. First, the Tiers indicate how an organization has implemented its practices and are useful for supporting improvement, but they **do not provide any certification**. Second, **the Tiers do not represent maturity levels**. An organization at Tier 1 is encouraged to consider moving higher, but the purpose of the Tiers is to support decision-making about how to manage cybersecurity risk and about which dimensions deserve additional resources. Progression to a higher Tier is encouraged only when a cost-benefit analysis indicates it would achieve a feasible and cost-effective reduction of risk — climbing the Tiers is a deliberate economic choice, not an automatic goal.

Each Tier can be understood across three dimensions: the **risk-management process**, the **integrated risk-management program**, and the degree of **external participation**.

Tier	Risk-management process	Integrated program	External participation
Tier 1 — Partial	Ad hoc, reactive, not formalized; prioritization not driven by risk objectives	Limited organization-level awareness; risk managed case by case; may lack information-sharing	None
Tier 2 — Risk Informed	Management-approved, but policies may be absent; prioritization may not flow from risk objectives	Aware but not organization-wide; informal internal sharing; assessments not repeatable	Low
Tier 3 — Repeatable	Policy formally approved, defined, and regularly updated	Organization-wide approach to managing risk	High
Tier 4 — Adaptive	Standards and best practices applied; continuous improvement	Organization-wide, risk-informed, adapts to events as they evolve	Full

7.5 Profiles

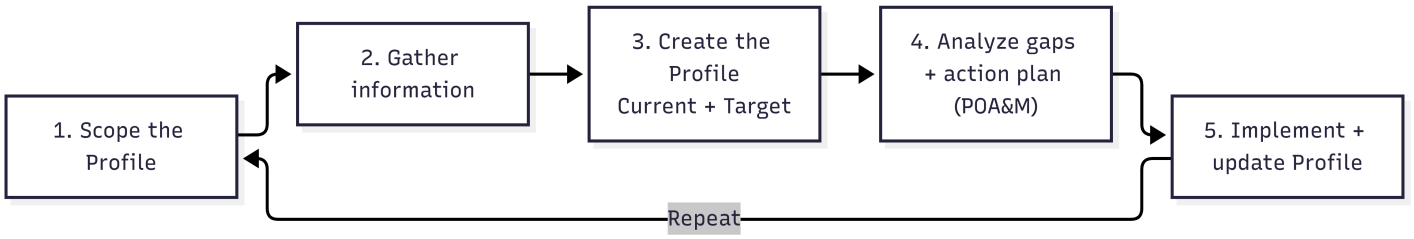
The third part of the framework is the **Profile**, the alignment of the Functions, Categories, and Subcategories with the particular business requirements, risk tolerance, and resources of an organization. The **Current Profile** indicates the cybersecurity outcomes presently being

achieved, while the **Target Profile** indicates the outcomes needed to meet the organization's desired risk-management goals. The comparison between the two enables a **gap analysis**: the difference between where the organization is and where it wishes to be defines the work that remains. The framework prescribes no Profile templates, preserving flexibility in how the comparison is implemented.

The Community Profile. Beyond the Current and Target Profiles of a single organization, the framework recognizes the **Community Profile**, a baseline of CSF outcomes created and published to address shared interests among a group of organizations. It is typically developed for a particular sector, subsector, technology, or threat type, and an organization can adopt it as the starting point for its own Target Profile rather than building one from nothing. The Community Profile is, in spirit, the NIST analogue of the sector-level Contextualization in the Italian National Framework.

The Organizational Profile lifecycle. Creating and using an Organizational Profile is a repeatable five-step cycle, the practical method by which the framework drives continuous improvement.

1. **Scope the Organizational Profile** — document the high-level facts and assumptions that define the Profile's scope. A Profile may cover an entire organization or a single concern, such as the financial systems or the handling of ransomware threats.
2. **Gather the information needed** — collect organizational policies, risk-management priorities and resources, enterprise risk profiles, business impact analysis (BIA) registers, the cybersecurity requirements and standards followed, and the practices, tools, and work roles already in place.
3. **Create the Organizational Profile** — determine which information to record for each selected outcome and document both the Current state and, considering the risk implications, the Target state.
4. **Analyze the gaps and create an action plan** — conduct the gap analysis between Current and Target Profiles and develop a prioritized action plan, expressed for instance as a risk register, a risk detail report, or a Plan of Action and Milestones (POA&M).
5. **Implement the action plan and update the Profile** — execute the plan to move toward the Target state, then repeat the cycle as often as needed.



7.6 How to Use the NIST CSF

The framework supports a range of practical uses: as a basic review of existing practices; as the foundation for establishing or improving a cybersecurity program; as a vehicle for

communicating cybersecurity requirements with stakeholders; as an input to buying decisions; as a means of identifying opportunities for new or revised informative references; as a methodology for protecting privacy and civil liberties; and as an instrument for self-assessing cybersecurity risk.

Risk communication across the organization. One of the framework's central purposes is to improve the flow of cybersecurity information between the three layers of an organization, and this communication is **bidirectional**. Executives set mission priorities, risk appetite, and budget, which cascade downward; managers translate these into Target Profiles and track the action plan; and practitioners implement the target state and measure operational risk, feeding key performance and key risk indicators back upward. Practitioners report implementation progress and emerging risks upward, while executives communicate expectations and resources back down — the loop that keeps strategy and operations aligned.

Integration with enterprise risk management. Cybersecurity risk is one of several risks an enterprise faces, alongside financial, privacy, supply-chain, reputational, and technological risks. The framework is explicitly designed to be used *in conjunction with* other risk instruments and to feed into **Enterprise Risk Management (ERM)**, translating cybersecurity terminology into the general risk language executives understand so that cybersecurity can be balanced as one element in a portfolio of enterprise risk. It also complements the **NIST Risk Management Framework (RMF)** — notably SP 800-37 and SP 800-30 — and its approach to selecting controls from SP 800-53.

Cybersecurity risk versus privacy risk. Cybersecurity and privacy are independent disciplines whose objectives overlap. Cybersecurity risks arise from the *loss* of confidentiality, integrity, or availability of systems and data; privacy risks arise from the *processing* of individuals' data and can occur even in the absence of any security incident. The two overlap in the region of **cybersecurity-related privacy events** — for example, a data breach that leads to identity theft. Privacy harms range from dignity-type effects, such as embarrassment or stigma, to tangible harms, such as discrimination, economic loss, or physical harm. The NIST Privacy Framework and the Cybersecurity Framework are intended to be used together to address these distinct but intersecting aspects.



This convergence of cybersecurity and data protection is what the Italian National Framework later operationalizes through its dedicated GDPR contextualization. The framework also extends to newer risk domains: **supply-chain risk** is addressed through the Govern function's C-SCRM category (with deeper guidance in SP 800-161), and **risks from emerging technologies** such as artificial intelligence are addressed in coordination with the NIST AI

Risk Management Framework (AI RMF), which borrows the same Functions-Categories-Subcategories structure.

7.7 The Framework in Action: A Worked Example

The abstract machinery of the CSF becomes concrete when applied to a scenario. Consider a **university research department** that wishes to protect sensitive research data, cloud services, staff accounts, and laboratory systems. The first governance step is to define the **organizational scope**, which here encompasses the research servers and laboratory systems; the cloud storage and collaboration platforms; the research data and personal information; the staff, researchers, students, suppliers, and external partners; and the legal, contractual, and university security requirements that apply.

With scope established, the department constructs its **Profiles** by examining each relevant Function and Category, recording the current state, the target state, and the priority. The result is a structured gap analysis.

Function and category	Current state	Target state	Priority
GV.OC — Organizational Context	Critical services and external dependencies not fully documented	Document critical services, stakeholders, dependencies, requirements	High
GV.RR — Roles and Responsibilities	Security responsibilities informally assigned to IT staff	Approve clear responsibilities across management, IT, researchers, suppliers, IR	High
GV.SC — Supply Chain Risk Mgmt	Cloud and software suppliers reviewed inconsistently	Establish supplier security requirements, due diligence, monitoring, obligations	High
ID.AM — Asset Management	Hardware inventoried; cloud services and datasets incomplete	Maintain updated inventory with owners, classifications, criticality	High
ID.RA — Risk Assessment	Performed only after major incidents or projects	Perform periodic and event-driven assessments via documented method	High
PR.AA — Access Control	MFA protects admins but not all users	Require MFA for all accounts; review privileged access regularly	High
PR.DS — Data Security	Backups exist; encryption and restore testing vary	Encrypt sensitive data, define retention, test restoration regularly	High
DE.CM — Continuous Monitoring	Logs collected from selected systems, reviewed manually	Centralize logs, generate alerts for suspicious activity	Medium

Function and category	Current state	Target state	Priority
RS.MA — Incident Management	General IT procedures only; no research-specific playbook	Establish, test, maintain IR procedures and escalation paths	High
RC.RP — Recovery Plan Execution	Services restorable but priorities and objectives unclear	Define recovery priorities, responsibilities, RTOs, RPOs, exercises	Medium

The department then situates itself on the Tier scale, and it is best characterized as **Tier 2 — Risk Informed**: it recognizes cybersecurity risks and performs some risk-management activities, but its practices are not yet consistently established or applied across the organization. The evidence is telling: security risks are discussed in IT and project meetings; some systems use MFA, backups, logging, and access reviews; risk assessments occur for selected projects or incidents; supplier risks are known but not managed through a consistent process; information flows mainly through informal communication; and procedures vary between laboratories and research teams.

The chosen target is **Tier 3 — Repeatable**, in which risk-management practices are formally approved, documented, consistently implemented, and regularly reviewed across the department. Reaching it requires a concrete set of actions: approving an organization-wide risk-management policy; assigning formal roles and responsibilities; maintaining complete asset and data inventories; applying a single risk-assessment method to all relevant systems; requiring MFA and periodic access reviews; standardizing supplier-security requirements; centralizing monitoring and incident reporting; testing incident-response and disaster-recovery plans; and using metrics and management reviews to track improvement.

7.8 What Tier Was WannaCry?

The framework also offers a diagnostic lens on the WannaCry failure. Reconsider the organization in the aftermath of the attack, having discovered that IT knew some machines were unpatched, that there was no complete inventory of Windows systems, that patching was performed manually whenever administrators happened to do it, that the security team had previously warned management about the risk, that there was no organization-wide process for prioritizing vulnerabilities, that some systems could not be patched immediately because of compatibility or availability concerns, and that although an incident-response procedure existed, it had never been tested through a large-scale exercise — prompting management to ask, 'How did we not know this was going to happen?'

This profile corresponds unmistakably to **Tier 1 — Partial**. The reactive, ad hoc patching, the absence of a complete inventory, the lack of any organization-wide vulnerability-prioritization process, and the informal, unheeded internal warnings are the textbook signature of the Partial Tier. Improving beyond it would mean precisely the moves that define the ascent toward Tier 3: building a formal asset inventory, instituting a documented and repeatable

vulnerability-management process, assigning clear responsibilities, and establishing an organization-wide, management-endorsed approach to prioritizing and treating risk.

The scenario therefore sharpens the chapter's recurring lesson: the organization failed not for lack of documented policies and controls, but because those controls were partial, untested, and unverified in operation — the gap between a control's *design* and its demonstrable *operation* that the implementation layer exists to expose.

8. The Italian National Cybersecurity Framework (NCF)

Having examined one international standard (ISO 27001) and one international framework (NIST CSF), the natural next step is a **national** framework: the Italian National Cybersecurity Framework, formally the *Framework Nazionale per la Cybersecurity e la Data Protection*. Its existence illustrates an important principle of governance — that a framework may need to be adapted to the economic and institutional realities of the country in which it operates, without abandoning international compatibility.

8.1 Motivations

The Italian NCF was created to pursue several interlocking objectives, each responding to a feature of the national context.

The first and most elevated aim is to bring **cyber-risk awareness to the highest levels of the business**, so that cybersecurity is no longer treated merely as a matter for technical staff. The framework seeks to make organizations fold cyber risk into their general risk-management processes, treating it as a component of *economic* risk rather than a purely technical nuisance.

This ambition is shaped decisively by the **structure of the Italian economic landscape**. A striking **69% of the national GDP** is generated by small and medium-sized enterprises (SMEs), and large national enterprises make up very few — on the order of **0.1%**. A framework intended for Italy cannot be designed around the resources and sophistication of large corporations; it must be usable by a vast population of small organizations with limited budgets and limited specialist staff.

The framework further aims to create something **recognized at the international level**, to **improve information-sharing capacity**, and to **increase the duty of care at the national level**. Finally, it embraces a principle of economy and pragmatism: **do not reinvent the wheel**. There is no sense in building a wholly new framework from scratch when a recognized, well-tested one already exists. Accordingly, the Italian NCF takes the **NIST CSF** as its starting point, inheriting its structure of Functions, Categories, and Subcategories and adapting it to the Italian situation.

8.2 What the National Framework Is and Is Not

A precise understanding of the NCF requires a sharp boundary around what it is *not*. The National Framework is **not a standard**, and consequently **it is not certifiable**. This distinguishes it fundamentally from ISO 27001.

What it *is* instead is a **tool for self-assessment**. It allows an organization to assess its **current profile** and to define its **target profile**, then supports the **gap analysis** between the two and the definition of a **roadmap** to close that gap. In spirit and mechanism it is far closer to the NIST CSF than to ISO 27001: it organizes and communicates risk and guides improvement, rather than imposing auditable requirements leading to a certificate.

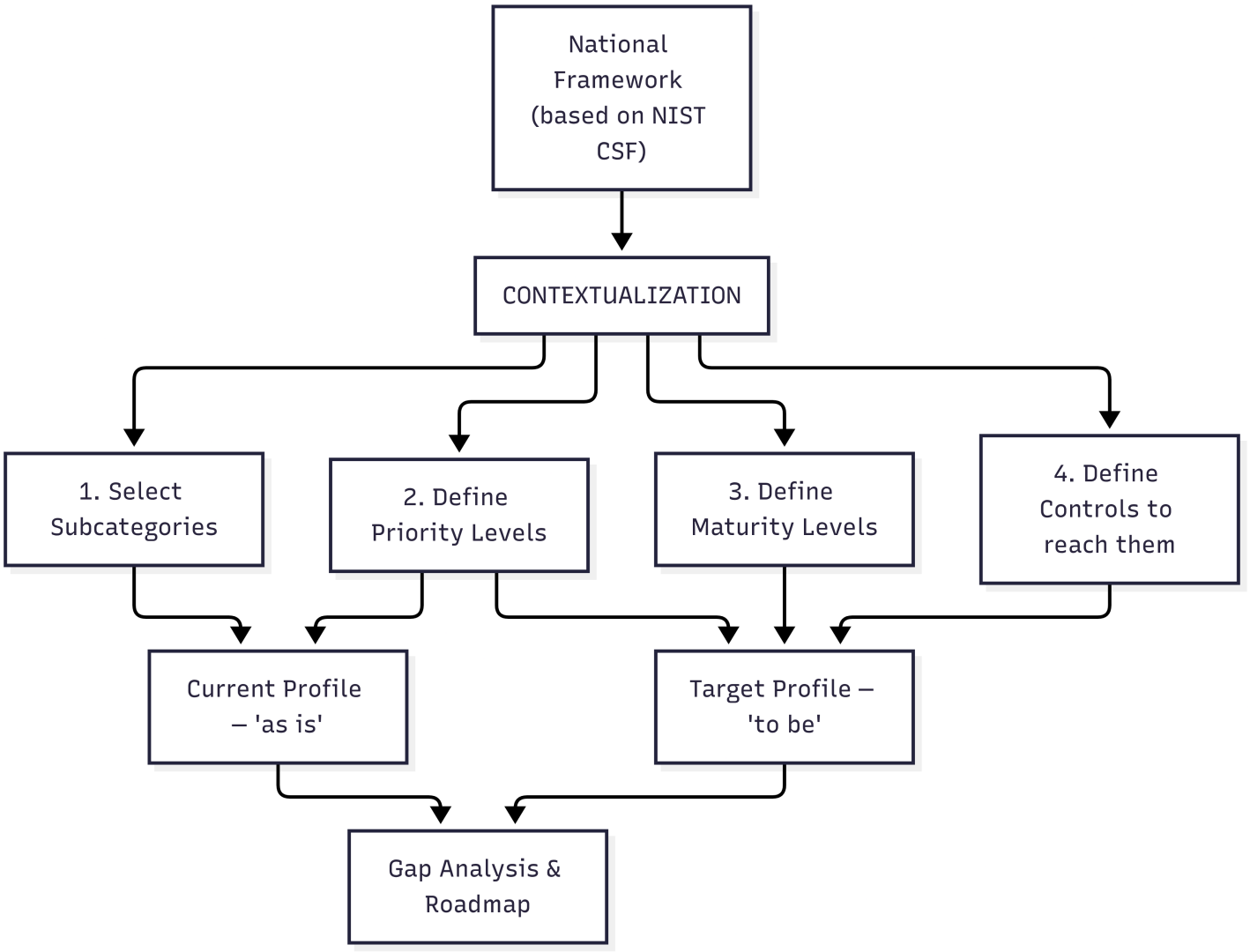
8.3 Contextualization

The central and distinctive mechanism of the Italian NCF is **Contextualization**: the process by which the framework is 'customized' to adapt it to a specific context, whether a sector, a public administration, or an individual organization. It is the operation that turns a generic national framework into a concrete, tailored instrument.

A Contextualization is defined through four steps.

1. **Selection of Subcategories** — choosing, from the full catalogue inherited from the CSF structure, those relevant to the context at hand.
2. **Definition of a Priority Level** for each selected Subcategory.
3. **Definition of Maturity Levels.**
4. **Definition of the Controls** needed to reach those maturity levels.

In this way the framework layers two additional dimensions — **priority** and **maturity** — onto the Functions, Categories, Subcategories, and Informative References it inherits. Once a Contextualization exists, the two profiles familiar from the CSF can be defined: the **Current Profile** ('as is') and the **Target Profile** ('to be'), whose gap again drives the improvement roadmap.

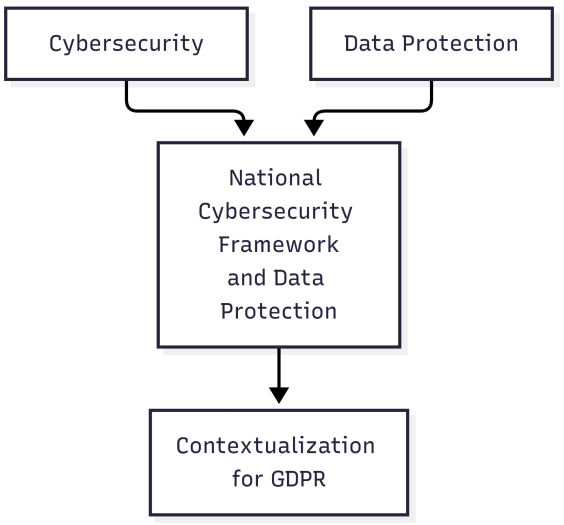


The flexibility of Contextualization is deliberately broad. A Contextualization can be defined for a single organization; for an entire economic or productive sector; for groups of organizations grouped by scale, such as SMEs or large enterprises; for regulated sectors, such as public administrations or banks; and for critical infrastructures. This directly answers the structural concern noted earlier: because a Contextualization can be built specifically for SMEs, the framework can serve the overwhelming majority of the Italian economy without demanding the resources of a large enterprise.

> [**Photo: Contextualization architecture diagram**] The architecture places the National Cyber Security Framework across the top as the common reference. On its left sits the **Implementation Tier** dimension, drawing on enterprise risk-management references (CAS, COSO, ISO); on its right sits the **Priority Levels / Maturity Levels** dimension, drawing on IT and security standards (COBIT, SANS, ISO). Between these flanks, the central body stratifies the economy into layers — Critical Infrastructures at the top, Large Organizations in the middle, Small and Medium Enterprises at the base — crossed vertically by production and regulated sectors. The theoretical message is that the single national framework can be contextualized along two axes at once: *vertically* for a given productive or regulated sector, and *horizontally* as a general contextualization driven by enterprise risk management on one side and security standards on the other.

8.4 Contextualization for GDPR

A particularly significant Contextualization integrates **data protection** into cybersecurity. Cybersecurity and data protection, though historically distinct disciplines, converge on the same assets and the same risks. By combining them, the framework produces a unified **National Cybersecurity Framework and Data Protection**, and this combined instrument can then be contextualized for the **GDPR**, the European General Data Protection Regulation.

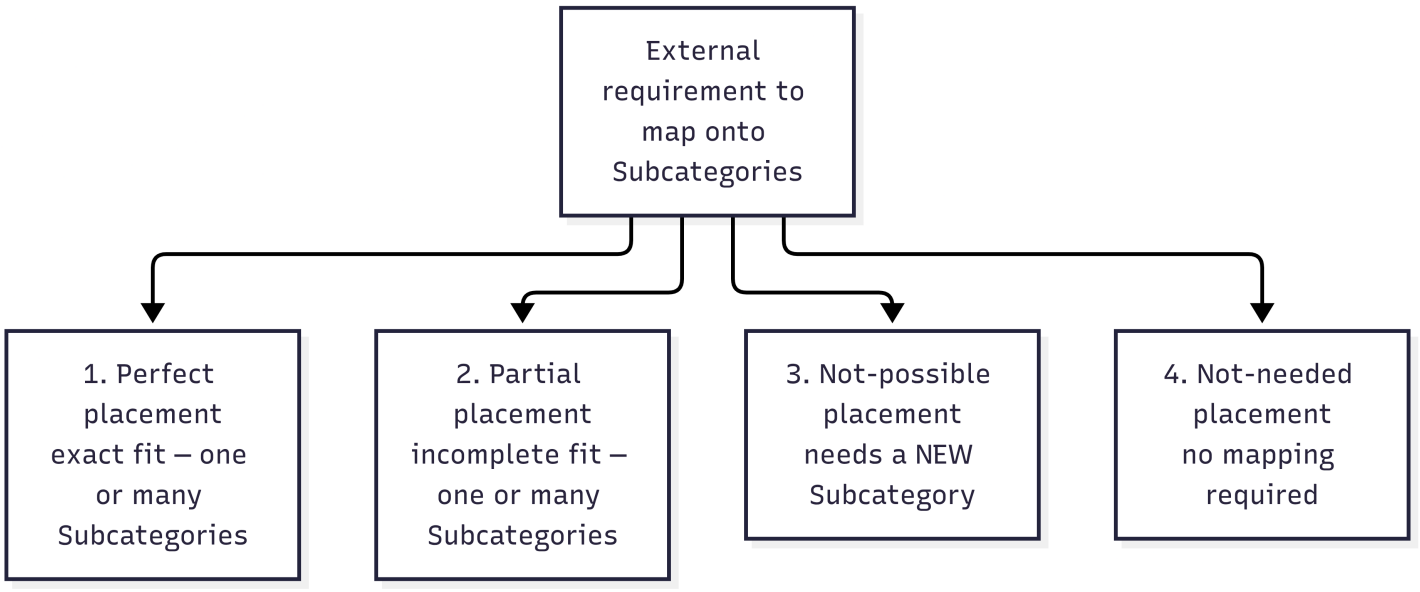


This integration is reflected in the evolution to **Framework 2.0**, whose architecture adds an explicit **Data Protection Regulations** dimension — with GDPR as its leading entry — alongside the existing IT and security standards. Data-protection regulation is thus elevated to a first-class reference axis of the framework, standing on equal footing with the technical security standards that inform priority and maturity levels.

8.5 Framework 2.0: The Four Placement Cases

When an external requirement, control, or regulatory obligation must be mapped onto the framework's structure of Subcategories, four distinct cases can arise. Understanding them is essential to performing a Contextualization correctly, because they determine whether the existing structure suffices or must be extended.

- **Perfect placement** — the requirement maps cleanly and completely onto the existing structure, whether on a single Subcategory or across several; the fit is exact.
- **Partial placement** — the requirement maps onto the existing structure only incompletely, covering part but not all of a Subcategory, again on one or more Subcategories.
- **Not-possible placement** — the requirement cannot be accommodated by any existing Subcategory, signalling a genuine gap and creating the need for a **new Subcategory**.
- **Not-needed placement** — a mapping is simply not required; the requirement does not call for placement within the framework.



9. Implementation: ISO/IEC 27002

With the framework layer complete, the chapter turns to the **implementation layer** — the instruments that answer the question of *how* to apply the strategy. The first is **ISO/IEC 27002**, the natural companion to ISO 27001.

9.1 Nature and Purpose

ISO/IEC 27002 is the **companion standard on information security controls**. Where ISO 27001 specifies *what* must be managed, ISO 27002 explains *how* the controls can be understood and implemented. It provides a reference set of generic controls together with detailed implementation guidance, and it is designed to be used within an ISMS based on ISO 27001, for implementing commonly accepted controls, or for developing organization-specific guidance. In practical terms, it gives security teams the detailed direction they need to select and apply controls consistently.

A defining characteristic sets 27002 apart from 27001: **it is not a certification standard.** Certification is always against ISO 27001, never against ISO 27002, because **there is no single correct way to implement a control.** Since implementation legitimately varies according to an organization's circumstances, technologies, and risk profile, 27002 can offer guidelines and best practices but cannot impose the uniform, verifiable requirements that certification demands. It describes the detail of each control, starting from Annex A of ISO 27001 and expanding each entry into actionable guidance.

9.2 Structure of ISO/IEC 27002:2022

The 2022 edition reorganized the control catalogue into **four themes**, or macro-areas, to make the catalogue easier to navigate and apply. The total control set comprises **93 controls**.

Theme	Number of controls
Organizational	37

Theme	Number of controls
People	8
Physical	14
Technological	34
Total	**93**

Comparing the themes is instructive: the **People** controls tend to be nearly the same for every organization — background screening, training, and disciplinary processes look broadly similar regardless of industry — whereas the **Technological** controls vary considerably from one organization to another, reflecting differences in systems, architectures, and technical environments.

Organizational controls (5.1–5.37) include policies for information security; information security roles and responsibilities; segregation of duties; management responsibilities; contact with authorities and with special interest groups; threat intelligence (new in 2022); information security in project management; inventory of information and assets; acceptable use and return of assets; information classification and labelling; access control and identity management; supplier security policies; information security in cloud services (new); incident management and ICT continuity; and legal, compliance, documentation, and audit requirements.

People controls (6.1–6.8) comprise screening (background verification of candidates prior to employment); terms and conditions of employment; information security awareness, education, and training; a disciplinary process for policy violations; responsibilities that remain after termination of employment; confidentiality or non-disclosure agreements; remote-working security controls; and a mechanism for the prompt reporting of security events.

Physical controls (7.1–7.14) cover physical security perimeters; physical entry controls; securing offices, rooms, and facilities; physical security monitoring (new); protection against physical and environmental threats; working in secure areas; clear desk and clear screen; equipment siting and protection; security of assets off-premises; storage media; supporting utilities; cabling security; equipment maintenance; and secure disposal or re-use of equipment.

Technological controls (8.1–8.34) are grouped into four clusters: *Access and Identity* (8.1–8.6) — user endpoint devices, privileged access rights, access control, authentication, identity management, and privileged utilities; *Data Protection* (8.10–8.12, 8.24) — information deletion (new), data masking (new), data leakage prevention (new), and cryptography; *Security Operations* (8.15–8.22) — logging and monitoring, clock synchronization, web filtering (new), network security, configuration management (new), and vulnerability management; and *Secure Development and Acquisition* (8.25–8.34) — the secure development lifecycle, secure coding (new), security testing, outsourced development, and separation of environments.

9.3 Annex A versus ISO 27002: Normative versus Guidance

The relationship between ISO 27001's Annex A and ISO 27002 rests on a single distinction: **Annex A is normative; ISO 27002 is guidance.**

Annex A of ISO 27001 contains a catalogue of control objectives and controls that organizations select as part of their ISMS. It is **normative** — binding and linked directly to the risk-treatment process — but also terse, typically stating only the control's name and a one-sentence statement of what it requires. ISO 27002:2022, by contrast, provides **detailed implementation** guidance, purpose statements, and attributes for the *same 93 controls* referenced in ISO 27001, translating the austere Annex A entries into actionable operative measures.

The difference is vividly illustrated by a single control. In Annex A, the control on access to program source code appears as little more than a title and the normative statement that access to program source code shall be restricted. In ISO 27002, the same control is accompanied by extensive guidance: that source code and associated items such as designs, specifications, and verification plans should be strictly controlled to prevent the introduction of unauthorized functionality and preserve confidentiality; that source libraries should preferably not be held in operational systems; that source code and libraries should be managed according to established procedures; that support personnel should not have unrestricted access; that updates and the issuing of source to programmers should occur only after appropriate authorization; that program listings should be held securely; that an audit log of all accesses should be maintained; that maintenance and copying should be subject to strict change-control procedures; and that, where source code is to be published, additional integrity controls such as digital signatures should be considered. The same control exists in both documents, but Annex A declares the *obligation* while ISO 27002 supplies the *method.*

10. Implementation: NIST SP 800-53

The second instrument of the implementation layer is **NIST Special Publication 800-53**. If ISO 27002 is the implementation companion to the ISO 27001 management-system standard, SP 800-53 plays an analogous role on the NIST side, translating high-level framework outcomes into a concrete, detailed catalogue of controls — with far greater granularity and a distinctly American, federal orientation.

10.1 The Special Publications Series

NIST periodically publishes technical reports providing guidelines, recommendations, and reference material, particularly in information security. These are the **Special Publications (SPs)**. The **SP 800 series** is NIST's primary channel for publishing computer, cyber, and information security guidelines, recommendations, and technical specifications, and SP 800-53 is among its most important members.

NIST Special Publication 800-53 provides a comprehensive catalogue of **security and privacy controls** for information systems and organizations. Its controls are deliberately **flexible and customizable**: organizations select and tailor them according to their mission,

business needs, risk tolerance, and operating environment. Like ISO 27002, SP 800-53 is **a control catalogue and implementation reference, not a certification standard**. It supplies the raw material of controls; it does not issue certificates.

The publication pursues three explicit aims:

- provide a **catalogue of security and privacy controls** for federal organizations and information systems;
- provide a **process to select controls** that protect an organization's mission, functions, image, reputation, assets, and individuals — and the state itself — from cyberattacks, disasters, malfunctions, and human errors;
- provide a **methodology for selecting critical controls**, which is the concept of the **baseline**.

10.2 Key Concepts

- **Security controls** — the safeguards and countermeasures that protect the confidentiality, integrity, and availability of systems and information.
- **Privacy controls** — the administrative, technical, and physical safeguards that manage privacy risks and support compliance with applicable privacy requirements.
- **Control families** — groups of controls organized by topic, such as access control, incident response, risk assessment, and supply-chain risk management.
- **Control enhancements** — additional requirements that strengthen or extend a base control for higher-risk environments.
- **Tailoring** — the process of adapting controls to a specific organization, system, mission, technology, or risk profile.

10.3 Revision 5 and the Twenty Control Families

The current edition, **Revision 5**, marks a significant structural change: it **integrates security and privacy controls into a single consolidated catalogue** rather than treating them as separate concerns, reflecting the growing recognition that security and privacy are deeply intertwined.

Revision 5 organizes the catalogue into **20 control families**. Each family contains the security controls related to its general topic, and the organization into families exists specifically to ease the control-selection and specification process.

ID	Family	ID	Family
AC	Access Control	PE	Physical and Environmental Protection
AT	Awareness and Training	PL	Planning
AU	Audit and Accountability	PM	Program Management

ID	Family	ID	Family
CA	Assessment, Authorization, and Monitoring	PS	Personnel Security
CM	Configuration Management	PT	PII Processing and Transparency
CP	Contingency Planning	RA	Risk Assessment
IA	Identification and Authentication	SA	System and Services Acquisition
IR	Incident Response	SC	System and Communications Protection
MA	Maintenance	SI	System and Information Integrity
MP	Media Protection	SR	Supply Chain Risk Management

Within this structure, each control generally includes a base control, a discussion, related controls, control enhancements, and references. As with the NIST CSF Core, the controls are intended to be **outcome-oriented and adaptable** rather than implemented as an identical checklist in every organization — a philosophy consistent across all the NIST instruments.

10.4 The Structure of a Security Control

Each security control in SP 800-53 follows a consistent internal structure of five components.

- **Control** — prescribes the specific security-related activities or actions to be carried out by organizations or information systems.
- **Supplemental Guidance** — non-prescriptive additional information that helps interpret and apply the control.
- **Control Enhancements** — statements of additional security capability that add functionality and specificity to the base control or increase its strength, for higher-risk environments.
- **References** — the applicable federal laws, Executive Orders, directives, policies, regulations, standards, and guidelines relevant to the control.
- **Priority and Baseline Allocation** — the recommended priority codes used to sequence implementation decisions and the initial allocation of controls and enhancements to the baselines.

Consider a representative control from the Audit and Accountability family, **AU-3, Content of Audit Records**. Its *Control* statement requires that the information system generate audit records establishing what type of event occurred, when and where, the source and outcome of the event, and the identity of any individuals associated with it. Its *Supplemental Guidance* clarifies that the satisfying content may include time stamps, source and destination addresses, user or process identifiers, event descriptions, success or failure indications, filenames involved, and the access-control or flow-control rules invoked. Its *Control Enhancements* add, among others, the capability to generate records containing additional organization-defined detailed information and the capability to centrally manage and configure the content captured in audit

records. Its *References* in this instance are none. Its *Priority and Baseline Allocation* assigns priority P1 and allocates the base control AU-3 to the LOW baseline, AU-3 with enhancement (1) to the MODERATE baseline, and AU-3 with enhancements (1) and (2) to the HIGH baseline. The five components together move from a terse requirement to a fully specified, risk-calibrated control.

10.5 Baselines: SP 800-53B

The mechanism for selecting controls in a principled way is provided by a companion document, **SP 800-53B**, devoted to the **security control baseline**, whose aim is to assist organizations in making the appropriate selection of controls for their information systems.

Baseline controls are the starting point for control selection. They are chosen on the basis of the **security category** and associated **impact level** of the information system in question. Three baselines exist, corresponding to **low-impact, moderate-impact, and high-impact** systems. An organization first categorizes its system by impact level, adopts the matching baseline as its point of departure, and afterward tailors the controls to its specific circumstances.

The baseline concept carries notable **limitations** that bound its general applicability. It is tightly tied to *federal* information systems and to U.S. legislation, policies, directives, instructions, regulations, and memoranda — more than fifty such instruments in total, all U.S.-based. It is also **very complex** and oriented toward operational personnel rather than strategic decision-makers. These limitations explain why SP 800-53, for all its thoroughness, is best understood as a deeply detailed federal control catalogue rather than a lightweight, universally portable instrument.

A worked scenario clarifies how baselines operate. Suppose an organization runs an internal web application for a federal agency that processes sensitive but unclassified data, where a breach would cause **serious — though not catastrophic — impact** on confidentiality, integrity, or availability. Under FIPS 199, such a system is categorized as **Moderate-impact**. The organization turns to SP 800-53B and selects the **Moderate security control baseline**, which includes roughly **287 security controls and enhancements** drawn from SP 800-53, then tailors the set to its particular environment.

11. Implementation: CIS Controls

The third and final instrument of the implementation layer is the **CIS Controls**, published by the **Center for Internet Security (CIS)**, a U.S. non-profit focused on improving global cybersecurity standards and practices. CIS collaborates with experts from industry, government, and academia to develop best practices that help organizations secure their IT systems and data. Where SP 800-53 is exhaustive and federal, the CIS Controls are **prioritized and pragmatic**, designed to tell an organization not merely *what* could be done but *what to do first.*

The CIS Controls are a **prioritized set of actions** developed by a global community of cybersecurity experts to provide clear, actionable steps against the most prevalent threats. They are organized into categories and **ranked by effectiveness and ease of implementation**, which makes them practical for organizations of every size.

11.1 Design Principles and Structure

Three design principles govern the construction of the CIS Controls.

- **Offense informs defense** — controls are selected and prioritized using *empirical knowledge* of how attackers actually behave and how their behavior can be stopped.
- **Focus and feasibility** — each safeguard is specific and practical to implement, avoiding 'nice-to-have' or purely theoretical requirements.
- **Measurable and aligned** — safeguards are written to be measurable and to align with other frameworks, including the NIST CSF, ISO 27001, OWASP, and MITRE ATT&CK.

Structurally, **CIS Controls v8.1** consists of **18 top-level controls** broken down into **153 safeguards**, each describing a single actionable requirement. The controls are **numbered by priority** — the numbering reflects the order in which they should be addressed — and grouped into three tiers of ascending sophistication:

1. Inventory and Control of Enterprise Assets
2. Inventory and Control of Software Assets
3. Data Protection
4. Secure Configuration of Enterprise Assets and Software
5. Account Management
6. Access Control Management *(Controls 1–6: **Basic** — fundamental measures)*
7. Continuous Vulnerability Management
8. Audit Log Management
9. Email and Web Browser Protections
10. Malware Defenses
11. Data Recovery
12. Network Infrastructure Management
13. Network Monitoring and Defense
14. Security Awareness and Skills Training
15. Service Provider Management
16. Application Software Security *(Controls 7–16: **Foundational** — more advanced practices)*
17. Incident Response Management
18. Penetration Testing *(Controls 17–18: **Organizational** — policies and governing processes)*

The lowest-numbered controls address the most fundamental, highest-priority hygiene — knowing what assets you have — while the higher numbers progress toward advanced and organizational practices.

Cutting across the eighteen controls is a second organizing dimension: every Safeguard is associated with one of six **asset classes**, which identify the part of the enterprise the Safeguard acts upon.

- **Devices** — enterprise assets such as end-user devices, servers, IoT and non-computing devices, network devices, and removable media.
- **Software** — applications, operating systems, and their services, libraries, and APIs.
- **Data** — sensitive data, log data, and physical data.
- **Users** — the workforce, service providers, and user, administrator, and service accounts.
- **Network** — network infrastructure and architecture.
- **Documentation** — plans, policies, processes, and procedures.

Pairing each Safeguard with an asset class makes explicit *what* is being protected and lets an organization reason about coverage across its real inventory rather than against an abstract list.

11.2 Implementation Groups

To make the 153 safeguards usable by organizations of radically different capacity, CIS assigns them to **Implementation Groups (IG)** — IG1, IG2, and IG3 — based on size, risk, and resources.

IG1, described as **basic cyber hygiene**, comprises approximately **56 safeguards** that *every* organization should implement, focused on the most common attack vectors. **IG2 and IG3** progressively add more advanced safeguards for organizations with higher risk, more complex environments, or regulatory obligations.

The IG1 safeguards cover the essentials of defensive hygiene: maintaining an **inventory** of devices and software (Controls 1–2); applying **secure configuration** of systems and applications (Control 4); enforcing **multi-factor authentication** and least privilege (Controls 5–6); performing **patch and vulnerability management** (Control 7); and sustaining **backup and tested data recovery, basic logging, anti-malware defenses, and awareness training** (Controls 8–14). These are the **first controls to implement** when building or improving a security program, a principle CIS captures in its slogan: *"start with IG1."*



11.3 Mapping to Other Frameworks and the CIS Workflow

A major practical virtue of the CIS Controls is their **interoperability**. CIS Controls v8.1 are mapped to and referenced by multiple legal, regulatory, and policy frameworks, including NIST CSF 2.0, ISO 27001, PCI DSS, SOC 2, and CMMC. The addition of a Govern function and updated mappings in v8.1 improve consistency with the NIST CSF 2.0 functions. The consequence is powerful: an organization can **implement the CIS Controls once and then use the mappings to demonstrate alignment across several governance and compliance regimes simultaneously**, avoiding duplicated effort.

This leads to a recommended **CIS Controls workflow**.

1. Use the CIS Controls as a **prioritized roadmap**, starting with IG1 and extending to IG2 and IG3 according to risk profile and regulatory drivers.
2. Treat each safeguard as a **measurable requirement** backed by evidence of both its *design* and its *operation*, rather than as a generic 'good practice.'
3. Use the Controls to structure security governance, then plug in the other frameworks (NIST CSF, ISO 27001, the Italian NCF) as overlays and mappings for reporting.

11.4 Benefits of the CIS Controls

- **Prioritized, actionable steps** — effort is focused on the most impactful risk-reduction measures.
- **Attack-surface and risk reduction** — directly targets common attack vectors such as unmanaged assets, misconfigurations, missing patches, and weak access control.
- **Essential cyber-hygiene baseline** — IG1 provides a realistic minimum program for an organization of any size.
- **Efficient use of limited resources** — helps allocate scarce budget and staff to the highest-value controls first.
- **Multi-framework alignment** — maps cleanly to NIST CSF, SP 800-53, ISO 27001, PCI DSS, and HIPAA, easing compliance.
- **A widely recognized "reasonable" standard** — used globally and increasingly cited by regulators and insurers as evidence of due diligence.

11.5 Treating the WannaCry Risk

The CIS Controls bring the WannaCry narrative to its operational conclusion. Having *identified* the central risk — that unpatched Windows systems could be exploited and cause widespread compromise — and having *decided* that the risk must be treated, the security team faces the final, concrete question: which controls to actually implement, given a single month in which to act.

The appropriate measures are precisely those drawn from the prioritized CIS hygiene: a **formal patch-management process**, a complete **asset inventory**, **network segmentation**, and **vulnerability scanning**. These directly dismantle the conditions that made the attack possible. By contrast, measures such as a new company logo or an annual security presentation, whatever their other merits, do nothing to reduce the technical risk at hand and would be mis-prioritized responses.

A final lesson concerns the difference between *claiming* a control and *evidencing* it. Suppose the organization asserts that it performs vulnerability scanning regularly. What would actually convince an auditor? Statements such as 'we have a vulnerability-management policy,' 'the CISO says scanning is performed,' 'a scanner is installed,' or 'the vendor says the scanner works' are all insufficient, because a policy is not its execution, an assertion is not proof, and an installed or vendor-endorsed tool is not evidence of operation. What genuinely demonstrates the control are **scan reports from the last six months** together with **evidence that identified vulnerabilities were assigned and remediated**. This distinction — between the *design* of a control and its demonstrable *operation* — is the heart of auditability, and it is exactly the discipline whose absence allowed a known, patchable vulnerability to bring hospitals to a standstill.

Chapter 3: Risk Management

An Introduction to Risk Management

Security governance rests on a simple but demanding idea: an organization must be able to decide, deliberately and repeatedly, what could go wrong, how badly, and what to do about it. Everything technical — firewalls, patches, access controls — is downstream of that decision-making. This chapter builds the conceptual machinery that makes those decisions disciplined rather than improvised, beginning with the vocabulary of risk, moving to what it means to *manage* risk as a continuous organizational function, and settling on the reference standard that structures the whole activity, **ISO 31000**, with its three interlocking layers of principles, framework, and process.

1. What Is Risk?

Informally, a **risk** is the potential that something goes wrong and thereby causes harm or loss. That intuition is correct but useless for analysis, because it cannot be measured or compared. The working definition is sharper: a risk is **the likelihood of an incident combined with its consequence for an asset**. Risk is therefore not a single quantity but a *pairing* — how probable an unwanted event is, weighed against how much damage it would do to something we care about.

Each term in that definition is a precise object.

- **Asset** — anything of value to a party. The crucial word is *anything*. It is a mistake to count only tangible, material elements such as servers, buildings, and money; intangible

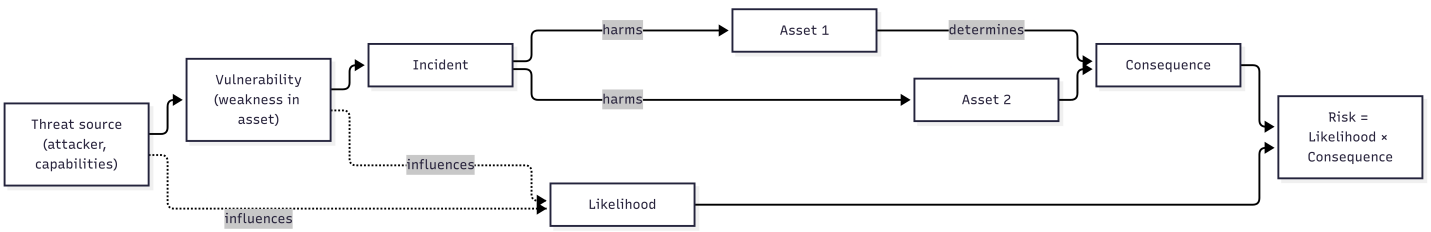
assets are frequently the most valuable and the most fragile. **Reputation** is the canonical example: if word spreads that a hospital suffered a serious incident, patients lose trust and avoid it, and that loss can outweigh any direct physical damage. The analyst must enumerate *all* the elements that contribute to the standing of the company, not just those on an inventory.

- **Party** — the organization, company, person, group, or body on whose behalf the assessment is conducted, simply *whoever is interested in performing the analysis.* Identifying the party is not a formality: it fixes the boundary of the analysis, determines which assets are relevant, and shapes how they are classified. Change the party, and the list of assets changes with it — the same physical system looks like a different problem depending on who is asking.
- **Incident** — an event that harms or reduces the value of an asset. An incident is never considered in the abstract; it is always related back to the assets it touches.
- **Likelihood** — the chance of something occurring, a measurement of the probability of an event. In practice it is an analysis of all the events that could lead to a given incident actually manifesting.
- **Consequence** — the impact of an incident on an asset, expressed as harm or reduced asset value.

Putting these together: for every incident we relate it to the assets it could affect; for each asset we estimate the **probability** the incident happens and the **consequences** it would produce; and from likelihood and impact we derive the **risk** on that asset.

1.1 Why Risk Management Is Hard: the web of relationships

The definitions hide a structural difficulty. There is a list of assets, and for each asset a list of incidents — but a single incident may harm several assets at once. For every incident the likelihood and consequence are not free parameters; they depend on the **vulnerability** present in the asset and on the **type of attacker** that wishes to exploit it, since not all attackers possess the same capabilities. The result is a dense, many-to-many (N–N) network of relationships in which every factor influences the others and much of the behaviour is, from the analyst's seat, effectively random. Managing risk is therefore not a bookkeeping task; it is the disciplined handling of this tangled, uncertain web.



2. What Is Risk Management?

Risk management comprises the coordinated activities used to direct and control an organization with regard to risk. In governance terms, it is nothing other than implementing the **direct-and-control loop** with risk as its subject: a structured set of processes that lets an organization evaluate risks, mitigate them, and monitor the effects of the mitigation to verify that the intervention produced the intended outcome.

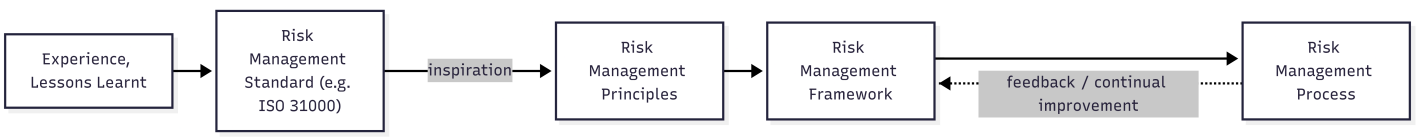
A risk management process is only worth running if it satisfies three joint requirements.

- **Adequate** — the process must fit the specific company under assessment. There is no single fixed process valid for every organization; it must be sized and shaped to the context, matching the available resources and the outcomes that matter.
- **Efficient** — it must evolve smoothly across the life of the company without blocking normal activity or being intrusive. A process that grinds the business to a halt defeats its own purpose.
- **Effective** — the benefit the process brings must be *quantifiable*. If the effect cannot be measured, the loop cannot be closed.

These requirements have a hard economic edge. Performing an assessment has a real cost: the people who perform it, the people who must step away from normal work to supply information, and possibly contracted third parties. For the exercise to be justified, it must ultimately yield a **positive return** — the value protected or created must exceed the cost of protecting it. 'Value creation' here is not merely monetary: preventing damage, avoiding the expense of handling an incident, and preserving reputation are all forms of value, even when they never appear directly as revenue.

2.1 Where Processes Come From: the inspiration pyramid

A company does not invent its risk management process from scratch each time. The process is customized, but the customization starts from something. The chain of inspiration runs as follows: **standards** encode accumulated experience and lessons learnt; from standards come risk management **principles**; principles inspire a **framework**; and the framework lets an organization design its concrete **process**. Feedback flows back up the chain — once a process is running, it is observed to check whether it remains adequate, efficient, and effective, and wherever something is unclear or not working, the process is understood and improved.



3. A Standard for Risk Management: ISO 31000

The first and most general standard in this space is **ISO 31000**. It is a *generic* standard for the risk management process: it does not confine itself to cyber risk but addresses risk of every kind. Inside it is the description of the main sub-processes — working in synergy — that an

organization must embrace to speak meaningfully of managing risk. It can be applied at every moment in the life of an organization: when the processes are first designed, during everyday operation, and whenever something changes in the scope or business of the organization. It applies to any activity, including decision-making at all levels.

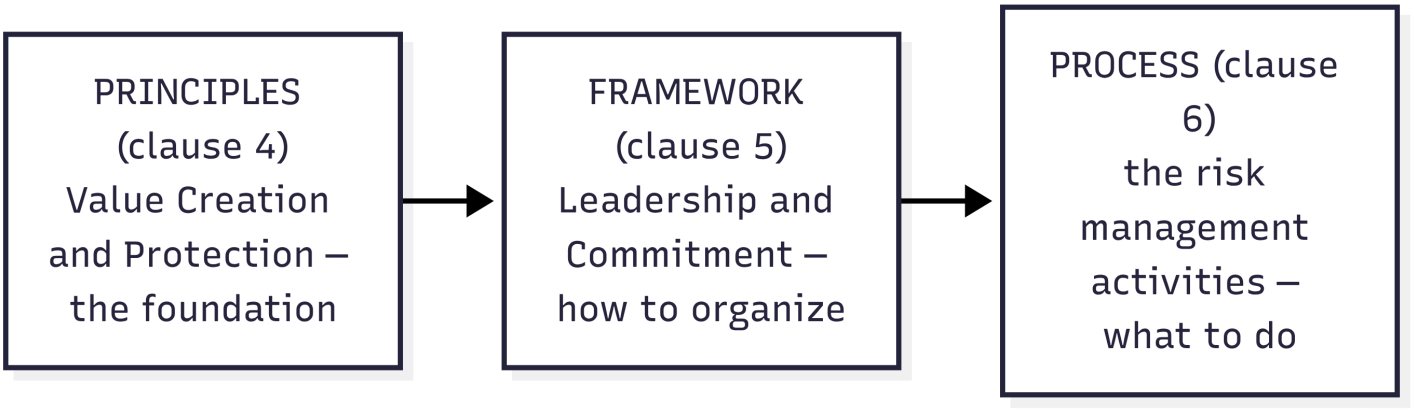
Several properties define its character.

- **It cannot be used for certification.** ISO 31000 is a *recommendation* — it states the principles and activities an organization should implement and provides guidance for internal or external audit programmes, but a company cannot be 'ISO 31000 certified.'
- **It is customizable** to any organization and context, and it is **not industry- or sector-specific**, offering a common approach to managing any type of risk.
- **It is international.** As part of the ISO family, it serves as a shared benchmark against which an organization can compare its own practices, which also makes it an instrument for understanding another company's risk *posture*.
- **Its last release is 2018.**

The standard's content is organized into three tightly related parts, each a clause and each depicted as one of three interconnected wheels.

1. **Principles (clause 4)** — the base ground, the big ideas kept in mind when identifying the activities of the process.
2. **Framework (clause 5)** — how the organization structures itself, with deadlines and responsibilities, to support the process cycle.
3. **Process (clause 6)** — the concrete sub-processes to be implemented.

The principles are the foundation over which everything is built; the framework tells us *how* to design the sub-processes; and the process is the set of activities themselves. One cannot design any sub-process without a framework telling us how, and one cannot build a sensible framework without the principles underneath.



4. ISO 31000 — Principles (Clause 4)

The principles are drawn as a wheel: one central hub surrounded by eight spokes. The hub is the overriding purpose, and the spokes are the qualities every good risk management effort

must display.

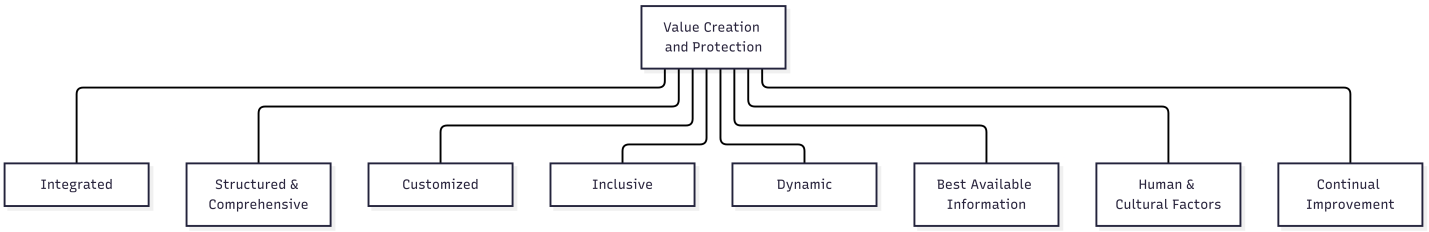
4.1 The Hub: Value Creation and Protection

At the centre sits **Value Creation and Protection**. The main objective of risk management is always to protect what represents value for the company and to increase that value as far as possible, putting in place actions that prevent damage and thereby avoid the expense of dealing with incidents. Value is not only money: indirect value such as reputation counts, and although an improved reputation is not itself revenue, revenue tends to follow it as a side effect. The task is to deploy all the instruments that strengthen the processes bringing value to the organization.

4.2 The Eight Principles

Each of the eight principles answers the question: *what must be true of the risk management effort for it to actually serve value creation and protection?*

- **Integrated** — risk management is an integral part of *all* organizational activities, working in synergy with everything the company does rather than as a bolt-on, with data collection woven into operations. Integration is what makes it non-intrusive and transparent. For example, a machine running software on a vulnerability-riddled OS invites the obvious move of installing a new OS — but doing so may break compatibility and leave the system non-functional; only a risk process integrated with knowledge of how the system actually works can foresee and weigh that trade-off, lest a 'fix' become a new incident.
- **Structured and Comprehensive** — a structured, repeatable methodology produces consistent and comparable results. Because the direct-and-control loop requires that different iterations be *comparable*, improvisation defeats it. Comprehensiveness also means not restricting attention to the technical layer alone — roles and business aspects must be included.
- **Customized** — the framework and processes are tailored and proportionate to the organization's external and internal context and objectives. One starts from the general principles and then customizes.
- **Inclusive** — involving stakeholders increases awareness and leads to better-informed risk management; all with an interest in the assessment should be brought in.
- **Dynamic** — risks and context are not static; the process must follow internal and external changes, anticipating and managing them in a timely manner.
- **Best Available Information** — an assessment is an *estimation* made to support decision-making, so decisions must rest on the best information available: timely, clear, and available to the relevant stakeholders.
- **Human and Cultural Factors** — human and cultural factors influence all aspects of risk management; the behaviours that shape how people actually carry out the work must be taken into account rather than assumed away.
- **Continual Improvement** — risk management is improved continually through learning and experience, evolving toward its best possible form.

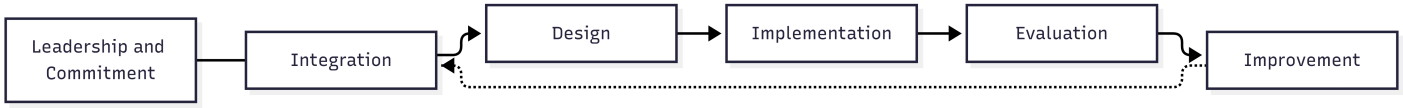


5. ISO 31000 — Framework (Clause 5)

If the principles say *why* and *what qualities*, the framework says *how the organization arranges itself* so the process can run and keep running. Its **purpose** is to help the organization integrate risk management into its significant activities and functions. Its **effectiveness** depends on how well it is integrated into the **governance** of the organization, including its decision-making — risk management that sits outside governance is ornamental, while risk management embedded in governance shapes real decisions. And as with everything in ISO 31000, the framework **must be customized** to the organization rather than taken off the shelf.

The framework is drawn as a wheel with a hub and five surrounding elements. The hub is **Leadership and Commitment**: the framework works only with explicit endorsement from top management, which assigns roles and responsibilities inside the company. Without that endorsement, the process has no authority and no owners. The five surrounding elements form a cycle around it.

- **Integration** — embedding risk management into the organization's structure and activities.
- **Design** — designing the framework appropriately for the context.
- **Implementation** — putting the designed framework into operation.
- **Evaluation** — assessing whether the framework is performing as intended.
- **Improvement** — refining the framework continually based on what evaluation reveals.



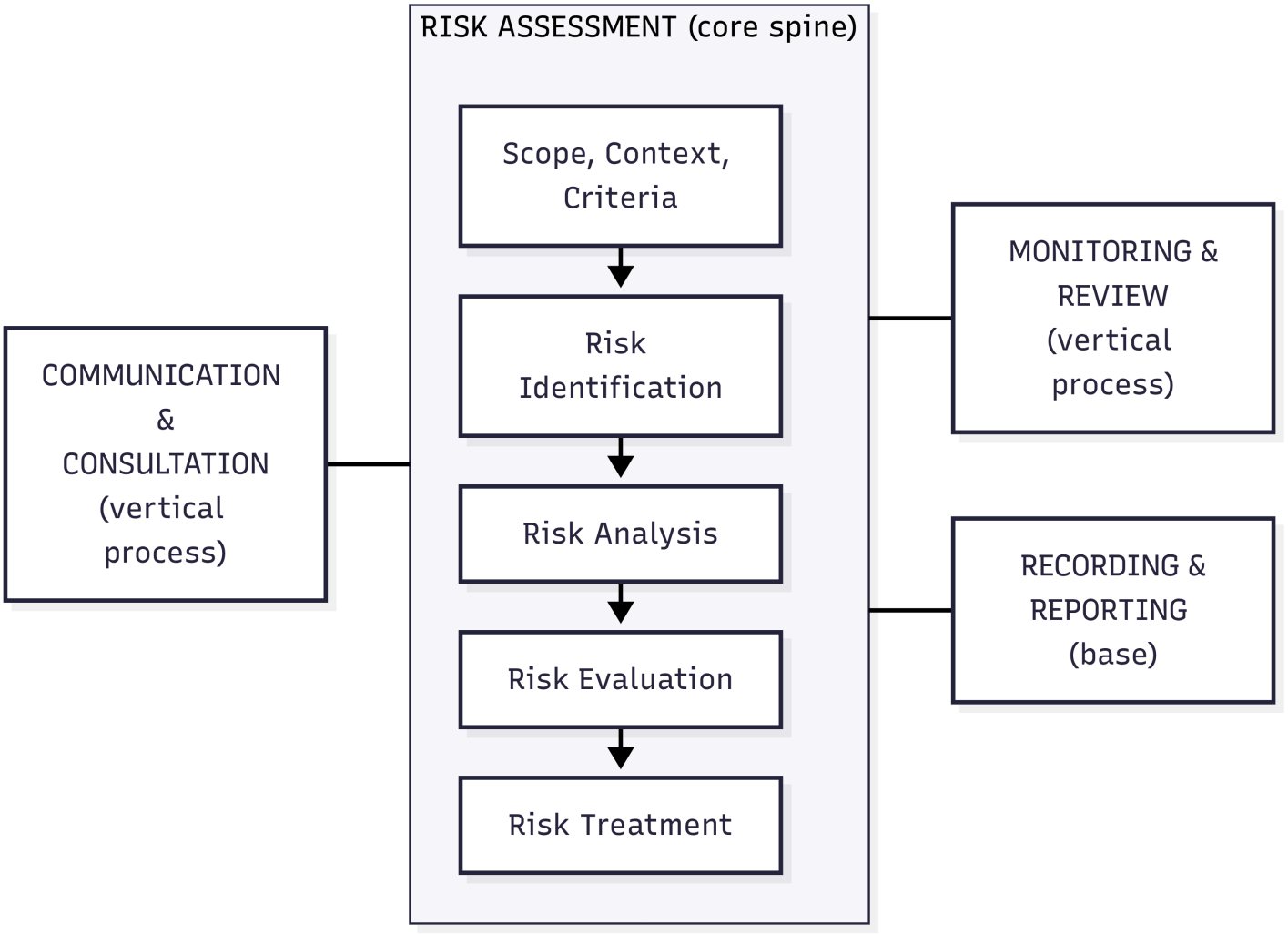
6. ISO 31000 — Process (Clause 6)

The process clause describes how to structure the concrete processes involved in risk management. It is drawn as a single circle whose internals reveal its anatomy: a vertical spine of core activities flanked on the left by **Communication and Consultation**, on the right by **Monitoring and Review**, and resting on a base band of **Recording and Reporting**. The whole picture decomposes into a small number of deliberately interconnected sub-processes — none stands alone.

At the top level, the process comprises three main sub-processes.

- **Communication and Consultation** — the activities aiming to provide, share, or obtain information and to interact with stakeholders about the management of risk; everything describing *how communication happens*, the management of all information flowing behind the assessment and every way of acquiring and transferring it between the people involved.
- **Risk Assessment** — the activities aiming to understand and document the risk picture for specific parts or aspects of a system or organization; the analytical core.
- **Monitoring and Review** — checking the current status to identify deviations from the expected one, and determining the effectiveness of the process, framework, risks, and treatments; it gathers data from the *process's own* point of view and supports continual improvement.

Communication and Monitoring are called **vertical processes** because they run alongside the entire assessment, flanking it from start to finish; the assessment itself is the central spine. Beneath everything sits **Recording and Reporting**, which documents what was done and its outcomes.



7. Communication and Consultation

This sub-process supports all the information management behind the assessment. Efficient and adequate communication and consultation support **awareness** throughout the organization, and the sub-process may concern any part or activity of the overall risk management effort. It is realized through four main activities, in sequence.

7.1 Establish a Consultative Team

The first activity is to identify everyone who contributes to the process and assemble them into a **consultative team** (or, in a small company, a single responsible person) with clearly defined responsibilities. The team supplies the knowledge that feeds the process and must therefore include **experts who know where to retrieve the relevant information**. Typically it includes internal stakeholders — decision makers, risk managers, employees with insight into the organization, board members, and customers — and it must also incorporate **stakeholders who provide information from the external domain**.

Two practical constraints matter: the team must **not be too large**, or reaching consensus becomes unworkable (up to around ten people is a reasonable ceiling), and the **roles and responsibilities** of each member must be clearly defined.

7.2 Define a Plan for Communication and Consultation

The consultative team should itself help define the plan and procedures. The point is to fix *how* communication happens rather than leaving it ad hoc — not one day by email, the next by phone, in an improvised fashion. The organization establishes procedures ensuring, for example, that different areas of expertise are brought together during assessments, that the interests of all relevant stakeholders are considered, that the risk evaluation criteria are appropriate, and that decision-making is informed. The plan must be **shared**, and its criteria checked for appropriateness to the type of assessment being performed.

7.3 Ensure Endorsement of the Risk Management Process

Good communication and consultation procedures help secure **endorsement of and support for** the risk management process. This requires the approval of top management and, just as importantly, a **common agreement and mutual understanding** of how risk should be managed across the people who must do the work.

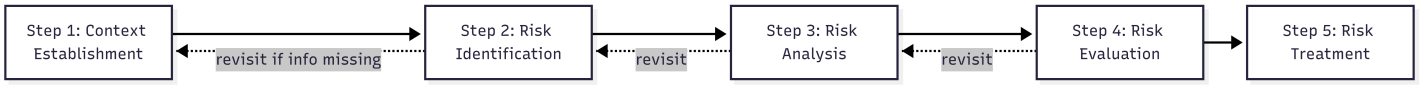
7.4 Communicate Risk Assessment Results

Once the assessment is performed, its results must be communicated — not only to the parties directly related to it but to all relevant parties, so that people can adapt their behaviour to the risks (the awareness objective). Communicating results can demonstrate **policy adherence or compliance** with directives and regulations, and it **justifies treatment plans**, including the resources required for mitigation. By letting those with a vested interest understand the basis on which decisions were made, communication helps ensure endorsement of the **risk treatment plans** from key stakeholders.

8. Risk Assessment

Risk assessment is the analytical heart of the process, and ISO 31000 divides it into **five steps**: Context Establishment, Risk Identification, Risk Analysis, Risk Evaluation, and Risk

Treatment. Typically this runs as a **linear progression**, but because the process must remain adequate, efficient, and effective, every step is carried out while holding all relevant information in view; if the information proves insufficiently accurate, or something is discovered missing from an earlier phase, the analyst **goes back and redoes** the affected step. The linearity is the default path, not a prohibition on iteration.



8.1 Step 1 — Context Establishment

The goal is to **identify and describe the context** of the risk assessment, split along two perspectives.

- **External Context** — the societal, legal, regulatory, and financial environment and the relationships with external stakeholders; all the factors that influence the organization from the outside, including regulations and any **legal requirements** to satisfy.
- **Internal Context** — the relevant goals, objectives, policies, and capabilities that determine how risk should be assessed; the specific details of what happens inside the company.

In both directions all the **constraints** must be listed, those coming from outside and those arising inside. The goals and objectives of the assessment itself must then be defined, which requires the **participation of decision makers**.

Context establishment rests on three sharp distinctions plus an explicit statement of assumptions.

- **Target of assessment** — the parts and aspects of the system that are the subject of the assessment.
- **Scope of the assessment** — its extent or range; it defines **what is held inside and what is held outside**, that is, the boundaries.
- **Focus of the assessment** — the central area of attention: the **identification and documentation of the assets** with respect to which the assessment is conducted. The focus is a *refinement within* the scope — one narrows from the broad range down to the specific assets and risk dimensions that matter. For one asset we might care only about confidentiality; for another, about accountability.
- **Assumptions** — the things taken for granted or accepted as true about the system. Stating them explicitly keeps the analysis honest and prevents hidden premises from distorting the result.

The output of this step is a **context description**, the input to the rest of the assessment, which must contain, in order:

1. identification of the **party** involved — from the party we derive the focus and the target, and from there the assets;
2. the **declaration of assumptions**;
3. identification of the **relevant assets** (the focus) — here a methodology distinguishes **primary assets from secondary ones**, which requires a **dependence analysis**;
4. the definition of **risk scales** for consequences and likelihoods — quantitative or qualitative, continuous, discrete, or given as intervals;
5. the **risk evaluation criteria**, the terms of reference by which the significance of a risk is judged — fixing them means deciding how many levels the **risk matrix** will have and where the boundaries of each column lie, the combination of the two scales that yields the final risk level.

8.2 Step 2 — Risk Identification

Risk identification is the set of activities aiming to identify, describe, and document risks and their possible causes. Two observations frame it. First, **a risk is always associated with an incident** — there is no free-floating risk; every risk is anchored to a specific unwanted event. Second, **there are three elements without which there can be no risk — Asset, Vulnerability, and Threat** — and removing any one makes risk vanish.

- **Without an asset** there is nothing to harm.
- **Without a vulnerability** there is no way to cause harm. If an asset faces a threat but holds no vulnerability, the incident cannot materialise. In the cyber domain this reasoning is treacherous, because unknown **zero-day vulnerabilities** may exist even when none are visible.
- **Without a threat** there are no causes of harm. An asset may be genuinely vulnerable, yet if nothing is trying to harm that vulnerability, no risk arises.

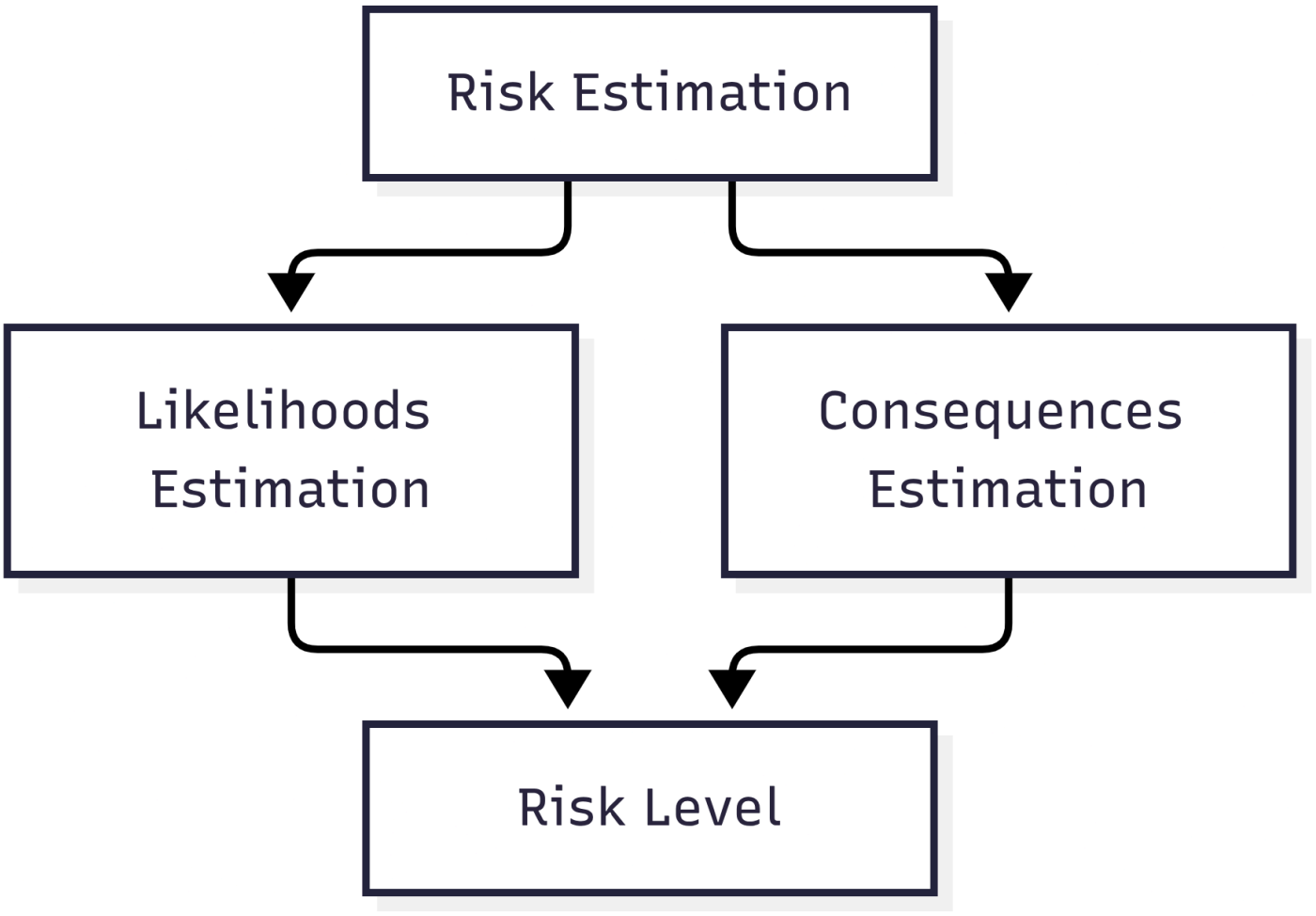
This yields a precise terminology that reads as a chain: a **threat source** is the potential cause of an incident (the attacker, the actor); a **threat** is an action or event, caused by a threat source, that may lead to an incident; a **vulnerability** is a weakness, flaw, or deficiency that can be **exploited** by a threat to harm an asset; and when a threat exploits a vulnerability, it **leads** to an incident, the incident on a valued asset being the **risk**.



The scope of identification is to work *with respect to the identified assets* by identifying the **threats** and understanding how those threats may lead to incidents — and thereby to risks — by **exploiting vulnerabilities**. Operationally: find all the possible open doors (vulnerabilities), then all the potential actors interested in entering (threat sources and their threats), then draw the links showing how an attacker could materialise a threat by exploiting a vulnerability. A single vulnerability may translate into **several different threats**, and every such element must be analysed, because all of them feed the later estimation of likelihood.

8.3 Step 3 — Risk Analysis

Risk analysis estimates and determines the level of the identified risks, its governing observation being that **the risk level is derived from the combination of likelihood and consequence**. The composite activity, **Risk Estimation**, is the sum of two estimations — Likelihoods Estimation plus Consequences Estimation.



Likelihoods Estimation determines the frequency or probability of incidents occurring, using the likelihood scale defined during context establishment. It requires techniques for gathering empirical data, and many risk-modelling techniques — **Bayesian networks, attack trees, and CORAS diagrams** — support both the estimation and its documentation. The aim is to understand how risks are most likely to arise and which threat sources matter most: one estimates the likelihood that threat sources initiate threats, then the likelihood that those threats lead to incidents. The natural approach formalises everything as a probability; the difficulty is that the factors are many and often hard to quantify.

Consequences Estimation rests on a key recollection: an incident represents **one risk for each of the assets it harms**, so the consequence must be estimated separately for *each* affected asset, measured on the appropriate scale. The estimation should be performed by a **walk-through of all identified incidents**, assigning estimates with the involvement of personnel who represent the party, or someone who can judge consequences on its behalf. Bridging all the layers is essential — the infrastructure, the people working, and what happens

if a specific part stops functioning. In complex systems a failure at one point can cascade into failures elsewhere, producing a **domino effect** that the analysis must try to capture.

8.4 Step 4 — Risk Evaluation

Risk evaluation compares the risk analysis results against the risk evaluation criteria to determine which risks should be considered for treatment. Given the estimates and criteria the step is largely straightforward — it is where risks are placed into the **risk matrix** and prioritised — but four sub-steps structure it.

1. **Consolidation of risk analysis results** — look at each risk individually, revisit its analysis, and plug it into the risk matrix, whose two dimensions (likelihood and consequence) are crossed to find the correct cell. Special attention goes to the estimates we are **uncertain** about, where that uncertainty casts doubt on the actual risk level.
2. **Evaluation of risk level** — for each risk, judge whether the level, and the error we can tolerate in it, is acceptable. This is the moment to decide whether to **go back to the analysis phase** to revisit a shaky estimate.
3. **Risk aggregation** — investigate whether certain sets of risks should be aggregated and evaluated as one, to **avoid accepting a set of risks that are individually non-critical yet unacceptable in combination**. Two natural aggregation points: several incidents that harm the **same asset**, and distinct incidents that share the **same threat source**.
4. **Risk grouping** — group risks that share elements, most usefully a **common mitigation**, so they can be treated together.

The output is a prioritised list of risks handed to the treatment step.

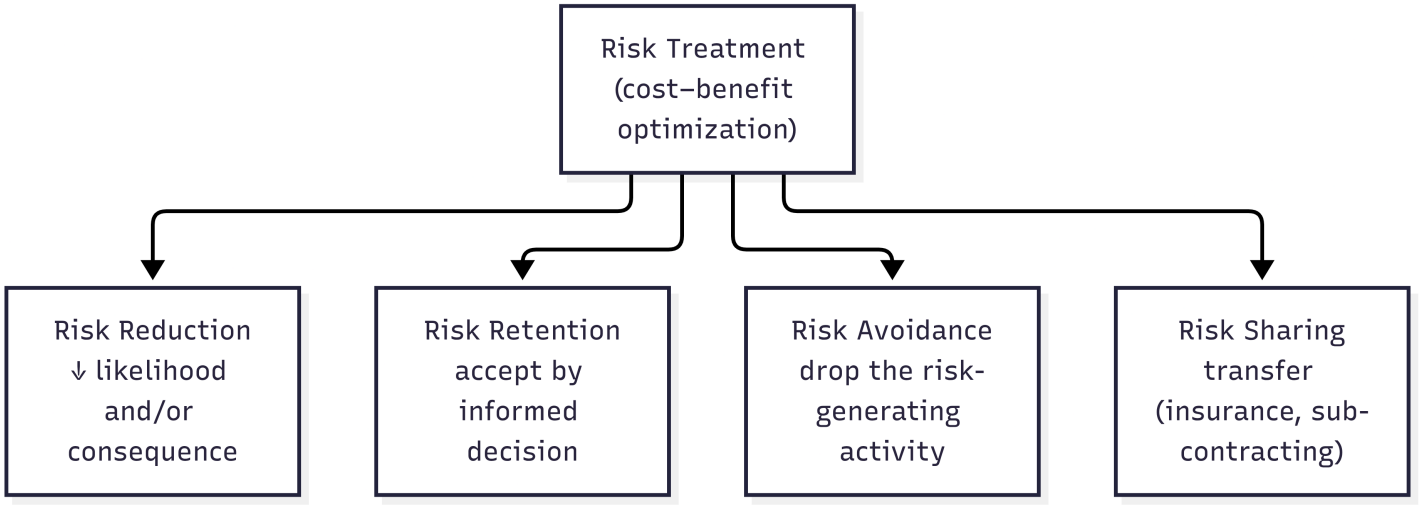
8.5 Step 5 — Risk Treatment

Risk treatment identifies and selects the means for risk mitigation and reduction. In principle one seeks to treat every **unacceptable** risk, but in the end this is always a question of **cost and benefit**, regardless of the nominal level. If a low, acceptable risk is very cheap to eliminate, we might eliminate it anyway; if the cost of treating a very high risk is unbearable, there may be no option but to accept it. In practice we rarely have a fixing strategy for every risk, because of constraints on time and budget, so treatment is in effect an **optimization problem**: reduce the overall level of risk as much as possible given the resources available, working on the risks aggregated by group and finding a suitable set of mitigation actions for each.

Because treatment can have side effects, the activity must involve both the **identification and the analysis** of treatments: some treatments **create new risks**, and some groups of treatments **reduce each other's isolated effect**, so their interactions must be weighed rather than each treatment judged alone. There are four main options.

- **Risk Reduction** — put in place actions that reduce the **likelihood and/or the consequence** of incidents.

- **Risk Retention** — accept the risk by an informed decision.
- **Risk Avoidance** — avoid the activity that gives rise to the risk altogether; sometimes the only option for an unacceptable risk.
- **Risk Sharing** — transfer the risk, or part of it, to another party, for example through insurance or sub-contracting.



9. Monitoring and Review

Monitoring is the continual checking, supervising, and observing of the current status to identify deviations from the expected or required one. **Review** is the activity of determining the suitability, adequacy, and effectiveness of the risk management process and framework, as well as of the risks and their treatments. Together they apply at four levels: the **risk management framework**, the **risk management process**, the **identified risks**, and the **measures** implemented to treat those risks.

The main purposes of Monitoring and Review are to ensure that controls remain effective and efficient; to obtain further information that improves the risk assessment; to analyse and learn lessons from incidents, changes, trends, successes, and failures; to detect changes; and to identify emerging risks.

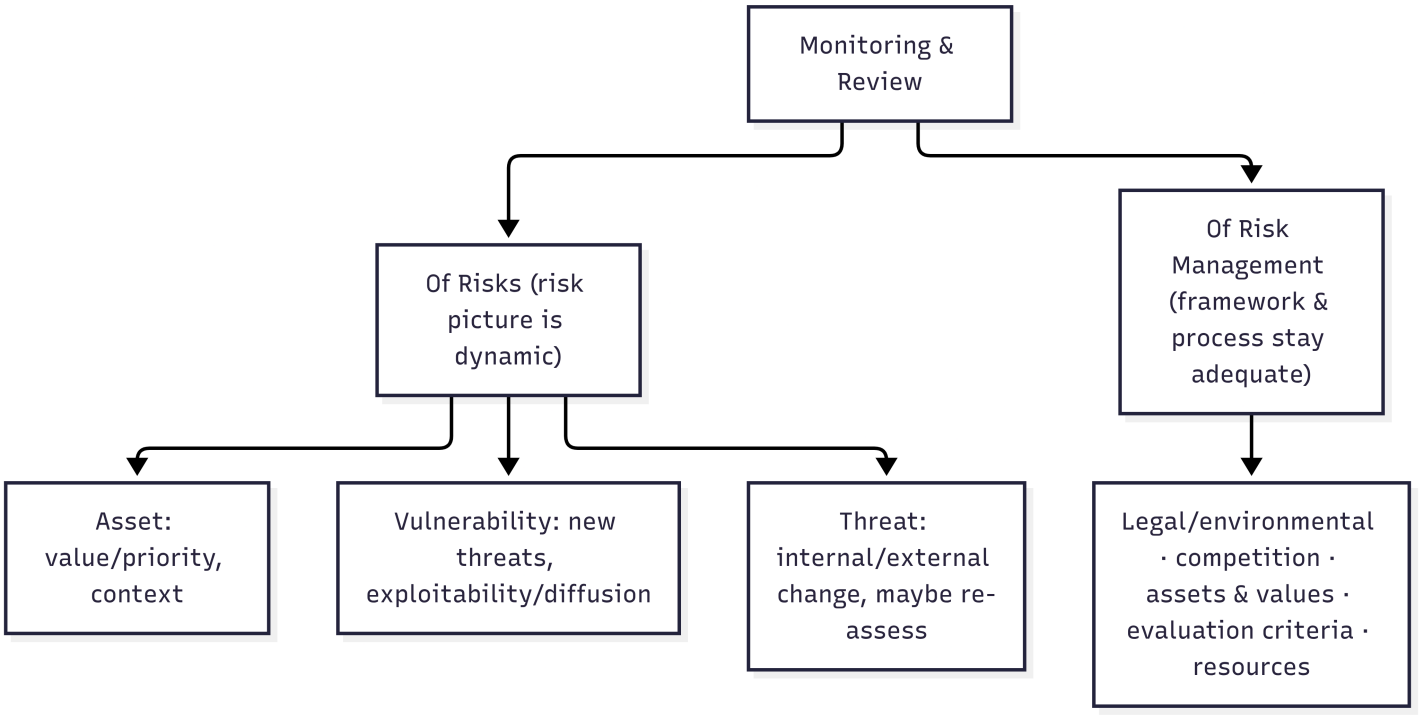
9.1 Monitoring and Review of Risks

Risks are **not static** and must be monitored and reviewed, this monitoring serving as the basis for taking action — modifying the risk picture or conducting new assessments. The monitoring maps directly onto the three irreducible elements of risk.

- **Asset** — check for changes in asset **value or priority over time**, and in the internal or external context.
- **Vulnerability** — check for vulnerabilities that could be exposed to **new threats**, and for changes in vulnerability **parameters** such as exploitability and diffusion.
- **Threat** — check for internal or external changes; sometimes new threats can be observed directly, while sometimes a **new risk assessment** is needed to surface them.

9.2 Monitoring and Review of Risk Management

Beyond the risks themselves, the **risk management apparatus** must be monitored, to ensure that the framework, the process, and all related activities, procedures, roles, and responsibilities remain **relevant, appropriate, and adequate**. What should be monitored are the changes in internal or external context that may affect that adequacy, including the **legal and environmental context**, the **competition context**, the **assets and their values**, the **risk evaluation criteria**, and the **resources required** for adhering to the risk management framework.



Chapter 3 bis: Risk Management pt. 2

The previous chapter built the general theory of risk management on a deliberately neutral footing: the definitions of asset, incident, likelihood, and consequence, and the ISO 31000 machinery of principles, framework, and process, apply to *any* kind of risk. Financial, operational, and physical risk all share the same skeleton — a risk is always associated with an incident, which is associated with an asset, and to evaluate its probability and consequence we must understand the relation between the asset, the vulnerability, and the threat that materialises the incident. Nothing in that structure is specific to computers. This chapter asks what changes when the risk lives in the **cyber domain**, and the answer is not that the theory changes but that each step becomes harder, larger, and more adversarial — hard enough to demand specialised procedures, catalogues, and scoring systems of its own.

1. Cyber-Systems and the Cyber Domain

To speak precisely about cyber-risk, three nested definitions are needed.

A **cyberspace** is a collection of interconnected computerized networks — services, computer systems, embedded processors, and controllers — together with the information held in storage

or in transit across them. Informally, it is everything an organization puts into operation using digital information to support its business processes: the networks, software, and hardware that connect everything, and crucially the information that flows over them. The emphasis on *information* rather than mere data is deliberate: a cyberspace carries not only raw data but the information that can be *inferred* by correlating that data.

A **cyber-system** is a system that makes use of a cyberspace — anything digital that requires data, allows information to be transferred, and at its lowest level rests on the network, software, and hardware tying the pieces together.

A **cyber-physical system** is a particular, important class of cyber-system: one that **controls and responds to physical entities through actuators and sensors**. These systems bridge the digital and the physical world, with a digital component (the network and control infrastructure) and a genuine physical component they monitor and act upon. The canonical examples are the critical infrastructures that control physical services: a **water distribution network** or an **energy distribution network** (the electrical grid), controlled remotely through monitors and actuators that modify the service in the physical world. A failure or attack here does not merely corrupt data; it can stop water or electricity from flowing.

2. Cyber-Security and Cyber-Threats

Cybersecurity is the protection of cyber-systems against **cyber-threats**. The definition narrows the universe of adversaries: a cyber-threat is a threat that **exploits a cyberspace**. We are therefore restricting attention to those who inject problems *through* the cyberspace, rather than to threats of every conceivable origin.

Within this restricted universe, cyber-threats split into two classes, and the distinction organizes the rest of the chapter.

- **Malicious** threats are caused by **intention** — raised by an attacker who deliberately wants to damage an infrastructure, as in denial-of-service (DoS) and injection attacks. A crucial observation sets the cyber domain apart: in a normal, non-cyber scenario the number of malicious actors is small relative to benign ones, whereas in cyberspace malicious actors are abundant and may sit anywhere in the world.
- **Non-malicious** threats are **accidental** — things that happen without intent, through mistakes, configuration errors, programming defects, or environmental factors, such as a crash due to a programming error or the loss of an Internet connection through wear and tear of hardware.

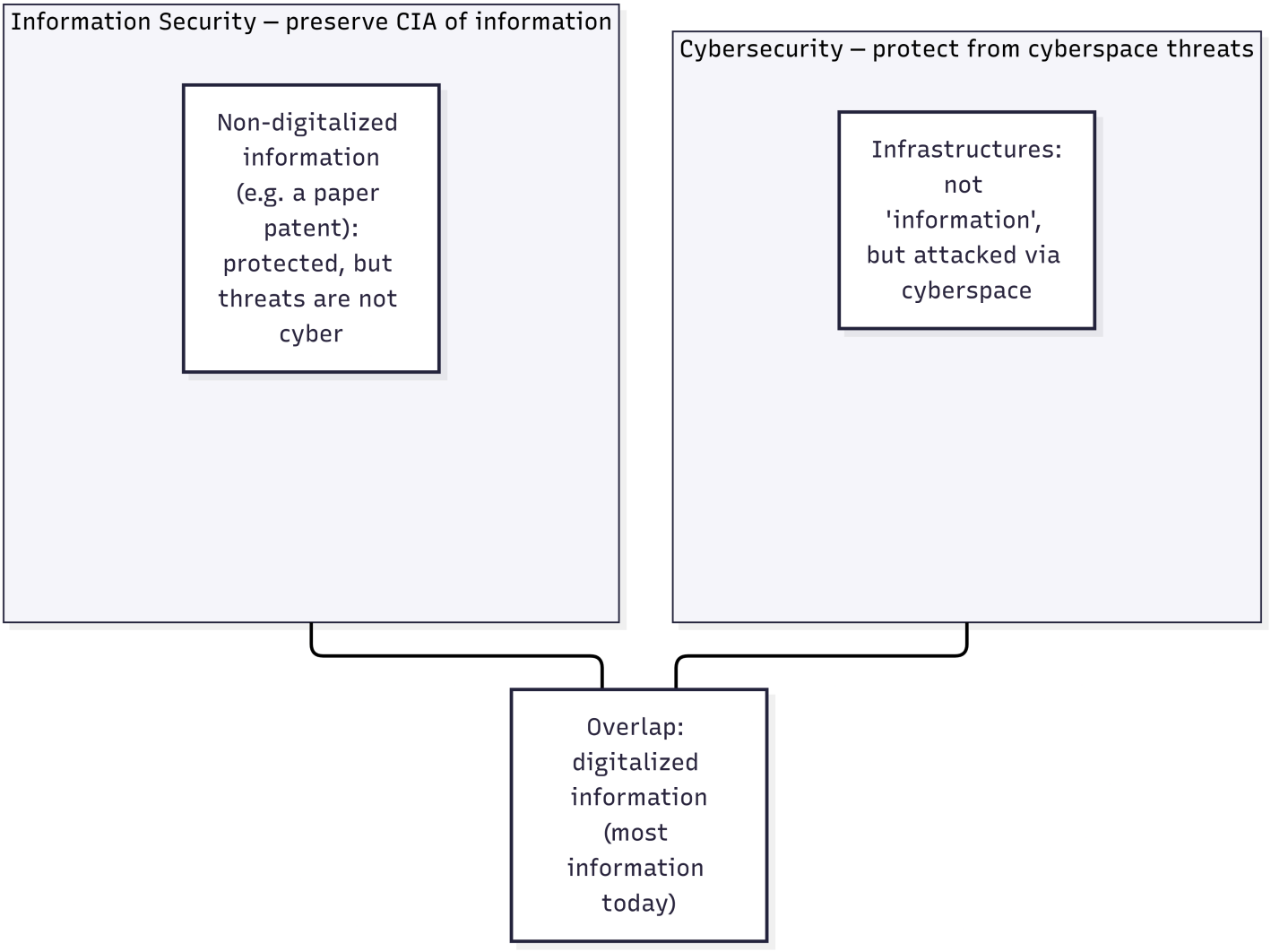
2.1 Cybersecurity versus Information Security

Cybersecurity and **information security** are closely related but not identical, and the relationship is best seen as two overlapping circles with a large intersection. Information security is the preservation of the **CIA properties** of information — Confidentiality, Integrity, and Availability — *regardless of the form* that information takes. Cybersecurity concerns

protection from threats that use a cyberspace, *regardless of whether* the protected thing is 'information' in the classical sense.

The overlap is large because today most information is digitalized and therefore lives in a cyberspace. But neither circle contains the other.

- There exists **information that must be protected but whose threats do not come from the cyberspace** — a patent or trade secret that is not digitalized falls under information security but outside cybersecurity.
- Conversely, **infrastructures** are an asset that is not 'information' yet must be protected from cyberspace threats — falling under cybersecurity but outside classical information security.



2.2 Cyber-Risk

A **cyber-risk** is a risk caused by a cyber-threat — one that materialises in the cyberspace, with assets living in the cyberspace. Mirroring the threat taxonomy, cyber-risks are **malicious** (behind them a malicious cyber-threat) or **non-malicious** (behind them an accidental cyber-threat).

The decisive subtlety is that the **same risk may sit in the intersection**, reachable by both a malicious and a non-malicious threat. Consider a loss of confidentiality of sensitive data: the identical incident — unauthorized disclosure — can be caused by a **hacker** who breaks in to

steal the data (malicious) or by the **accidental posting** of the data on an open website, or an employee leaving a laptop open for anyone to see (non-malicious). Because one incident can have roots on both sides, analysing cyber-risk means examining problems from two perspectives at once, which makes the **aggregation and grouping** of risks especially important in the cyber setting, since a single risk may legitimately belong to different categories.

2.3 Why Cyber-Risk Assessment Is Different

Two structural features make cyber-risk assessment harder than generic risk assessment.

1. The **potentially far-reaching extent of a cyberspace** means the origins of threats are widespread, possibly **global** — an attacker may act from anywhere on the planet.
2. The **number of potential threat sources and threats**, both malicious and non-malicious, is **very large**.

The practical response to this explosion of possibilities is the same throughout the chapter: **define procedures and techniques that provide guidance and direction**, and lean heavily on shared, public knowledge rather than trying to enumerate every threat unaided.

3. Communication and Consultation of Cyber-Risk

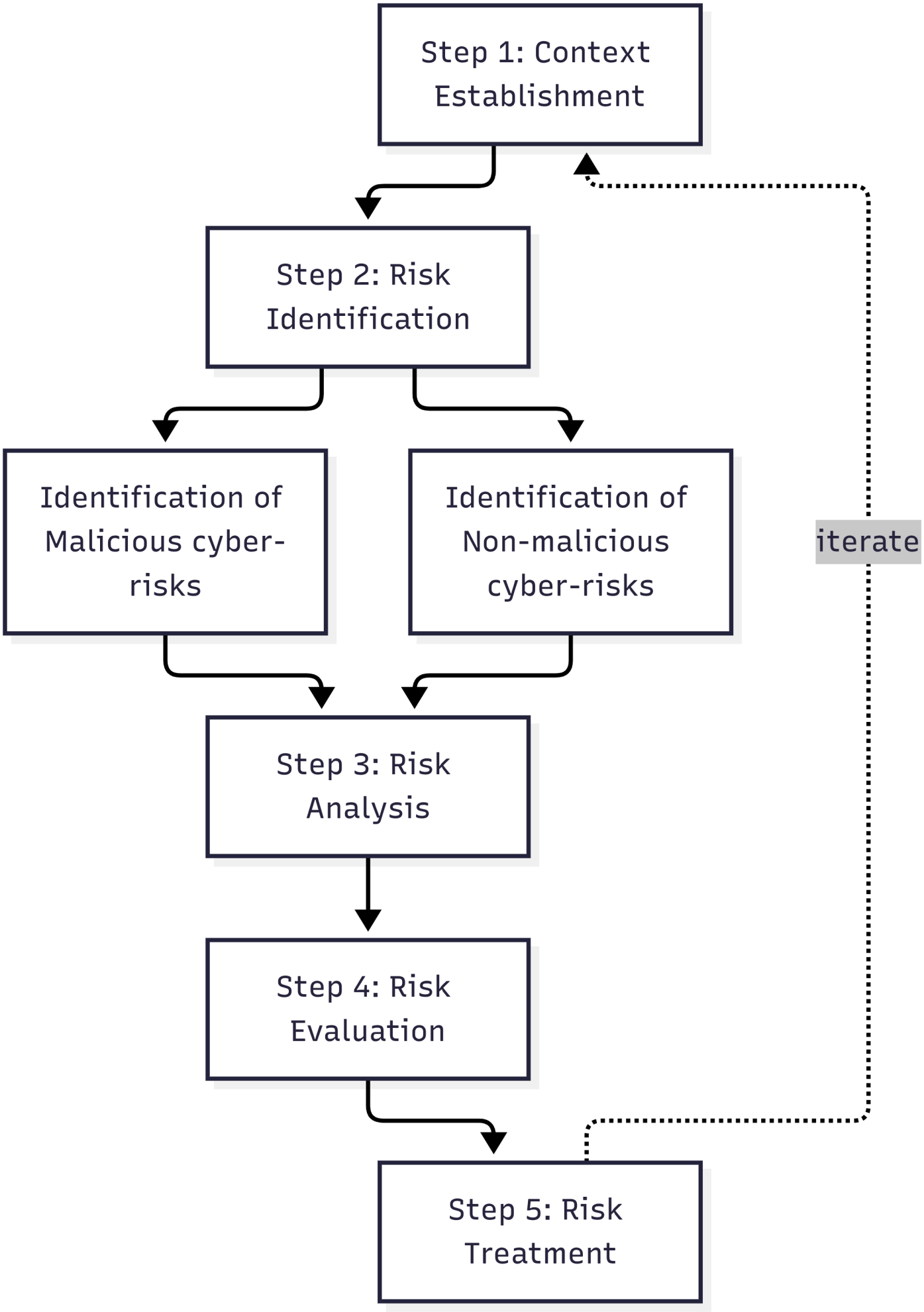
The general process is unchanged in form — communication and consultation still defines how information is gathered, managed, and distributed, built around a consultative team (including management) that fixes the procedures for feeding the outcome back into decision-making and awareness. But two additional challenges appear when the system is a cyber-system.

1. Cyber-systems may have **stakeholders everywhere** — in a networked service, essentially any user could be a stakeholder, which makes stakeholders hard to identify and bound.
2. There may be **adversaries everywhere**, with specific and varied intentions.

The way out of both difficulties is to **leverage common knowledge** — to push hard for **information sharing**, both consuming and contributing public information about cyber-threats, vulnerabilities, incidents, confirmed adversary profiles, and current mitigation strategies. Dedicated repositories of up-to-date information exist precisely for this. The obstacle is that everyone shares information in their own way, so the organization must still **define procedures and techniques** that turn this flood of heterogeneous public knowledge into usable guidance.

4. Cyber-Risk Assessment: the Five Steps Revisited

The five-step structure carries over exactly — Context Establishment, Risk Identification, Risk Analysis, Risk Evaluation, Risk Treatment — and it remains iterative. The first visible divergence appears in **Step 2, Risk Identification**, which **splits into two parallel sub-activities**: the identification of *malicious* cyber-risks and of *non-malicious* cyber-risks. These feed back together before the analysis in Step 3.



Before the split matters, recall the attacker's economics. Every company has a **mission** — it provides services to earn money. From the attacker's point of view, the target is **most often**

not the business itself but the data; the attacker cares only about the ways through which it can interfere with the business or extract value from it. This reframing guides where the defender should look.

4.1 Step 1 — Context Establishment for Cyber-Risk

Context establishment retains its purpose: the preliminary documentation of internal and external context, assets, and constraints. In the cyber setting its hardest and most dangerous part is **understanding the boundary of the context**, which in the digital world is anything but obvious — a networked system has no clean physical perimeter.

The device that makes the boundary tractable is the **attack surface**: all the points where an attacker or other threat source could get *into* the cyber-system, and where information or data could get *out*. Because we cannot be precise about where the system ends, we confine the infrastructure by identifying its **entry points** — the attack surface — and then reason about every asset in terms of **its placement with respect to that attack surface**.

Typical assets of concern fall into two families.

- **Information** — for example a specific set of data. A hospital managing patient records holds sensitive data that must be protected from loss of confidentiality and integrity.
- **Information infrastructures** — the 'more material' elements, including **software, services, and networks** that make up the critical infrastructure.

4.2 Step 2a — Identification of Malicious Cyber-Risk

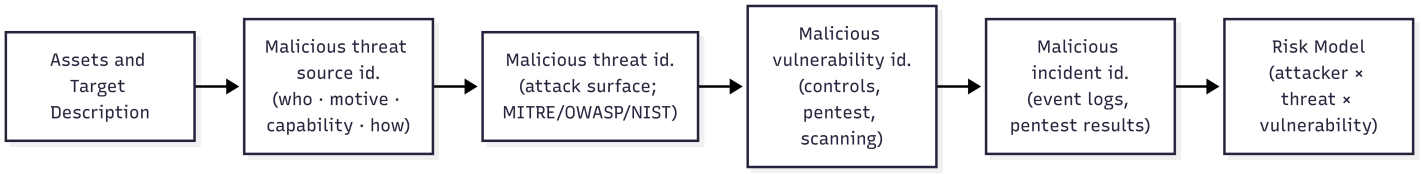
Here the analysis takes the shape of a **game of strategy**. On one side sits a **rational attacker** with a strategy and an objective; on the other sits the **defender** — the company, characterised by a target and a set of assets — who must guess what the attacker intends in order to defend. Standing beside the defender is the **risk assessor**, whose job is to anticipate how the attacker wants to play and to suggest the defender's next move. Each time the attacker makes a step, the assessor reassesses the situation, weighs the risk of every action the defender could take, and plans the response.

The governing observation is that what we can expect from the adversary depends on four things: the adversary's **(i) motives, (ii) abilities, (iii) helpers, and (iv) resources**. To make this tractable, malicious identification proceeds through four stages, flowing from an initial *Assets and Target Description* and producing a **Risk Model** that expresses the risk in terms of the category of attacker, threat, and vulnerability.

- **Malicious threat source identification** — ask who could realistically want to attack, narrowing to the sources with a stronger interest in this particular company. Four questions structure it: **who** may initiate attacks (a human, software, an automated agent); **what motivates them** (often the hardest to determine, from a competitor seeking to destroy our reputation to ideological activists); **what their capabilities and intentions are** (an adversary able only to launch a small DoS with an off-the-shelf tool like Metasploit is a

different proposition from a well-resourced terrorist group); and **how the attack can be launched** (some attacks are easy to detect, others far subtler, involving social engineering). A warning in red: **threats and threat sources are not just external** — the attacker usually arrives from outside through the attack surface, but may equally come from inside, a disgruntled employee already within the cyber-system.

- **Malicious threat identification** — reason about how those sources could become actual threats, paying attention to the **interface to the cyberspace** and the documented **attack surface**, and using the target-of-assessment description to investigate *where and how* attacks can be launched. Here public knowledge becomes indispensable: catalogues and repositories from **MITRE, OWASP, and NIST** are used to avoid reinventing the threat landscape.
- **Malicious vulnerability identification** — search for the vulnerabilities that could let threats become incidents, focusing on the attack surface, across technical and non-technical layers. Technically, **investigate existing controls and defence mechanisms** for strength and adequacy, and run **security testing — penetration testing and vulnerability scanning** — to check whether and how easily a threat source could attack, gauge the severity of known vulnerabilities, hunt for potential ones, and foresee the incidents they might lead to. Beyond the technical infrastructure (addressed with scanning tools and SIEM), examine **non-technical weaknesses** such as untrained new employees and vulnerabilities in the **processes** themselves.
- **Malicious incident identification** — investigate how the malicious threats can actually harm the identified assets given the identified vulnerabilities, using **event logs about previous incidents** and reasoning about the incidents the threats and vulnerabilities may produce, informed by the penetration-testing results and **always focusing on what is more probable in our own environment**.



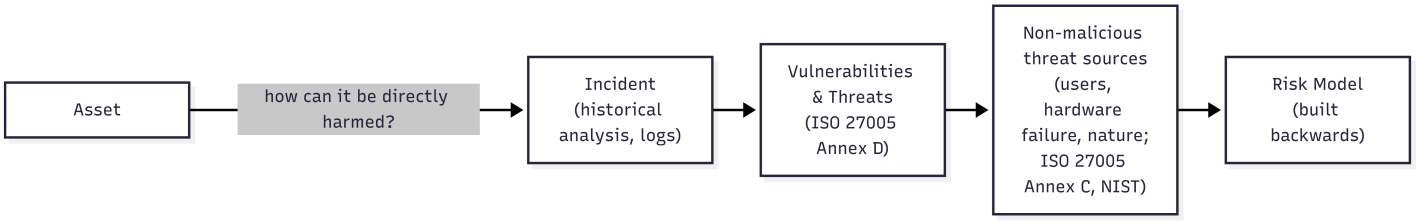
4.3 Step 2b — Identification of Non-Malicious Cyber-Risk

For non-malicious risk the attacker-first method collapses, because there is **no intent or motive** to impersonate and therefore no adversary to start from. There is still a threat source — a hardware failure, an act of nature, a careless user — but we cannot begin by hunting for it. Instead the analysis **runs backwards**: we start from the valuables to be defended and the incidents, and reconstruct the risk model in reverse.

1. **Answer "In which way may the considered asset be directly harmed?"** Each distinct possibility corresponds to an **incident**. Non-malicious incidents are typically fewer and more easily standardised than malicious ones, and usually **recurring and known**, so incident identification leans on **historical analysis** — logs, monitored data, and historical

records. For example, a loss of data caused not by an attacker but by a **blackout** affecting the datacenter: the effect is the same (the data is gone) but the source is accidental, and such incidents are listed by looking at the history.

2. **Identify the vulnerabilities and threats that may cause these incidents**, focusing only on the parts of the target relevant to those incidents. Continuing the example: the machine storing the data was **not placed behind a continuity (UPS) server**. Public knowledge supplies candidate vulnerabilities — notably **ISO 27005**, whose informative **Annex D** lists examples of vulnerabilities organised by type (Hardware, Software, and so on), each with the example threats that might exploit them.
3. **Identify the non-malicious threat sources that can cause the threats** — ask who the users are and how they can cause unintended events, deliberately including **non-human threat sources** (hardware failure, wear and tear, acts of nature). Here too **ISO 27005 (Annex C, typical threats)** and the **NIST risk assessment guide** provide categorized lists. Annex C classifies each threat by origin with the codes **D (deliberate), A (accidental), E (environmental)** — *Fire* is marked A, D, E (arson, accident, or natural), while *Seismic phenomenon* is purely E — exactly the information needed to separate the non-malicious (A, E) contributions from the malicious (D) ones.



At the end of Step 2 the assessor holds **two risk models** — one malicious, one non-malicious — ready to carry into the analysis.

5. Analysis of Cyber-Risk

With two risk models in hand, the third step — estimating likelihood and consequence to derive the risk level — proceeds much as in the general case, but two aspects distinguish cyber-risk analysis, one a difficulty and one an advantage.

1. **Likelihood is hard to estimate for malicious threats.** Where there is **human intent and motive**, estimating the probability of occurrence is genuinely difficult: the information is incomplete and, decisively, it **changes very quickly** — an attacker's capabilities, the availability of exploit code, and the fashion for particular attacks all shift on short timescales, so a likelihood estimated today may be stale tomorrow.
2. **Cyber-systems offer rich instrumentation.** The digital nature of the system gives many options for **logging, monitoring, and testing**, and this empirical data can be fed directly into the analysis to ground estimates that would otherwise be guesswork.

The technique that reconciles the two is **threat modelling**. Rather than asserting a bare probability, we build **threat models and attack models** describing the relevant factors — attack prerequisites, the attacker skill or knowledge required, the resources required, the

attacker's motive, the attack opportunity, and so forth. Similar descriptive models are built for **vulnerabilities**, capturing attributes such as **ease of discovery** and **ease of exploit**. We then attach the likelihood of exploiting individual vulnerabilities (and groups of them), together with **temporal information** about how those likelihoods evolve. In combination, this structured information lets us **derive the likelihoods** of threats and incidents defensibly, instead of pulling a number from the air. Consequence estimation is unchanged: an incident still represents one risk per harmed asset, estimated on the appropriate scale.

6. Evaluation of Cyber-Risk

Evaluation keeps its four sub-steps, each carrying a cyber-specific nuance, and the unifying theme is that **malicious and non-malicious risks are kept as two separate entities** for as long as it is useful to do so.

1. **Consolidation of risk analysis results** — focus on the cyber-risks whose estimates are **uncertain**. The cyber addition is the explicit need to **estimate the level of uncertainty** in both the likelihood and the impact estimates; if it is too great, go back and acquire additional data before trusting the result. From the outset the consolidation **distinguishes malicious from non-malicious** risks.
2. **Evaluation of risk level** — for convenience it is possible and recommended to **evaluate malicious and non-malicious cyber-risks separately**, maintaining **two distinct risk matrices**, one per class, **merged only later**.
3. **Risk aggregation** — exactly as in the general case: look for sets of risks to treat as one, where several incidents harm the **same asset** or share the **same threat source**.
4. **Risk grouping** — here the malicious/non-malicious separation is deliberately **undone**: because grouping looks ahead to *treatment*, we **merge malicious and non-malicious risks that share a common mitigation**, grouping them to improve the selection of treatments.

The two classes are held apart through consolidation and level-evaluation, where their differing natures matter for estimating likelihood and uncertainty, and brought back together at grouping, where what matters is the shared remedy.

7. Treatment of Cyber-Risk

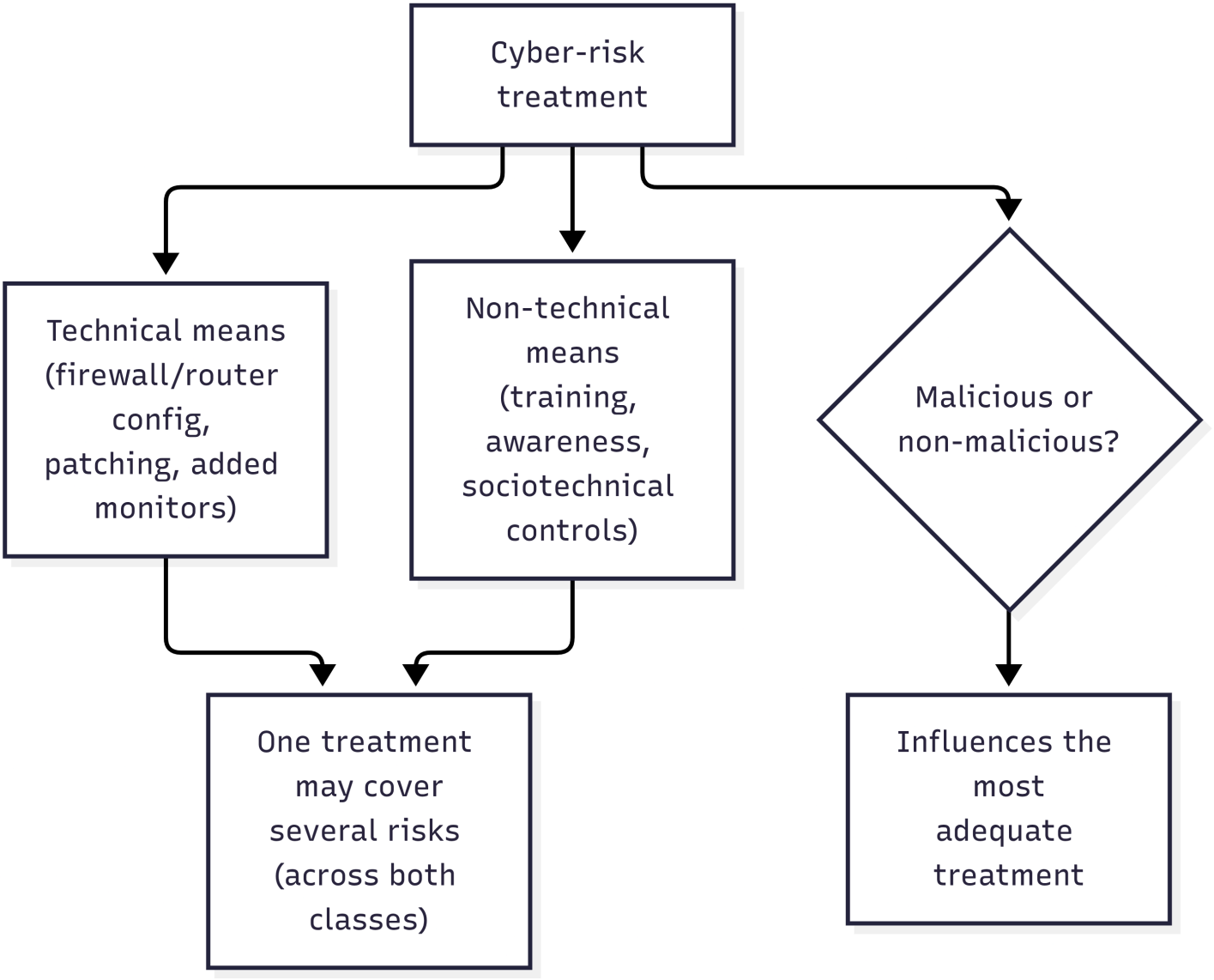
Two features distinguish the treatment of cyber-risk from the general case.

The treatment options are themselves highly technical. Because cyber-systems are technical, the natural levers are technical too: acting directly on the infrastructure, for example by changing the configuration of a **firewall** or updating the **policies of routers**. But the technical lever is not always available or sufficient, and the **sociotechnical aspects and human involvement** must also be considered. The archetypal case is a server that **cannot be upgraded because of compatibility constraints**: unable to patch it, we turn to **non-technical** treatment — training the people who interact with the machine, while **in parallel** adding compensating controls such as additional monitors around the vulnerable component.

Treatment therefore ranges across the full four options (reduction, retention, avoidance, sharing), realised through a technical or non-technical means as the situation allows.

The malicious/non-malicious distinction has implications for the most adequate treatment. The class of a risk influences which treatment fits best. At the same time — and this is why grouping merged the two classes — **a single treatment may benefit more than one risk across both categories**, which is precisely the efficiency that grouping is meant to capture.

For **Monitoring and Review** there is no significant difference from the general case: risks remain dynamic and must be watched for changes in assets, vulnerabilities, and threats, and the framework and process must be checked for continuing adequacy.



8. Supporting Repositories and Catalogues

The recurring prescription of the whole chapter — lean on shared public knowledge — is realised through a family of community-maintained catalogues. Each occupies a distinct position on the chain **threat source → threat → vulnerability/weakness → incident**, and knowing which catalogue answers which question is itself a useful skill.

8.1 MITRE CAPEC — attack patterns

CAPEC (Common Attack Pattern Enumeration and Classification) is a publicly available catalogue of **attack patterns**, with a comprehensive schema and classification taxonomy, created to assist in building secure software. Its premise is that understanding how the adversary operates is essential to defence, so it provides a dictionary of known attack patterns used to exploit known weaknesses in cyber-enabled capabilities. An **attack pattern** describes a common method for exploiting software **from the attacker's perspective**, with guidance on how to mitigate it. CAPEC can be browsed **by Mechanisms of Attack** or **by Domains of Attack**, and it is the natural reference for the **malicious threat identification** stage.

A representative entry, *HTTP Response Splitting*, shows the richness of each pattern.

Field	Value
Typical severity	High
Description	A vulnerable web server is coaxed into emitting a response stream interpreted as two separate responses; exploitable when user-controlled input is used unvalidated in response headers
Attack prerequisites	User-controlled input in an HTTP header, ability to inject custom strings, insufficient input validation
Typical likelihood of exploit	Medium
Methods of attack	Injection, Protocol Manipulation
Attacker skill required	High (solid understanding of HTTP)
Also records	Examples/instances (with CVE reference), resources required, probing techniques, indicators/warnings, solutions and mitigations, attack motivation-consequences
CIA impact	Confidentiality High, Integrity High, Availability Low
Related weaknesses	CWE identifiers — the bridge to the underlying software weaknesses

8.2 MITRE CWE — software weaknesses

CWE (Common Weakness Enumeration) is a community-developed list of common software security weaknesses. It serves three roles: a **common language** for discussing weaknesses, a **measuring stick** for software security tools, and a **baseline** for weakness identification, mitigation, and prevention. Its list can be viewed by **Research**, **Development**, or **Architectural** concepts.

Where CAPEC describes *how an attack is carried out*, CWE describes *the underlying flaw that makes the attack possible* — the two are explicitly linked, as the CAPEC entry's 'related weaknesses' field shows. A CWE entry (for example **CWE-415, Double Free**) includes the

name and description of the weakness type, alternate terms, descriptions of the **behavior** and the **exploit**, the **likelihood of exploit**, the **consequences**, **potential mitigations**, demonstrative examples, **observed examples** (real CVEs exhibiting the weakness), **node relationship** information placing it in the CWE hierarchy, source taxonomies, and code samples for the relevant languages and architectures.

8.3 CVE and the NVD — disclosed vulnerabilities

Weaknesses (CWE) are generic *types* of flaw; a **vulnerability** is a concrete *instance* of such a flaw in a specific product. Vulnerabilities may affect a system at many levels — **Hardware, Software, Network, Personnel, Organizational**. The MITRE Corporation maintains a list of disclosed vulnerabilities in a system called **CVE** (Common Vulnerabilities and Exposures), in which each vulnerability is scored using the **CVSS**. NIST collects these scored vulnerabilities and makes them available through the **National Vulnerability Database (NVD)**.

A CVE/NVD entry (for example **CVE-2003-0062**) records an original release date and last-revised date, the source, an **overview** of the flaw (a buffer overflow allowing local users to execute arbitrary code via a long path name), and the **impact** expressed as a **CVSS severity** — a base score with a qualitative rating (e.g. 7.2, HIGH), a **vector string** encoding the metric choices, impact and exploitability subscores, and metrics such as **access vector, access complexity, and authentication**.

8.4 CVSS — scoring a vulnerability's severity

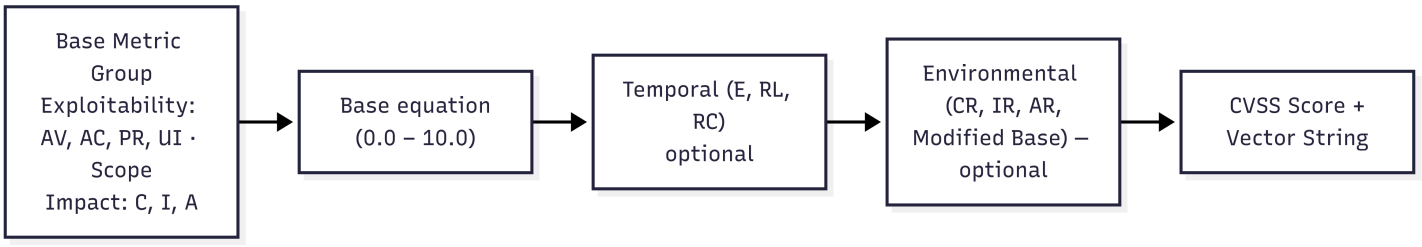
The **Common Vulnerability Scoring System (CVSS)** captures the principal **technical characteristics** of software, hardware, and firmware vulnerabilities and outputs a **numerical score** indicating a vulnerability's severity *relative to others*. Its value to the risk assessor is a **standardized, vendor- and platform-agnostic** methodology within an **open framework** — the score is transparent, because the vector string that produced it is published alongside it.

CVSS is composed of three metric groups.

- **Base metrics** — reflect the severity arising from a vulnerability's **intrinsic characteristics**, constant over time and assuming the reasonable worst-case impact across environments. They split into **Exploitability metrics** (Attack Vector, Attack Complexity, Privileges Required, User Interaction) and **Impact metrics** (Confidentiality, Integrity, Availability), together with **Scope**.
- **Temporal metrics** — adjust the base severity for factors that **change over time**, such as the availability of exploit code: Exploit Code Maturity, Remediation Level, and Report Confidence. These capture exactly the 'information changes quickly' problem from the analysis step.
- **Environmental metrics** — adjust the base and temporal severities to a **specific computing environment**, accounting for mitigations and the relative importance of the

vulnerable system: Modified Base Metrics plus the Confidentiality, Integrity, and Availability Requirements.

When an analyst assigns values to the **Base** metrics, the **Base equation** computes a score from **0.0 to 10.0**, itself derived from an **Exploitability sub-score** and an **Impact sub-score**. The Temporal and Environmental groups are **optional** refinements layered on top, and the final result is reported as a score **together with its vector string**, so any consumer can see precisely how it was derived.



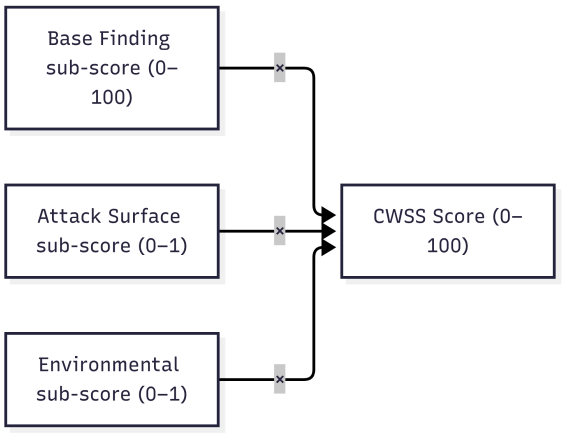
8.5 CWSS — scoring a weakness

Just as CVSS scores concrete vulnerabilities, the **Common Weakness Scoring System (CWSS)** provides a mechanism for **prioritizing software weaknesses** (CWE entries) in a consistent, flexible, and open manner. It is **distinct from, but not a competitor to, CVSS** — CVSS scores a disclosed vulnerability, CWSS prioritizes a weakness, often one found during development — and the two can be leveraged **together**. CWSS offers **quantitative measurements** of the unfixed weaknesses in an application, a **common framework** for prioritizing security errors, and **customized prioritization**.

CWSS is organized into three metric groups.

- **Base Finding** — the **inherent risk** of the weakness, the confidence in the finding, and the strength of controls: Technical Impact, Acquired Privilege, Acquired Privilege Layer, Internal Control Effectiveness, Finding Confidence.
- **Attack Surface** — the **barriers an attacker must overcome**: Required Privilege, Required Privilege Layer, Access Vector, Authentication Strength, Level of Interaction, Deployment Scope.
- **Environmental** — characteristics **specific to a particular environment or operational context**: Business Impact, Likelihood of Discovery, Likelihood of Exploit, External Control Effectiveness, Prevalence.

The scoring differs from CVSS in how the groups combine. Each factor in the **Base Finding** group is assigned a value, the values are converted to weights, and a Base Finding sub-score is computed between **0 and 100**. The same method applies to the **Attack Surface** and **Environmental** groups, but their sub-scores range between **0 and 1**. The three are then **multiplied together** to yield a final **CWSS score between 0 and 100** — so the Attack Surface and Environmental groups act as **multiplicative discounts** on the Base Finding score, reducing it to the extent that attack barriers and environmental factors make the weakness less exploitable or less consequential.



8.6 How the Catalogues Fit Together

The five resources are not interchangeable; each answers a different question along the risk chain, and together they operationalise the 'use public knowledge' prescription end to end.

Resource	Maintainer	What it catalogues	Where it helps in the assessment
CAPEC	MITRE	Attack **patterns** (how adversaries operate)	Malicious **threat** identification
CWE	MITRE	Software **weakness** types	**Vulnerability** identification (the generic flaw)
CVE / NVD	MITRE / NIST	Disclosed concrete **vulnerabilities**	Vulnerability identification (the specific instance)
CVSS	FIRST	**Severity score** of a vulnerability (0–10)	**Analysis** — grounding likelihood/impact of a CVE
CWSS	MITRE	**Priority score** of a weakness (0–100)	Analysis/prioritisation of CWE findings

The chain reads naturally: **CAPEC** tells us how an attacker might strike; those patterns point to **CWE** weakness types; a weakness instantiated in a real product becomes a **CVE** recorded in the **NVD**; **CVSS** scores how severe that vulnerability is, with temporal metrics tracking the fast-changing information that made malicious likelihood so hard to estimate; and **CWSS** prioritises weaknesses still being addressed in software under development. Feeding these standardized, shared scores into the threat and attack models is what lets the cyber-risk assessor attach defensible likelihoods to threats and incidents, closing the gap that the adversarial, fast-moving, globally-sourced nature of cyber-risk had opened.

Chapter 3 tris: A Case Study

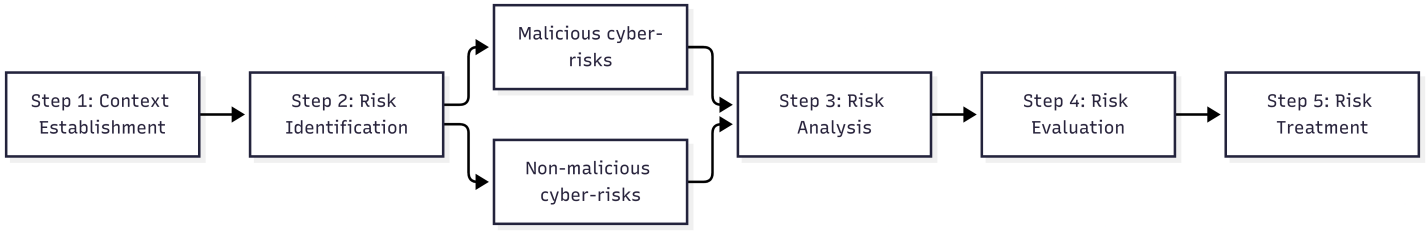
The two preceding chapters built the theory twice over — once in the neutral language of ISO 31000, once sharpened for the cyber domain. Theory, however, only becomes usable when walked through end to end on a concrete system, where every abstract instruction ('identify the attack surface,' 'estimate the likelihood,' 'aggregate risks that pull in the same direction')

must be turned into an actual table, number, or decision. This chapter does exactly that. It takes a single realistic target — the **Advanced Metering Infrastructure (AMI)** of a smart grid — and carries it through all five steps of the cyber-risk assessment, showing not only *what* each step produces but *why* each judgement is made the way it is. Nothing new is added to the method; what is added is the discipline of applying it completely.

1. The Scenario: AMI in a Smart Grid

A **smart grid** is an electricity distribution network that can monitor the flow of electricity within itself and automatically adjust to changing conditions — the textbook **cyber-physical system**, a digital control layer superimposed on a physical service of national importance. A central component is the **Advanced Metering Infrastructure (AMI)**, a fleet of power meters that use **two-way communication** both to **collect** information about electric-power usage and to **provide** information back to customers. Because the communication runs in two directions, the infrastructure does not merely *read* consumption; it can also *act* — switching power on or off, or **choking** it (limiting the power delivered).

That capacity to act is what makes the system worth assessing. The stated **aim** is to assess the risks for this infrastructure, including the components that switch power on and off and the component that limits the power provided. The **party** — the entity on whose behalf the assessment is run — is the **distribution system operator (DSO)**. This choice is decisive and must be held in mind throughout: the whole analysis looks at the infrastructure *from the point of view of the organization that manages it*, not the customer. The DSO is the **sole party**, so **every consequence estimate and every risk-evaluation criterion is defined from the operator's perspective** — the overall business impact to the operator, not the inconvenience to any individual customer.



Most of the analytical heavy lifting is done with the **OWASP Risk Rating Methodology**, and along the way the assessment leans on public catalogues and guidelines — CAPEC, CWE, **NISTIR 7628** (smart-grid cybersecurity), and **ISO 27005** (its annexes of vulnerabilities and threats).

2. Step 1 — Context Establishment

Context establishment bounds the assessment and furnishes everything the later steps consume: the internal and external context, the assessment goals, the target/scope/focus, the assumptions, the assets, the likelihood and consequence scales, and the risk-evaluation criteria. The guiding difficulty, as always in the cyber domain, is **fixing the boundary** of a system with no clean physical perimeter.

2.1 External Context

The external context describes the societal, legal, regulatory, and financial environment and the relationships with external stakeholders. Because the smart grid is a cyber-physical system forming part of a **critical infrastructure**, the DSO is subject to national laws and regulations governing its operation — in Europe, the **NIS directive** on the security of network and information systems. Identifying these obligations is not optional: **failure to comply may carry significant legal and financial consequences**, in the worst case putting the operator out of business. The regulation's logic, for a critical-service provider, is that **critical incidents must be avoided**, and when they occur they must be **reported within a predefined time window** and **resolved within a predefined window**.

Two external concerns follow. First, **power outages** threaten the availability of a critical service. Second, incidents such as **charging the wrong amount** or **disclosing customer data** damage the **reputation** and public trust in the operator — and reputation is itself a protected value. The external context therefore already points at the assets that will matter: availability of the service, and the reputation that rides on it.

2.2 Internal Context

The internal context describes the goals, objectives, policies, and capabilities that shape how risk should be assessed. The DSO's **mission** is to distribute electrical power, and from it flow three goals: to **provide power reliably**, so customers do not suffer unintended failures; to **exchange correct and timely information**, so they are charged the right amount; and to **protect the privacy** of customers, so their data is not disclosed. Read as security properties, these yield the three dimensions the assessment will track: the **availability** of the service, the **integrity** of information in transit, and the **privacy** of customers.

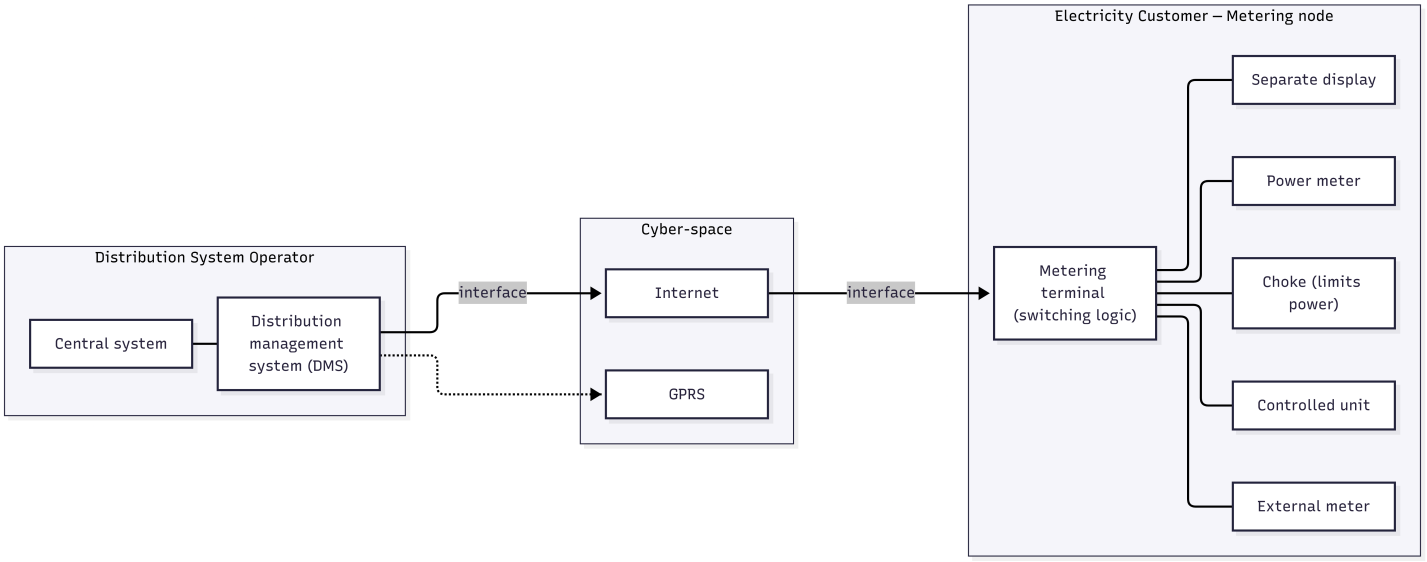
One internal fact about people carries real analytical weight. **Most employees of the DSO have strong technical competence, but only a few have received special training in risk assessment.** The division that will run the assessment is technically excellent yet unaccustomed to risk analysis — a limitation that itself becomes a vulnerability later, and a reason the results must be documented in plain, non-technical language.

2.3 Goals and Objectives of the Assessment

Three objectives are set for the assessment itself. The primary one is to **assess risk with respect to business continuity** — reducing the risk of incidents that may impact the DSO's business by identifying appropriate treatments for the important risks. The second is **law and regulation compliance** — analysing the risks that would leave the operator non-compliant. The third is to **improve situation awareness**: the assessment must be documented so that a wide range of internal and external stakeholders can understand it, **including those who are not experts**, which means technical detail should be avoided as far as possible in the reporting. The latter two are, in effect, documentation requirements layered on top of the core continuity analysis.

2.4 Target, Scope, and Focus

The **target** is pinned down with a diagram supplied by the party; the system divides into two sides.



On the **operator side** sit the central system and the **distribution management system (DMS)** — the latter the real target, since it decides how power is provided. To act, it connects to the central system over a network, either the **Internet** or **GPRS**. On the **customer side** sits the **metering node**, installed in the electricity box, containing the **power meter**; the **metering terminal**, which holds the switching logic and connects to a separate display, a controlled unit, and an external meter; and the **choke**, which regulates power intensity. The metering terminal and choke together perform switching and choking.

The **scope** fixes what is in and out: the assessment is limited to **risks due to attacks on or via the target**. Attacks via back-end systems are **out of scope**, and communication over **GPRS** is assumed covered by a separate assessment and is also **out of scope**. The scope thus narrows to the **Internet path**.

The **focus** is the refinement within that scope: the **exchange of meter data and control data via the Internet**, and how this may affect the **provisioning of power**. Within the scope but *not* the main focus are attacks via **physical access**. Both malicious and non-malicious threat sources are considered. In functional terms the focus rests on the basic AMI functions: **registering meter data, transferring data** between customer and operator, and **switching on/off or choking** power.

The assessment therefore trusts the internal infrastructure and treats the **attack surface** as the points where the two sides meet the external world — the **Internet (and GPRS) interfaces** on both operator and customer components. An attacker may enter through any of these; and, because the distribution side is secured but the customer side cannot be guaranteed, the attacker may equally be **internal to the customer component**.

2.5 Assumptions

The analysis is made honest by stating what is taken for granted. The assessment assumes:

- threat sources may be **both internal and external**;
- malicious and non-malicious threats alike may be internal or external;
- the target may be attacked not only by actors with a financial or personal motive but also by actors who wish to **disrupt society** (terrorists, activists);
- **all meter data and control data** sent between the central system and metering nodes are **encrypted**.

There is also a working presumption that the probability of an external source may differ from that of an internal one — a distinction the likelihood analysis will later weigh.

2.6 Assets

From the party, the mission, and the functions in scope, the assessment derives the assets as **security properties over specific components**. Three result, each defined from the operator's business perspective.

Asset	Description
Integrity of meter data	The integrity of meter data must be protected all the way from the power meter to the distribution system operator.
Availability of meter data	Meter data from the metering node must be available to the distribution system operator at all times.
Provisioning of power to electricity customers	Power should be switched off or choked *only* as the result of legitimate control signals from the central system.

2.7 The Likelihood Scale

Likelihood is expressed in terms of **frequencies**, using a **five-step scale of intervals** rather than exact values. The analyst need only decide *which interval* an incident's frequency falls into, a simple and honest way of expressing uncertainty; the granularity chosen depends on data availability and the decision makers' preferences. The five levels, in ascending order, are **Rare, Unlikely, Possible, Likely, Certain**, each mapped to a frequency band (events per unit of time) read off from historical data.

2.8 The Consequence Scales

Because consequences are tied to assets, a single universal scale is impossible: the assessment defines **one consequence scale per asset**, even though all three share the same five-level granularity (Insignificant, Minor, Moderate, Major, Critical). Every scale is written **from the DSO's perspective**, measuring overall business impact, and calibrated against the legal duty of a critical provider to guarantee certain levels of availability and integrity. The scales quantify impact by the **number of affected customers** and, for availability, the **duration** of the disruption.

Integrity of meter data — measured by how many customers' data is wrong:

Value	Description
Insignificant	Errors for up to 100 customers
Minor	Errors for 101–2,000 customers
Moderate	Errors for 2,001–20,000 customers
Major	Errors for 20,001–50,000 customers
Critical	Errors for more than 50,000 customers

Availability of meter data — measured by number of customers *and* duration of unavailability:

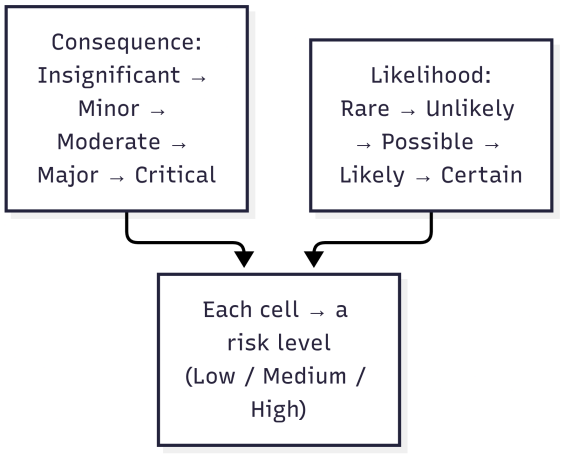
Value	Description
Insignificant	Up to 1,000 customers unavailable for 1–24 hours
Minor	Up to 1,000 customers for more than 1 day, or 1,001–10,000 for 1–24 hours
Moderate	1,001–10,000 customers for more than 1 day, or more than 10,000 for 1–24 hours
Major	More than 10,000 customers for 25 hours–7 days
Critical	More than 10,000 customers for more than 7 days

Provisioning of power — measured by number of customers suffering an outage and its duration:

Value	Description
Insignificant	Up to 100 customers for 1–24 hours
Minor	Up to 100 customers for more than 24 hours, or 101–1,000 for 1–24 hours
Moderate	101–1,000 customers for more than 24 hours, or 1,001–10,000 for 1–24 hours
Major	1,001–10,000 customers for 25–72 hours, or more than 10,000 for 1–24 hours
Critical	1,001–10,000 customers for more than 72 hours, or more than 10,000 for more than 24 hours

2.9 Risk Evaluation Criteria

The criteria are fixed by drawing the **risk matrix**: a 5×5 grid crossing the five likelihood levels against the five consequence levels, with each cell coloured to a risk level (**Low, Medium, High**). Deciding the colouring — how many bands, and where each boundary runs — *is* the act of setting the evaluation criteria, and it is done **now, before any risk is seen**, so that the later placement of risks is fair and not swayed by whichever risk happens to look most alarming.

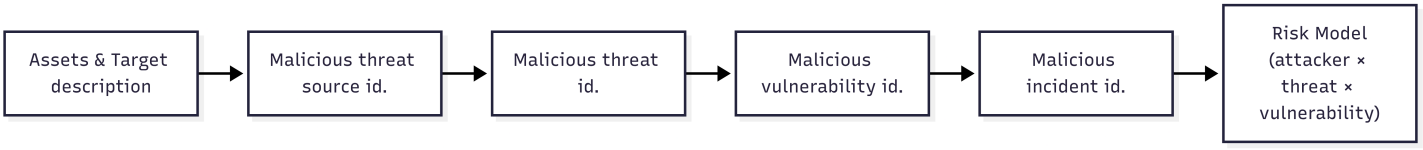


3. Step 2 — Risk Identification

The goal of risk identification is to arrive at a collection of **threat sources, threats, vulnerabilities, incidents, and risks** that is as correct and complete as possible for *this* target and *these* assets. To populate it, the assessment first gathers information from **logs, intrusion-detection and monitoring tools, vulnerability scanners, penetration tests, and source-code reviews**; from **external vulnerability and threat repositories**; and from **people who know the target well**. A warning attaches to the historical sources: when using event logs one must **take great care not to assume that tomorrow will be like yesterday**, since the cyber threat landscape shifts quickly. Identification then proceeds along the two mandated tracks — malicious first (attacker-forward), non-malicious second (incident-backward).

3.1 Identification of Malicious Cyber-Risk

The malicious track is a game of strategy against a **rational attacker**, running through four stages from the assets-and-target description to a **risk model** expressing each risk as a combination of attacker category, threat, and vulnerability.



Threat-source identification asks *who* may want to attack and *why*, documenting for each candidate the **motive and intention**, the **capabilities**, and the **available resources** — the four things on which the potential for harm depends (recorded in a table whose columns are *Threat source · Motive and intention · Capability and resources*). Among the sources considered are the **script kiddie** (a naive, relatively unskilled attacker), the **cyber-terrorist**, and the **hacktivist**. The hacktivist entry illustrates the reasoning: motivated by a political, ideological, or religious agenda, hacktivists are assumed — though the line from cyber-terrorists is admittedly fuzzy — to be **less willing to go to extremes**, aiming to harm selected groups, politicians, or individuals rather than society as a whole; their skill and resources vary widely, since some operate alone or in small groups but a well-organized group

can command significant competence. Threat sources may be **external** (arriving through the attack surface) or **internal** (a disgruntled employee).

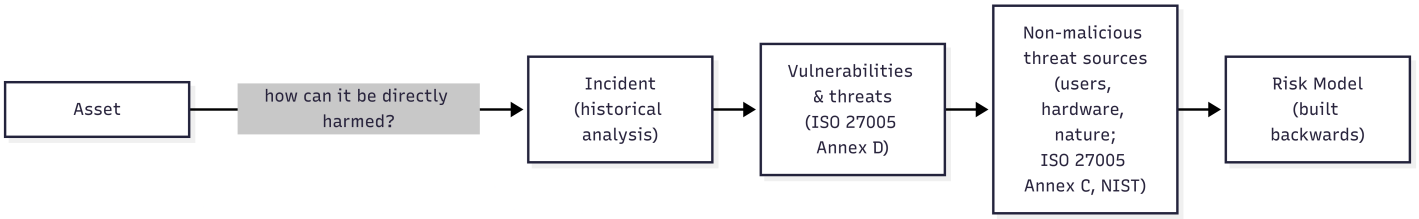
Threat identification asks, for each source, which threats it may initiate, concentrating on how the source could exploit the documented **attack surface** and drawing on public catalogues such as **CAPEC**.

Vulnerability identification searches, for each threat, the weaknesses it could exploit — beginning from vulnerability lists (**NISTIR 7628**, **ISO 27005**'s catalogue of hardware/software/network/personnel/site/organization vulnerabilities, **OWASP**, and **CWE**) and continuing with the output of **scanners and security testing**. For example, the threat *DDoS attack on the central system* is paired with *Inadequate attack detection and response on the central system*, explained by the fact that new DDoS techniques constantly emerge to defeat existing countermeasures, which in a power-distribution domain not historically operated in cyberspace are rarely updated. The tampering threats are paired with *weak encryption and integrity check*, especially acute at the metering nodes, which have little computing power and are rarely replaced.

Incident identification investigates how those threats, through those vulnerabilities, actually harm the assets — guided by **event logs** and by what is **most probable in this environment**. A single threat source, *metering node infected by malware*, produces **three distinct incidents** against three assets (malware *compromises* meter data → integrity; *disrupts transmission* → availability; *disrupts the choke functionality* → provisioning of power), a reminder that one event can be several risks at once.

3.2 Identification of Non-Malicious Cyber-Risk

For the non-malicious track the attacker-forward method collapses, because there is **no intent or motive** to impersonate. The analysis **runs backwards**: start from the incident, then find the vulnerabilities and threats, then the threat sources.



Incident identification here leans on **historical analysis**, because non-malicious incidents are fewer, more easily standardized, and usually recurring and known. The result is organized asset-first. The incidents include *communication between the central system and metering terminal is lost* (harming both provisioning and availability, since control data flows one way and meter data the other); the three *software bug on the metering terminal* incidents (compromising meter-data integrity, disrupting its transmission, and disrupting the choke respectively); two *mistakes during maintenance of the central system* incidents (disrupting control data to the choke, and preventing reception of metering-node data); and *the metering terminal goes down due to damage from lightning* — a purely environmental cause. Candidate

vulnerabilities and threats come from **ISO 27005's Annex D** (vulnerabilities by type) and **Annex C** (typical threats, each tagged **D/A/E** for deliberate, accidental, environmental, which separates the non-malicious A and E causes from the malicious D ones).

At the close of Step 2 the assessor holds **two risk models** and the full enumeration of incidents the analysis will now quantify.

4. Step 3 — Risk Analysis

Risk analysis estimates the **likelihood** and **consequence** of every identified incident so that a risk level can be derived. The information sources are the same as in identification; the difference is that the assessment now needs the **severity of vulnerabilities**, the **likelihood of threats and incidents**, and the **consequence of incidents**. It proceeds by answering four questions in order, each against a scale fixed during context establishment.



4.1 Analysing Malicious Threats — the OWASP Method

For malicious threats the likelihood of a threat materializing is estimated from **event logs** and **expert judgement**, structured by an approach inspired by the **OWASP Risk Rating Methodology**. Four factors are rated on a **0–9** scale.

Factor	Question	Example anchors (score)
Skill Level	How technically skilled is this group?	no skills (1), some (3), advanced (5), network/programming (6), security-penetration (9)
Motive	How motivated are they to find and exploit this?	low/no reward (1), possible reward (4), high reward (9)
Opportunity	What resources/opportunities are required?	full access/expensive (0), special (4), some (7), none required (9)
Size	How large is this group?	developers (2), sysadmins (2), intranet users (4), partners (5), authenticated (6), anonymous Internet (9)

The factors are averaged per *(threat, threat-source)* pair. For the threat **DDoS attack on the central system**:

- **Script kiddie** — Skill 3, Motive 1, Opportunity 7, Size 7 → **AVG 4.5**.
- **Cyber-terrorist** — Skill 7, Motive 8, Opportunity 7, Size 3 → **AVG 6.25**.

The 0–9 range is mapped **uniformly** onto the five-level likelihood scale, in equal bands of 1.8.

Interval	Likelihood
0 – 1.8	Rare
1.8 – 3.6	Unlikely
3.6 – 5.4	Possible
5.4 – 7.2	Likely
7.2 – 9	Certain

Taking the **worse case** of the two scores (6.25 for the cyber-terrorist falls in 5.4–7.2), the DDoS threat is estimated **Likely** — then checked against the event logs and confirmed by the operator's participants. Iterating over all threats yields:

Threat	Likelihood
DDoS attack on the central system	Likely
Tampering with control data to the choke (all/most)	Possible
Tampering with data from the metering terminal	Possible
Malware installed via the external meter	Possible
Metering node infected by malware	Rare
Tampering with control data to the choke (selected customers)	Unlikely
Illegitimate control data from the central system	Unlikely

4.2 Analysing Non-Malicious Threats

For non-malicious threats OWASP is inapplicable — **there is no attacker** whose skill, motive, opportunity, and size could be scored. The likelihood is instead taken from **event logs, expert judgement, interviews or questionnaires, and published statistics** about how often similar threats occur in comparable organizations. Historical data carries the estimate; for instance, *mistakes during update/maintenance of the central system* is judged **Certain** on the basis of the operator's own record.

4.3 Analysing Vulnerabilities

Vulnerability severity is rated on a three-step scale — **Low (0–3), Medium (3–6), High (6–9)** — using expert judgement, statistics, open repositories, and the output of **vulnerability scans, security testing, penetration testing, and code review**. For *malicious* threats the OWASP-inspired rating uses four factors on 0–9.

Factor	Question	Example anchors (score)
Ease of discovery	How easy is it to discover?	practically impossible (1), difficult (3), easy (7), automated tools (9)

Factor	Question	Example anchors (score)
Ease of exploit	How easy is it to exploit?	theoretical (1), difficult (3), easy (5), automated tools (9)
Awareness	How well known is it to the agents?	unknown (1), hidden (4), obvious (6), public knowledge (9)
Intrusion detection	How likely is an exploit to be detected?	active detection (1), logged and reviewed (3), logged without review (8), not logged (9)

Scoring *Inadequate attack detection and response* gives ease-of-discovery 7, ease-of-exploit 5, awareness 6, intrusion-detection 7 → **AVG 6.25 → High**. Iterating yields:

Malicious-threat vulnerability	Severity
Inadequate attack detection and response	High
Weak encryption and integrity check	Medium
Unprotected local network, no sanitation of input from external meter	Medium
Outdated antivirus protection on the metering node	High
Four-eyes principle not implemented, no logging of operator actions	High

For *non-malicious* threats there is **no intent to discover and exploit**, so the four OWASP factors make no sense; instead the analyst gauges **the extent to which barriers are missing** that would stop a threat from becoming an incident.

Non-malicious vulnerability	Severity	Note
Single communication channel (central system ↔ metering terminal)	High	The Internet is the sole channel for many customers
Poor testing	Medium	Recurring undiscovered bugs; updates not tested before deployment
Poor training and heavy workload	Medium	Low awareness and overloaded staff, though central-system routines are strong
Inadequate overvoltage protection	High	Metering-terminal hardware is not robust to transient overvoltage

4.4 Likelihood of Incidents

An incident's likelihood is derived by combining the likelihood of the **threat** that causes it with the severity of the **vulnerability** it exploits. For the malicious incident *Data from metering nodes cannot be received due to DDoS*, the logs show only two such incidents in three years (which alone would read Possible), but an **increasing trend** pushes the estimate up;

even allowing that successful attacks are fewer than attempts, the frequency is judged **Likely** — matching its threat (DDoS, Likely) and vulnerability (Inadequate detection, High).

The non-malicious case shows how the **same threat yields different incident likelihoods**. The threat *mistakes during maintenance* is Certain, and its two incidents — disrupting control signals to the choke, and preventing reception of metering-node data — look equally frequent at first. But the operator has **routines for reviewing and testing changes before launch**, stronger for changes that may affect control data because provisioning of power is more critical than meter reading. This lowers the choke-control incident to **Unlikely** while leaving the meter-data-reception incident at **Possible** — identical threat, different incident likelihoods, because the protective barriers differ.

4.5 Consequences of Incidents

Each incident's consequence is judged **for every asset it harms**, read off the relevant per-asset scale, which requires estimating the **expected time to detect and respond** and the **number of affected customers**. For the DDoS incident, the operator's investigation reports show past DDoS attacks **never caused unavailability for more than one day**; but because the **customer base has grown**, the number of customers affected can now be higher — netting out to a consequence of **Moderate**.

Carrying this through all incidents produces the two master tables, numbered 1–8 (malicious) and 9–17 (non-malicious).

No.	Incident (malicious)	Asset	Likelihood	Consequence
1	Metering-node data not received due to DDoS	Availability of meter data	Likely	Moderate
2	False control data received by all/most choke components	Provisioning of power	Unlikely	Critical
3	False meter data for a limited number of customers	Integrity of meter data	Likely	Minor
4	Malware compromises meter data	Integrity of meter data	Rare	Moderate
5	Malware disrupts transmission of meter data	Availability of meter data	Rare	Moderate
6	Malware disrupts the choke functionality	Provisioning of power	Rare	Major
7	False control data to choke for selected customers	Provisioning of power	Rare	Insignificant
8	Power switched off without legitimate reason	Provisioning of power	Unlikely	Moderate

No.	Incident (non-malicious)	Asset	Likelihood	Consequence
9	Communication central ↔ metering terminal is lost	Provisioning of power	Certain	Minor
10	Same (communication lost)	Availability of meter data	Certain	Insignificant
11	Software bug compromises meter data	Integrity of meter data	Unlikely	Moderate
12	Software bug disrupts transmission of meter data	Availability of meter data	Unlikely	Moderate
13	Software bug disrupts the choke functionality	Provisioning of power	Rare	Major
14	Maintenance mistakes disrupt control data to choke	Provisioning of power	Unlikely	Moderate
15	Maintenance mistakes prevent reception of meter data	Availability of meter data	Possible	Minor
16	Metering terminal down due to lightning	Provisioning of power	Likely	Insignificant
17	Same (lightning)	Availability of meter data	Likely	Insignificant

5. Step 4 — Risk Evaluation

Evaluation keeps its four sub-steps, applied here with the cyber habit of holding malicious and non-malicious risks apart until grouping brings them back together.

Consolidation of risk analysis results. The goal is to confirm the **correct risk level** is assigned to each risk — the central question being not whether every estimate is individually perfect, but whether the resulting **risk level** is right. Two contrasting examples make the point. For risk 4, *Malware compromises meter data* (Rare, Moderate → **Low**), even raising the likelihood to Unlikely leaves the level Low, so that distinction is immaterial. For risk 15, *Maintenance mistakes prevent reception of meter data*, uncertainty over whether the consequence is Minor or Moderate **does** matter, because it moves the level from **Low to Medium** — so it must be resolved. Consolidation also checks for risks that are **both malicious and non-malicious** — where a malicious and a non-malicious threat lead to the *same incident*, appearing in both tables. There the likelihood and consequence estimates must be **consistent**, with both causes accounted for in the likelihood, an easy thing to overlook precisely because the two classes were handled separately for so long.

Evaluation of risk level. Each risk is plotted in the risk matrix by its likelihood and consequence, producing **two matrices**, one per class.

Malicious risks (1–8):

Consequence \ Likelihood	Rare	Unlikely	Possible	Likely	Certain
Critical		2			
Major	6				
Moderate	4, 5	8		1	
Minor				3	
Insignificant	7				

Non-malicious risks (9–17):

Consequence \ Likelihood	Rare	Unlikely	Possible	Likely	Certain
Major	13				
Moderate		11, 12, 14			
Minor			15		9
Insignificant				16, 17	10

Risk aggregation. Some risks pull in the same direction strongly enough to be evaluated as one. **Case 1** arises when a single incident harms two assets: each may be low alone, yet their **joint consequence** may warrant a higher level (the aggregated risk keeps the same likelihood but takes the combined consequence). **Case 2** arises when **several incidents of the same nature, triggered by the same threat, harm the same asset**. Going through the tables, no single incident harms more than one asset, so Case 1 does not apply. Case 2 does: risk 4 (*Malware compromises meter data*) and risk 11 (*Software bug compromises meter data*) both concern software on the metering node harming integrity, so they merge into a generic *Software on the metering node compromises meter data* — with **likelihood rising from Rare/Unlikely to Possible** while consequence stays Moderate. By the same reasoning risks 5 and 12, and 6 and 13, are aggregated; the rest are retained.

Risk	Likelihood	Consequence
(4) Malware compromises meter data	Rare	Moderate
(11) Software bug compromises meter data	Unlikely	Moderate
(4+11) Software on the metering node compromises meter data	**Possible**	**Moderate**

The matrix after aggregation merges the two classes:

Consequence \ Likelihood	Rare	Unlikely	Possible	Likely	Certain
Critical		2			

Consequence \ Likelihood	Rare	Unlikely	Possible	Likely	Certain
Major	6, 13	(6+13)			
Moderate		8, 11, 12, 14	(4+11), (5+12)	1	
Minor			15	3	9
Insignificant				16, 17	10

Risk grouping. Finally, risks that would **benefit from the same treatment** are grouped so that treatment selection can consider them together. The assessment asks, for each pair, whether they share something pointing to a common remedy — for example, two risks each sitting one step away from Medium, where a single treatment addressing both is likely worth its cost. Grouping is deliberately where the malicious/non-malicious wall comes down, because the only thing that matters now is the **shared mitigation**.

6. Step 5 — Risk Treatment

Treatment begins by **identifying treatments for selected risks**, then assessing their effect and deciding whether the **residual risk** is acceptable; if it is, the documentation is finalized and the process ends, otherwise another iteration follows.

Selecting the risks to treat. Since time and resources are limited, the assessment cannot treat everything and so **selects the most important risks** — those in the higher-risk cells of the aggregated matrix.

Documenting each treatment. For each selected risk a small table records the risk number, incident, asset, threat source, threat, attack point, vulnerability, and proposed treatment. For risk 1:

Element	Value
Risk n.	1
Incident	Data from metering nodes cannot be received by the central system due to DDoS
Asset	Availability of meter data
Threat Source	Script kiddie; Cyber-terrorist
Threat	DDoS attack on the central system
Attack Point	Internet connection to the central system
Vulnerability	Inadequate attack detection and response on the central system
Treatment	Implement state-of-the-art DDoS detection and response mechanism on the central system

Here the two distinctive features of cyber-treatment appear. First, treatments are **highly technical** — reconfiguring a firewall, updating router policies, deploying detectors — but when the technical lever is unavailable (a server that cannot be upgraded for compatibility reasons), the assessment turns to **non-technical** means: training the staff who operate the machine and, in parallel, adding compensating monitors. Second, the malicious/non-malicious class influences which treatment fits best, yet **a single treatment may benefit several risks across both classes** — exactly the efficiency grouping was designed to surface.

Risk acceptance and cost–benefit analysis. Every treatment carries a cost, so its **effect** (reduced risk level) is weighed against that cost. For the DDoS treatment on risk 1 (level **High**), the reasoning is instructive: the treatment will **hardly prevent** script kiddies or cyber-terrorists from *launching* attacks — no effect on the threat — but **early detection** reduces the likelihood that an attack becomes the incident, moving likelihood from **Likely to Possible**, and a **prompt response** means fewer customers affected for a shorter time, moving the consequence from **Moderate to Minor**. The combined effect drops the level from **High to Low**. The cost, however, is **High**: significant investment in hardware and network infrastructure, plus assembling an adequate set of detectors (ideally combining anomaly-based and signature-based approaches).

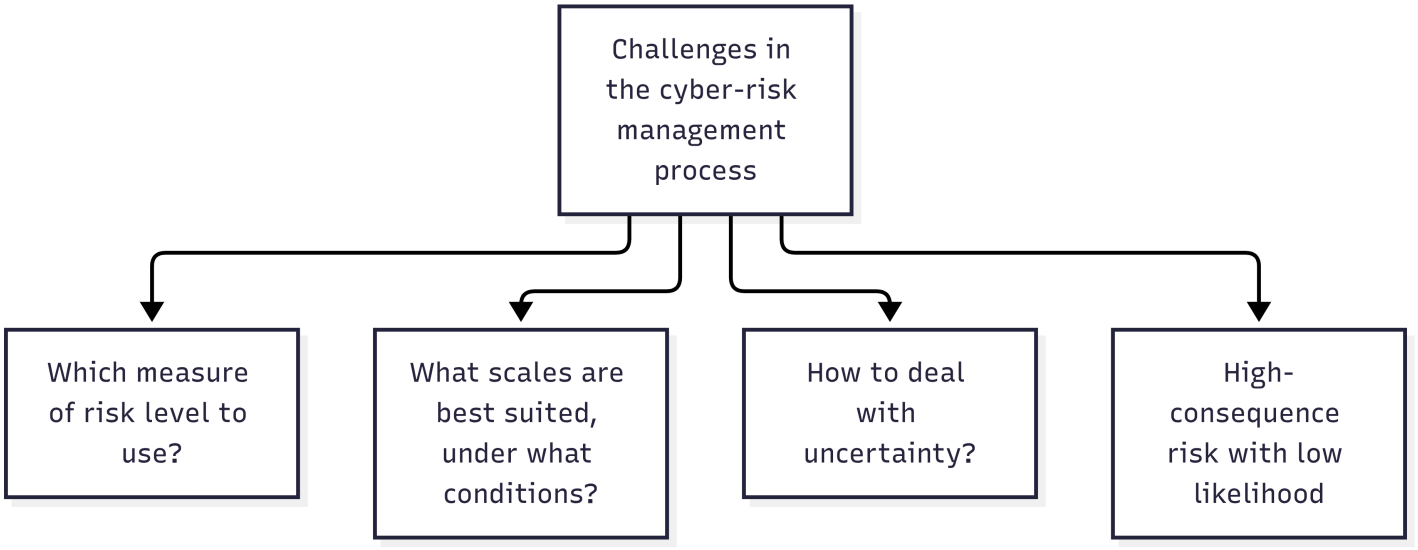
Iterating over the selected risks produces the final **effect-of-treatments** table, the deliverable the whole assessment has been building toward.

Treatment	Risk	Effect	Cost
Implement state-of-the-art DDoS detection and response on central system	1	High → Low	High
Stronger integrity checking of received meter data on central system	4; 11; (4+11)	Low→Low; Low→Low; **Medium→Low**	High
Hire more staff	14, 15	Low → Low	High
Develop executable scripts for routine maintenance tasks	14, 15	Low → Low	Low

The table captures the economic reality that runs through the entire discipline. The DDoS treatment buys a dramatic reduction (High → Low) but at high cost, and must be justified against the operator's budget. **'Hire more staff'** and **'develop executable scripts'** both address risks 14 and 15 with the same Low→Low effect, but at **High versus Low cost** — so the scripting treatment is plainly the better buy for the same benefit, a decision only visible because the risks were **grouped by shared mitigation**. The residual-risk logic closes the loop: the risks remaining in the **green (Low)** cells are typically **treated by acceptance** — an informed decision to retain them because the cost of further treatment is not justified by the small reduction it would bring. With treatments selected, their effects estimated, and the residual risks judged acceptable, the documentation is finalized and the cyber-risk assessment of the AMI is complete.

Chapter 3 tetrakis: Risk Management — Challenges and Guidelines

Having walked a complete assessment end to end, it is tempting to think the method is now simply a matter of execution. It is not. Every estimation in that case study concealed a choice that could have been made differently, and each choice is a place where a real assessment can quietly go wrong. This chapter steps back from the mechanics to confront the **main challenges that arise during the estimation process** and to set out guidelines for handling them. Four questions organize the discussion, corresponding to the four genuine decision points an assessor faces: **which measure of risk level to use, what scales are best suited under what conditions, how to deal with uncertainty, and how to handle high-consequence risks of low likelihood**.

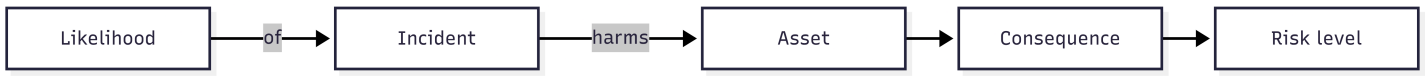


1. Which Measure of Risk Level to Use?

Risk can be described from several points of view, and the first challenge is to choose the right **factors** through which the risk level will be measured — because the measurement, once chosen, fixes what the rest of the assessment can say. The choice is essentially one of **how many factors** to decompose the risk into: two, three, or many. The three models are not rivals so much as instruments suited to different heights in the organization.

1.1 The Two-Factor (Classical) Measure

The most general definition characterizes risk by just two factors: the **consequence** (the loss of asset value when a potential incident occurs) and the **likelihood** (how often this happens). This is the classical measure used throughout the case study, and when only two factors are in play things are simple — the risk level is fully determined by them. The likelihood is tied to a **specific incident**, while the consequence is defined by looking at the **asset** in connection with that incident.



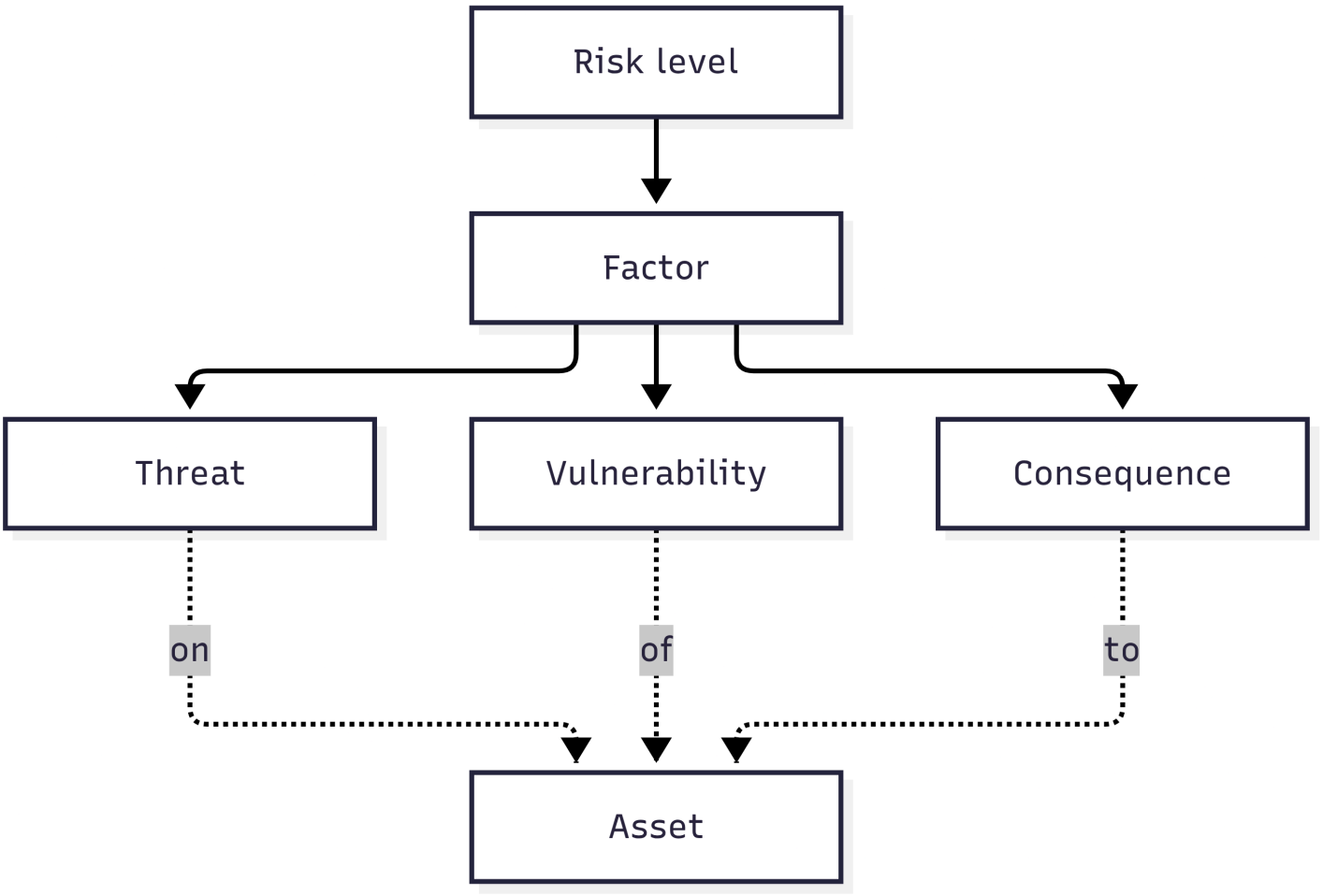
This characterization belongs at the **top of the hierarchy**, with strategic management. When risk must be reported at a high level, more elaborate models are useless: following the **Von Solms** principle that information should be **aggregated as it rises**, it is better to collapse everything into few factors than to hand executives a count of ten vulnerabilities. The price is a **loss of sensitivity**: collapsed into two factors, many distinct risks come to look equal. At the top this is not a problem — strategic management still grasps that there *are* issues and can act. The trouble only appears lower down.

1.2 The Three-Factor Measure

Sometimes it helps to say more precisely what 'likelihood' means, by decomposing the probability itself. The **NIACAP** standard defines risk as *'a combination of the likelihood that a threat will occur, the likelihood that a threat occurrence will result in an adverse impact, and the severity of the resulting impact.'* The key move is splitting likelihood into **two distinct probabilities**:

- the probability that **a threat will occur** — the extent to which the target is *subject to* a certain threat, estimated by studying the company and its environment;
- the probability that **the threat occurrence will result in an incident** — the **vulnerability** of the target with respect to that threat.

The distinction matters because the mere existence of a threat does not guarantee an incident: the attacker *tries* to materialize it, but there is a probability the attempt succeeds and a probability it does not. The result is a three-factor model built on **Consequence, Threat, and Vulnerability**, where threat and vulnerability are analysed **independently** and only then combined into the likelihood, with the vulnerability becoming relevant only when considered together with a specific threat on a specific asset. This is exactly the model used in the AMI case study, and it sits at a **middle point** in the hierarchy.



1.3 The Many-Factor (Multi-Factor) Measure

At the operational layer still more points of view open up, and some methodologies — **OWASP** is the example — analyse **threats and vulnerabilities under separate factors**. The likelihood can be decomposed into the probability that a specific threat agent is *interested* in a threat, that the threat *happens*, that it *materializes*, and so on, each combined back together; and the consequence can be split into **technical impact** and **business impact** factors (a technical damage may cost little in business terms). Considering all these perspectives **refines the model** and yields additional information — but only if there is **data** to quantify each factor.

The payoff is **richer differentiation**. Where the aggregated models make many risks look equal, a multi-factor model gives many variables to hold in mind and so supports **different prioritizations** of otherwise similar risks. Since the ultimate aim is to protect the infrastructure **without losing money** — to optimize treatment in a cost–benefit manner — this finer prioritization is precisely what the people 'close to the stuff' need. The multi-factor model is used close to the operational layer.



The contrast between the two ends is worth stating plainly. The **three-factor** model is simple and conceptual (Threat–Vulnerability–Impact), good for awareness and high-level decisions, but offers limited differentiation — many risks look 'equal.' The **multi-factor** model breaks probability and impact into measurable sub-factors, giving more objective and repeatable scoring and enabling smarter prioritization. In a phrase, the multi-factor model is the move *from understanding the risk to managing it effectively.*

1.4 Which Measure to Use for Cyber-Risk

There is no universally correct answer: the approach and the way it is used **depend on the context** and the risk-assessment situation, with the position in the pyramid as a first guide and several practical considerations layered on top.

The dominant parameter is **data availability**. With good data on frequency and consequence, one will probably go for the **two-factor** approach; to use a multi-factor model one *needs* the data to instantiate it, and without that data the model collapses. Within cybersecurity, the popularity of multi-factor measures is **growing**, precisely because at the bottom of the stack cyber-systems generate logs automatically for many indicators — though measuring likelihood with reasonable uncertainty remains difficult. A sharper way to put the data problem: it is usually **not the lack of data, but the lack of the *right kind* of data** with respect to the predefined factors. When the logged indicators do not match the factors, one may **define one's own risk function** from factors that match those indicators — a move that, as the slides warn in red, **requires experience and great care**. Cybersecurity also offers technologies for **data correlation** that help bridge raw data to the chosen factors. Finally, information comes not only from the environment but from **experts and stakeholders**, and when it does, two guidelines apply: make the factors **carefully defined and easy to keep apart**, and **select the right kind of scale** for each factor — the next challenge.

2. What Scales Are Best Suited, Under What Conditions?

Choosing the number of factors is only half the measurement problem; each factor still needs a **scale**. Its suitability depends on the **factor in question, the kind of risk assessment, the target of assessment, and the available data sources**. Beyond fitting the factor, a scale must suit the *type* of assessment: if the analysis is multi-factor but its output must reach **top**

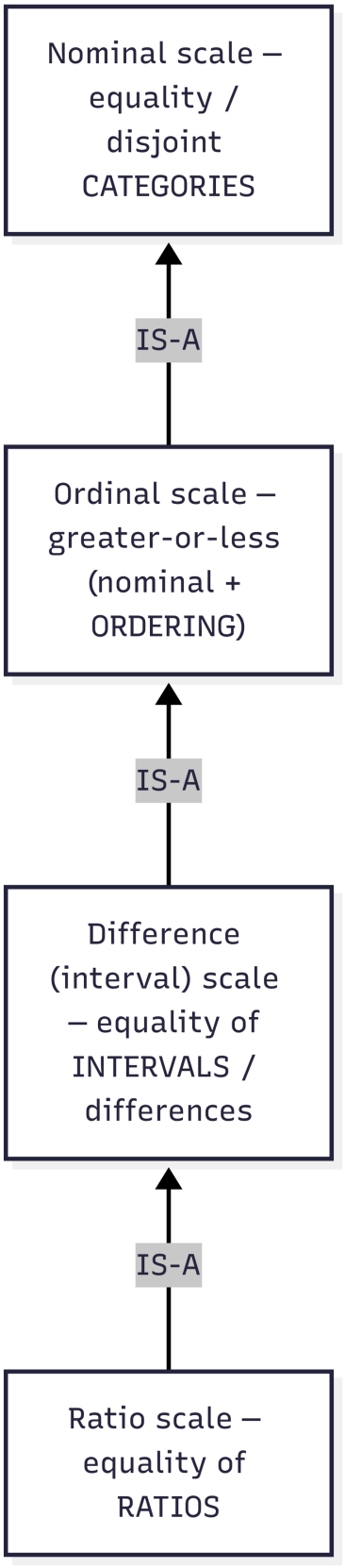
management, risk cannot be reported as raw numbers, because executives do not know what those numbers mean. The range of values must therefore be chosen **deliberately** — a preliminary assessment need not be precise and can stay **qualitative**, whereas greater refinement calls for **quantitative** evaluation with more values per scale — and there must be enough data to actually distinguish the levels of whatever scale is chosen.

2.1 Classification of Scales

Scales form an **IS-A hierarchy**, each level adding a capability to the one below it.

- **Nominal scale** — allows the determination of **equality** and represents **disjoint categories** (e.g. 'non-human threat source,' 'accidental human threat source,' 'intentional human threat source'). It groups elements into classes and tells which risks look equivalent, but says nothing about order.
- **Ordinal scale** — adds **ordering** (greater or less) on top of the nominal categories (e.g. 'minor,' 'moderate,' 'major').
- **Difference (interval) scale** — allows the determination of **equality of intervals or differences**: equal differences correspond to equally distinct phenomena. The **Celsius** scale is canonical — a 10 °C difference reflects the same temperature gap whether the pair is (−10, 0) or (100, 110) — but it has **no meaningful zero**, so ratios are undefined.
- **Ratio scale** — adds a **true zero** and so allows the determination of **equality of ratios** (e.g. 'identity thefts in Europe per year,' or 'financial loss (€) caused by cyber incidents per year'), where statements like 'twice as many' are meaningful.

The first two levels (nominal, ordinal) are **qualitative**; the latter two (difference, ratio) are **quantitative**.



2.2 Qualitative Versus Quantitative Assessment

The qualitative/quantitative split is not a matter of taste but of **what the information allows**. **Quantitative** assessment fits when the information to be quantified is **homogeneous**, and it works better at a more technical level or when fine granularity is required. **Qualitative** assessment is for information that is **not homogeneous** — reputation, political impact, public perception — where a single number would be a fiction.

A crucial asymmetry governs movement between the two: **going from quantitative to qualitative is easy, but the reverse is difficult.** Quantitative is attractive in theory, since it lets one define functions and algorithms and set up optimization problems, yet in practice it often fails for want of the data to instantiate the model — and then everything collapses. A purely qualitative approach, on the other hand, is **not satisfactory on its own**, because the result of any risk assessment must ultimately be converted into a quantitative scale (typically **costs measured in money**) to support a cost–benefit decision. The guideline is therefore to **merge the two**: stay quantitative as far down as the data allows, switch to qualitative where limitations arise, and aggregate toward the qualitative as one moves up the pyramid. The deeper one is in the operational layer, the more quantitative the assessment; every risk must eventually be translated into numbers — cost, time, or impact avoided — and the art is **balancing quantitative precision with qualitative practicality**.

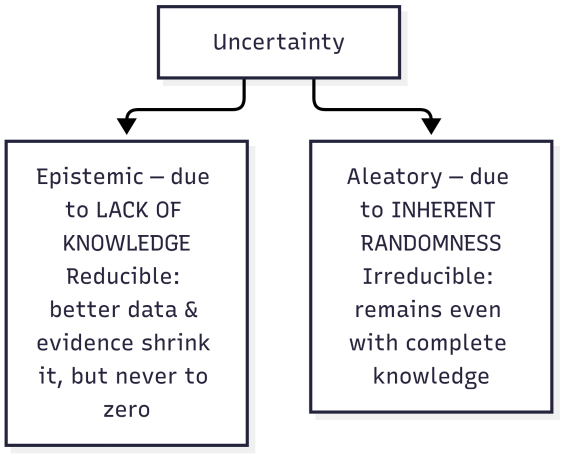
For **likelihood** specifically, there is much uncertainty in the measurements, and estimates often rest on historical data that are not always accurate enough. The advice is to **avoid complicated probabilities**; where probabilities are used, accompany them with explicit information about the uncertainty decision makers can use; and **avoid continuous scales in favour of ranges** (e.g. Rare / Unlikely / Possible / Likely / Almost Certain), because broad categories reduce the illusion of precision and support more reliable decisions. For **consequence**, recall that it is ultimately tied to the **treatment** phase — we weigh the cost and gain of treating, an indirect evaluation of the consequence — so when defining a consequence scale one should keep in mind the criteria of the later cost–benefit analysis, and **define a specialized scale for each asset** fitted to its intended usage. The difficulty is uneven: if the asset is a **bag of diamonds**, the consequence equals the monetary value stolen; if it is the **integrity of a customer database**, one can count the records harmed but not easily say what that means in euros; if it is the company's **reputation**, both the impact and its euro equivalent are hard even to characterize. The typical cyber practice is to associate consequences with **direct and indirect costs** not directly in money but through a **cost function** later mapped onto economic factors.

A closing note to the previous section (**2.2**) before moving on: cyber-risk has one **simplifying** feature and one **complicating** one. The simplifying feature is that cyber-risk concerns systems which are **largely computerized**, and computerized parts are well suited to automatic measurement and logging. The complicating feature is the **openness of cyberspace** and the frequent need to measure **human intentions and skills**, which resist quantification.

3. How to Deal with Uncertainty?

No amount of quantitative rigour removes uncertainty; it will always be present, and the task is to **represent it correctly**. ISO 31000 defines uncertainty as *'the state, even partial, of deficiency of information related to, understanding or knowledge of an event, its consequence, or likelihood.'*

3.1 Two Kinds of Uncertainty



Epistemic uncertainty arises from a **lack of knowledge**: if knowledge were perfect it would disappear, and better data and evidence can reduce it — though in practice knowledge is never perfect, so some always remains. **Aleatory uncertainty** arises from **inherent randomness** and cannot be controlled; the clean test is that *if uncertainty remains even with complete knowledge, it is aleatory, not epistemic.* The discipline concentrates on the **epistemic** kind, which it attacks through data collection and analysis, while measuring how much uncertainty remains and how much it affects the results.

3.2 Representing Uncertainty

Representation depends on the scale. With a **quantitative** scale, uncertainty is represented by **intervals**, the **width** of the interval expressing the level of uncertainty; how much is tolerable depends on how it impacts the **decision procedure**. Plotted in the risk matrix, a risk acquires a **confidence interval** in each direction — one horizontal (likelihood) and one vertical (consequence) — defining a region of how many mistakes can be tolerated. The dangerous case is a risk whose interval **straddles a boundary** so that the uncertainty could change its risk level (a risk hovering between Moderate and Minor): to judge whether the uncertainty is tolerable, the information must be represented *together with* its uncertainty.

With a **qualitative** scale, uncertainty is instead expressed as a **separate attribute** — a natural-language label on its own ordinal scale — rather than plotted. A consequence table can carry an extra **Uncertainty** column, so that two incidents with the *same* consequence are distinguished by our confidence in the estimate. This documentation belongs in the report accompanying the risk assessment; the visual and the explicit representations are **not alternatives but complementary tools**.

Incident	Consequence	Uncertainty
Information leakage	Low	Some
Breakdown of server	High	No
Identity theft	High	Considerable
Spyware installed	Medium	Some

3.3 Reducing Uncertainty

Beyond representing it, epistemic uncertainty can be actively **reduced** using data already acquired, by several methods:

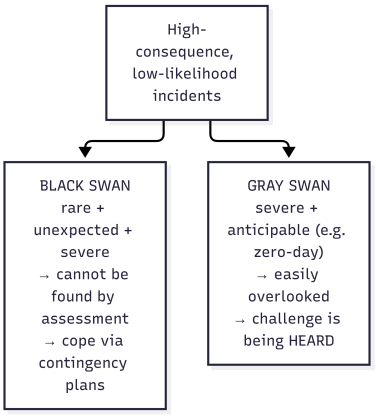
- **fuzzy logic**;
- **iterating data collection** (improving the quality of the data);
- **comparative analysis** (comparing the situation under investigation with what is already known);
- **testing** the risk model against historical data, or conducting surveys across larger groups of stakeholders — in effect, testing whether the issue is really there.

4. High-Consequence Risk with Low Likelihood

The last challenge concerns the risks the matrix is worst at handling: those that are **unlikely but devastating**, sitting in the 'yellow' borderland where a routine assessment is tempted to let them pass. Two species must be told apart.

A **black swan** is an incident that is **extremely rare and unexpected** but has **very significant consequences**. Black swans are, by their nature, **not likely to be discovered by risk assessment at all** — they are the genuinely unforeseen. The right response is therefore not better estimation but **preparation**: developing good **contingency plans** is the best way to cope with them, and the existence of a risk assessment does *not* make contingency planning obsolete. Their danger is exactly that they occupy the low-likelihood region yet carry critical impact.

A **gray swan** also has far-reaching consequences, but unlike a black swan it **can be anticipated** to a certain degree — the archetype being a **zero-day vulnerability**: we assume our system is not free of them and try to estimate what could happen. The characteristic danger of gray swans is that they are **easily overlooked** — they may not appear in the documentation we take as input, we may be interacting with the wrong stakeholders, or we may be unable to extract the required information — even though, **by definition, a relevant gray swan is known *somewhere* in the organization**, if not documented then implicitly in stakeholders' knowledge or inferable from existing data.



Because a gray swan is in principle knowable, the real challenge is **not estimating it but making sure it is heard**. Communicating it well means defining an effective process to convey the message to managers: **using probability is not recommended**; when a quantitative scale is used it should **always carry a reference**; and if there is **considerable uncertainty**, that must be communicated to the relevant decision makers.

Handling a gray swan then becomes a **treatment decision** aimed at helping the decision makers, and the lever is **cost of treatment**. When the cost is **Low**, simply treat the gray swan — reducing both its likelihood and its consequence. When the cost is **High**, there is no free move, and the decision is a genuine **trade-off between the cost of treatment and the residual severity of the risk**. The textbook example is a **healthcare data breach**: strong but costly **prevention** weighed against **rapid detection and response planning**, which is less costly but leaves more residual risk. For gray swans the decision is exactly that balance — what it costs to reduce the risk against how severe the risk remains after treatment.

CHAPTER 4: Auditing

1. Definitions

Audit is the independent, objective evaluation of whether controls, processes, or activities meet defined criteria. The criteria are essential: without a fixed standard to measure against, there is no basis on which to judge what is being examined.

Compliance is the degree to which an organization follows applicable rules, standards, or internal policies. It quantifies, as precisely as possible, the alignment between the defined criteria and what is actually implemented.

Assurance is the independent confidence that information, controls, or processes are reliable and effective. It is the central purpose of auditing: the confidence that an evaluator provides to the organization about the state of the work that has been assessed.

Auditing is the systematic and independent **evaluation** of processes, systems, and controls to ensure that **objectives are achieved** effectively and securely. It bridges classical audit theory and contemporary cybersecurity practice, applying risk-based reasoning, evidence collection, and control verification as instruments of organizational assurance.

2. Why Auditing Matters

Digital trust depends on the demonstrable integrity of systems and data. Auditing is one of the core security-governance processes that produce this trust, because it converts technical **compliance** into credible **assurance**, enabling transparency, accountability, and stakeholder confidence in an organization's cybersecurity posture.

Auditing is an **evidence-based**, risk-focused process that evaluates the design and effectiveness of controls. The defining discipline of the auditor is that nothing is left to chance and nothing is left unexplained: every conclusion must be supported by concrete evidence. This

is what makes an audit opinion objective and independent rather than a matter of personal judgment.

This distinguishes an **audit** from an **inspection**. Inspections are rule-based, often punitive, and typically one-off events performed by inspectors against a checklist. Audits instead aim to understand whether governance, risk management, and control mechanisms adequately support strategic objectives, and they ground that understanding in evidence. The separation becomes clearest during certification, where the roles of the party being examined and the party giving assurance are kept strictly apart.

3. History

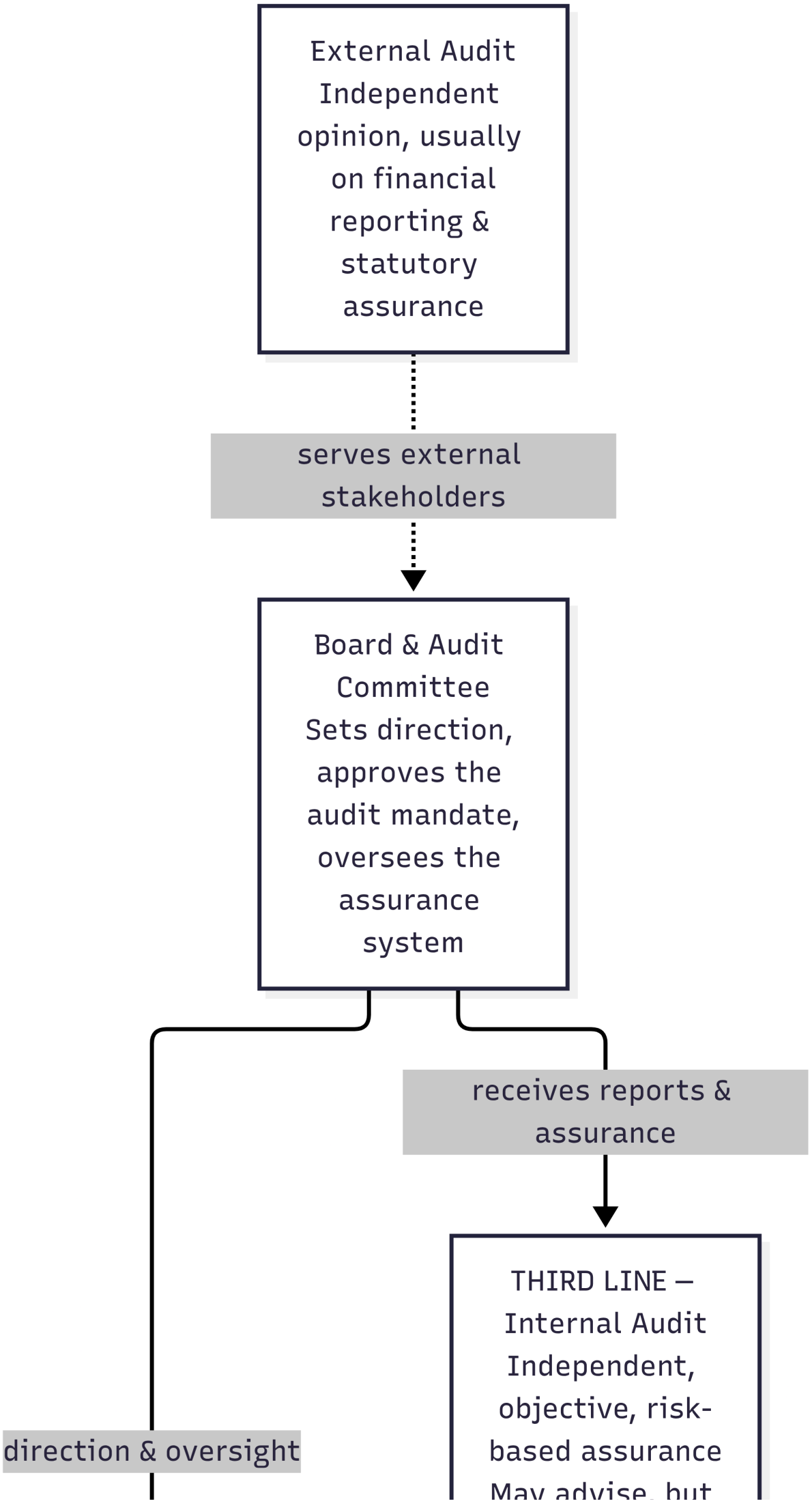
Auditing began as a way to verify records and reduce errors or fraud in early economic systems, where it mainly checked transactions and confirmed that accounts were accurate. As organizations grew larger and more complex, this simple verification activity became more structured, and modern internal auditing emerged in the early 20th century when organizations needed a more formal way to review operations and internal controls.

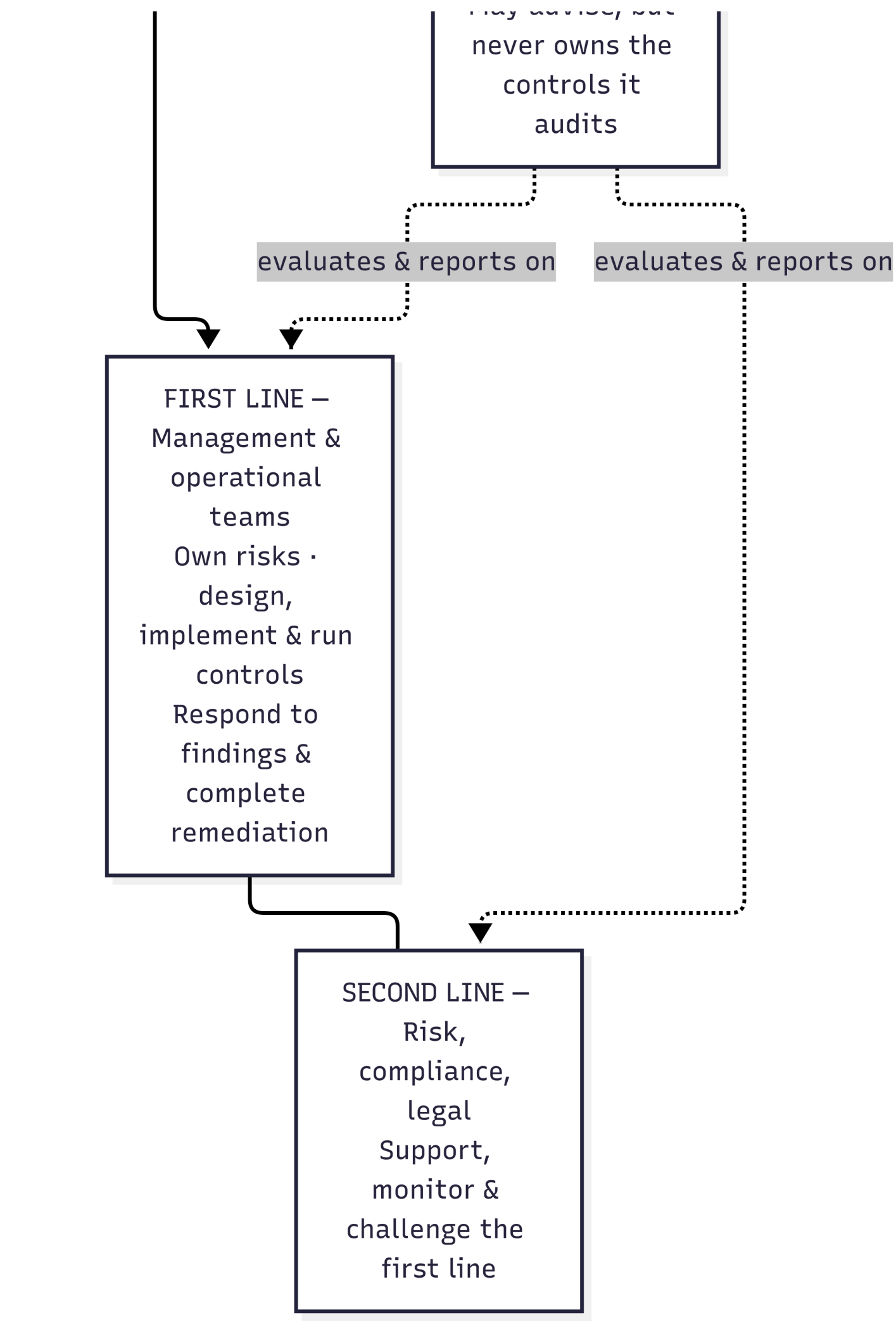
A major milestone came in **1941**, when the **Institute of Internal Auditors (IIA)** was founded and the first major book on internal auditing was published. This marked the beginning of internal auditing as a recognized profession with its own identity, language, and standards. The IIA is the global professional association and standard-setting body for the profession: it defines what internal auditing is, sets the rules and guidance practitioners must follow, and supports auditors through certification, education, research, and advocacy. Today, auditing is strongly connected to governance, assurance, and accountability, especially in complex and digital organizations.

4. Roles and Responsibilities

Auditing works through clearly separated roles. The organizing principle is that **management owns the controls**, while **audit provides independent assurance** on whether those controls work. These responsibilities are arranged hierarchically in the **Three Lines Model**.

- **First line** — management and operational teams own and manage risks, and they design and run the controls. They are the **owners of the risk** and the first accountable party when weaknesses are found.
- **Second line** — risk, compliance, and legal functions support, monitor, and challenge the first line. The board and audit committee sit at this governance level.
- **Third line** — internal audit gives independent, objective assurance to the governing body and senior management.


External Audit
Independent opinion, usually on financial reporting & statutory assurance
serves external stakeholders
Board & Audit Committee
Sets direction, approves the audit mandate, oversees the assurance system
receives reports & assurance
direction & oversight
THIRD LINE — Internal Audit
Independent, objective, risk-based assurance
May advise, but



Senior management (first line) is responsible for designing, implementing, and maintaining controls. It responds to audit findings, defines corrective actions, and ensures remediation is

completed. Because it owns the risk, it is the first accountable party when weaknesses are identified.

The board and audit committee set direction, approve the audit mandate, and oversee the whole assurance system. The audit committee is the main body that receives internal audit reports, monitors significant findings, and follows up on remediation. Its role is **oversight, not execution**: it must challenge management and use audit results to make informed decisions, translating the organization's concerns into the language of auditing and passing execution to the third line.

Internal auditors (third line) are independent, objective, and risk-based. Their job is to evaluate governance, risk management, internal controls, and compliance processes, then report findings to the board and management. They may advise on improvements, but they must not become responsible for the controls they audit, as that would destroy their independence.

External auditors (also a third-line function) provide an independent opinion, usually focused on financial reporting and statutory assurance. They may coordinate with internal audit but are not a substitute for it. The guiding distinction is that external audit serves external stakeholders, while internal audit serves governance inside the organization. External auditors are frequently the same parties that handle the organization's financial statements.

5. Internal vs External Auditing

Dimension	External Audit	Internal Audit
Who performs it	Independent auditors or assurance firms, external to the organization	An independent function **within** the organization, reporting to the board or audit committee
Standards	International standards such as **ISA**, and **ISAE 3402 / SSAE 18** for assurance engagements	Control frameworks such as **COSO**, **COBIT**, **NIST CSF**
Continuity	Typically periodic, **not continuous**; called when needed (e.g., certification); subject to formal reporting obligations	Operates on a **continuous**, improvement-oriented basis
Objective & focus	External accountability, regulatory compliance, public trust; credibility for investors and regulators	Evaluate and improve governance, risk management, and internal-control effectiveness; internal value creation, control maturity, process improvement

Because external auditors must carry international recognition, the standards they follow are generally more formal and more standardized than those an internal function uses. Internal auditing is freer: it can rely on special, non-certified publications — for example NIST special publications — which are more technical and specific and therefore more useful for identifying detailed control weaknesses. The separation is not absolute: internal auditors may also use

recognized standards and certifications; they are simply not bound to the same standardized set.

6. Certification

Professional certifications such as **CISA**, **ISO 27001 Lead Auditor**, and relevant **ISACA** frameworks establish recognized competence benchmarks for auditing and cybersecurity assurance. They validate not only technical knowledge but also the ability to plan audits, gather and evaluate evidence, report findings, and follow up on remediation. They demonstrate familiarity with professional ethics — independence, objectivity, due care, confidentiality, and accountability — and they standardize audit quality across organizations and industries by defining common expectations for skills and conduct. Many also require ongoing professional education, which keeps skills current as threats, standards, and regulations evolve.

At the point of certification, external auditing becomes mandatory, and a certified result must be as precise and unambiguous as possible. This is the structural reason auditing is so demanding: every assertion about a security control must be accompanied by the reasoning and evidence behind it. Audits are commonly classified by the relationship between the auditor and the audited party.

Audit type	Performed by	Examples
1st-party audit	The organization itself	Internal audit
2nd-party audit	An external but interested party	External provider audit; audit by another interested party
3rd-party audit	An independent certification body	Certification and/or accreditation audit; statutory, regulatory, and similar audit

The Auditing Process

7. ISO 19011 and the Nature of an Audit

The reference framework for how an audit is run is **ISO 19011 — Guidelines for Auditing Management Systems**. It defines an audit as a *systematic, independent, and documented process for obtaining evidence and evaluating it objectively against established criteria.* Each word of that definition is load-bearing: the process must be repeatable and structured, free from the influence of those being examined, recorded so it can be defended later, and anchored to evidence measured against pre-agreed criteria.

8. Audit Types and Approaches

Audits share a common methodology but differ in focus. The classic types are **financial** (validates the accuracy of financial statements), **operational** (evaluates efficiency and effectiveness of processes), **compliance** (tests adherence to laws, regulations, or standards),

performance (measures achievement of objectives), and **IT** (examines technological infrastructure and the control environment). There is no single fixed taxonomy; audits can equally be grouped by domain, such as healthcare or financial auditing.

In a cybersecurity context, a more granular set of approaches is useful. Each answers a different core question and concentrates on a different kind of evidence.

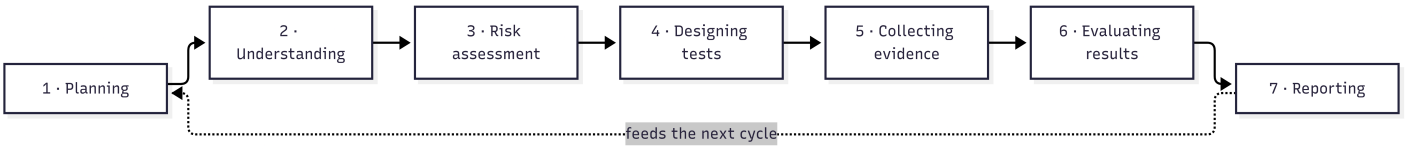
Audit approach	Main question	Typical focus
Compliance-based	Are we complying with required standards and regulations?	ISO 27001, NIST, GDPR, NIS2, DORA
Controls-based	Are security controls properly designed and implemented?	Access control, logging, incident response, encryption
Risk-based	Are the most significant risks adequately addressed?	Risk exposure, likelihood, impact, prioritization
Process-based	Are processes correctly defined and consistently executed?	Change management, vulnerability management, incident handling
Performance-based	Are security measures achieving their intended outcomes?	KPIs, control effectiveness, incident rates, remediation time
Continuous	Can deviations and control failures be detected continuously or near real time?	Automated checks, telemetry, configuration drift, alerts
Technical	Do systems satisfy technical security requirements?	Configuration reviews, vulnerability scanning, penetration testing
Forensic	What happened, how, and who or what was responsible?	Incident reconstruction, digital evidence, attribution

The **risk-based** approach is the one most relevant to cybersecurity, because it concentrates effort on the areas where the probability and impact of failure are greatest.

9. Auditing as a Seven-Phase Process

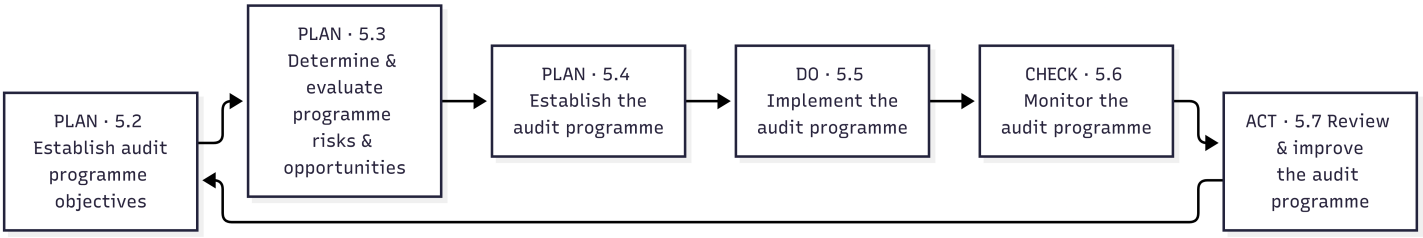
An audit unfolds as a sequence of seven phases, each building on the previous one.

1. **Planning** — define objectives, scope, and criteria.
2. **Understanding** the system — gather context and identify controls.
3. **Risk assessment** — determine inherent and control risks.
4. **Designing tests** — establish audit procedures and sampling.
5. **Collecting evidence** — execute fieldwork and document results.
6. **Evaluating results** — compare evidence to criteria.
7. **Reporting** — summarize findings, conclusions, and recommendations.



10. The Audit Programme as a Deming Cycle

Like every security-governance process, the management of an audit programme follows the **Plan–Do–Check–Act (PDCA)** cycle, which is the organizing logic of ISO 19011. The feedback loop means objectives are set, carried out, monitored, and continually improved rather than executed once and abandoned.



A second, more operational view describes the conduct of a single audit: **initiating** the audit, **preparing** the audit activities, **conducting** them, **preparing and distributing** the audit report, **completing** the audit, and **conducting the follow-up**. The follow-up feeds back into the monitoring of the overall programme. The two views are nested: programme management sits above the execution of each individual engagement.

11. The Seven Phases in Detail

A running example — the audit of an organization's exposure to **PHP2Shell** — illustrates each phase. PHP2Shell is a pair of critical flaws in the PHP interpreter (CVE-2026-109618 and CVE-2026-109619, rated CVSS 9.2) that allow unauthenticated remote code execution before any application code runs. They were publicly disclosed on 20 October 2026; fixes shipped the next day as PHP 8.5.12, 8.4.27, 8.3.36, and 8.2.35, while branch 8.1 and older received no release. The organization runs 120 PHP hosts, and its policy requires critical fixes within 7 days.

Planning

The auditor defines the objectives, scope, and criteria before any fieldwork begins, determining *what* will be examined, *why* it matters, and against which *standards* the audit will be judged. Good planning also identifies the resources, timing, and evidence needs of the engagement.

PHP2Shell — The objective is to verify that critical PHP vulnerabilities are identified, prioritized, and fixed within the deadline in the patch policy. The scope covers all internet-facing PHP applications, the patch-management platform, and vendor-hosted PHP systems, for the period from 20 October 2026 (public disclosure) to the audit date. The criteria are the internal vulnerability-management policy (critical fixes within 7 days), the asset-inventory procedure, and the vendor-contract SLAs — the last of these matters because the service belongs to the vendor, a fact that must be weighed when assessing the risk.

Understanding the System

The auditor gathers background information about the organization, its processes, and its environment, and identifies the key activities, responsibilities, and existing controls relevant to the scope. This phase builds context so the auditor does not test controls without understanding how the system actually works, and it deliberately reaches beyond the organization's internal context to relevant public information.

PHP2Shell — The auditor learns how alerts arrive (vendor advisories, CERT notices), who triages them, and who deploys the fix. A typical flow is: the security team receives the advisory, IT checks the inventory, system owners approve the change, and operations deploys the patch. The auditor also establishes which patched branches exist and that 8.1 and older receive no release — so some systems need an **upgrade, not a patch**. The control points identified are the asset inventory, advisory triage, emergency change approval, firewall rules, and web-shell detection.

Risk Assessment

The auditor determines the main risks in the area under review and assesses **inherent risk** and **control risk** to prioritize what matters most. The output is a clearer focus on the areas where evidence and testing are most needed.

PHP2Shell — The main risk is that an unpatched PHP server is compromised through a single unauthenticated request, because the flaws affect the PHP interpreter itself rather than any one application. A second risk is that unknown or vendor-hosted PHP systems are missing from the inventory and never patched. A third is that unsupported branches (8.1 and older) have no available fix. The auditor concludes this is a **critical risk area**: inherent risk is high, and the impact includes data exposure, service disruption, and loss of server control.

Designing Tests

The auditor translates the audit objectives into specific procedures and sampling methods, defining how each control will be tested, what evidence will be collected, and how the sample will be selected. This phase makes the plan operational and keeps every test tied back to the criteria.

PHP2Shell — The auditor decides to test three things: whether the inventory is complete, whether patches were applied within the deadline, and whether detection would have caught exploitation. For example, 20 of the 120 PHP hosts are selected by risk (internet-facing, running mod_php, on an unsupported branch, or vendor-managed), plus 5 random hosts to reduce bias, together with 3 emergency change tickets and the advisory triage log. The procedures include re-performing a PHP version check on the sampled hosts and reviewing the firewall and web-shell detection rules.

Collecting Evidence

The auditor executes the fieldwork and gathers **sufficient, appropriate, and relevant** evidence from documents, logs, interviews, observations, or testing results. All work is documented carefully so the findings can be supported later, and the integrity of the evidence itself must be confirmed — for instance by taking checksums of exported files so the evidence cannot have been tampered with.

PHP2Shell — The auditor collects the patch-management export with installed PHP versions, the emergency change tickets, the triage log, and the vendors' written exposure statements, then runs or witnesses a version check on the sampled hosts and requests file-integrity alerts for new web shells. The fieldwork surfaces that one host still shows PHP 8.4.26 after the fixed 8.4.27 release, while another host appears in no inventory export at all.

Evaluating Results

The auditor compares the evidence collected with the audit criteria and identifies the **gaps** between what should happen and what is actually happening. This is where findings begin to take shape, as the auditor interprets whether the evidence supports compliance, effectiveness, or a control weakness.

PHP2Shell — Most sampled hosts were patched within 7 days, but 3 of the 25 were not, including one internet-facing server. Two vendor-hosted systems have no exposure statement, and one application runs PHP 8.1 with no available fix. The evidence shows that the process **exists but is not consistently operating**, especially for assets outside the central inventory.

Reporting

The auditor summarizes the findings, conclusions, and recommendations in a clear report that explains what was examined, what was found, why it matters, and what should improve. A strong report turns audit work into action by helping management understand priorities for remediation. Each finding is expressed through a fixed chain.



PHP2Shell — A typical recommendation is to complete the PHP inventory (including vendor-hosted systems), upgrade or isolate unsupported branches, obtain vendor attestations, and add automated version scanning. Every recommendation is assigned an **owner** and a **deadline** (for example, 30 days for the unsupported system) so that remediation can be tracked and validated. While judgment is unavoidable in reaching a conclusion, during the audit it is fundamental that every conclusion remains supported by evidence.

Audit Methods, Principles, and Challenges

12. Audit Methods

Once the process is understood, the auditor needs a repertoire of **evaluation methods**, chosen according to the kind of objective and the kind of evidence being sought.

Method	Purpose	Examples
Review of records	Collect and examine what is already documented	Analysis of records, logs, reports, configuration files; verification of credentials and history
Feedback	Gauge how performance is perceived	Surveys, questionnaires, references, complaints, performance evaluations, peer review
Interview	Verify information and test knowledge through prepared questions	Structured personal interviews, sometimes recorded
Observation	Watch control execution directly, against a well-defined scope and metrics	Witnessed audits, on-the-job observation, role playing
Testing	Evaluate behaviour and controls in practice	Penetration testing, oral and written exams
Post-audit review	Review prior audit work and outcomes	Review of earlier reports, used in continuous auditing to confirm ongoing compliance

Methods also vary by the **location** of the auditor (on-site or remote) and the **extent of human interaction** (interactive or not). The trade-off is between directness and effort.

	On-site	Remote
Human interaction	Interviews, checklists and questionnaires with the auditee, document review with the auditee, sampling	Interactive communication: interviews, observing work with a remote guide, checklists, document review
No human interaction	Document review and data analysis, observing work performed, on-site visits, sampling of products	Document review and data analysis, observing via surveillance means

Methods that involve people give a more direct and less biased collection of evidence, but they are time-consuming; document and log review scales better but is heavier to process, while an interview delivers a faster, more immediate result. The central difficulty of human-based methods is that the people interviewed are usually not auditors, so the auditor must **translate** what a stakeholder claims into the precise language of auditing. This borrows techniques from human–computer interaction and stakeholder analysis, combining field study, interviews, remote feedback, and data analysis.

13. Audit Principles

An audit is only as trustworthy as the principles observed while performing it. If any of these are violated at the end of the report, the evaluation may not be reliable. Six principles govern audit quality: **integrity and ethical conduct**, **independence and objectivity**, **due professional care and competence**, an **evidence-based approach**, **confidentiality and discretion**, and **professional skepticism and judgment**.

Assurance

Assurance is the independent and objective evaluation that provides confidence to stakeholders about the reliability, integrity, and effectiveness of an organization's processes, controls, or information. It is both an **engagement** and its **outcome**: the auditor must be confident throughout the work, and the final report must be effective, leaving no unanswered question. Its defining features are that it is **independent** (performed by parties not involved in the operations), **objective** (evidence-based and free from bias), **evidence-driven** (every conclusion is tied to verifiable data), and focused on **purpose** (anchored to the defined objectives; pursuing side concerns without a clear rationale undermines it).

Risk Management vs Assurance

Risk management **creates** confidence; assurance **validates** that confidence. Risk management identifies, assesses, and mitigates threats, producing the organization's view of its exposure; assurance then provides independent evidence that the risk-assessment process is actually effective, rather than resting on the assessor's intuition.

	Risk Management	Assurance (Internal Audit)
Objective	Identify, assess, and mitigate threats to achieving objectives	Provide independent evaluation of how risks and controls are managed
Responsibility	Management and operational staff	Internal audit and other assurance providers
Nature of work	Proactive and continuous	Reactive and periodic — on time, *during* the process, not before or after
Focus	Managing risk exposure and implementing controls	Testing, verifying, and reporting on control effectiveness
Output	Risk registers, mitigation plans, risk-appetite statements	Audit reports, assurance opinions, recommendations
Key question	"Are we managing our risks effectively?"	"Is our risk management effective?"

Level of Assurance

Assurance comes in three levels along a common scale:

1. **Reasonable assurance** — high but not absolute confidence, based on sampling and sufficient testing.

2. **Limited assurance** — moderate confidence, typical of review engagements and of methods such as interviews, where the auditor cannot be fully certain of what a stakeholder reports.
3. **No assurance** — purely descriptive or advisory observations.

Assurance does not mean a positive result. It represents confidence based on evidence, whether the findings are good or bad. When an audit reveals weaknesses or non-compliance, it still provides assurance — it assures stakeholders that the controls do **not** meet the expected standards. The value of assurance lies in transparency and reliability, not in favourable outcomes, so the **quality of the security process** and the **assurance about that process** must never be conflated: they are distinct concepts.

Ethics and Quality

Auditors must uphold ethical codes emphasizing confidentiality, objectivity, and competence, handling all information with appropriate expertise. Ethical lapses undermine trust, damage professional credibility, and may invalidate the assurance engagement. Quality-assurance mechanisms — internal reviews, peer evaluations, and external assessments against recognized quality standards — ensure conformity with professional standards, and continuous improvement reinforces the reliability and relevance of the audit function.

Independence and Objectivity

Auditors must remain independent in **fact** and in **appearance**, avoiding conflicts of interest that could bias judgment. Objectivity means making balanced assessments based on facts, not on loyalty, assumptions, or personal preference. Independence does **not** require the auditor to be external to the organization: an internal auditor is objective when the work is evidence-based, and independent when no one audits their own work. The one who runs a penetration test cannot also judge the quality of that test, because the result would be biased; another person inside the organization can perform that evaluation. This principle protects the audit's credibility and prevents auditors from auditing work they are responsible for managing.

Confidentiality and Discretion

Auditors routinely handle sensitive operational, financial, and security information, so they must protect it carefully. Confidentiality means not disclosing information without proper authorization, unless there is a legal or professional duty to do so; in practice this is enforced through non-disclosure agreements — by contract for external auditors and through employment terms for internal ones. Discretion also sustains the trust between auditors and stakeholders that makes open evidence collection possible.

Professional Skepticism and Judgment

Professional skepticism means keeping a questioning mind and critically assessing evidence, especially when something looks incomplete, inconsistent, or too convenient. A practical technique is to keep asking *why* until no further question remains, because the auditor then

holds all the arguments needed to support the evaluation — which is also why practitioners are required to stay certified and up to date. **Professional judgment** is the ability to interpret evidence, weigh alternatives, and decide what is material and what is not. A concrete case: when a threat-intelligence vendor claims no large-scale exploitation of PHP2Shell, skepticism means not relying on unverified media reporting to reach a conclusion; verifiable sources are acceptable, but a media claim must not shift the assessment. Together these traits stop auditors from accepting explanations too quickly.

Evidence-Based Approach

Audit evidence must be **sufficient** (in quantity) and **appropriate** (in quality) to support conclusions, which must rest on verifiable data and documented testing rather than intuition or managerial claims. Reliable evidence strengthens the credibility of the auditor's opinion. Evidence is gathered through several means:

- **Documentation and records** — policies, logs, reports, and configuration files.
- **Observation of processes** — direct review of control execution in real time.
- **Interviews and inquiries** — discussions with process owners to corroborate evidence.
- **Analytical procedures** — data comparisons, trend analysis, and correlation testing.
- **Re-performance of controls** — independent repetition of an activity to confirm effectiveness.

The collection and verification of information follows a defined sequence.



14. Challenges in Modern Auditing

Modern auditing faces new complexities that make the discipline harder. The volume of data is **massive and heterogeneous**: logs may hold millions of records, and even with log-inspection systems it can be impractical for a team to analyse everything, so the auditor may review only the most critical hosts to estimate the broader state. This is why **sampling** is unavoidable — but any sample must have a clear rationale justifying why that subset was chosen.

Two further pressures compound the problem. Cybersecurity threats and cloud environments are **constantly evolving**, and the integration of **AI and analytics** into control testing changes how evidence is produced and judged. There is also the problem of **evidence freshness**: evidence collected on one day may not reflect the state on another, which matters sharply around a disclosure event — an audit run the day before PHP2Shell was disclosed would not capture the new exposure, and budget or operational constraints may prevent an immediate re-audit. This is the reason audits are periodic and governed by defined timing.

The discipline adapts through continuous auditing, data-driven testing, and risk intelligence. In essence, auditing in cybersecurity translates traditional assurance principles to **digital**

evidence.

15. Consolidation Scenario — Auditing PHP2Shell

The lecture's reasoning comes together in a single scenario. PHP2Shell is two critical flaws (CVSS 9.2) with fixes released on 21 October 2026; the organization has 120 PHP hosts and a policy to apply critical fixes within 7 days. Six questions, one per stage of the process, test the reasoning.

Planning — which scope best supports the objective "critical PHP vulnerabilities are fixed within policy"? The correct scope is **inventory, advisory triage, patching, vendor-hosted systems, and detection**. Firewall rules alone do not tell you the vulnerabilities are patched and ignore hosts missing from the inventory and the vulnerability-management rules; scoping to the text of the security policy alone is useless, because it leaves the entire judgment to the human and tests nothing.

Understanding — fixes exist for 8.5, 8.4, 8.3, and 8.2 but not for 8.1 and older; why must the auditor know this before testing? Because it is itself a **risk**: some hosts cannot be updated, and that risk must be evaluated. It also differentiates compliance from non-compliance, since an organization may remain compliant through a **mitigation** — isolating a host it cannot patch — so the auditor must distinguish "update" from "isolate" as the appropriate response.

Designing tests — you can examine 25 of the 120 hosts; how do you choose? Select by **exposure and criticality**: internet-facing hosts are more critical than internal ones, a judgment that comes from the asset inventory and the risk assessment, which already flagged the high-risk hosts. Add 5 **random** hosts to avoid bias and to surface side-effect weaknesses the risk-based sample might miss, such as corrupted PHP files running in the background.

Evaluating — 3 of 25 sampled hosts were unpatched, one of them internet-facing; is the control effective, partially effective, or ineffective? Partially effective. It cannot be fully effective, because three hosts (including an internet-facing one) are unpatched; it is not ineffective, because that would require essentially all hosts to be unpatched. The process exists but does not operate consistently, and the partial effectiveness should be quantified.

Skepticism — sources disagree on in-the-wild exploitation (a honeypot recorded working shells; a vendor reported none). Does this change your risk rating? No. Missing or conflicting media evidence does not lower a risk that has already been assessed; an absence of confirmed exploitation is not evidence of safety, and unverified news must not move the assessment.

Independence — the team that applies the patches also produces the compliance report; what is the problem, and which line should re-perform the check? The team is **biased**: having applied the patches, it will attest that they are correct, which makes its self-

check worthless. The re-performance must come from an independent party — the **third line** — rather than from the team that found or fixed the vulnerability.

Risk-based Auditing

Risk-based auditing concentrates audit effort on the areas where the **probability** and **impact** of failure are greatest. By prioritizing high-risk domains, components, and subsystems, it ensures efficient resource allocation, handles the most dangerous areas first, and prepares the organization for catastrophic events. Its defining activity is producing evidence for computing the likelihood and impact of the risks an organization faces, aligning audit scope with organizational objectives and emphasizing proactive assurance over reactive verification.

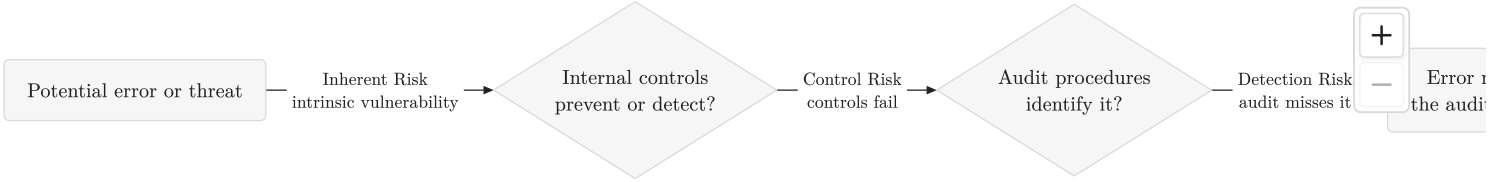
The Audit Risk Model

The base definition of risk is **likelihood** $\times$ **impact**. Audit risk refines this into three interacting components whose product is the overall audit risk.

$$\text{Audit Risk (AR)} = \text{Inherent Risk (IR)} \times \text{Control Risk (CR)} \times \text{Detection Risk (DR)}$$

- **Inherent Risk** — the risk that an error or irregularity occurs in the **absence** of internal controls. It reflects the intrinsic vulnerability of a process or system, such as the baseline cyber risk of exposed software.
- **Control Risk** — the risk that existing internal controls fail to **prevent or detect** such an error. It depends on the design and operational effectiveness of the safeguards, and rises when controls are poorly designed or misinterpreted.
- **Detection Risk** — the risk that the **audit procedures** themselves fail to identify the remaining error. It is the risk that the audit is planned or executed badly.

These components form a cascade, which is why audit risk is their product rather than detection risk alone: each layer depends on the previous one, and looking only at the audit procedures gives partial information.



Inherent and control risks are properties the organization manages. Detection risk is the auditor's to manage, and it is **inversely related to the level of assurance**: lower detection risk means higher confidence. It is controlled through **sample size**, **test depth**, and **corroborating evidence**, and is influenced by auditor competence, automation tools, and time constraints. Balancing detection risk against inherent and control risks keeps overall audit risk at an acceptable level. When inherent and control risks are high, detection risk must be reduced by applying more extensive or rigorous audit procedures.

Materiality

Materiality is the **threshold** beyond which an error or control weakness becomes significant enough to influence decisions or audit conclusions. A risk **materializes** when it actually occurs, so materiality marks the point at which a risk starts becoming an incident. Judging materiality has three aspects:

- **Quantitative** — the measurable impact or size of the error.
- **Qualitative** — the sensitivity or nature of the issue, which may not be quantifiable but still carries impact.
- **Contextual** — dependence on risk appetite, regulatory expectations, and which policies are violated.

Even a single critical vulnerability can be material if it threatens core systems or sensitive data, and even a low-likelihood event such as a privileged-access breach may be material if its potential impact is critical.

Materiality interacts directly with the audit risk model. Higher inherent or control risk, or a limited detection capability (higher detection risk), requires a **lower materiality threshold** and expanded testing. Lowering the threshold in practice means protecting the system so that the probability of the risk materializing stays low, which in turn reduces the risk. The following scale guides the judgment.

Materiality judgment	Quantitative impact	Qualitative factors	Possible audit response
Likely immaterial	Small	Non-production system; no sensitive data	Document and monitor remediation
Usually immaterial	Low	Limited effect on detection capability	Recommend correction
Material or significant deficiency	Moderate	Indicates a systemic control deficiency	Test the full access-governance process
Material	Potentially high	Core system; exploitable weakness; regulatory implications	Immediate escalation and expanded testing
Potentially material	Difficult to quantify	May prevent timely detection and regulatory reporting	Review incident-response readiness and conduct scenario testing

Assertions

Assertions are **claims made by management** about data, processes, or systems. They make materiality judgments more objective by giving the auditor a defined statement to test against. The common categories are:

- **Existence** — the item or control actually exists.
- **Completeness** — all relevant elements are included; for example, all hosts in a subnet run a vulnerability-management agent.
- **Accuracy** — information is recorded correctly; for example, the detection rate stays below a defined threshold.
- **Authorization** — actions are properly approved; for example, certain privileges are required on certain hosts.
- **Presentation** — data are appropriately disclosed or formatted.

Auditors design tests to verify each assertion objectively.

Materiality — PHP2Shell Example

Consider an internet-facing server still running a vulnerable PHP version. The **quantitative dimension** is high: exploitation yields root access and therefore full control, and because the host runs web applications, many customers are affected. The **qualitative dimension** is severe: the weakness is exploitable without authentication, so by anyone, and exploitation attempts were actually observed, since a threat-intelligence vendor noticed some exploitation while denying worldwide spread. The **likelihood** is moderate-to-high because exploitation is easy and has been attempted, and the **potential impact** is critical because unauthenticated remote code execution grants the highest privilege.

The **materiality judgment** is therefore *material*: even without proof of compromise, an attacker needs only one anonymous request to take full control of a server holding customer data. The scenario sits exactly on the materiality threshold, between a risk and an incident. The appropriate **audit response** is to escalate, verify with logs and file-integrity checks, expand the testing of patching, and recheck the inventory and detection procedures — the reasoning behind temporarily blocking exposed internal web interfaces during such an event.

The COSO Model

Origin and Purpose

COSO stands for the **Committee of Sponsoring Organizations of the Treadway Commission**, a private U.S.-based initiative established in 1985 to improve the quality of financial reporting and internal control. In 1992 it released the Internal Control–Integrated Framework, later expanded to Enterprise Risk Management (2004, 2017). Although it originates in the financial domain, it is now the standard model for **internal auditing** across general IT, because it is designed to analyze the internal controls of an organization. This is also why internal and external auditors may use different frameworks: models like COSO are built for internal use, while international and regulatory standards serve external auditing.

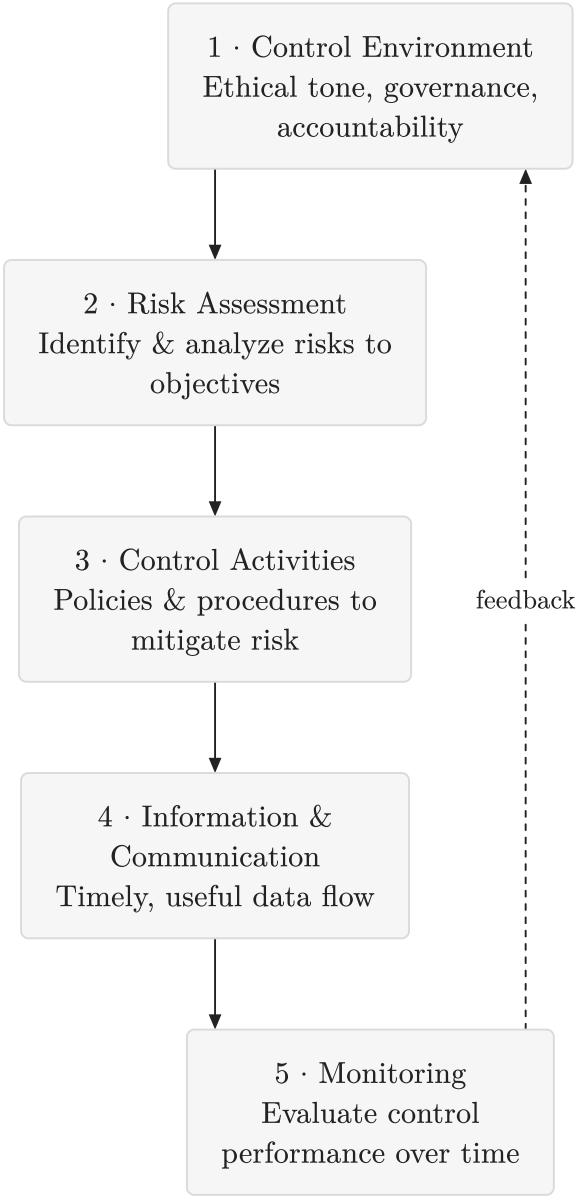
The model defines the processes, policies, and procedures that ensure objectives are achieved effectively, efficiently, and ethically. Its purpose is to protect assets, ensure reliable reporting, and support legal and policy compliance, and it must be ongoing and embedded in daily

activities rather than applied once. In the COSO view, internal control is *a process effected by the board, management, and staff to provide reasonable assurance on operations, reporting, and compliance.* It is studied because it is the global foundation for internal auditing and provides a common procedure that applies across every audit domain and type.

The Five Components

COSO defines five interrelated components of internal control:

1. **Control Environment** — ethical tone, governance, and accountability.
2. **Risk Assessment** — identifying and evaluating risks to objectives.
3. **Control Activities** — policies and procedures mitigating those risks.
4. **Information & Communication** — ensuring reliable, timely data flow.
5. **Monitoring** — evaluating control performance over time.



The components operate as one integrated system, so a weakness in an upper layer is reflected in the layers below. If risk assessment fails to identify ransomware as a major threat, management may never implement adequate backup, access-control, or recovery procedures. From the audit perspective, the auditor evaluates whether each component is **present**,

functioning, and **operating together** effectively, and effective internal control yields reasonable assurance over operations, reporting, and compliance.

Control Environment

The control environment is the foundation of the model. It reflects the organization's ethical values, governance structure, management philosophy, and assignment of authority and responsibility, setting the **tone at the top** that makes internal control credible. It is here that management decides whether the audit will be less formal — a check on small components — or more formal, structured, and in-depth, and here that the credibility of the final evaluation is established. Because it is the foundation, a weakness at this point propagates into every other layer. A strong control environment shows a clear code of conduct, active board oversight, and defined accountability for security and compliance.

Risk Assessment

Risk assessment is the process of identifying and analyzing the risks that could prevent the organization from achieving its objectives, based on the controls selected. It is dynamic and must be continuously updated to reflect changes in the business, the technology environment, and external threats. A typical example is identifying the risk of unauthorized access to a sensitive system and estimating its likelihood and impact.

Control Activities

Control activities are the policies and procedures that help ensure management directives are carried out, and they come **after** risk assessment. They include approvals, authorizations, reconciliations, segregation of duties, and other preventive or detective measures. A representative example is requiring managerial approval before privileged access is granted, and reviewing that access periodically. Because they depend on the prior phase, controls cannot be chosen well if the risks were not assessed well.

Objectives of Internal Control and the CIA Triad

Control activities serve three objectives, which map onto the cybersecurity CIA triad. COSO defines *what* the organization must achieve, while the CIA triad defines *how* IT controls make it possible; the mapping must be made explicitly when applying COSO so that both the business and the security views are preserved.

COSO objective	Focus	Supporting CIA objective	Example controls
Operations	Efficiency, effectiveness, and continuity of business processes	**Availability** — systems and data accessible when needed	Backup schedules, failover systems, uptime monitoring

COSO objective	Focus	Supporting CIA objective	Example controls
Reporting	Reliability and integrity of financial and non-financial data	**Integrity** — accuracy and completeness of information	Input validation, audit trails, checksum verification
Compliance	Adherence to laws, regulations, and policies	**Confidentiality** — protect sensitive information from unauthorized access	Access control, encryption, segregation of duties

Control Types

COSO classifies controls along two dimensions. By **purpose**, which reflects the timeframe of an attack — before, during, or after:

- **Preventive** — stop errors or incidents before they occur: least privilege, multifactor authentication, firewalls, network segmentation, input validation.
- **Detective** — identify events, deviations, or failures as they occur or afterward: log monitoring, SIEM alerts, intrusion-detection systems, vulnerability scans, periodic access reviews.
- **Corrective** — contain the impact, eliminate the cause, and restore operations after an event: patch deployment, account revocation, malware quarantine, incident-response procedures, backup restoration.

By **execution**:

- **Manual** — performed by people: manager approval of privileged access, manual log review, periodic control self-assessment.
- **Automated** — performed consistently by systems: automatic account lockout, firewall-rule enforcement, real-time alerting, automated patch deployment.
- **Hybrid** — combine automated execution with human judgment: a SIEM detects anomalous activity while an analyst validates and investigates it.

Control	Purpose	Execution
Role-based access control	Preventive	Automated
Manager approval for administrator access	Preventive	Manual
SIEM alert for unusual administrator activity	Detective	Automated
Analyst investigation of the alert	Detective	Manual
Disable the account and rotate credentials	Corrective	Hybrid

Information and Communication

This component ensures that relevant information is identified, captured, and communicated in a timely and useful form, so people understand their control responsibilities and can make informed decisions. The communication format must be agreed **before** events occur, so that everyone involved knows how to communicate with each other, and timeliness is critical because prompt decision-making is essential at the end of the audit. Examples include security incident reports, risk dashboards, and policy updates shared with managers and staff.

Monitoring Activities

Monitoring assesses whether internal controls continue to work **over time**, through ongoing monitoring, separate evaluations, or both. The organization must establish procedures, systems, or tools that keep this analysis continuous rather than performed only once or twice, which requires monitoring what actually happens. Examples include periodic internal audits, control self-assessments, and follow-up reviews of remediation actions.

Sampling in Auditing

Testing every transaction or record is impractical — logs may hold millions of records — so **sampling** takes a subset of the available evidence. It can be done two ways:

- **Statistical sampling** — based on a probability distribution. A uniform distribution avoids bias; a normal (Gaussian) distribution fits data that follows it, drawing on statistics such as the mean, median, or maximum. When analyzing risk, the worst-case scenario corresponds to taking the maximum.
- **Judgmental (non-statistical) sampling** — selecting high-risk or more-exposed entities first, such as all internet-facing hosts in PHP2Shell, since these are the likely entry points.

Sampling must balance efficiency with assurance, keep the evidence sufficient and appropriate, use risk as the priority guide for what to sample, and preserve professional judgment without losing reliability.

COSO Model in PHP2Shell

The model's components map directly onto the failures in the scenario.

Which COSO component failed?

- *"Nobody knew which systems ran PHP"* → the **control environment**, where no formal ownership or assignment existed, or **risk assessment**, where the risk was never identified.
- *"Emergency patching required no approval or testing record"* → **control activities**: an emergency-change pipeline must be prepared beforehand and used only in emergencies, and here that formal process was missing.
- *"The advisory reached IT but not the business owners"* → an **information and communication** failure.
- *"Patch compliance was never reviewed after the policy was written"* → **monitoring**.

Classifying the controls by purpose and execution: automatic PHP updates are **preventive and automated**; a firewall rule blocking the exploit pattern is **preventive** and can be automated, manual, or hybrid depending on how the rules are managed; an alert on a new web-shell file is **detective** (real-time) and typically **automated**, since intrusion detection is now largely automated; isolating and rebuilding a compromised server is **corrective** and **hybrid**, because it involves many sub-activities, only some of which are automated.

Which COSO objective and CIA property is most threatened? All of them. For a critical event like this, confidentiality, integrity, and availability are all attacked, and all three COSO objectives — operations, reporting, and compliance — are at risk. Worst-case reasoning is the guiding principle: even when the objectives are not all impacted to the same degree, every one of them is affected.

Auditing Frameworks

This part of auditing concerns how the work is **reported**. The process framework itself — ISO 19011 — was already established; the two frameworks here, **ISAE** and the **SOC** suite, focus on reporting the results of control audits. Because all risk-based frameworks are interconnected, auditors must map them to one another and to the audit process rather than treating any one in isolation.

ISO 19011 and Control Frameworks

ISO 19011:2018 — Guidelines for Auditing Management Systems is the standard guideline for designing audit management systems, covering the management of an audit programme, the conduct of an audit, and the competence and evaluation of auditors. It is the backbone of the audit process.

Control frameworks translate governance principles into **structured control sets** and promote consistency, comparability, and accountability across organizations. Frameworks such as **COSO, COBIT, ISO 27001, and NIST CSF** help auditors benchmark practices and align with regulatory expectations. COSO integrates objectives, components, and principles into a holistic approach adaptable to both financial and IT domains, and it provides the conceptual foundation for later frameworks like COBIT. Within this set, the ISAE standards occupy the place dedicated to assurance reporting.

International Standard on Assurance Engagements (ISAE)

ISAE is a category of **professional standards** issued by the International Auditing and Assurance Standards Board (IAASB), part of the International Federation of Accountants (IFAC). It comprises several documents on assurance and supports a structured, evidence-based conclusion on whether a subject matter meets defined criteria. Born in the financial domain, it now applies to both financial and non-financial assurance engagements, including internal controls, outsourced services, cybersecurity, sustainability information, and compliance

reporting. It requires the practitioner to plan the engagement, assess risks, obtain sufficient appropriate evidence, evaluate identified exceptions, and issue an assurance report. Two standards are central:

- **ISAE 3000** — the general, principles-based standard for assurance engagements other than audits or reviews of historical financial information.
- **ISAE 3402** — assurance reporting on controls at a **third-party service organization** relevant to customers' internal control over financial reporting.

ISAE 3402

ISAE 3402 defines how an **independent auditor** evaluates and reports on the design and operating effectiveness of controls at a service organization. Its purpose is to provide assurance to **user entities** and their auditors that outsourced processes are reliable and well-controlled. It applies to services that impact financial reporting — payroll, accounting, or data-hosting for ERP systems — and its scope is the controls at third-party providers relevant to financial reporting. This introduces the relationship between an organization and the third-party services it depends on, and the question of how to audit the risks those services carry. It distinguishes two report types.

	ISAE 3402 Type 1	**ISAE 3402 Type 2**
Scope	Description and design of controls	Description, design, and operating effectiveness of controls
Time focus	A specified **point in time**	A specified **period** of time
Core question	*Are* the controls suitably designed and implemented as of the report date?	*Were* the controls suitably designed and implemented, and did they operate effectively throughout the period?
Testing of operation	No testing of consistent operation over time	Yes — the auditor tests control performance, usually using samples
Level of assurance	Shows the control framework exists and is appropriately designed	Provides stronger evidence that controls worked consistently in practice
Typical use	New service, newly implemented controls, or initial readiness assessment	Ongoing assurance requested by customers and their auditors

Each report includes a **description** of the service organization's system, management's written **assertion** about the system and controls, and the service auditor's reasonable-assurance **opinion**. For a Type 2 report, it also includes the **tests of controls** performed and their results, including relevant exceptions. Assertions play a central role, because they state objectively what the provider guarantees and what the user is entitled to expect. A Type 1 report is used when the context changes — a new server, asset, or control enters the inventory

— while a Type 2 report is used after an incident or over a significant period, for continuous monitoring across a fixed window.

AICPA SOC Reports

The **AICPA** (American Institute of Certified Public Accountants), founded in 1887, is the U.S. professional body of CPAs and sets U.S. auditing and ethical standards. **SOC** (System and Organization Controls) is a suite of CPA service offerings covering the system-level controls of a service organization or the entity-level controls of other organizations. SOC reports provide independent evidence that relevant controls have been evaluated, support third-party risk management and vendor due diligence, and reduce the need for every customer to perform a separate full-scope audit of the provider. They help customers understand control responsibilities, identified exceptions, and remaining risks, but they do **not** certify that an organization is completely secure: the assurance is limited to the defined system, criteria, period, and controls.

SOC report	Focus	Equivalent / basis
SOC 1	Controls relevant to financial reporting	U.S. equivalent of ISAE 3402
SOC 2	Security, availability, processing integrity, confidentiality, and/or privacy	Cybersecurity controls, akin to the risk-based frameworks
SOC 3	High-level, general-use assurance	Based on the Trust Services Criteria

SOC 2 and SOC 3 introduce the **CUEC (Complementary User Entity Control)** — a control that the service provider (the vendor) expects the **customer** to implement so that the vendor's own controls can work as designed. These controls are, in effect, the assertions the vendor relies on to deliver the agreed service level. ISAE 3402 and SOC 1 answer a financial-reporting question, while SOC 2 and SOC 3 answer a technology-and-trust question, so the appropriate report depends on the risks and decisions of the report users.

Why ISAE 3402 and SOC Reports Matter

The goal of both is to give a **customer-independent**, evidence-based assurance that a service provider's relevant controls are properly designed and actually operated over a period, reducing the need for every customer to audit the same provider separately. As organizations increasingly rely on third-party providers for critical processes, these reports give confidence that providers maintain documented and effective controls, preserve the integrity, confidentiality, and availability of data, and support financial-reporting reliability and regulatory compliance. User entities can leverage the reports instead of duplicating control testing on their suppliers. The outcome is transparency, reduced audit duplication, and trusted outsourcing.

Mapping Frameworks and the Report's Structure

Auditors frequently **cross-reference** controls between frameworks, for example mapping NIST CSF to ISO 27001 Annex A or COBIT. These mappings ensure comprehensive coverage, reduce redundancy, and enhance interoperability across compliance regimes, drawing on a diverse toolkit of policy and documentation review, walkthroughs and interviews, sampling and re-performance, configuration inspection, data analytics, and observation of control operation.

A real ISAE 3402 or SOC 2 report exposes the same concepts used throughout the audit process — control objectives, control tests, and the results of those tests, as in a Type 2 report. The report is formalized into defined fields, decided as part of information and communication, while the content within each field reflects the auditor's expertise. A complete report contains:

- The auditor's and management's **responsibilities**.
- The **quality-assurance assessment**.
- The scope **limitations**.
- The auditor's **opinion**.
- The **discussion of tested controls** and their results.
- The **intended users and purpose**.
- The **signatures**.

A SOC 2 report adds the independent service auditor's report, management's assertion, the system description, and the information provided by the service provider, where the assertion page explains what the provider expects the user to do to uphold the contract.

Audit Report in PHP2Shell

The reporting concepts come together in a final scenario around PHP2Shell (CVE-2026-109618 / 109619), the critical PHP flaw whose fixes shipped on 21 October 2026 with CISA urging immediate patching.

A Type 1 report dated 30 June 2026 — does it tell you anything about PHP2Shell? No. It is a point-in-time snapshot from a date before disclosure, and it does not cover a period, so it cannot speak to a vulnerability disclosed months later.

The vendor's SOC 2 report covers "the hosting platform" — does it cover the PHP runtime? It depends on what "hosting platform" includes. If the platform bundles the web framework and PHP runtime, such as a managed WordPress host, it may; if it is a bare server with no runtime unless one is installed, it does not. The scope must be checked.

The report lists "customer must apply security updates" as a CUEC — what does that mean for you? Responsibility for applying those updates falls on the customer, so as auditor you must verify the logs, the patching, and that the relevant controls and objectives are actually met. This is why assertions matter: they establish who is responsible for what.

The report notes two exceptions in change management — should you still rely on it? Possibly, depending on whether those exceptions affect the controls you depend on. If they touch controls relevant to your risk, the reliance must be reconsidered.

Your vendor sends a clean Type 2 SOC 2 report dated last month, and PHP2Shell is disclosed today — what do you do, and what can the report tell you? The report covers a period that ended before disclosure, so it cannot tell you anything about PHP2Shell. It may confirm that a PHP runtime existed, but a new server could have been added since, so you must re-run the audit cycle and request fresh, current evidence.

Case Study 1 — MediStore and HealthCloud

Context and Objective

MediStore is an online pharmacy. It owns 8 PHP servers, including 3 internet-facing systems that handle customer prescriptions and payment data, and it also uses a prescription-management portal hosted by **HealthCloud**, a managed healthcare IT provider. In the wake of PHP2Shell, HealthCloud sends MediStore an exposure statement claiming that *all hosted systems used by MediStore are patched and are no longer vulnerable to PHP2Shell.* The audit objective is narrow and practical: MediStore must decide whether this claim can be **relied upon**.

The engagement splits cleanly across the three lines. The **first line** builds the inventory of MediStore's own systems and the five HealthCloud-hosted systems and prepares the remediation plan for MediStore-owned servers. The **second line** performs the compliance assessment, noting that the hosted systems depend on HealthCloud's remediation. The **third line** produces the audit report, based on an independent, simulated version check of the three MediStore-owned internet-facing servers. The hosted systems themselves sit with an external stakeholder, so assurance for them must be requested from HealthCloud.

The Hosted Systems Under Review

System	PHP version	Internet-facing	Patch owner
RX-CART-01	8.4.26	Yes	HealthCloud
RX-CART-02	8.4.27	Yes	HealthCloud
RX-SEARCH-01	8.3.35	Yes	HealthCloud
RX-API-01	8.2.34	No	HealthCloud
RX-REPORT-01	8.1.9	No	HealthCloud

The Assurance Request and the Exposure Statement

MediStore's audit team sends HealthCloud a formal request to confirm the **PHP version** and **patch status** of the five hosted systems, asking specifically for patch records and remediation

dates as evidence, with a deadline of 13 November 2026. HealthCloud replies with its exposure statement — *all hosted systems have been patched and are no longer vulnerable to PHP2Shell* — supported by a single piece of evidence: a patch-management export showing the five systems marked as "patched."

This is the point at which professional skepticism applies. The claim is a management **assertion**, and the supporting evidence is the vendor's own export; neither is independent confirmation that the running software is actually fixed.

MediStore's Independent Audit

MediStore's internal audit team tests HealthCloud's claim against an independent fact sheet of the actual running versions. The patch export says "patched," but the runtime record shows older versions still in place, and no upgrade or isolation evidence is supplied for the systems that cannot simply be patched.

System	Claim in exposure statement	Fact sheet	Independent simulated check	Conclusion
RX-CART-01	Patched	PHP 8.4.26; fix 8.4.27 available	PHP 8.4.26	**Not patched**
RX-CART-02	Patched	PHP 8.4.27	PHP 8.4.27	Patched
RX-SEARCH-01	Patched	PHP 8.3.35; fix 8.3.36 available	PHP 8.3.35	**Not patched**
RX-API-01	Patched	PHP 8.2.34; fix 8.2.35 available	PHP 8.2.34	**Not patched**
RX-REPORT-01	Patched	PHP 8.1.9; no fix available	PHP 8.1.9	**Unsupported version**

Only RX-CART-02 genuinely meets the requirement. Three systems still run a version below the available fix, and RX-REPORT-01 runs an unsupported branch with no fix at all, so it would need an **upgrade or isolation** rather than a patch — a mitigation for which no evidence was provided.

Audit Conclusion

HealthCloud's statement is not supported by the evidence, and three systems remain vulnerable. The finding is expressed through the standard reporting chain.

- **Criterion** — Critical PHP vulnerabilities must be remediated within 7 days of the fix release.
- **Condition** — HealthCloud states that all hosted systems are patched, but three systems (RX-CART-01, RX-SEARCH-01, and RX-API-01) still run vulnerable PHP versions.
- **Cause** — The exposure statement was never reconciled with the actual PHP versions of the hosted systems.
- **Effect** — MediStore cannot rely on HealthCloud's assurance, and internet-facing systems may remain exposed to unauthenticated remote code execution.
- **Recommendation** — HealthCloud should correct the exposure statement, complete remediation, and provide **timestamped evidence** of the running PHP versions.

The Challenge to the Vendor

MediStore replies to HealthCloud with a formal challenge: the independent review of the fact sheet shows that RX-CART-01, RX-SEARCH-01, and RX-API-01 still run vulnerable PHP versions, so HealthCloud must supply a corrected exposure statement, remediation dates, and timestamped evidence of the running PHP versions for each hosted system. Until that evidence arrives, MediStore cannot rely on the claim that all hosted systems are patched.

The case demonstrates the reporting frameworks in action. A vendor's assertion is not assurance until it is independently verified against evidence; patching a hosted system is the vendor's responsibility, but confirming it is a **complementary user entity control** that falls on the customer; and a claim marked "patched" in an export means nothing if the runtime record contradicts it. Evidence, not the statement, is what justifies reliance.

Case Study 2 — Audit of the Student Information System

Context, Objective, and Scope

A university relies on a **Student Information System (SIS)** to manage exams, grades, student records, and faculty access. It is used by administrative staff, professors, IT personnel, and external support vendors. The audit objective is to assess whether the key **IT General Controls (ITGCs)** are designed and operating effectively to protect the confidentiality, integrity, and availability of the SIS. The scope covers three ITGC domains — **access management**, **change management**, and **IT operations** — using selected evidence from the last six months.

What Happened

Four recent events frame the audit, each pointing to a different control domain.

- A former contractor account was still active **two months after** contract termination — an access-management concern.

- A professor reported that one exam grade was changed **without clear traceability** — a change-management (and access) concern.
- A scheduled backup **failed twice** last month, but no escalation ticket was opened — an IT-operations concern.
- Two urgent production fixes were deployed **directly on Friday evening** — a change-management concern.

Controls to Be Tested

Each domain has a defined set of controls the auditor must examine.

- **Access management** — Joiner–Mover–Leaver process; role-based access control and least privilege; MFA for privileged access; periodic review of admin accounts.
- **Change management** — approval before deployment; testing evidence; rollback plan; traceability in version control.
- **IT operations** — backup monitoring; incident escalation; patch and vulnerability tracking; logging and alerting.

Evidence Available to the Auditor

Each piece of evidence relates to a control, carries a different weight, and suggests a follow-up question.

Evidence	Relates to	Strength	Follow-up question
Admin account list showing 14 privileged accounts	Access management	Weak–moderate (a snapshot only)	Are all 14 approved and owned?
HR exit form for a contractor who left in April	Access management	Strong (authoritative record)	Was access revoked on that date?
Quarterly access-review spreadsheet with 3 blank owner fields	Access management	Moderate (shows a review gap)	Who owns those accounts?
Git log showing two direct commits to production	Change management	Strong (objective, immutable)	Were they approved and tested?
Change-ticket register with one emergency fix lacking approval	Change management	Strong	Why was there no approval?
Backup dashboard showing two failed jobs marked "warning"	IT operations	Strong	Why was no escalation raised?

Evidence	Relates to	Strength	Follow-up question
SIEM extract showing login activity from the old contractor account	Access management	Strong (shows actual misuse)	When, from where, and what actions?

Findings

Three findings emerge, one per domain.

Finding 1 — Access management. A former contractor retained privileged access after termination, and periodic access-review records show incomplete ownership for several admin accounts. This indicates that the Joiner–Mover–Leaver process and the privileged-access review control are not operating effectively.

Finding 2 — Change management. Two production changes were deployed without complete approval and testing evidence. This weakens traceability and increases the risk of unauthorized or unstable changes in production.

Finding 3 — IT operations. Backup failures were detected, but incident escalation did not occur. This suggests a monitoring weakness and may reduce service recoverability in the event of disruption.

Reporting

Each finding is reported through the Criterion → Condition → Cause → Effect → Recommendation chain. For the access-management finding:

- **Criterion** — Privileged accounts must be reviewed quarterly and deprovisioned promptly after termination.
- **Condition** — One contractor account remained active after departure, and three privileged accounts had no recorded owner.
- **Cause** — Access-review and offboarding processes were not consistently enforced.
- **Effect** — Increased risk of unauthorized access and a lack of accountability.
- **Recommendation** — Enforce automated deprovisioning triggers, assign account owners, and require documented quarterly review evidence.

Analysis

The access-management finding is the **most material**: it combines a control weakness with a SIEM extract showing the terminated account was actually used, so both likelihood and impact are high, and it touches confidentiality, integrity, and availability of student records at once. The **strongest evidence** is the objective, tamper-resistant kind — the SIEM login extract, the Git log, and the change-ticket register — rather than the point-in-time admin-account snapshot. The overall control environment is best rated **partially effective**: the

controls exist across all three domains but do not operate consistently. Management should remediate the **active contractor account** first, because it is the one weakness with confirmed exploitation.

Takeaways

Auditing is not just compliance checking. It is a systematic, independent, and evidence-based evaluation that produces assurance about whether controls, processes, and governance mechanisms actually work as intended. In security governance, audit creates **digital trust** by turning technical checks into accountable, risk-aware assurance for management, boards, regulators, and other stakeholders.

The most important structural idea is the **separation of roles**: management owns risk and controls, while internal audit provides independent assurance within the three-lines model. A good audit then follows a disciplined process — planning, understanding the system, assessing risk, designing tests, collecting evidence, evaluating results, and reporting findings with recommendations. Its quality rests on core professional principles such as integrity, independence, due care, confidentiality, skepticism, and above all an evidence-based approach, because findings must be traceable and defensible.

Modern auditing is increasingly **risk-based**, using inherent risk, control risk, detection risk, and materiality to guide scope and testing depth. In cybersecurity and IT audit, the practical focus is on control frameworks and real domains, where evidence usually comes from logs, configurations, tickets, dashboards, and technical artifacts rather than documents alone.

Key References

The material rests on a recognized body of standards: **ISO 19011:2018** (guidelines for auditing management systems), **COBIT 2019** (governance and management of enterprise IT), **ISO/IEC 27001:2022** (information security management systems), **NIST Cybersecurity Framework 2.0** and **NIST SP 800-53 Rev. 5** (risk management and security/privacy controls), **ISAE 3402 / SOC 2** (assurance reports on controls at a service organization), **ISACA IT Audit Fundamentals** (3rd ed., 2023), the **GDPR Regulation (EU) 2016/679**, **INTOSAI GOV 9140** (internal control standards for the public sector), and the **COSO** internal-control framework.

CHAPTER 5: Threat Modelling

What Threat Modelling Is

Threat modelling is a structured activity that identifies, characterises, and organises the threats that may arise inside a system. It abstracts away from the specific operational domain and groups concrete problems into general classes, so that each class can then be reasoned about, quantified, and mitigated. It is a *modelling* activity: nothing is attacked or tested in practice. The system is examined, its details are abstracted, and recurring problems that may

surface in different places are clustered together. The method mirrors conceptual data modelling — a system is described, the description is analysed, and a model is produced — except that here the model captures potential security problems rather than data structures.

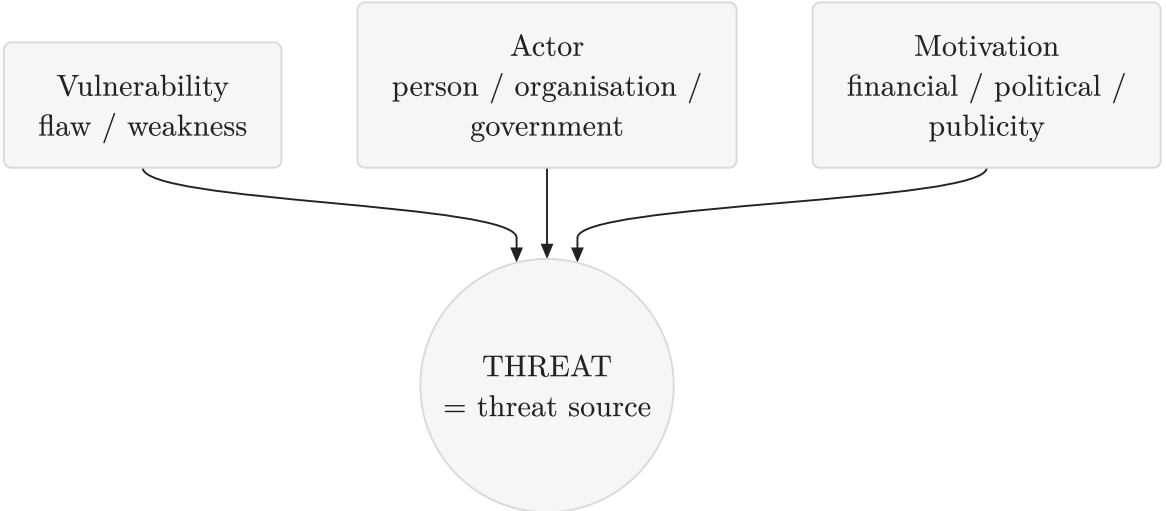
The activity sits downstream of vulnerability assessment. Vulnerabilities can be discovered with largely automated instruments: network and infrastructure scanning reveals exposed devices and open services, and standardised procedures supply additional information about each weakness. Threat analysis cannot be automated in the same way, because it requires reasoning about what an attacker wants and how a given vulnerability could be made to materialise into an incident. It therefore proceeds through structured approaches applied at different levels of granularity, and its results feed back into the risk evaluation.

Threat modelling supports risk management, but its usefulness is broader. It serves four purposes:

- **Strengthen protection (hardening).** The results can drive improvement of defensive barriers regardless of the level of risk a threat generates — the system is hardened because a threat exists, not only because the risk is high.
- **Improve preparedness.** When a system cannot be fully defended, the analysis guides mitigation and prepares a response plan for threats that may still materialise.
- **Increase awareness.** It conveys that a designed system carries latent weaknesses that must be addressed.
- **Support risk management.** High-quality threat information refines the estimation of likelihood and impact.

Threats, Vulnerabilities, and Risk

A threat is the joint view that connects three ingredients: a **vulnerability** (a flaw or weakness that can be exploited), an **actor** (a person, organisation, or government with the intention to act), and a **motivation** (financial, political, or reputational). A threat is the materialisation of a strategy by a specific actor, exploiting specific vulnerabilities, driven by a specific motivation. The point at which all three overlap is the **threat source**.



Because threat modelling clarifies all three ingredients, it enables a **three-factor risk model**. Threats come from the model, their probability follows from the existence of vulnerabilities, and impact is the third component:

$$\text{RISK} = \text{THREAT} \times \text{PROBABILITY} \times \text{IMPACT}$$

A well-executed threat analysis therefore raises the quality of the whole risk assessment, because each factor rests on solid information rather than guesswork.

The Four Questions

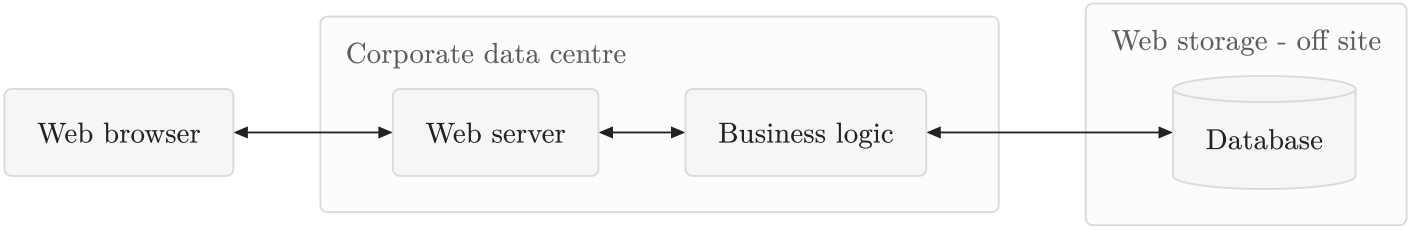
A threat model is built by answering four questions in order. The first two scope and populate the model; the last two turn it into action and quality control.

1. **What are you building?** Identify the context: the core functionalities the system must provide and the interactions between its parts. Because the subject is a cyber system made of hardware and software, this means the physical components, the applications running on them, and the services offered to users.
2. **What can go wrong?** Enumerate the ways the system may fail to meet its security requirements. A threat is simply a way in which things may go wrong.
3. **What should you do about it?** Choose mitigations. The available options depend on *when* the analysis is performed (see below).
4. **Did you do a decent job of analysis?** There is no guarantee the analysis is perfect. If the identified threats are good enough for the intended purpose, the work is complete; otherwise it is complemented with other models or with additional information about the system.

The third question is governed by timing within the development life cycle. Performed during design, mitigations are built into the system before deployment and cost little. Performed afterwards, when the system is already operational, the analysis becomes a reverse-engineering exercise: the components are already deployed, so mitigation requires costly recovery or redevelopment of parts that are already in place.

What Are You Building?

The first question is answered by documenting and designing the system, exactly as in ordinary software design: diagrams capture the main components and their interactions at a chosen level of granularity. A common high-level representation of a web-based application decomposes it into layers connected in a chain — a browser interacting with a web server, which drives the business logic, which reads and writes the data layer (the database). Documenting this makes it possible to reason about the assets, the environment, and the **trust boundaries**: the points that separate what can be trusted from where an adversary may come.



What Can Go Wrong? The STRIDE Mnemonic

STRIDE is a mnemonic introduced by Microsoft at the end of the 1990s that collects six general categories of threat worth considering in any software system. Each category is checked against the system diagram, component by component and interaction by interaction.

- **Spoofing:** pretending to be something or someone you are not — an entity impersonates another, typically where no authentication or identification mechanism exists. It threatens authenticity.
- **Tampering:** modifying something you are not supposed to modify, threatening integrity. It may be a software bug that produces unexpected output (a function that should add but subtracts), or data altered in transit over an unprotected channel. It can affect packets on the wire or wireless link, bits on disk, or bits in memory.
- **Repudiation:** claiming you did not do something, regardless of whether you did. It arises when an activity cannot be linked to an entity, so the threat is countered by accountability and traceability.
- **Information Disclosure:** exposing information to people not authorised to see it, threatening confidentiality.
- **Denial of Service:** attacks that harm availability — crashing the system, making it unusably slow, or filling its storage.
- **Elevation of Privilege:** gaining the technical ability to do things beyond your permitted role.

Worked example: STRIDE on the web application

Applying STRIDE to the three-layer diagram means attaching candidate threats to each component and each interaction:

- **Web browser → Spoofing, Information Disclosure.** With no authentication or access-control mechanism, anyone can access the system and impersonate another user.
- **Database → Tampering.** Without database access control, other users can alter the stored records.
- **Browser ↔ web server channel → Information Disclosure, Tampering.** If the network channel is not encrypted, anyone can read the data in transit and an attacker can manipulate it.

The timing effect is visible here. Discovered during design, the channel threat is fixed trivially by specifying an encrypted protocol — HTTPS instead of HTTP. Discovered once the system

is operational, the same fix requires rewriting deployed code. This is why the same exercise yields different criticalities depending on where in the development life cycle it is run.

Tips for Identifying Threats

Three working principles guide the exercise:

- **Start with external entities.** Begin from the parts of the system that are outside your control — the "doors" separating the core from the external world, through which external attackers try to enter. Protecting the boundaries well makes protecting the interior easier.
- **Never ignore a threat because it is not what you are looking for right now.** During design, implementation details are unknown, so a threat that does not yet exist is still *potential*. Consider the worst case, because preparing for it removes issues that implementation would otherwise introduce.
- **Focus on feasible threats.** A well-detailed, reasonably designed system produces very many threats. Do not inflate the list for the sake of being conservative: if a threat is not feasible in the given context, discard it, because the feasible threats alone already demand management.

Strategies for Threat Modeling

Once the system is documented, the search for threats begins. There is no single strategy that works for every purpose: the choice depends on the goal of the model and the granularity required. Threat models are classified into three main variants, though a given model may fit several patterns depending on how it is used.

- **Asset-centric:** the analysis starts from the assets that carry value.
- **Attacker-centric:** the analysis starts from the adversary and what the adversary wants.
- **Software-centric:** the analysis starts from the software components that make up the system.

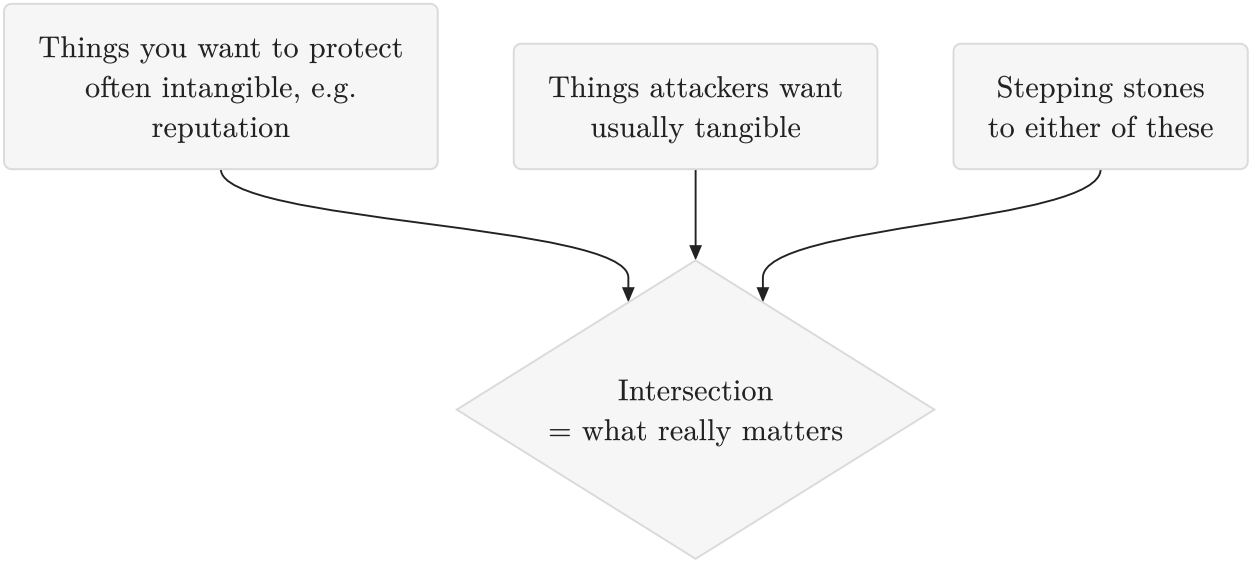
The three approaches reach the same elements from different directions, and in practice they are often combined.

Asset-Centric Threat Modeling

Asset-centric modelling puts the emphasis on assets. Starting from the system representation, the elements that constitute assets for the company are marked, and the analysis expands outward from each asset to the threats against it.

The central difficulty is a mismatch of perspective. Assets are usually identified from the company's point of view, but what has value for the company is not necessarily what the attacker wants, and the attacker may want something whose value the company underestimates. Three sets must therefore be distinguished: the things you want to protect, the things the attacker wants, and the stepping stones that lead to either. What truly deserves

analysis lies in the intersection. Spending effort on something valuable to the company that no one wants to take is wasted, and something the attacker may want but that carries no value for the company can simply be left to the attacker. Because resources are limited and not everything can be protected, the intersection is where attention is concentrated.



Implementing asset-centric modelling

The approach proceeds in three steps, and all three must be completed — stopping early leaves the analysis too high-level to be useful.

1. **List the assets and, for each, the protection needs and the attacker's objectives.** This produces a table that breaks down every asset into what must be protected and why an attacker might act against those protection needs.
2. **Build a dependency model** that connects each high-level asset to the concrete components behind it. An asset is often an abstract, intangible concept, and the threats against it can only be understood once it is mapped onto where it physically resides and how it is accessed.
3. **Draw the system** — assets, supporting components, and their interconnections — until you can tell a complete story about how the asset lives inside the company.

Only at that point can STRIDE, an attacker-centred brainstorm, or another model be applied, because only then is there enough structure to attach threats to.

Two dependency examples make the second step concrete. Company **data** is a typical asset, yet it is intangible and spread across several databases; the asset "data" must be mapped to the stores that hold it and the paths that reach them before any threat against it can be assessed. A **booking service** offered to customers — for instance a travel company's flight-booking site — is an asset because the company stops earning when it is down; but "booking service" is a high-level concept that resolves into software components (Java classes, for example) running on networked machines. Cutting the network cable stops the service, a threat invisible to anyone who looks only at the service and not at the hidden dependency between how it is deployed and how it is used from outside.

Attacker-Centric Threat Modeling

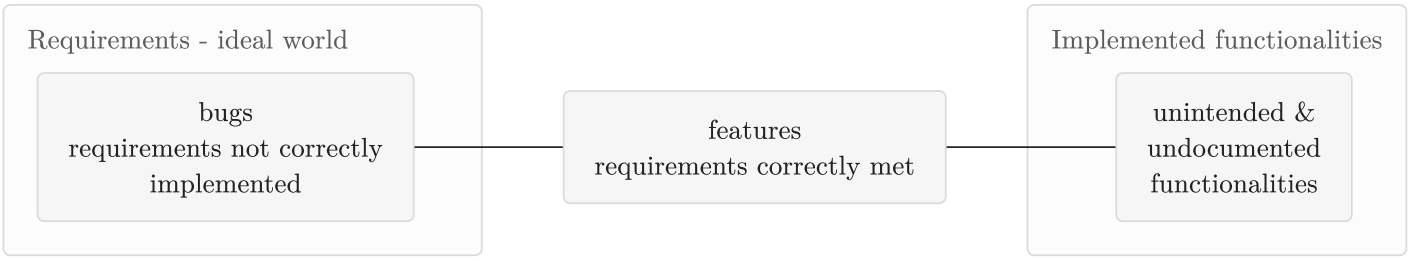
Attacker-centric modelling reverses the direction of reasoning. Instead of starting from assets and enlarging the view toward possible attackers, it starts from the adversaries and focuses inward on what they would want from the system. Lists of attacker types provide structure for a brainstorming approach, and because the analysis is driven by human agents, it naturally surfaces human-centred possibilities that a component-by-component scan would miss.

The approach has no administrative or rule-based procedure to follow. It is carried out through interviews — talking to people to understand the important points and which attackers typically target this kind of system — and is informed by issues occurring in the same domain and by prevailing trends. Its weakness is intrinsic: a defender is not the attacker and cannot know exactly what the attacker wants, so placing oneself in the attacker's position is difficult. For this reason attacker-centric analysis is used as a confirmatory complement to the other approaches rather than as the primary method for finding threats.

Software-Centric Threat Modeling

Software-centric modelling puts the emphasis on the software components running in the system. It presupposes a decomposition of the system into those components, and it begins from the **requirements**, which define the ideal world: exactly what the system is supposed to do and what it is not supposed to do.

In the ideal world there is a one-to-one correspondence between requirements and implemented functionalities — every requirement is covered, and nothing is implemented that the requirements did not specify. In the real world the correspondence breaks down, and the gap is where security issues arise.



Two kinds of mismatch appear. **Bugs** are requirements that the implementation fails to satisfy correctly. **Unintended and undocumented functionalities** are behaviours the system performs that no requirement asked for — extra capabilities added because they seemed useful, which introduce behaviour nobody specified and therefore nobody controls. Both are dangerous: the security-relevant problems grow precisely from the difference between what the system is supposed to do and what it really does. Testing and static analysis can shrink the undocumented region and expose some bugs, but they can never fully close the gap, because no technique guarantees the complete reality of the code.

Software is also more than the application code written in-house. It runs on an operating system, communicates over a network, and depends on firmware managing the physical

resources. Problems can arise from the interaction of the code with the operating system, with the network, and with other software, and all of these interactions must be considered.

What should you do?

The first obligation is to describe and document the system. Design documentation that may seem tedious to produce — naming components, defining dependencies — is reused later for security assessment, so doing it carefully pays off. In practice this documentation is rarely found in real company projects, which means a security expert must be able to reconstruct it before any assessment can proceed.

Iterating across the development life cycle

Ideally threat models are created during system design, before deployment. In practice they are often created for existing systems, which makes threat modelling part of maintenance. System designers with security experience are best equipped to identify the threats.

The strongest practice is to repeat the exercise at every stage of the development life cycle, each pass adding information that the previous one lacked:



- **Architecture and design:** the main components and their interactions are known, so a first assessment removes whole sets of threats — identifying an access-control issue leads to placing an access-control service.
- **Construction (development):** concrete implementation detail appears; a secure channel specified at design time may turn out to use weak cryptography that can be broken.
- **Testing:** the differences between requirements and implementation are examined and bugs are found and fixed.
- **Deployment:** operational facts emerge — co-located components that depend on one another can produce a domino failure and an availability problem.
- **Maintenance:** the exercise continues as the system evolves.

This reflects a historical shift. A decade or two ago security was treated as a wish rather than a requirement: systems were deployed first and firewalls and intrusion detection were bolted on afterwards. Today security within the development life cycle is mandatory. The activity is anticipated to the requirements phase by writing explicit **security requirements** that state the measures the system will adopt, which then feed directly into the risk analysis. The same requirement-driven threats are checked again at each later stage — code review during construction, security testing, penetration testing — so good work at the beginning makes the rest of development easier. Performed only once the system is in operation, the model is still valid for risk assessment but can no longer shape the security requirements.

These security activities run alongside the ordinary engineering activities at each phase:

Phase	Core activities	Security activities
Planning	—	—
Requirements & Analysis	Functional / non-functional / technical requirements	Security objectives
Architecture & Design	Design guidelines, architecture and design review	Threat modeling, architectural risk analysis
Development	Unit tests, code review, daily builds	Security code review
Testing	Integrated testing, system testing	Security testing
Deployment	Deployment review	Penetration testing
Maintenance	—	Risk assessment

Software-centric targets

Software-centric reasoning also organises threats by the layer they strike, from the network inward to the application:

Layer	Example threats
Network	Packet spoofing
Host	Buffer overflows, illicit paths
Application	SQL injection, XSS, input tampering

Data Flow Diagrams

Modelling the Software

Before threats can be derived, the software must be represented in a formalism that exposes its components and their interactions. Several instruments serve this purpose at different levels of refinement. **Flow charts** work at a high, outline level, showing interaction between functions, but they are less commonly used and make it difficult to overlay threats. **Data Flow Diagrams (DFDs)** are more refined: hierarchical in nature, focused on data input, and the representation favoured in Microsoft environments. **UML diagrams** go further into the design of each individual component, using object-oriented and component diagramming familiar to developers.

DFDs are the model to use together with STRIDE, because their focus is exactly the three things STRIDE examines: data, functionalities, and the flows between components. They deliberately contain few element types. UML, by contrast, details the internal design of each component. The two are inter-mappable — a DFD can be translated into a UML diagram and the reverse — so a system described in one formalism can be expressed in the other. DFDs are worth knowing because they are widely used in organisations built on the Microsoft stack,

which covers a large share of companies and most public administrations running Azure, Office, Teams, and SharePoint.

A Data Flow Diagram is a graphical representation of the flow of data through an information system. It is a conceptual model that moves a system from a textual description to a visual one centred on components and their interactions, and it can be drawn with dedicated tools or by hand. It visualises what data enters the system, what data it produces as output, the processing steps, and the data-persistence activities. Its value for threat modelling is the high-level view it gives of how an information system manages the assets that need protecting, together with its hierarchical character: it starts from a small number of components and refines them one by one through increasingly detailed diagrams.

DFD Elements

A DFD uses four graphical element types: external entities, data flows, processes, and data stores.

External Entity (Terminator)

An external entity has a role in the system but is not managed directly by the system itself. It may be a physical actor or an external software system, and it maps onto the **actor** of a UML use-case diagram — an entity that starts an interaction. It is drawn as a named rectangle.

- Can be duplicated on the diagram one or more times to avoid crossing lines when it interacts with several parts of the system.
- Is not part of the system being studied. If an element starts to require analysis, it is probably a process rather than an external entity, because mattering to the analysis implies belonging to the system; conversely, a part you do not care about is a candidate external entity.
- May belong to the same organisation but lie beyond the system's influence — "external" does not mean third-party. An internal payment unit managed by another department and an external service such as PayPal are both external entities for the sake of the analysis.
- Is something over which there is no direct control; it must be accounted for but cannot be changed.
- Must have at least one connection, since an external entity exists on the diagram only because it exchanges data with an internal element (a process or a data store); one with no connection is removed as useless.
- Is placed on the edges of the diagram, around the boundary.

The governing rule is that a DFD **never includes a direct data flow from one external entity to another**. Such entities are irrelevant to the system being described because they are external — like a conversation between two strangers, which is irrelevant to you — and the diagram must not be overloaded with information that is meaningless for the analysis.

Data Flow

A data flow represents data moving between elements of the system. It is drawn as a directed, named arrow, the label identifying the type of communication. A data flow may be a physical network communication or a function call between two software modules, depending on the level of granularity; there is a data flow wherever information moves from one part to another. Communication in two directions is drawn as two separate arrows — one from A to B and one from B to A — because each typically carries a different type of data.

- **Rule 2: only represent data, not material goods.** The diagram captures the movement of information, not physical movement of things.
- **Rule 3: only one type of data per arrow.** Each arrow carries a single named kind of data, such as "Orders" or "Customer Data". Parallel flows are kept separate even when they run alongside each other, because different data have different protection requirements — some may be freely disclosed while others must be protected.
- A **fork** means the same data goes to two destinations; the same data arriving from several locations can likewise be **joined**.

Process

A process is an abstract component that does something — the real application logic, usually representing a functionality of the system. Processes can be modelled coarse-grained or fine-grained depending on what is to be assessed, and the DFD's layered structure allows starting general and refining later.

- **Rule 4: a process must have at least one data flow in and one data flow out.**
- Named with a verb and the object of the verb — the thing being processed — for example "Calculate (verb) average income (object)".
- Each process should represent only one function or action.
- Processes must be univocally numbered; breaking down a composite process `x` generates sub-processes numbered `x.y`.

A process drawn with a single circling line is fine-grained. A process drawn with a **double circling line is composite**: it aggregates several functions and can be broken down into sub-processes connected by flows, its internal structure detailed at a lower DFD level. This parallels a class diagram, where one object can be expanded into the detailed definition of its sub-elements.



The composite process on the left is a **Level 0 DFD**; its decomposition into numbered sub-processes on the right is a **Level 1 DFD**.

Data Store

A data store is where data is stored — databases, files, directories, and similar formats representing information. It is drawn as an open-ended rectangle with a name.

- The name is usually the plural form of the data flowed into it.
- Can be duplicated one or more times to avoid crossing lines.
- Can be shared by two or more systems.
- Its contents are detailed elsewhere, in a data dictionary.

Trust Boundaries

A trust boundary, drawn as a dotted line, marks the division between diagram elements that can be trusted and those that cannot. A boundary is crossed either because an element is external, or because, although internal, it is reached by a flow that cannot be trusted. It represents the limits in the trustability of subparts of the system.

A remote-maintenance scenario illustrates it. A system split between a central side and a customer site has two components that are both trusted, but they are connected by an external network. A dotted line drawn around the network component expresses that everything traversing the public internet is untrusted, while the two endpoints remain trusted.

Forbidden DFD design choices

Three constructions are always wrong, because each conceals a missing process:

- **No direct data flow between two data stores.** Data stores are passive; a flow between two of them implies a procedure that mediates it, which must be drawn as a process in the middle.
- **No direct data flow from an external entity to a data store.** A mediating process internal to the system must sit between them.
- **No direct data flow between two external entities.**

Building a DFD

A DFD is constructed top-down, refining the system level by level while preserving its external interface:

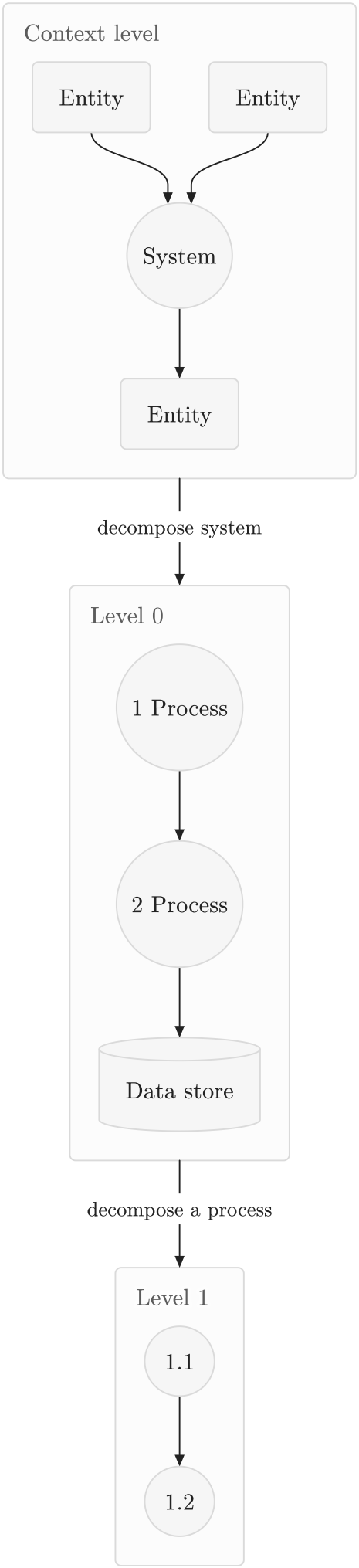
1. **Identify actions and actors from the use cases** — this is the requirement analysis.
2. **Build the context-level DFD,** separating the external entities from the system.
3. **Build the Level-0 DFD,** identifying the processes inside the system.
4. **Build the Level-1 DFD,** breaking down processes, identifying inter-process data flows, and identifying data stores.
5. **Add trust boundaries.**

The **context-level DFD** contains only one composite process, which can be called "System" or given a more meaningful name. Its purpose is to focus on the external entities — which

actors can interact with the system — and the data flows between each external entity and the process. It is a star-like diagram, built mainly from the use cases, and it does not yet analyse the system; it fixes the interactions between the core of the analysis and the outside world so that later decomposition stays clean.

The **Level-0 DFD** opens the box and breaks the system down, identifying the main business processes, connecting them with appropriate data flows, and possibly including data stores at a high level of abstraction.

Lower-level DFDs further detail specific composite processes through **functional decomposition** — an iterative process of breaking a system description into finer and finer detail across a series of increasingly detailed diagrams. This decomposition obeys **balancing**: the conservation of inputs and outputs when a process is decomposed to a lower level. The input and output data flows of the parent DFD must be maintained on the child DFD. Choosing how far to decompose is a trade-off between the detail represented and the purpose of the analysis, exactly as a class diagram may stay high-level or descend to the interaction between individual functions.

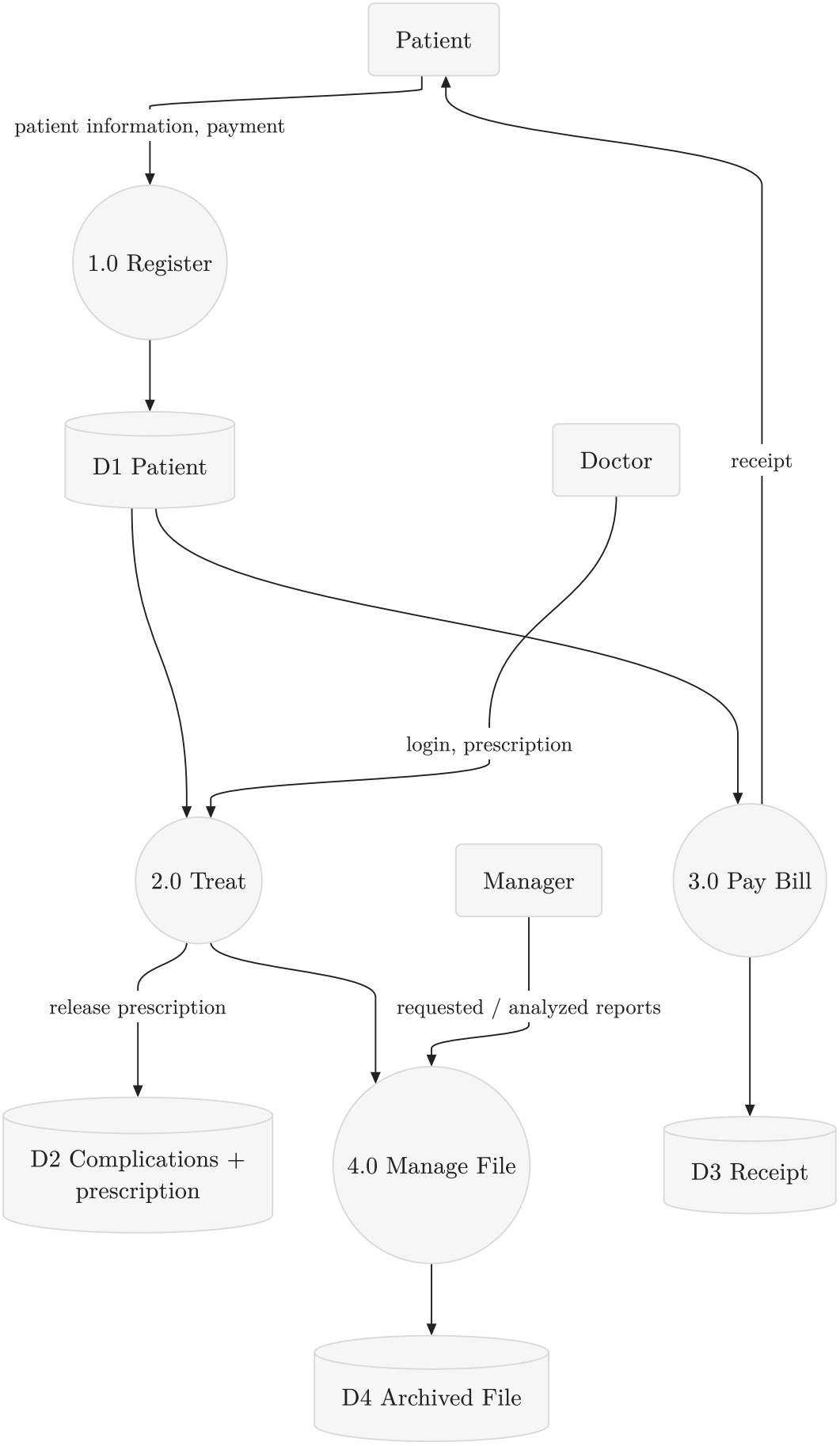


Worked Example: Dental Clinic Transaction System

The first example models the software a dentist uses to manage appointments and patients.

The **context-level and Level-0 diagrams** place a single central system among the external entities that interact with it. A **patient** sends personal information, receives a receipt, and pays. A **doctor** logs in with username and password, inserts prescriptions (the receipt taken to the pharmacy), retrieves patient information, and sees which doctors are logged in. Further external entities — **manager** (typically the software administrator who checks the system), **accounting**, **owner**, **assistant**, and **supplier** — each hold a role in the interaction. The analysis focuses on the central software system and models it down to the Level-0 reaction.

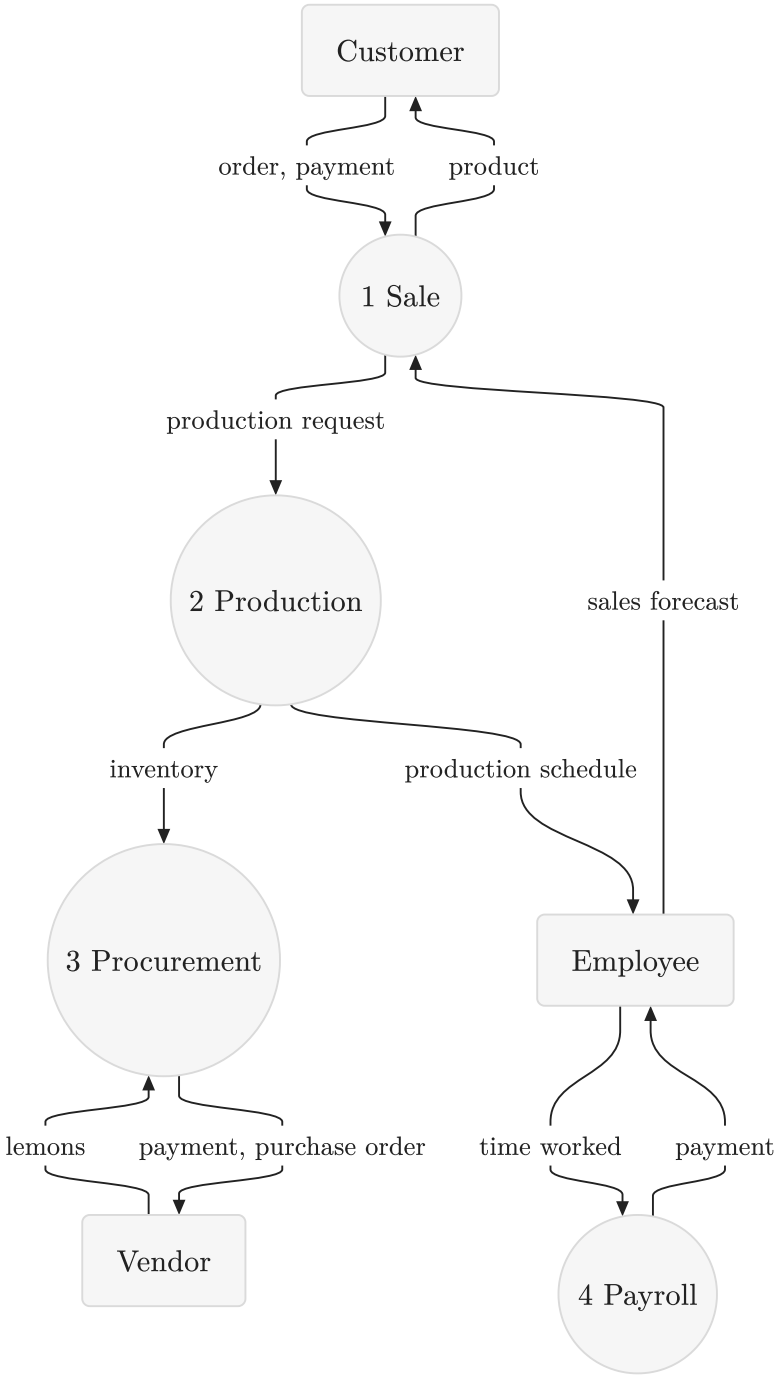
The **Level-0 decomposition** expresses what the system does as functionalities. A registration process handles patient data and feeds a patient **archive** data store, accessed through a file-management functionality offering the usual database operations (update, store, delete). A treatment process manages patient care, and a payment process handles billing. The design deliberately uses two components wherever data is persisted — the process (application logic, the DBMS interface) and the data store it manages. Three data stores are kept separate: patient records, receipts, and patient complications, so that **personal data and clinical data are not mixed**.



Worked Example: Lemonade Stand

The second example models the software system that controls the provisioning of a lemonade (vending) machine — comparable to a coffee machine, serving customers, interacting with vendors, and requiring production of the product.

Its **context diagram** mirrors the dental one: a single system surrounded by external entities — Customer, Employee, and Vendor — exchanging orders, products, payments, purchase orders, time worked, and sales forecasts. The **Level-0 DFD** separates the main functions of the business into sale, production, and procurement — the typical phases of a production system, with payroll added. The **Level-1 DFD** then refines one function; for the order part, "Record order", "Receive payment", and "Produce sales forecast" become numbered sub-processes exchanging data with Customers, Orders, and Payments stores.

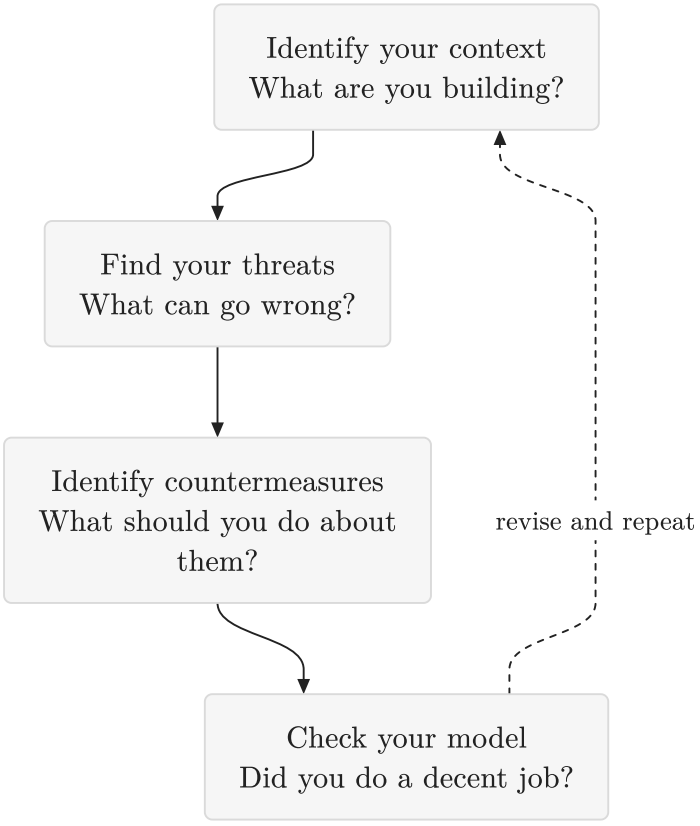


The DFD is the starting point of the whole method. It produces the formal representation — DFD or UML — that must exist before threats are extracted, and the next stage applies a methodology such as STRIDE to this diagram to derive the list of threats.

CHAPTER 6: STRIDE & Attack Trees

Overview

Threat modelling proceeds by answering four questions in order. The first, *what are we building?*, fixes the context and is answered by a model of the system — for software-centric work, a data-flow diagram (DFD) that names the processes, data stores, data flows, external entities, and trust boundaries. The second, *what can go wrong?*, enumerates the threats against that model and is the subject of this chapter. The third, *what should we do about the things that can go wrong?*, selects countermeasures, and the fourth, *did we do a decent job?*, checks the analysis for completeness. The output of the second step is a list of threats that feeds risk assessment, drives the early selection of countermeasures during design, and gives the team situational awareness before any cost–benefit trade-off is made.



This step can be applied at any stage of the system's life cycle. Running it during design catches threats early, when they are cheapest to fix; the analysis is then repeated after implementation to confirm that the threats believed solved are in fact solved and that development introduced no new ones. Threat modelling is therefore iterative across the life cycle, not a single pass.

---

1. STRIDE

1.1 STRIDE is elicitation, not modelling

STRIDE was proposed by Microsoft at the end of the 1990s as a framework aimed primarily at software-centric analysis: its threat definitions and categories are tuned to how a system is implemented and deployed. When STRIDE was introduced in 1999, security was rarely a design requirement — the dominant concerns were scalability and performance for newly networked, internet-facing services — so the technique was in practice applied late, once a

system was already in production. Current practice pushes it to the design phase, where it is run iteratively alongside development.

STRIDE falls under the umbrella of threat modelling, but it is not itself a modelling technique; it is a threat **elicitation** (enumeration) technique. The distinction matters. The model is not produced by STRIDE — the model is the DFD, and STRIDE supplies a fixed categorization of threats. The analyst does not invent a model; they take the system's DFD and, for each STRIDE category, list the concrete threat instances that could occur. Because the categorization is given and only the instances are discovered, the activity is elicitation rather than modelling.

STRIDE is an acronym for the six categories of threat relevant to software systems: **S**poofing, **T**ampering, **R**epudiation, **I**nformation disclosure, **D**enial of service, and **E**levation of privilege.

1.2 The classification schema

For each category STRIDE specifies three things: the **security property** the threat violates, the **typical victims** — the DFD element types the threat usually applies to — and a **taxonomy of concrete examples** to prompt the analyst. Linking each threat class to a violated property is what ties the analysis back to security requirements.

Threat	Violated property	Definition	Typical victims
Spoofing	Authentication	Pretending to be something or someone other than yourself	Processes, external entities
Tampering	Integrity	Unauthorized modification of data on disk, on a network, or in memory	Data stores, data flows, processes
Repudiation	Non-repudiation	Claiming you didn't do something or weren't responsible (honestly or falsely)	Processes
Information disclosure	Confidentiality	Providing information to someone not authorized to see it	Processes, data stores, data flows
Denial of service	Availability	Absorbing the resources needed to provide service	Processes, data stores, data flows
Elevation of privilege	Authorization	Doing something you're not authorized to do	Processes

The "typical victims" are not absolute rules but starting points: the analysis begins from the elements a category usually affects and may later consider the others. The reasoning behind the typical victims recurs throughout. A **data flow** is information in motion and cannot itself be impersonated, so it is not a spoofing victim; a **data store** is a passive component whose access is mediated by a process, so the active logic (the process) and the external entities are

the sides that get impersonated. Integrity, confidentiality, and availability threats, by contrast, bear on data wherever it lives — at rest in a store, in transit on a flow, or while handled by a process — so those three categories list all data-bearing elements as victims.

1.3 Spoofing

Spoofing violates **authentication**: an entity impersonates another, pretending to be something or someone it is not. The typical victims are processes and external entities, because impersonation happens where two sides interact. The example taxonomy descends from the general class to concrete sub-cases.

Threat example	What the attacker does	Notes
Spoofing a process on the same machine	Creates a file before the real process does	Output picked up as if from the genuine process
	Renaming / linking	Creating a Trojan `su` and altering the path
	Renaming	Naming your process `sshd`
Spoofing a file	Creates a file in the local directory	Can be a library, executable, or config file
	Creates a link and changes it	The swap should occur between the link being checked and being accessed (a time-of-check/time-of-use race)
	Creates many files in the expected directory	Automation makes it easy to create 10,000 files in `/tmp` named `/tmp/"pid.NNNN` or similar
Spoofing a machine	ARP spoofing, IP spoofing, DNS spoofing (forward or reverse)	
	DNS compromise	Compromise the TLD, registrar, or DNS operator
	IP redirection	At the switch or router level
Spoofing a person	Sets the e-mail display name; takes over a real account	The classic vector behind phishing
Spoofing a role	Declares themselves to be that role	Impersonating the *figure* (e.g. administrator), not a specific person

Two sub-cases are worth separating. Spoofing a **person** impersonates a specific legitimate user so the application believes it is interacting with the real individual — the mechanism underlying phishing. Spoofing a **role** impersonates not the person but the privileges attached to a role, such as presenting oneself as a system administrator to gain what that figure may do.

1.4 Tampering

Tampering violates **integrity**: the concern is that every modification of the system be authorized and confined to what the actor's permissions allow. Because integrity bears on data — the essence of an information system — it can affect every element: data stores (passive stores of data, the primary integrity target), data flows (data corrupted in transit), and processes (where data is generated and manipulated).

Threat example	What the attacker does	Notes
Tampering with a file	Modifies a file they own and on which you rely	
	Modifies a file you own	
	Modifies a file on a file server that you own	
	Modifies a file on *their* file server	Relevant when you include files or XML schemas from remote domains
	Modifies links or redirects	Redirects a victim to a fake application
Tampering with memory	Modifies your code	Hard to defend once the attacker runs code as the same user
	Modifies data they've supplied to your API	Pass by value, not by reference, when crossing a trust boundary
Tampering with a network	Redirects the flow of data to their machine	Often stage 1 of tampering
	Modifies data flowing over the network	Easier and more effective on wireless (Wi-Fi, mobile); unencrypted data in transit can be altered freely
	Enhances spoofing attacks	

1.5 Repudiation

Repudiation violates **non-repudiation**: an actor claims not to have performed an action for which they were responsible. The claim may be honest or false, and it often sits at the business level rather than the technical one — confirming a transaction that never happened, or denying one that did. The key question for the designer is *what evidence do you have?* Repudiation threats are fundamentally about the ability to audit and log actions; every action that goes unlogged is a potential repudiation issue, because unattributable actions cannot be pinned to anyone.

Threat example	What the attacker does	Notes
Repudiating an action	Claims to have not clicked	Maybe they really didn't
	Claims to have not received	Receipt is ambiguous — did a phone downloading mail, or a proxy pre-fetching images, count as "received"? Did someone leave a package on the porch?
	Claims to have been a fraud victim	
	Uses someone else's account	
	Uses someone else's payment instrument without authorization	
Attacking the logs	Notices you have no logs	
	Puts attacks in the logs	To confuse the logs, the log-reading code, or a person reading the logs

The two sub-classes sit at different places in the system. Repudiation **of an action** lives at the boundary where the system meets a real user — the user-interface side. **Attacking the logs** lives in the inner system, where actions must be attributed to entities but the logs are missing or have been compromised.

1.6 Information Disclosure

Information disclosure violates **confidentiality**: information reaches someone not authorized to have it. Every data-manipulating element is a typical victim — processes, data stores, and data flows.

Threat example	What the attacker does	Notes
Against a process	Extracts secrets from error messages	A verbose error can leak internal behaviour, enabling reverse engineering by interacting with the system
	Reads error messages revealing username/passwords up to entire database tables	Never answer a failed login with "that's not your password, the password is X"; even confirming a valid username aids a brute-force attack
	Extracts machine secrets from error cases	Can make a defence such as ASLR far less useful

Threat example	What the attacker does	Notes
	Extracts business/personal secrets from error cases	
Against a data store	Takes advantage of inappropriate or missing ACLs	Access-control violation; under GDPR, disclosing personal data without consent carries financial penalties
	Takes advantage of bad database permissions	
	Finds files protected by obscurity	Obfuscation is used by defenders to resist reverse engineering — and by attackers to hide malicious code
	Finds crypto keys on disk (or in memory)	
	Sees interesting information in filenames; reads files as they traverse the network	
	Gets data from logs or temp files; from swap or other temp storage	Poorly managed logs let an attacker infer usage patterns and user behaviour
	Extracts data by obtaining the device, changing the OS	
Against a data flow	Reads data on the network; redirects traffic to enable reading	The classic case of an unencrypted link; captured ciphertext can be decrypted offline at leisure
	Learns secrets by analysing traffic	Traffic analysis correlates multiple observed segments to reconstruct a secret no single segment reveals
	Learns who's talking to whom by watching DNS	
	Learns who's talking to whom by social-network inference	Reconstructs the communication graph, exposing central figures for targeted social-engineering

1.7 Denial of Service

Denial of service violates **availability**: the attacker consumes a resource the application needs until it can no longer function — processing power, bandwidth, or memory. The affected elements are processes (consuming compute), data stores (filling memory, e.g. an SQL injection that writes a huge amount of data), and data flows (saturating communication links).

Threat example	What the attacker does
DoS against a process	Absorbs memory (RAM or disk); absorbs CPU; uses the process as an amplifier
DoS against a data store	Fills the data store up; makes enough requests to slow down the system
DoS against a data flow	Consumes network resources

1.8 Elevation of Privilege

Elevation of privilege violates **authorization**: an actor with a given role and permission set escalates to acquire additional privileges. Escalation can be of user level (common user to administrator) or of capability (read access to read-and-write). The typical victim is the process — the active part of the system.

Threat example	What the attacker does	Notes
EoP against a process by corrupting the process	Sends inputs the code doesn't handle properly	These errors are very common and usually high impact
	Gains access to read or write memory inappropriately	Writing memory is (obviously) bad; reading memory can enable further attacks
Elevation through missed authorization checks	No mechanism checks whether the action is authorized	A configuration issue — granting read/write to everyone is like having no permission at all
Elevation through buggy authorization checks	Exploits a bug in the check	Centralizing such checks makes the bugs easier to manage
Elevation through data tampering	Modifies bits on disk to do things other than what the authorized user intends	E.g. editing one's own role in the access-control database to gain privileges

Local privilege escalation through the exploitation of a machine-local vulnerability (there are many such CVEs, executable once some level of access on the machine is held) is the archetypal case, and tampering-driven escalation shows how one STRIDE category can produce a threat in another.

1.9 Applying STRIDE: the per-element method

STRIDE is applied against the DFD, so the diagram must be well-defined and carry enough detail for the analysis. The first methodology works **element by element**: take each component of the DFD and, category by category, identify the threats of each kind that apply

to it. The typical-victims rule prunes the work — a category that cannot affect an element type is skipped. The applicability matrix below fixes which categories are considered for each element type.

Victim element	S	T	R	I	D	E
External entity	X		X			
Process	X	X	X	X	X	X
Data flow		X		X	X	
Data store		X	?	X	X	

A process is prone to all six categories, so every process is checked against the full STRIDE set. A data store or data flow is examined only for tampering, information disclosure, and denial of service; an external entity only for spoofing and repudiation. (Repudiation against a data store is marginal, hence the question mark.) The analyst walks the DFD, applies the relevant categories to each element, and moves to the next.

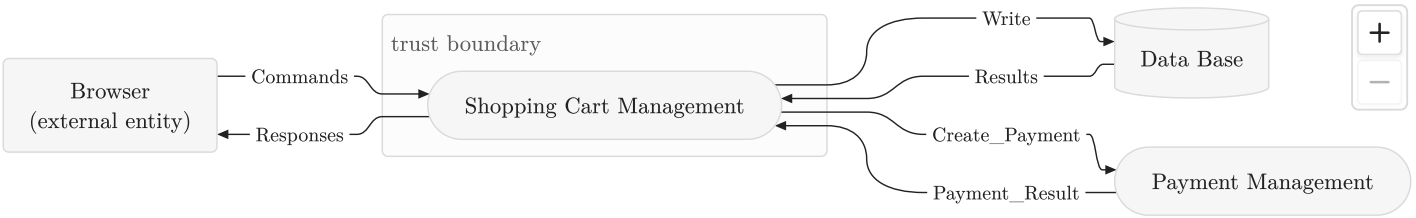
The method is not strictly sequential. Threats against one element can have side effects on others: tampering with a data flow in transit can redirect a process to a fake component, so a threat discovered on the flow forces a new threat onto a process that was already analyzed. Completing an element does not freeze it — later findings may require going back and adding threats to components already examined.

1.10 Applying STRIDE: the per-iteration method

The per-element method's blind spot — interactions between components — is addressed by **STRIDE-per-iteration**, which enumerates threats over tuples of *(origin, destination, interaction)*. Instead of analyzing a component in isolation, fix one element as a focal point and analyze each of its interactions with the rest of the system in turn: a chosen process first alone, then in connection with each data flow it handles, then with each process it talks to. For every interaction, the relevant STRIDE categories are listed together, which surfaces the cross-component side effects the per-element pass misses.

1.11 Worked example: a shopping-cart service

Consider a shopping-cart service of the kind every e-commerce site runs. A **Shopping Cart Management** process lets the user add and remove items; when the user checks out it calls an external **Payment Management** service; a **Database** stores the cart and the orders; and the user drives everything through a **Browser**.



The only trusted component is the Shopping Cart Management process, so the **trust boundary** is drawn around it. The Browser is an external entity — it interacts with the service but is not managed by us. The Payment Management service is managed by the company but not under our direct control, so it sits outside the boundary. The Database is internal but, placed on a remote machine, is treated here as external. Where the boundary falls is a deployment decision: were the database co-located on the same machine as the process, or the payment service run entirely in-house, either could be brought inside the boundary.

Applying per-iteration analysis with the Shopping Cart Management process as the focal point, each interaction is examined in turn. The interaction *process → database (write)* admits **spoofing** — the database is outside the trust boundary and no authentication was implemented, so an attacker could impersonate it and the process would write to the wrong place — and **information disclosure**, since data written to a spoofed database is handed to a third party. It carries no availability threat if the bandwidth was provisioned for the interaction frequency. The interaction *process ↔ payment service (data flow)* admits **tampering** via a man-in-the-middle with no prevention in place, **information disclosure** if the flow is not properly encrypted, and **denial of service** if the connection is mismanaged — for instance the payment service injecting oversized messages until the channel fails. Proceeding over every interaction yields the full threat list.

#	Element	Interaction	S	T	R	I	D	E
1	Shopping Cart Management	Process has outbound data flow to data store (write)	X			X		
2		Process sends output to another process	X		X	X	X	X
i	Data Flow (Payment/Result)	Crosses machine boundary		X		X	X	
j	Data Base	Process has outbound data flow to data store		X	X	X	X	

#	Element	Interaction	S	T	R	I	D	E
k	Browser	External interactor passes input to process	X		X	X		

Which threats land in the table depends on the implementation. Run on an already-built system, the entries reflect the real configuration. Run during design, the analysis assumes the worst case — an X in every applicable cell — because design time is the right time to fix issues.

1.12 The DESIST rebranding

A second acronym, **DESIST**, is sometimes encountered. It is not a different model — it is STRIDE with the categories renamed and reordered: **D**ispute (= Repudiation), **E**levation of privilege, **S**poofing, **I**nformation disclosure, **S**ervice denial (= Denial of Service), and **T**ampering. An analysis labelled DESIST is a STRIDE analysis.

1.13 Checking the model

The output of STRIDE is the filled table — a cross in each applicable category, with one or more concrete vectors behind every cross. Judging whether that result is good enough is genuinely hard, because the categories overlap, a single issue may fall into two of them depending on perspective, and one threat may generate another; the list can be revised indefinitely, each pass adding something. Three checks give confidence without ever proving completeness.

1. **At least one threat in every STRIDE category across the whole system.** It is likely that somewhere in any real system a threat of each category occurs, so a category left empty is a prompt to look again — not proof of a bad analysis, but a guard against forgetting a category while jumping between them.
2. **At least one threat per element of the diagram.** An element that comes out completely clean is worth reviewing, especially a data flow in a complex system with many parallel flows, where it is easy to miss one. Again this speaks to coverage, not quality.
3. **For more comprehensiveness, run STRIDE-per-element and ensure one threat per check.** Use the per-element pass first to avoid forgetting anything, then refine with the per-iteration pass.

Passing all three leaves you reasonably comfortable that nothing was forgotten; failing one is a reason to go back. The asymmetry is the crucial point: **not meeting these criteria tells you that you are not done, but meeting them is not a guarantee of completeness.** Threats may remain however many times the list is revised, because completeness cannot be ensured for an elicitation technique.

1.14 Observations

Three observations close the STRIDE treatment. First, STRIDE only **enumerates** what might go wrong; the exact mechanism of each threat is analyzed later. Second, it is useful to **record every possible attack even when a mitigation is already in place**, listing the mitigations in parallel with the threats — an unencrypted flow that could be tampered with or disclosed is recorded alongside its fix, "use an encrypted channel." Proceeding this way delivers each problem together with a candidate solution, and because the mitigation is a testable feature, a test case should exist for it. Third, **STRIDE is not a taxonomy or classification mechanism.** Its classes are not disjoint: one attack can generate threats in several categories, and forcing each issue into a single class breaks correlations that should be preserved. STRIDE provides classes as an indication of where problems lie; used that way — without insisting on a strict one-issue-one-class mapping — the results are good enough.

2. Attack Trees

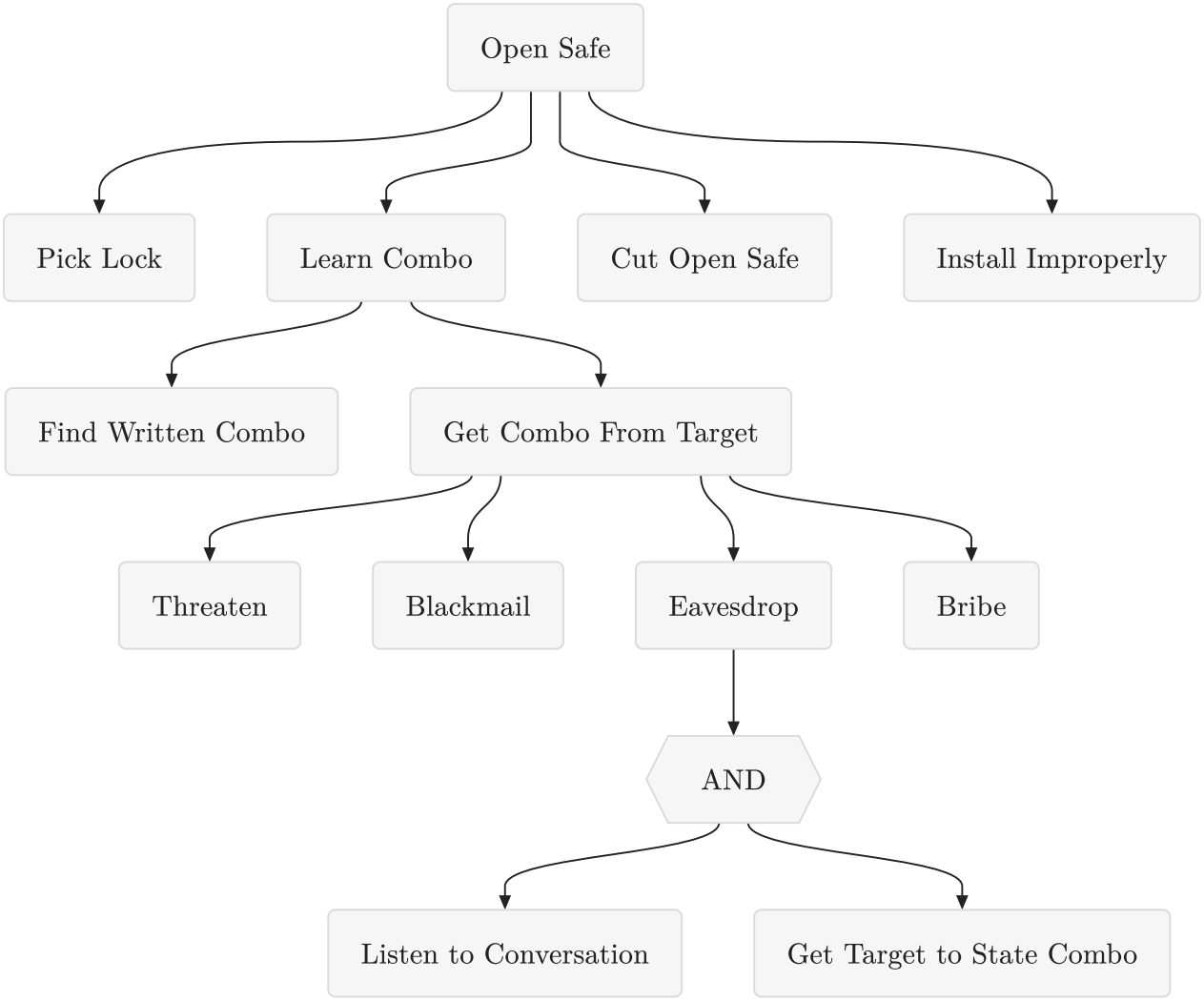
2.1 What an attack tree is

An attack tree is a graphical framework for reasoning about the attacks that may affect a system. It is built **top-down**: start from a concept and refine it, decomposing a goal into the ways it can be achieved, down to concrete leaf actions. The concepts modelled are the attacks or threats against the system under analysis. Attack trees are flexible — they can describe the security of an entire system, decompose a system into a hierarchical view, serve as a reusable knowledge base of common issues, or be labelled with quantitative information (probability, cost, feasibility) to support decision-making. As an input to risk analysis, a labelled tree lets you infer a level of risk and, combined with mitigations, support a cost–benefit analysis.

Two perspectives meet in the construction. An **asset-centric** decomposition starts from the system's functionalities and breaks them down until the leaves are the assets worth protecting. Each asset — expressed as a security property over a specific component — then becomes the **root of an attacker-centric tree** whose branches are the ways an attacker could compromise it, and whose leaves are the concrete attack actions. A complex system is therefore described not by one tree but by many, analyzed together: one tree decomposes the system (mirroring the levels of its DFD), and each asset-leaf spawns its own attack tree.

2.2 Worked example: opening a safe

A non-security example conveys the structure. The asset is a safe holding things of value; the attacker's goal — the root — is to open it. (In the cyber domain the equivalent root might be *gain root privileges on a machine.*) The attacker has alternative strategies: pick the lock, learn the combination, cut the safe open, or exploit an improper installation. Some strategies are atomic leaves; others refine further. Learning the combination splits into finding it written down (a post-it left somewhere) or getting it from the target, which in turn splits into threatening, blackmailing, bribing, or eavesdropping. Eavesdropping is special: it requires two actions **together** — listening to a conversation *and* getting the target to state the combination — a conjunction that must both hold for the technique to work.



Every leaf is a specific attack the attacker may launch, and every path from a leaf up to the root is an **attack strategy** — a distinct potential incident. The likelihood of the root goal is a combination of the likelihoods along the paths. Once the tree exists, it can be used to enumerate all leaf-to-root paths and find every attack, to attach probabilities and impacts and compute risk, and to define countermeasures.

2.3 AND vs OR representation

A tree's semantics are carried by how a parent relates to its children.

- **OR trees** — a node is satisfied if *any* child is satisfied. They represent different ways to achieve the same goal (to break into a house, pick the lock OR break a window). This is the default and the most common representation, because reasoning about an attacker is naturally reasoning about alternatives.
- **AND trees** — a node is satisfied only if *all* children are satisfied. They represent different steps in achieving a goal (to enter through a window, break the window AND climb through the opening).

By convention the default is OR, and any AND node is marked explicitly on its edges, as the eavesdrop node is above. The safe example is thus an OR tree with a single explicit AND.

2.4 Building a tree, and reusing existing ones

Creating a complete attack tree from scratch is hard even for experts, so the practical approach is to reuse. Standard attack trees already exist — including trees defined for the STRIDE taxonomy — and the sensible workflow is to take the available trees and contextualize them rather than redo everything, which wastes effort and risks an inaccurate model. In practice: start from the DFD, identify the assets, identify the targets, link predefined attack trees to those assets, and combine standard knowledge with contextual knowledge into a new tree for your system. The edges are then labelled with the feasibility and cost conditions specific to your domain, producing a customized tree. Because the contextual part — asset values, operating context — changes over time while the standard taxonomies stay relatively stable, the model must be monitored and updated continuously to stay fresh.

The construction itself follows six steps.

1. **Decide on a representation** — AND, OR, or a mix, and if mixed, which is the default and which edges must be marked explicitly.
2. **Create a root node** — the attacker's goal, instantiated to the context.
3. **Create sub-nodes** — by analyzing the system directly, or by attaching existing sub-trees to the root.
4. **Consider completeness** — check that the analysis is adequate.
5. **Prune the tree** — remove branches ruled out by context (e.g. an infeasible condition).
6. **Check the presentation** — the model is read by human eyes, so a poorly structured diagram leads to analysis mistakes.

2.5 Choosing the root node

The root can be the **component that prompts the analysis** (sub-nodes labelled with what can go wrong for it), an **adversary's goal** (sub-nodes labelled with ways to achieve it), or a **problematic state** to be reached. The problematic-state form appears when security and safety are analyzed together. The two properties are distinct: **security** means the system does nothing bad relative to its requirements, while **safety** means the system takes no action that endangers its users — a central concern for cyber-physical and critical systems such as railways, where one wants both that an attacker cannot remotely control the trains and that the train cannot itself create a dangerous situation. A combined attack tree models both, and the scenarios that lie in the intersection — attacks that break security *and* safety — become the priority; there the root represents the incident (the dangerous state) directly.

The practical suggestions follow from the usual purpose, which is to reason about risk: make the root represent the attacker's objective, tied to the notion of an incident, so that risk can be computed once the incident is represented; prefer OR trees, which are easier for a human to understand; and render the tree in a human-readable format, since these models are still read and double-checked by people even as automation of the analysis is being developed.

2.6 Valuing the tree: boolean labels and pruning

A node's value is a function of its children's values, computed from the **leaves upward to the root**, with AND and OR nodes requiring different calculations. The simplest labels are boolean — **Possible (P) vs Impossible (I)** — assigned to each leaf from the context, then propagated:

- at an **OR** node, the result is **possible if any child is possible**;
- at an **AND** node, the result is **impossible if any child is impossible**.

In the safe tree, suppose the lock is magnetic (no lock to pick) so *Pick Lock* is impossible, while *Cut Open Safe* is possible. Eavesdrop is an AND of *Listen to Conversation* (possible) and *Get Target to State Combo* (impossible), so eavesdrop is impossible. Propagating upward marks which root goals are reachable and, crucially, **which branches can be pruned**: an impossible branch is removed because it is useless to assign a remediation cost to a direction the attacker cannot take.

2.7 Other boolean labels and querying

Any boolean attribute can be codified in the leaves and used to prune or filter:

- **Easy / Complex**
- **Expensive / Cheap**
- **Intrusive / Non-intrusive**
- **Legal / Illegal** — a legal action has a different probability of being attempted than an illegal one
- **Special equipment required / not required**

Labels combine, which turns the tree into a queryable structure. After filtering out the impossible attacks, one can ask among the remainder which require no special equipment — first filtering by feasibility, then by equipment — to narrow the set of realistic attacks.

2.8 Continuous values and what-if analysis

Leaves can also carry continuous values: **cost of the attack, cost of defense, time to achieve, resources needed, probability of attack success, and likelihood that an attacker will try it**. Propagation mirrors the boolean case — at an **OR** node take the **minimum** (the cheapest alternative), at an **AND** node take the **sum** (all steps are needed).

With per-leaf costs attached, the tree answers quantitative queries. Taking the safe tree with the costs below:

Leaf	Cost	Leaf	Cost
Pick Lock	$30K	Threaten	$60K
Cut Open Safe	$10K	Blackmail	$100K
Install Improperly	$100K	Bribe	$20K

Leaf	Cost	Leaf	Cost
Find Written Combo	$75K	Listen to Conversation	$20K
		Get Target to State Combo	$40K

The cheapest attack is found by propagating minimums upward: *Eavesdrop* costs $20K + $40K = $60K (AND/sum); *Get Combo From Target* is the minimum of its children ($20K via Bribe); *Learn Combo* is $20K; and *Open Safe* is the minimum over Pick ($30K), Learn ($20K), Cut ($10K), Install ($100K) = **$10K, by cutting the safe open**. The same model answers which attacks cost less than a given budget, and — combining the cost labels with the equipment labels — the cheapest attack requiring no special equipment. Countermeasures enter as contextual edits: applying a countermeasure that raises the cost of *Bribe* from $20K to $80K changes the cheapest path, which is exactly the **what-if analysis** the labelled tree supports. Once labels are injected, the model becomes a flexible instrument for querying attack feasibility, cost, and risk.

2.9 Human-viewable representations

Because attack trees are read by people, presentation is part of correctness. A tree can be drawn graphically or written in outline (indented) form, but in either case it should be **compact — no more than one page**; a tree too large to fit is split into multiple trees. Graphical representations must be **information-rich and communicative**: the same logical tree drawn with ambiguous or cluttered connectors invites analysis mistakes, whereas a clearly structured layout with explicit AND/OR markings lets the eye track the logic. The quality of the drawing, not just the content, determines whether the analysis can be trusted.

3. Attack Libraries

3.1 What attack libraries are, and the abstraction spectrum

An attack library is a public, pre-built collection of attack knowledge from which the knowledge needed for a specific analysis can be extracted and turned into a concrete attack technique. The predefined attack trees for the STRIDE taxonomy are an example: a reference such as a threat-modelling text ships, in its appendix, a tree for each STRIDE class (a *Spoof a file* tree, for instance). Reuse is then a matter of matching the tree's root to the corresponding element of your contextual attack tree, which immediately supplies all the ways that threat can be realized — after which you assign the context-specific labels and prune what is irrelevant.

Libraries differ along three axes: the **level of detail** they provide, their **target audience**, and their **scope**. They span a spectrum from abstract to detailed.

Abstraction	Library	Character
Abstract	**STRIDE**	Six threat classes; you list specific instances under each

Abstraction	Library	Character
	OWASP Top 10	The ten most critical issues for a domain, as general headings
	CAPEC	Structured catalogue of concrete attack patterns
Detailed	**Check lists**	Specific controls to verify

An abstract library gives general concepts and leaves it to the analyst to link them to the system; a detailed one tells exactly how an attack works and often references available exploits. Which to use depends on the task. MITRE ATT&CK, used earlier in the course, sits among these with its own granularity. The different libraries are complementary data sources for the threat-identification stage.

3.2 CAPEC

CAPEC (Common Attack Pattern Enumeration and Classification) is a publicly available catalogue of attack patterns with a comprehensive schema and classification taxonomy, created to assist in building secure software. It is organized as a **tree-shaped structure**: a root concept refines down to leaves, where each leaf is a specific attack pattern. The whole repository (a few hundred megabytes) can be exported and imported into a local database, then connected to your contextual attack tree, so that starting from a class of threat you obtain directly the specific pattern that realizes it — solving the threat-identification problem.

Each CAPEC entry is rich with fields usable both to understand an attack and to **label and prune an attack tree**: typical severity; a description with a summary and an attack execution flow; prerequisites; method(s) of attack; examples; attacker skills or knowledge required; resources required; probing techniques; indicators/warnings of attack; solutions and mitigations; attack motivation/consequences; vector; payload; relevant security requirements, principles and guidance; technical context; and bookkeeping fields linking related patterns and weaknesses. The prerequisites let you prune by the privilege level an attacker must already hold; the resources-required and impact fields quantify what the attack needs and the CIA violation it causes on the asset if it succeeds.

A concrete entry, **HTTP Response Splitting** (typical severity *High*), illustrates the schema. The attack causes a vulnerable server to emit a response that is interpreted as two responses, by injecting unvalidated user-controlled input into the response headers; the attacker identifies the controllable input, crafts input that terminates the real response and starts a second attacker-controlled one, and causes the victim to interpret the split response as belonging to a dummy request (real-world instance CVE-2006-0207). Its method is injection and protocol manipulation; its CIA impact is confidentiality High, integrity High, availability Low; and it is tied to related weaknesses CWE-113 (HTTP Response Splitting), CWE-74, CWE-697, CWE-707, and CWE-713. The mitigation is to keep data separate from headers and validate all data used in response headers.

CAPEC groups its patterns into browsable categories — among them Abuse of Functionality, Architectural Manipulation, Authentication Attacks, Code Inclusion, Data Leakage, Data Structure Attacks, Elevation of Privilege, File Manipulation, Flow Manipulation, Injection, Manipulating User Interface, Networking Attacks, Probabilistic Techniques, Resource Depletion, and Social Engineering. Used this way, CAPEC informs the attack-enumeration process, helps identify problems, and builds awareness of the breadth of threats.

3.3 CAPEC vs STRIDE

CAPEC and STRIDE are different kinds of thing and are most effective in combination. STRIDE is a **set of security properties** — a property-based classification that is easy to apply and, because it is property-based, makes defences easier to find than attacks. CAPEC is a **classification of common attacks** — richer and more detailed, going to how attacks actually work, and correspondingly more complex to address. The natural workflow runs STRIDE first against the DFD to identify the classes of threat, then matches each identified class to the specific CAPEC pattern(s) that realize it.

	STRIDE	**CAPEC**
Nature	Set of security properties	Classification of common attacks
Ease	Easier to find defences than attacks	More complex to address attacks
Role	Property-based	Richer elicitation technique

3.4 OWASP Top 10

The OWASP Top 10 is an awareness document representing a broad consensus on the most critical security risks to web applications; produced by security experts worldwide, it has become a de facto application-security standard. In threat modelling it serves as a checklist of priorities: take each of the ten entries as the **root of an attack tree** in your system and investigate whether your environment offers a path that leads an attacker to that goal. The list is reviewed roughly every four years.

#	**OWASP Top 10 — 2021**	**OWASP Top 10 — 2017**
1	Broken Access Control	Injection
2	Cryptographic Failure	Broken Authentication
3	Injection	Sensitive Data Exposure
4	Insecure Design	XML External Entities (XXE)
5	Security Misconfiguration	Broken Access Control
6	Vulnerable and Outdated Components	Security Misconfiguration
7	Identification and Authentication Failures	Cross-Site Scripting (XSS)

#	OWASP Top 10 — 2021	OWASP Top 10 — 2017
8	Software and Data Integrity Failures	Insecure Deserialization
9	Security Logging and Monitoring Failures	Using Components with Known Vulnerabilities
10	Server-Side Request Forgery	Insufficient Logging & Monitoring

Comparing editions makes a durable point: the rankings shift, but the issues persist. Broken Access Control remains at or near the top; the overlap between 2017 and 2021 is substantial. Problems move up or down because technology and the defences deployed against them change, and because in production a system cannot be rebuilt from scratch — once an issue is present, one must often live with it. The persistence of the catalogue is precisely why investing in these libraries for strategy identification and risk assessment is worthwhile: when the list changes every few years, you adapt an existing structure rather than start over.

3.5 The OWASP risk-rating methodology and per-issue data

For each issue, OWASP supplies the threat agents and attack vectors, the security weaknesses involved, the controls that mitigate it, and the business impact — a set of fields that can be extracted to feed a risk analysis. OWASP also provides its own **risk-rating methodology**, consistent with the taxonomy, so the same tool estimates risk directly from the classification. Each factor is scored on a three-point scale.

Threat agents	Exploitability	Weakness prevalence	Weakness detectability	Technical impacts	Business impacts
Application-specific	Easy: 3	Widespread: 3	Easy: 3	Severe: 3	Business-specific
	Average: 2	Common: 2	Average: 2	Moderate: 2	
	Difficult: 1	Uncommon: 1	Difficult: 1	Minor: 1	

Each entry comes with a per-issue sheet — as in the *A1: Injection* entry, which walks threat agents → attack vectors → security weakness → technical/business impacts, states what makes the application vulnerable (unvalidated, unfiltered, or unsanitized user input; dynamic queries without context-aware escaping) and how to prevent it (a safe parameterized API, positive server-side input validation, escaping, and `LIMIT` to bound disclosure). These characteristics are exactly what you extract to label the attack tree you are building.

3.6 Integrating libraries into threat modelling

The libraries interconnect, which is what makes them powerful in practice. The OWASP classification links to the CWE (Common Weakness Enumeration) repository, and CWE in turn connects to CAPEC. Downloading these datasets offline and correlating them lets you build tooling that attaches the pre-constructed, standard trees to the domain-specific attack

tree built for your organization. The value is not in memorizing the Top 10 or the identifiers of individual patterns, but in understanding the **role** these external libraries play with respect to threat modelling and risk analysis: they supply the standard knowledge — classes of threat, concrete attack patterns, weaknesses, controls, and risk-rating scales — that you contextualize and label onto your own models, so that knowing the structure of each source tells you exactly where to find the information a given task needs.

CHAPTER 7: Attack graphs

Multi-Step Attacks and Attack Graphs

From Single-Host Security to Distributed Systems

Classical host security reasons about one machine at a time. A server exposes some vulnerabilities, and the goal is to secure it: enumerate its weaknesses, consult catalogues such as CWE to understand each weakness in depth, and associate mitigation actions and security controls to patch it. This model works when the object of study is a single critical host.

Modern systems are rarely isolated. They are distributed, with machines that communicate and depend on each other, and this interconnection changes the nature of the threat. A vulnerability no longer matters only for the host that carries it; it matters for everything that host can reach.

Motivating example: remote exploitation of GitLab

Project-management platforms like GitLab host source code, pipelines, and stored credentials, and a flaw in such a service can expose everything it manages. A concrete case is **CVE-2024-45409**, an authentication bypass in the Ruby-SAML library used by GitLab. The SAML protocol exchange carries an authentication token inside the response; by inspecting the page source, an attacker retrieves this token and uses it to authenticate into an existing account, including an administrator account. Once authenticated as an admin, the attacker can read, alter, or delete any project.

Three properties make this exploit severe. It is **remotely exploitable**, so no local or physical access to the host is needed. It is **trivial to perform**, requiring only a few seconds and a browser. And its **impact is total**, granting root-equivalent control over the account. Yet this is still a single exploit against a single service. The subject of this chapter is the space that opens up *after* such a foothold: how an attacker moves through a distributed network to reach a target that is not directly accessible.

Multi-Step Attacks

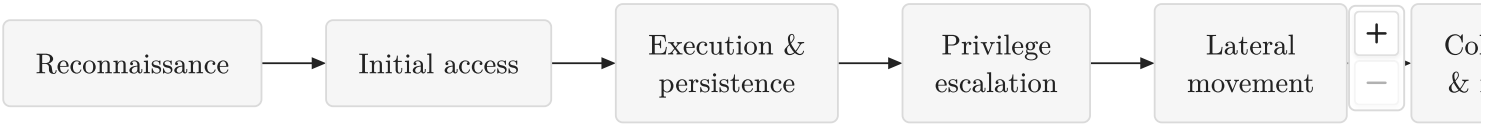
A **multi-step attack** is an attack that requires the attacker to perform several distinct steps to reach a specific goal. In a network with many hosts, the number of strategies available to the attacker is large: from an entry host she might pass through a web server to reach a

database server, or traverse two firewalls along a different route. Every one of these combinations of hops is a candidate attack.

This is hard to model precisely because each jump from one step to the next requires exploiting a vulnerability. The analysis must therefore manage many vulnerabilities and exploits at once and, crucially, understand how they connect to one another.

The common attack chain

Multi-step attacks are described through a common attacker lifecycle, with particular emphasis on the middle stages where movement happens.



- **Reconnaissance:** identify exposed services, software versions, identities, endpoints, and trust relationships. This stage is symmetric: the attacker looks for instruments to exploit, while the defender looks for exposed services and open ports.
- **Initial access:** gain the first foothold by exploiting a public-facing application, stealing credentials, abusing a misconfigured service, or compromising a supplier.
- **Execution and persistence:** run malicious code, establish a backdoor, create tokens, or modify authentication mechanisms.
- **Privilege escalation:** obtain stronger permissions. Entering a system is not enough; the attacker needs sufficient privileges to perform disruptive actions. If she lands with only guest rights, her actions are limited. Modeling these privileges is what lets the analysis quantify the risk of the threat.
- **Lateral movement:** move across hosts, containers, services, network zones, or cloud accounts, exploiting a different vulnerability at each hop to progress toward the goal.
- **Collection and impact:** access sensitive data, deploy ransomware, disrupt availability, or manipulate business processes.

The question that drives this analysis is therefore not *"is this component vulnerable"* — in practice every component is vulnerable in some way — but *"what can an attacker reach after exploiting this component."*

Single-step versus multi-step attacks

Dimension	Single-step	Multi-step
Scope	A specific asset or a specific vulnerability	Multiple assets and services, with impact spread across several security controls

Dimension	Single-step	Multi-step
Objective	Compromise one target	Reach the goal *and* sustain lateral movement; the objective is tied to the infrastructure, not a single host
Required knowledge	Limited: one environment to study and test	Large: many systems, firewalls, domains, and rules to understand
Typical defenses	Single-device controls: patching, input validation, filtering (CWE mitigations)	Network-level controls: segmentation, firewalls, intrusion detection and prevention, monitoring
Detection	Easier: bound to one host, so monitoring can focus	Harder: distributed and deliberately crafted to bypass monitoring
Risk assessment	Vulnerability-centric	Path-centric

The objective of a single-step and a multi-step attack can look identical, since the attacker's ultimate goal may be the same. The difference is that when a target is reachable directly, a single step suffices; a multi-step attack is needed precisely when the target sits inside a distributed system with no direct access path, forcing progressive movement. When every vulnerability in the chain is trivial to exploit, the overall complexity of a multi-step attack can even approach that of a single-step one.

The detection gap deserves emphasis. The same host may be effectively unreachable through a single-step attack because it is heavily monitored, yet become easy to compromise through a multi-step path: after gaining root on an intermediate host, the attacker can operate from a position that bypasses the monitoring entirely.

Why multi-step attacks matter in distributed systems

Distributed systems introduce characteristics that make multi-step attacks both more likely and harder to analyze:

- **Scale:** a system may comprise a huge number of interconnected machines, infeasible to model by hand with threat models such as STRIDE or attack trees unless the process is automated.
- **Remote exploitation:** some vulnerabilities let the attacker reach a goal without entering the system, and remoteness has degrees — within a LAN, between LANs, or fully across the internet — each of which must be modeled.
- **Replication:** identical machines carry identical vulnerabilities, offering multiple equivalent entry points with the same behavior and the same risk.
- **Multiple domains:** each domain may enforce its own firewall and access rules.
- **Dynamicity:** the model must tolerate changes in the network over time.
- **Complex trust assumptions:** trust boundaries exist not only at firewalls but also inside a single LAN, and these internal boundaries must be modeled and assessed.

Attack Graphs

An **attack graph** is the threat model used to represent multi-step attacks. It is a graph-based structure that represents all the possible ways a potential attacker can intrude into a target network by exploiting a series of vulnerabilities. Each such path through the graph *is* a multi-step attack.

Typically, **nodes represent the privileges** the attacker holds on a specific host, and **edges represent the exploit of a vulnerability**. An edge from one node to another means the attacker can move from the source privilege to the target privilege by exploiting the vulnerability labeling that edge. Building such a graph requires heterogeneous information: the reachability within the network, the vulnerabilities present, and the privileges each vulnerability grants.

Formalization

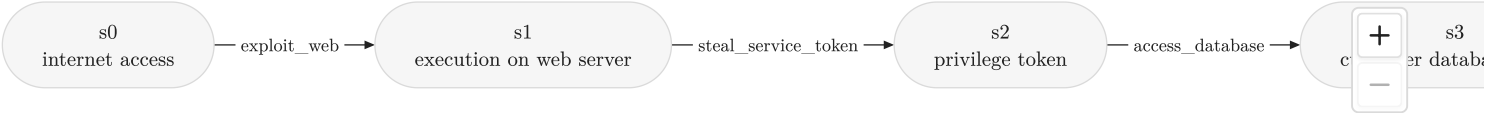
A state-based attack graph is defined as:

$$AG = \langle S,\ \tau,\ S_0,\ S_G,\ L \rangle$$

- S — a finite set of security-relevant system states. Most often a state models the set of privileges on a specific host, but it can represent any security-relevant situation.
- $\tau \subseteq S \times S$ — the transition relation, describing how the attacker can move from one state to another.
- $S_0 \subseteq S$ — the set of initial capabilities, i.e. the attacker's starting privileges and the source of every attack path.
- $S_G \subseteq S$ — the set of goal states the attacker wants to reach. Both S_0 and S_G are subsets of S, because the reasoning happens over the nodes and edges of the graph.
- $L : S \to 2^{AP}$ — a labeling function mapping each state to a set of atomic propositions (AP). Because its codomain is the power set of AP, a state can be labeled with any combination of atomic propositions.

Example: a four-state attack graph

Consider four states describing an attacker's progression from the internet to a customer database:



- s_0 — the attacker has only internet access (the initial state).
- s_1 — the attacker has code execution on the web server, reached via the atomic proposition `exploit_web`.
- s_2 — the attacker possesses a privileged service token, reached via `steal_service_token`.
- s_3 — the attacker can access the customer database, reached via `access_database`.

The states here model situations, and the edges model the actions that shift the attacker between them. Both choices are free: a state can encode anything security-relevant, and an action can be any mechanism of transition, though in practice it is almost always the exploitation of a vulnerability.

Preconditions and postconditions

To decide what labels an edge — how to model a transition $s_i \xrightarrow{a} s_j$ — each attack action a is described as a pair of conditions:

$$a = \langle Pre(a),\ Post(a) \rangle$$

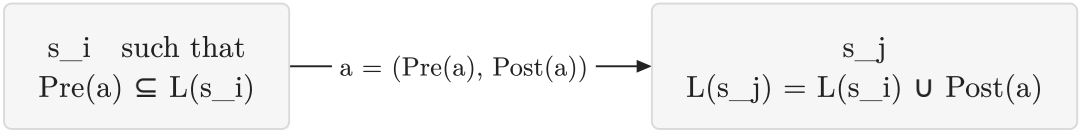
- $Pre(a)$ — the set of facts the attacker must already satisfy to perform action a.
- $Post(a)$ — the set of facts made true after a is successfully executed (the gains the attacker obtains).

An action a is **enabled** in state s when its preconditions hold in that state:

$$Pre(a) \subseteq L(s)$$

After the action, the resulting state s' accumulates the postconditions on top of what already held:

$$L(s') = L(s) \cup Post(a)$$



The union in the postcondition carries a consequence worth stating explicitly: whatever held before the exploit still holds after it, so a postcondition always implicitly contains its precondition.

Generating an Attack Graph

Like every threat model, attack graph generation starts from a model of the environment. The term "attack graph generation" simply names the procedure for building these graphs; once a graph exists, it can be used for risk assessment.

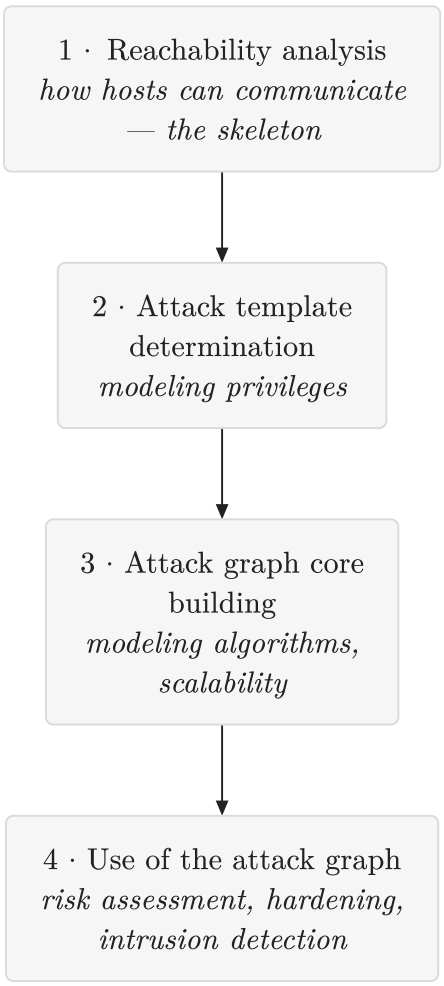
Required inputs

- **Network topology and connectivity** between hosts.
- **Hosts, services, and software versions** running in the system.
- **Known vulnerabilities**, which depend on the software and configuration present.
- **User and service identities**, including external entities such as the attacker.
- **Access-control policies and privileges.**

- **Trust relationships** — firewall rules, access rules, and the trust assumptions that let the labeling function model transitions.

The conceptual process is: *system facts + attack rules + initial attacker capabilities → attack graph → risk assessment.*

The four phases



1. **Reachability analysis** — collect how hosts communicate and which vulnerabilities each host carries. The reachability information forms the skeleton of the attack graph: it is the worst-case set of communication channels the attacker could exploit.
2. **Attack template determination (modeling privileges)** — decide how vulnerabilities link to one another and how privileges are represented, producing the preconditions and postconditions. The attack graph is effectively a subset of the reachability graph, keeping only the channels that can actually be exploited.
3. **Attack graph core building (modeling algorithms)** — given the reachability conditions and per-host privileges, run an algorithm that computes the possible attack paths, and address the scalability problem that arises in large distributed systems.
4. **Use of the attack graph** — apply the finished model to cyber risk assessment, network hardening, intrusion detection, and related tasks.

Phase 1 — Reachability Analysis

Reachability analysis investigates how the hosts in a network can communicate with one another, where "communication" means any kind of exchange two hosts may have. This matters because any such exchange is something an attacker can potentially exploit. The result is the skeleton of the attack graph: it fixes the worst-case set of channels along which the attacker could move.

Taxonomy: scope and content

The analysis is characterized by two components.

Reachability scope defines which parts of the network are analyzed. It can be the whole network, or selected subnetworks. Restricting the scope is useful for three reasons: to keep the analysis computationally feasible, to keep the resulting graph small enough to inspect visually, or simply because the network is already naturally divided into subnetworks.

Reachability content defines what kind of exchange is modeled as a reachability condition:

- **Access-control rules** — the rules enforced by firewalls or access-control lists.
- **Intrusions** — reachability inferred from detection data. When no direct monitoring is available, the source and destination IPs recorded in intrusion-detection alerts can be used to reconstruct which hosts talk to which.
- **Trust relationships** — the rules that permit trust, and therefore data exchange, between two devices or subnetworks.
- **Application relationships** — the network protocols, modeling whether one host can exchange packets with another over a given protocol.

Output: the reachability graph and matrix

The phase produces a **reachability graph** or a **reachability matrix**, either of which lets the analysis reason about the reachability conditions present in the network. In the graph, nodes are hosts and edges are the reachability conditions (for example the protocol each pair uses). The same information translates into a matrix whose rows are sources and columns are targets.

Consider four hosts. Each host is assumed able to communicate with itself, so the diagonal is set. The matrix can record either the mere **existence** of a reachability condition (a binary value) or the specific **protocol** that realizes it. Links need not be bidirectional.

source ↓ \ target →	H1	H2	H3	H4
H1	Y	Y	Y	0
H2	Y	Y	0	0
H3	Y	0	Y	Y
H4	0	0	0	Y

Here H3 can reach H4, but H4 cannot reply to H3: the relation is directional. A protocol-aware variant replaces each `Y` with the service that carries it, e.g. `HTTP` between H1 and H2.

Two modeling cautions follow. First, accuracy is critical: any error here propagates into the final attack graph, because a link omitted at this stage becomes a path the attacker is assumed unable to take. Second, the question to ask is not whether a target is directly reachable, but through which combinations of hops it can be reached. If one reasons only from direct links, a host with no inbound edge from the attacker looks safe — and that conclusion is exactly what multi-step analysis exists to overturn. For this reason an explicit attacker host H_0 is added as the external origin of all paths.

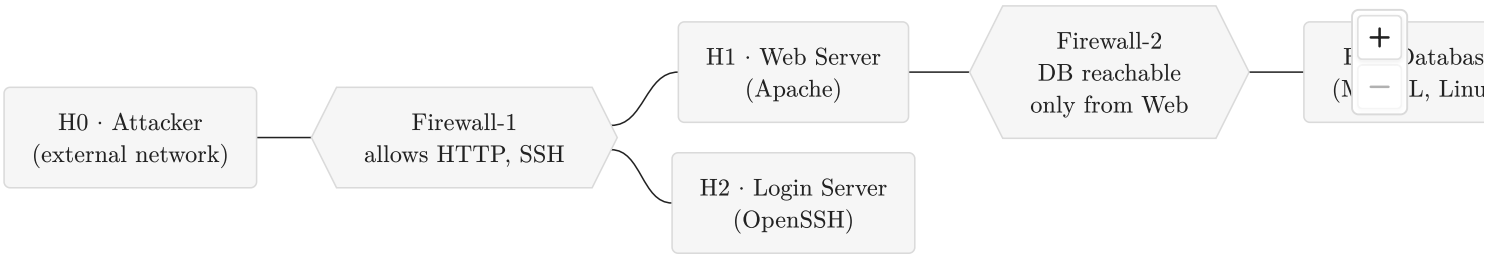
Information and tools

Computing the matrix requires, at minimum, the **network topology** (the network configuration). Depending on which reachability conditions are modeled, it may also require application and filtering rules, detection configuration, and trust relationships. In the ideal case the analyst has complete knowledge of the system and produces the most accurate matrix; in practice the available data is limited, and the data on hand determines which attack graph can be built.

Two tool families support the phase. **Topology builders** (such as Batfish, Minesweeper, and Tiros) reconstruct the network structure when the configuration is not directly available, often by probing or approximation. **Vulnerability scanners**, which are probe-based, associate each host with its set of vulnerabilities — the complete input the later phases need. Firewall rules interact with this directly: a link that exists physically may be rendered non-exploitable because a firewall blocks the specific application being modeled, in which case that entry is set to zero within the modeled context.

Running example: "NetSec"

This example is carried through the remaining phases.



The network has two firewalls protecting a demilitarized zone (DMZ). **Firewall-1** separates the DMZ from the external network where the attacker sits (host H_0); it allows HTTP and SSH traffic to the Web and Login servers and blocks everything else. **Firewall-2** separates the DMZ from the internal network containing the Database server (H_3); it allows access to the database only from the Web server. The DMZ contains the Web server (H_1) and the Login server (H_2); the internal network contains the Database server (H_3).

Each host runs vulnerable software:

Software	Host	Vulnerability
Apache Web Server	H1	V1 — CVE-2023-37941 (remote → user on Web server)
OpenSSH	H2	V2 — CVE-2021-34718 (remote → user)
MySQL	H3	V3 — CVE-2023-48712 (remote → user)
Linux Kernel	H3	V4 — CVE-2019-3819 (local → root)

Modeling all links as bidirectional data exchange among trusted devices, and removing the self-connections, yields the reachability matrix:

source ↓ \ target →	H0	H1	H2	H3
H0	—	1 (http, ssh)		
H1	1 (http, ssh)	—	1	1
H2		1	—	
H3		1		1

The decisive observation: H_0 cannot communicate with H_3. An analysis that stops at direct reachability therefore declares the database server safe from external attackers — true only for a single-step attack. Because H_0 can reach H_1, and H_1 can reach H_3, the multi-step path $H_0 \rightarrow H_1 \rightarrow H_3$ compromises the database. Any inaccuracy in this reachability model propagates directly into the later cyber-risk-assessment phase.

Phase 2 — Modeling Privileges (Attack Template Determination)

The attack graph models how privileges are gained and escalated by the attacker to reach a goal, so this phase determines the privileges associated with each vulnerability in the network. The mechanism used to model these privileges is the **attack template**: the skeleton of the attack graph, specifying the conditions an attacker must satisfy to *successfully* exploit a vulnerability and the privilege gained afterward. The emphasis on success is deliberate — many combinations of privileges could be written down, but only those that can actually be exploited belong in the graph, and preconditions and postconditions are what validate them.

Access levels

Privileges can be modeled in any custom scheme, but the common approximation uses the rights an attacker holds on a host, drawn from identity and access management:

- **guest** — minimal access,
- **user** — ordinary read/write rights,

- **root** — full control.

Finer-grained privilege models (for instance explicit formal properties) yield a more accurate attack graph, at the cost of enlarging the state space.

Three approaches to modeling privileges

1. **Manual.** A human expert reads each vulnerability description and assigns its preconditions and postconditions from expertise. This is feasible for a handful of vulnerabilities but is slow and biased by the analyst's judgment.
2. **Data-based.** Preconditions and postconditions are derived from standardized, structured data. This is the approach used here, based on CVSS metrics. Other data, such as historical IDS alerts, can similarly be mined to infer privileges from past suspicious movements.
3. **NLP/ML-based.** Models embed CVE descriptions as vectors and, using encoders pre-trained on security text (such as ATT&CK-BERT or SecureBERT), predict privileges for previously unseen vulnerabilities by similarity to known ones. Nominal accuracy is around 90% but is lower in practice, so a prediction should be re-weighted by the measured accuracy of the tool before it feeds the risk assessment.

CVSS-based modeling

With the data-based approach, each metric plays a distinct role. The **Access Vector** determines the *existence and shape of the edge* (how remote the attacker may be). The **authentication / privileges-required** metric determines the *privilege the attacker must hold on the source host* — the precondition. The **impact** metrics (Confidentiality, Integrity, Availability), taken at their worst case, determine the *privilege gained on the destination host* — the postcondition. Preconditions and postconditions are produced by two functions, which differ slightly between CVSS versions because the metrics changed; both versions must be supported, since a given CVE may be scored under only one of them.

CVSS v2

Version 2 uses the **Access Vector (AV)** and **Authentication (Au)** for the precondition.

AV	Meaning
Local (L)	Exploitable only with local or physical access (e.g. local privilege escalation via `sudo`, Firewire/USB DMA).
Adjacent Network (A)	Requires access to the broadcast or collision domain (same IP subnet, Bluetooth, 802.11, local Ethernet).
Network (N)	Remotely exploitable; the software is bound to the network stack (e.g. an RPC buffer overflow).

Au	Meaning
Multiple (M)	The attacker must authenticate two or more times.
Single (S)	The attacker must be logged into the system once.
None (N)	No authentication required.

```
function preCondition(AV, AC, Au):
    if AV == N:   reachability = REMOTE
    else if AV == A: reachability = ADJACENT_NETWORK
    else if AV == L: reachability = LOCAL_HOST_ACCESS

    if Au == N:   privilege = NONE      # guest
    else if Au == S: privilege = USER
    else if Au == M: privilege = ROOT

    return (reachability, AC, privilege)

function postCondition(C, I, A):
    impact = maximum(C, I, A)   using the order  N < P < C
    if impact == N:   return NONE        # guest
    else if impact == P: return USER
    else:             return ROOT
```

The reachability value gives the edge its shape (whether the source host may sit anywhere, in an adjacent LAN, or only on the same host); the precondition privilege is what the attacker must already hold on the source; the postcondition is the privilege obtained on the target, taken from the worst of the three impact dimensions.

CVSS v3.1 and v4

The later versions use the **Attack Vector (AV)**, **Privileges Required (PR)**, and **Scope (S)**, which allow more accurate reasoning. AV adds a fourth value, **Physical (P)**, which signals that an edge can exist on the same host without network movement — useful for local privilege escalation. PR maps almost one-to-one onto the privilege model: None → guest, Low → user, High → root. Scope records whether the exploit's impact stays within the vulnerable component's security authority (Unchanged) or crosses it (Changed).

AV	Reachability
Network (N)	REMOTE
Adjacent (A)	ADJACENT_NETWORK
Local (L)	LOCAL_HOST_ACCESS

AV	Reachability
Physical (P)	PHYSICAL_ACCESS

```
function preCondition(AV, PR):
    if AV == N:   reachability = REMOTE
    else if AV == A: reachability = ADJACENT_NETWORK
    else if AV == L: reachability = LOCAL_HOST_ACCESS
    else if AV == P: reachability = PHYSICAL_ACCESS
    else: return NOT_APPLICABLE

    if PR == N:   privilege = NONE       # guest
    else if PR == L: privilege = USER
    else if PR == H: privilege = ROOT
    else: return NOT_APPLICABLE

    return (reachability, privilege)

function postCondition(S, C, I, A):
    CIA_max = maximum(C, I, A)   using the order  N < L < H
    if CIA_max == N:                 return NONE    # guest
    if CIA_max == L:                 return USER
    if CIA_max == H and S == U:      return USER
    if CIA_max == H and S == C:      return ROOT
```

The scope refines the postcondition: a high impact confined to the same authority (S = Unchanged) is capped at **user**, whereas a high impact that crosses the boundary (S = Changed) yields **root**, reflecting a genuine privilege escalation.

Worked example: CVE-2023-37941 on NetSec

Take the Apache vulnerability on the Web server, scored under CVSS v3.1:

```
CVSS:3.1/AV:N/AC:H/PR:H/UI:N/S:U/C:H/I:H/A:H
```

Postcondition (privilege gained on the target host H_1). Because the vulnerability resides on H_1, its postcondition is the privilege the attacker gains there.

Step	Metric / condition	Value	Interpretation	Output
1	Confidentiality impact	C:H	High loss of confidentiality	—

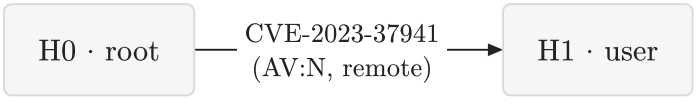
Step	Metric / condition	Value	Interpretation	Output
2	Integrity impact	I:H	High loss of integrity	—
3	Availability impact	A:H	High loss of availability	—
4	CIA aggregation	max(H,H,H)	Take the maximum impact	$CIA_{\max} = H$
5	Scope	S:U	Impact stays within the same security authority	scope = UNCHANGED
6	Resulting-privilege rule	$CIA_{\max} = H$, S=U	High impact, unchanged scope	**USER**

The attacker gains **user** on H_1.

Precondition (edge existence and required source privilege).

Step	Metric / condition	Value	Interpretation	Output
1	Firewall-1 rule	HTTP allowed $H_0 \to H_1$	H_1 is reachable over the network	—
2	Attack Vector	AV:N	Reachable remotely (assuming firewall/access rules satisfied)	reachability = REMOTE → **edge exists**
3	Attack Complexity	AC:H	Exploitation requires difficult, uncontrolled conditions	complexity = HIGH
4	Privileges Required	PR:H	Attacker must already hold high privileges on the source	privilege = ROOT
5	User Interaction	UI:N	No action by another user needed	—
6	preCondition(N, H)	—	Apply the required-privilege rule	**ROOT** on source

The resulting edge is therefore:



read as: an attacker holding **root on** H_0 can exploit CVE-2023-37941 over the network to obtain **user on** H_1. Applying the two functions to every vulnerability on every host produces

the full set of modeled edges; combining these edges with the reachability skeleton, and then chaining them, is what yields the complete attack graph.